



US012411867B2

(12) **United States Patent**
Karr et al.

(10) **Patent No.: US 12,411,867 B2**
(45) **Date of Patent: Sep. 9, 2025**

(54) **PROVIDING APPLICATION-SIDE
INFRASTRUCTURE TO CONTROL
CROSS-REGION REPLICATED OBJECT
STORES**

(56) **References Cited**

U.S. PATENT DOCUMENTS

5,706,210 A 1/1998 Kumano et al.
5,799,200 A 8/1998 Brant et al.
(Continued)

(71) Applicant: **PURE STORAGE, INC.**, Santa Clara,
CA (US)

FOREIGN PATENT DOCUMENTS

EP 0725324 A2 8/1996
WO 2012087648 A2 6/2012
(Continued)

(72) Inventors: **Ronald Karr**, Palo Alto, CA (US);
Naveen Neelakantam, Mountain View,
CA (US); **Ivan Jibaja**, San Jose, CA
(US)

(73) Assignee: **PURE STORAGE, INC.**, Santa Clara,
CA (US)

OTHER PUBLICATIONS

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 71 days.

Tunable consistency guarantees of selective data consistency model
by Phansalkar Journal of Cloud Computing 4.1: 1-12. Heidelberg:
Springer Nature B.V. (Jun. 2015) [https://dialog.proquest.com/
professional/cv_2188612/docview/1774943517/fulltextPDF/
19324A9A234/1?accountid=131444](https://dialog.proquest.com/professional/cv_2188612/docview/1774943517/fulltextPDF/19324A9A234/1?accountid=131444) (Year: 2015).*

(21) Appl. No.: **18/446,303**

(22) Filed: **Aug. 8, 2023**

(65) **Prior Publication Data**

US 2023/0394065 A1 Dec. 7, 2023

Primary Examiner — Joseph O Schell

(74) *Attorney, Agent, or Firm* — Replicated Object Stores

Related U.S. Application Data

(63) Continuation-in-part of application No. 18/152,148,
filed on Jan. 9, 2023, now Pat. No. 12,314,134.
(Continued)

(57) **ABSTRACT**

Providing application-side infrastructure to control cross-
region replicated object stores is disclosed. An embodiment
includes determining, for an object store replicated across a
plurality of storage systems at a plurality of locations, a
minimum time interval between requests to modify the
object store that are received by differing storage systems of
the plurality of storage systems that ensures a predictable
result on all storage systems among which the requested
modifications are replicated; and providing, through an
application programming interface (API) to a client that
utilizes the object store, one or more object store parameters
including the minimum time interval.

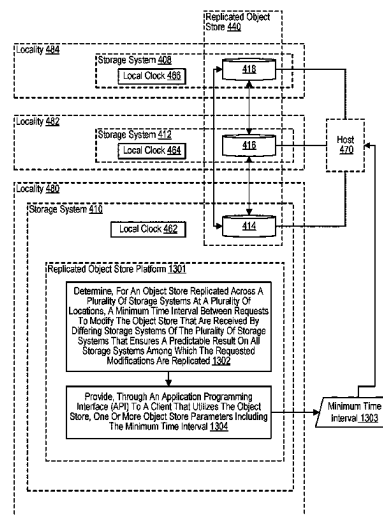
(51) **Int. Cl.**
G06F 11/08 (2006.01)
G06F 11/20 (2006.01)
G06F 16/27 (2019.01)

(52) **U.S. Cl.**
CPC **G06F 16/27** (2019.01); **G06F 11/2079**
(2013.01); **G06F 11/2064** (2013.01)

(58) **Field of Classification Search**
CPC G06F 11/1415; G06F 11/2064; G06F
11/2074; G06F 11/2079; G06F 16/27;
G06F 3/067; G06F 3/065; G06F 16/275

See application file for complete search history.

20 Claims, 41 Drawing Sheets



2003/0225961	A1	12/2003	Chow et al.	
2004/0030837	A1*	2/2004	Geiner	G06F 11/2076 714/E11.107
2004/0080985	A1	4/2004	Chang et al.	
2004/0111573	A1	6/2004	Garthwaite	
2004/0153844	A1	8/2004	Ghose et al.	
2004/0193814	A1	9/2004	Erickson et al.	
2004/0260967	A1	12/2004	Guha et al.	
2005/0160416	A1	7/2005	Jamison	
2005/0188246	A1	8/2005	Emberly et al.	
2005/0216800	A1	9/2005	Bicknell et al.	
2006/0015771	A1	1/2006	Van Gundy et al.	
2006/0129817	A1	6/2006	Borneman et al.	
2006/0161726	A1	7/2006	Lasser	
2006/0230245	A1	10/2006	Gounares et al.	
2006/0239075	A1	10/2006	Williams et al.	
2007/0022227	A1	1/2007	Miki	
2007/0028068	A1	2/2007	Golding et al.	
2007/0055702	A1	3/2007	Fridella et al.	
2007/0109856	A1	5/2007	Pellicone et al.	
2007/0150689	A1	6/2007	Pandit et al.	
2007/0168321	A1	7/2007	Saito et al.	
2007/0220227	A1	9/2007	Long	
2007/0294563	A1	12/2007	Bose	
2007/0294564	A1	12/2007	Reddin et al.	
2008/0005587	A1	1/2008	Ahlquist	
2008/0077825	A1	3/2008	Bello et al.	
2008/0162674	A1	7/2008	Dahiya	
2008/0195833	A1	8/2008	Park	
2008/0256141	A1	10/2008	Wayda et al.	
2008/0270678	A1	10/2008	Cornwell et al.	
2008/0282045	A1	11/2008	Biswas et al.	
2009/0077340	A1	3/2009	Johnson et al.	
2009/0100115	A1	4/2009	Park et al.	
2009/0198889	A1	8/2009	Ito et al.	
2010/0052625	A1	3/2010	Cagno et al.	
2010/0211723	A1	8/2010	Mukaida	
2010/0246266	A1	9/2010	Park et al.	
2010/0257142	A1	10/2010	Murphy et al.	
2010/0262764	A1	10/2010	Liu et al.	
2010/0306500	A1	12/2010	Mimatsu	
2010/0325345	A1	12/2010	Ohno et al.	
2010/0332754	A1	12/2010	Lai et al.	
2011/0035540	A1	2/2011	Fitzgerald et al.	
2011/0072290	A1	3/2011	Davis et al.	
2011/0125955	A1	5/2011	Chen	
2011/0131231	A1	6/2011	Haas et al.	
2011/0167221	A1	7/2011	Pangal et al.	
2012/0023144	A1	1/2012	Rub	
2012/0054264	A1	3/2012	Haugh et al.	
2012/0079318	A1	3/2012	Colgrove et al.	
2012/0131253	A1	5/2012	McKnight et al.	
2012/0303919	A1	11/2012	Hu et al.	
2012/0311000	A1	12/2012	Post et al.	
2013/0007845	A1	1/2013	Chang et al.	
2013/0031414	A1	1/2013	Dhuse et al.	
2013/0036272	A1	2/2013	Nelson	
2013/0071087	A1	3/2013	Motiwalla et al.	
2013/0145447	A1	6/2013	Maron	
2013/0166505	A1	6/2013	Peretz et al.	
2013/0191555	A1	7/2013	Liu	
2013/0198459	A1	8/2013	Joshi et al.	
2013/0205173	A1	8/2013	Yoneda	
2013/0219164	A1	8/2013	Hamid	
2013/0227201	A1	8/2013	Talagala et al.	
2013/0290607	A1	10/2013	Chang et al.	
2013/0311434	A1	11/2013	Jones	
2013/0318297	A1	11/2013	Jibbe et al.	
2013/0332614	A1	12/2013	Brunk et al.	
2013/0339407	A1*	12/2013	Sharpe	G06F 16/137 707/827

(56)

References Cited**U.S. PATENT DOCUMENTS**

2014/0173232	A1	6/2014	Reohr et al.	
2014/0195636	A1	7/2014	Karve et al.	
2014/0201512	A1	7/2014	Seethaler et al.	
2014/0201541	A1	7/2014	Paul et al.	
2014/0208155	A1	7/2014	Pan	
2014/0215590	A1	7/2014	Brand	
2014/0220561	A1	8/2014	Sukumar et al.	
2014/0229654	A1	8/2014	Goss et al.	
2014/0230017	A1	8/2014	Saib	
2014/0258526	A1	9/2014	Le Sant et al.	
2014/0282983	A1	9/2014	Ju et al.	
2014/0285917	A1	9/2014	Cudak et al.	
2014/0325262	A1	10/2014	Cooper et al.	
2014/0351627	A1	11/2014	Best et al.	
2014/0358858	A1 *	12/2014	Camble	G06F 11/0715 707/634
2014/0373104	A1	12/2014	Gaddam et al.	
2014/0373126	A1	12/2014	Hussain et al.	
2015/0026387	A1	1/2015	Sheredy et al.	
2015/0074463	A1	3/2015	Jacoby et al.	
2015/0089569	A1	3/2015	Sondhi et al.	
2015/0095515	A1	4/2015	Krithivas et al.	
2015/0113203	A1	4/2015	Dancho et al.	
2015/0121137	A1	4/2015	McKnight et al.	
2015/0134920	A1	5/2015	Anderson et al.	
2015/0149822	A1	5/2015	Coronado et al.	
2015/0154418	A1	6/2015	Redberg	
2015/0193169	A1	7/2015	Sundaram et al.	
2015/0378888	A1	12/2015	Zhang et al.	
2016/0026397	A1	1/2016	Nishikido et al.	
2016/0098323	A1	4/2016	Mutha et al.	
2016/0182542	A1	6/2016	Staniford	
2016/0248631	A1	8/2016	Duchesneau	
2016/0313944	A1 *	10/2016	Hodgdon	G06F 3/0652
2016/0350009	A1	12/2016	Cerreta et al.	
2016/0352720	A1	12/2016	Hu et al.	
2016/0352830	A1	12/2016	Borowiec et al.	
2016/0352834	A1	12/2016	Borowiec et al.	
2017/0262202	A1	9/2017	Seo	
2017/0322846	A1 *	11/2017	Gray	G06F 11/1076
2018/0054454	A1	2/2018	Astigarraga et al.	
2018/0081562	A1	3/2018	Vasudevan	
2018/0091595	A1 *	3/2018	Gray	G06F 11/1092
2018/0260125	A1	9/2018	Botes et al.	
2019/0220315	A1	7/2019	Vallala et al.	
2020/0034560	A1	1/2020	Natanzon et al.	
2020/0133582	A1 *	4/2020	Volvovski	G06F 3/067
2020/0326871	A1	10/2020	Wu et al.	
2021/0081433	A1 *	3/2021	Vig	G06F 16/2272
2021/0360833	A1	11/2021	Alissa et al.	
2021/0399868	A1	12/2021	Chen et al.	
2022/0358140	A1 *	11/2022	Syu	G06F 11/20

FOREIGN PATENT DOCUMENTS

WO	2013071087	A1	5/2013
WO	2014110137	A1	7/2014
WO	2016015008	A1	1/2016
WO	2016190938	A1	12/2016
WO	2016195759	A1	12/2016
WO	2016195958	A1	12/2016
WO	2016195961	A1	12/2016
WO	2023133590	A1	7/2023

OTHER PUBLICATIONS

Bellamy-McIntyre et al., "OpenID and the Enterprise: A Model-based Analysis of Single Sign-On Authentication", 15th IEEE International Enterprise Distributed Object Computing Conference (EDOC), Aug. 29, 2011, pp. 129-138, IEEE Computer Society, USA, DOI: 10.1109/EDOC.2011.26, ISBN: 978-1-4577-0362-1.

ETSI, "Network Function Virtualisation (NFV); Resiliency Requirements", ETSI GS NFV-REL 001, V1.1.1, Jan. 2015, 82 pages,

etsi.org (online), URL: www.etsi.org/deliver/etsi_gs/NFV-REL/001_099/001/01.01.01_60/gs_NFV-REL001v010101p.pdf.

Faith, "dictzip file format", GitHub.com (online), accessed Jul. 28, 2015, 1 page, URL: github.com/fidlej/idzip.

Google Search of "storage array define" performed by the Examiner on Nov. 4, 2015 for U.S. Appl. No. 14/725,278, Results limited to entries dated before 2012, 1 page.

Hota et al., "Capability-based Cryptographic Data Access Control in Cloud Computing", International Journal of Advanced Networking and Applications, col. 1, Issue 1, Aug. 2011, 10 pages, Esward Publications, India.

Hu et al., "Container Marking: Combining Data Placement, Garbage Collection and Wear Levelling for Flash", 19th Annual IEEE International Symposium on Modelling, Analysis, and Simulation of Computer and Telecommunications Systems, Jul. 25-27, 2011, 11 pages, ISBN: 978-0-7695-4430-4, DOI: 10.1109/MASCOTS.2011.50.

Hwang et al., "RAID-x: A New Distributed Disk Array for I/O-Centric Cluster Computing", Proceedings of the Ninth International Symposium on High-performance Distributed Computing, Aug. 2000, pp. 279-286, the Ninth International Symposium on High-Performance Distributed Computing, IEEE Computer Society, Los Alamitos, CA.

International Search Report and Written Opinion, PCT/US2016/015006, Apr. 29, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/015008, May 4, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/016333, Jun. 8, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/020410, Jul. 8, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/032052, Aug. 30, 2016, 17 pages.

International Search Report and Written Opinion, PCT/US2016/032084, Jul. 18, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/035492, Aug. 17, 2016, 10 pages.

International Search Report and Written Opinion, PCT/US2016/036693, Aug. 29, 2016, 10 pages.

International Search Report and Written Opinion, PCT/US2016/038758, Oct. 7, 2016, 10 pages.

International Search Report and Written Opinion, PCT/US2016/040393, Sep. 22, 2016, 10 pages.

International Search Report and Written Opinion, PCT/US2016/044020, Sep. 30, 2016, 11 pages.

International Search Report and Written Opinion, PCT/US2016/044874, Oct. 7, 2016, 11 pages.

International Search Report and Written Opinion, PCT/US2016/044875, Oct. 5, 2016, 13 pages.

International Search Report and Written Opinion, PCT/US2016/044876, Oct. 21, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/044877, Sep. 29, 2016, 13 pages.

International Search Report and Written Opinion, PCT/US2023/060398, Mar. 15, 2023, 13 pages.

Kong, "Using PCI Express as the Primary System Interconnect in Multiroot Compute, Storage, Communications and Embedded Systems", White Paper, IDT.com (online), Aug. 28, 2008, 12 pages, URL: www.idt.com/document/whp/idt-pcie-multi-root-white-paper.

Li et al., "Access Control for the Services Oriented Architecture", Proceedings of the 2007 ACM Workshop on Secure Web Services (SWS '07), Nov. 2007, pp. 9-17, ACM New York, NY.

Microsoft, "Hybrid for SharePoint Server 2013—Security Reference Architecture", Microsoft (online), Oct. 2014, 53 pages, URL: hybrid.office.com/img/Security_Reference_Architecture.pdf.

Microsoft, "Hybrid Identity Management", Microsoft (online), Apr. 2014, 2 pages, URL: download.microsoft.com/download/E/A/E/EAE57CD1-A80B-423C-96BB-142FAAC630B9/Hybrid_Identity_Datasheet.pdf.

PCmag, "Storage Array Definition", Published May 10, 2013, URL: <http://web.archive.org/web/20130510121646/http://www.pcmag.com/encyclopedia/term/52091/storage-array>, 2 pages.

(56)

References Cited

OTHER PUBLICATIONS

Stalzer, "FlashBlades: System Architecture and Applications", Proceedings of the 2nd Workshop on Architectures and Systems for Big Data, Jun. 2012, pp. 10-14, Association for Computing Machinery, New York, NY.

Storer et al., "Pergamum: Replacing Tape with Energy Efficient, Reliable, Disk-Based Archival Storage", FAST'08: Proceedings of the 6th USENIX Conference on File and Storage Technologies, Article No. 1, Feb. 2008, pp. 1-16, USENIX Association, Berkeley, CA.

Storer et al., "Secure Data Deduplication", Proceedings of the 4th ACM International Workshop on Storage Security and Survivability (StorageSS'08), Oct. 2008, 10 pages, ACM New York, NY, USA, DOI: 10.1145/1456469.1456471.

Sweere, "Creating Storage Class Persistent Memory with NVDIMM", Published in Aug. 2013, Flash Memory Summit 2013, URL: http://www.flashmemorysummit.com/English/Collaterals/Proceedings/2013/20130814_T2_Sweere.pdf, 22 pages.

Techopedia, "What is a disk array", techopedia.com (online), Jan. 13, 2012, 1 page, URL: web.archive.org/web/20120113053358/http://www.techopedia.com/definition/1009/disk-array.

Webopedia, "What is a disk array", webopedia.com (online), May 26, 2011, 2 pages, URL: web.archive.org/web/20110526081214/http://www.webopedia.com/TERM/D/disk_array.html.

Wikipedia, "Convergent Encryption", Wikipedia.org (online), accessed Sep. 8, 2015, 2 pages, URL: en.wikipedia.org/wiki/Convergent_encryption.

* cited by examiner

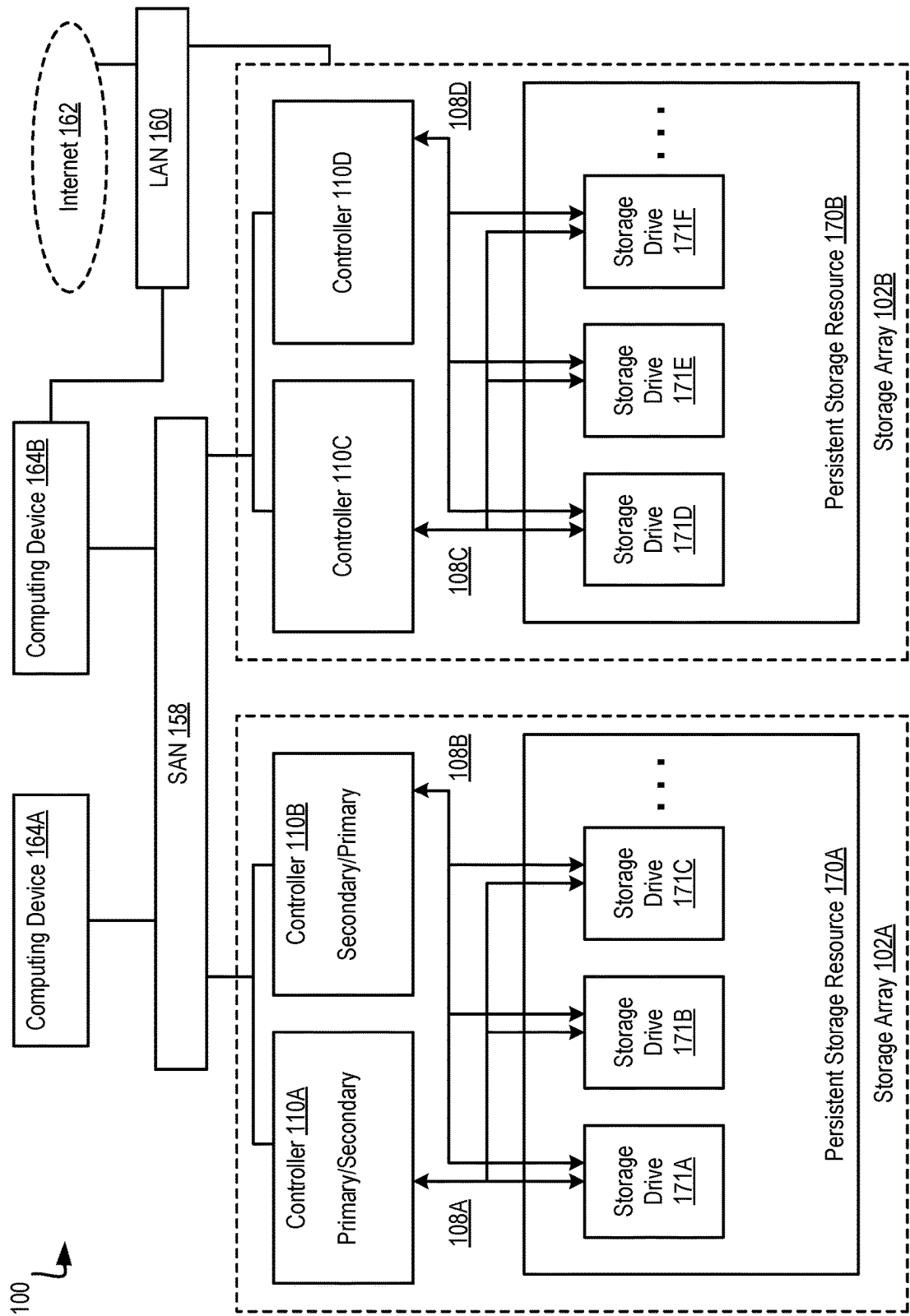


FIG. 1A

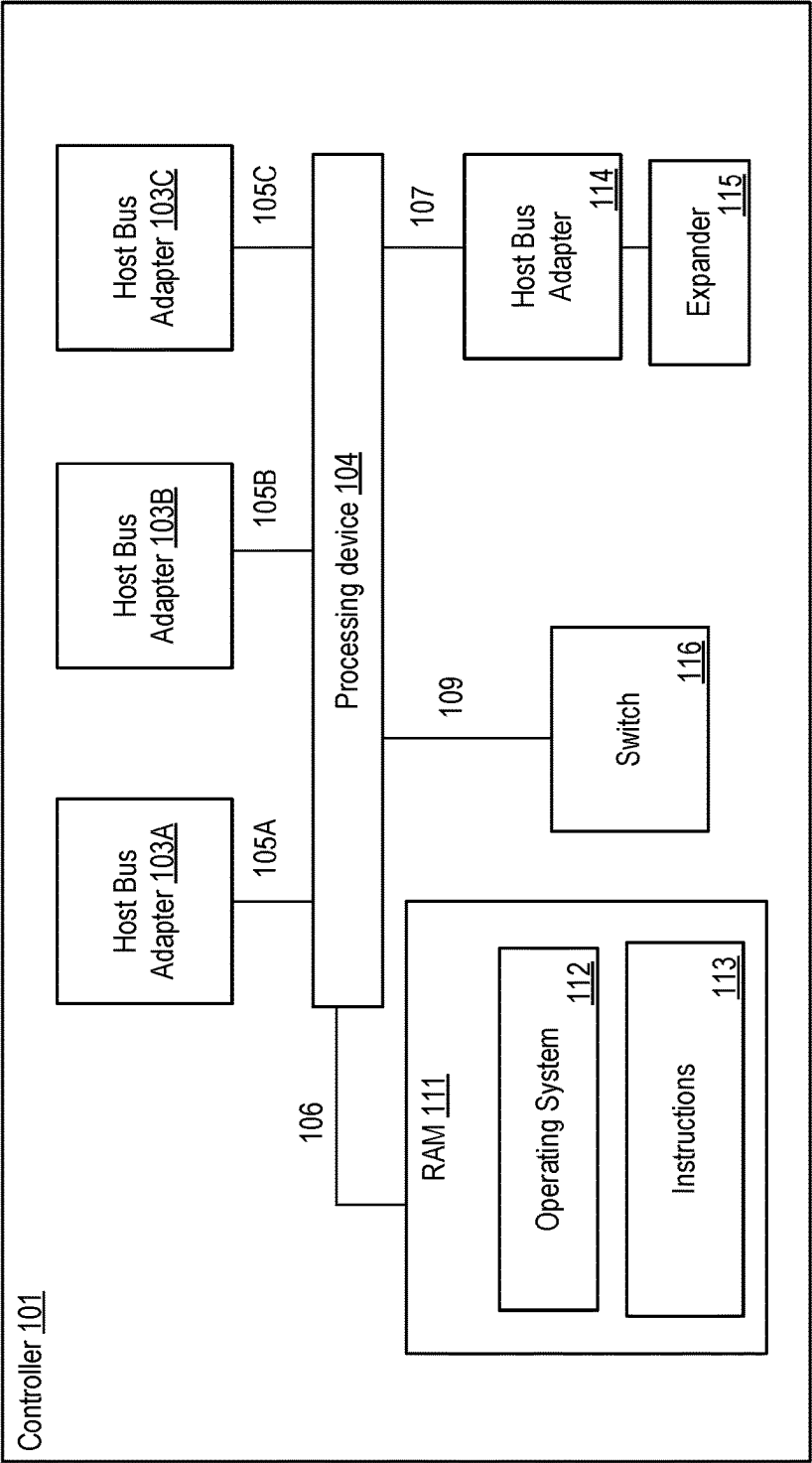


FIG. 1B

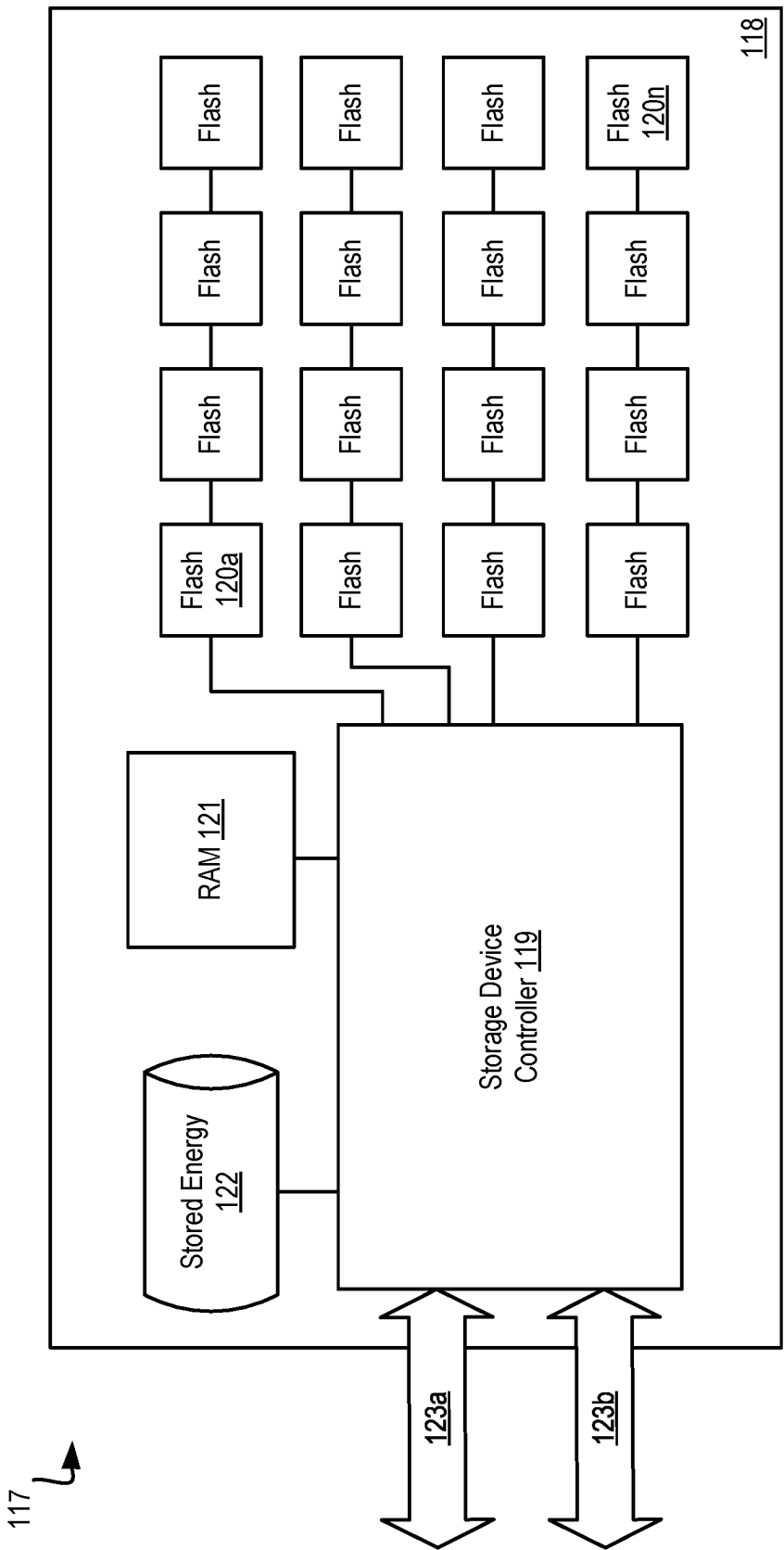


FIG. 1C

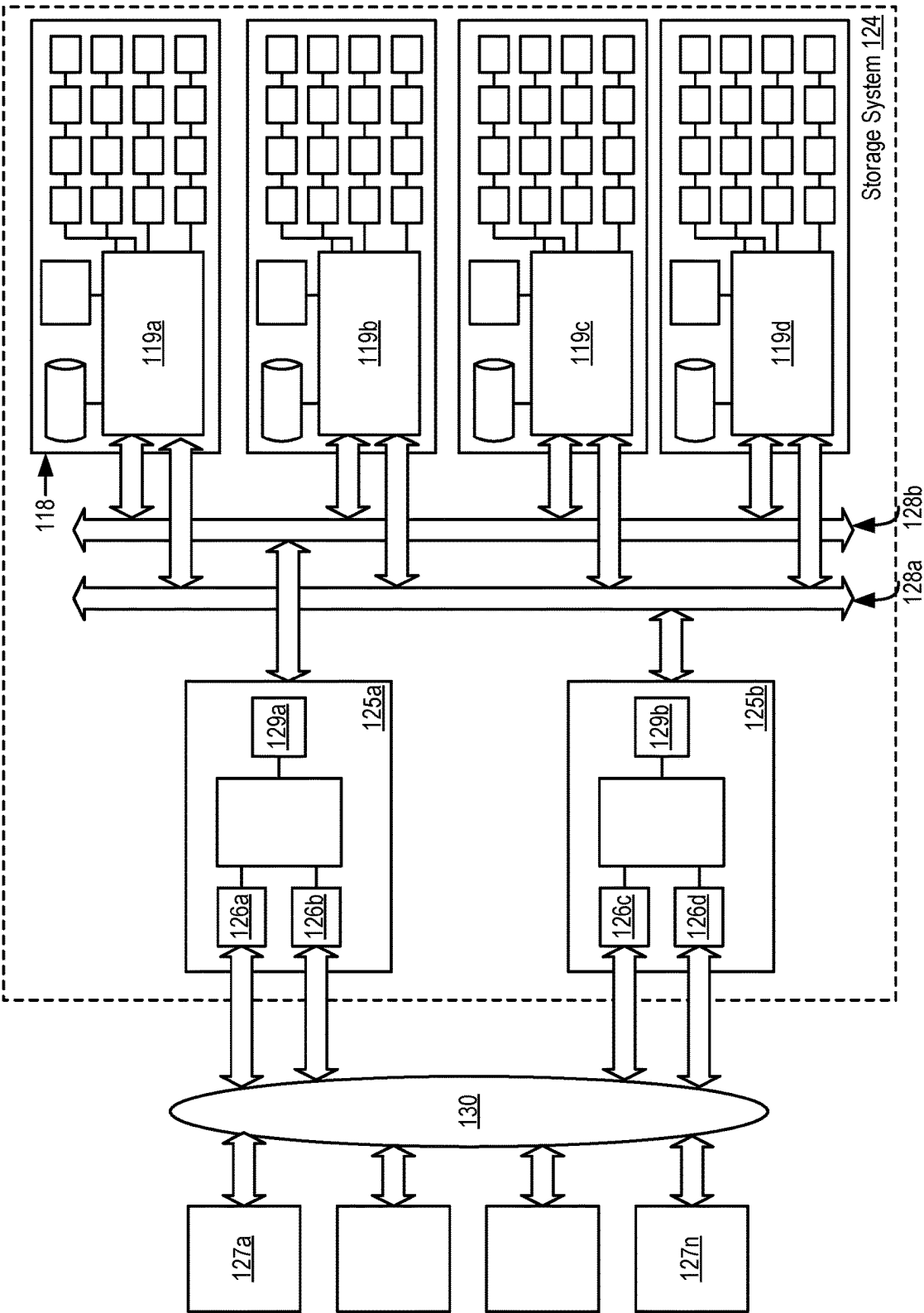


FIG. 1D

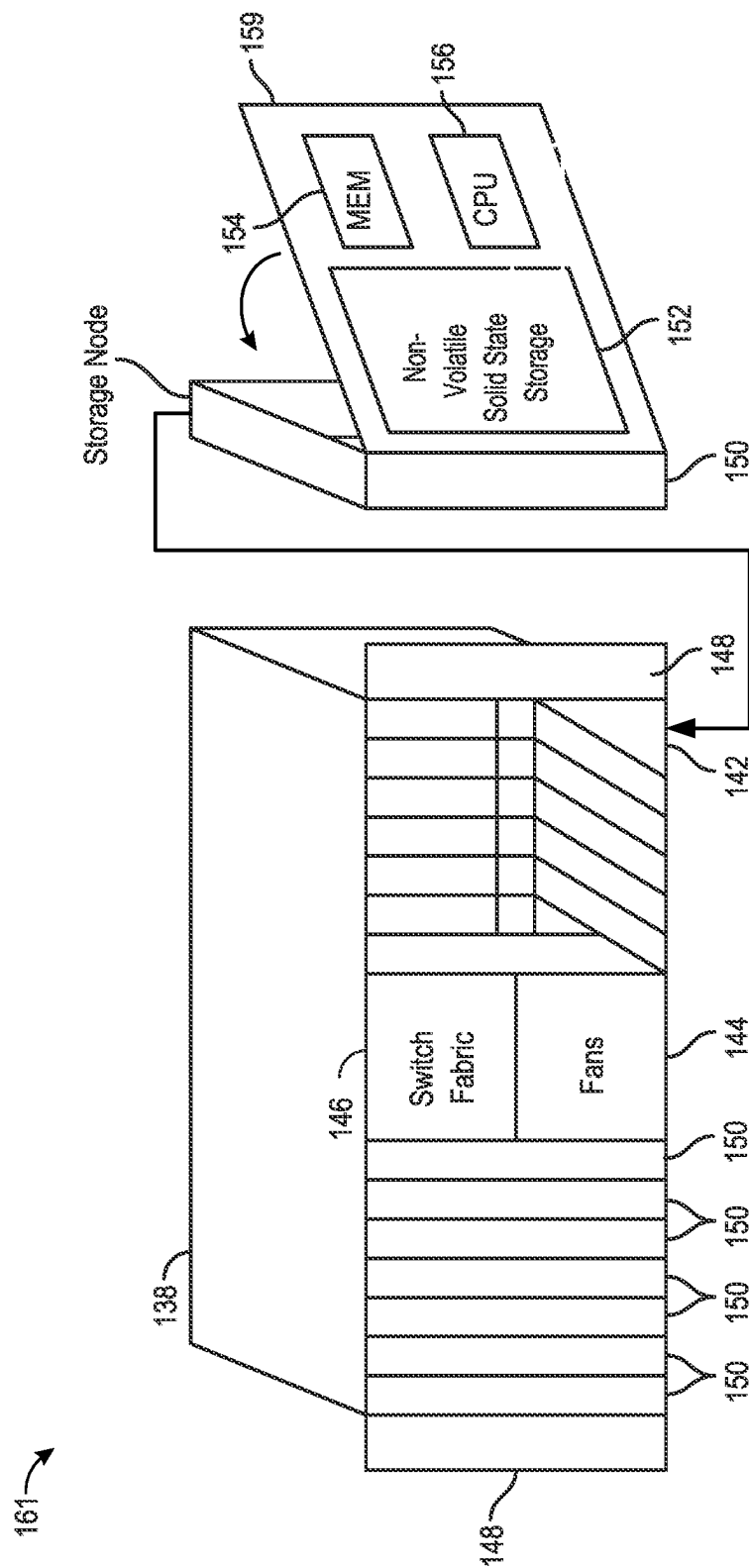


FIG. 2A

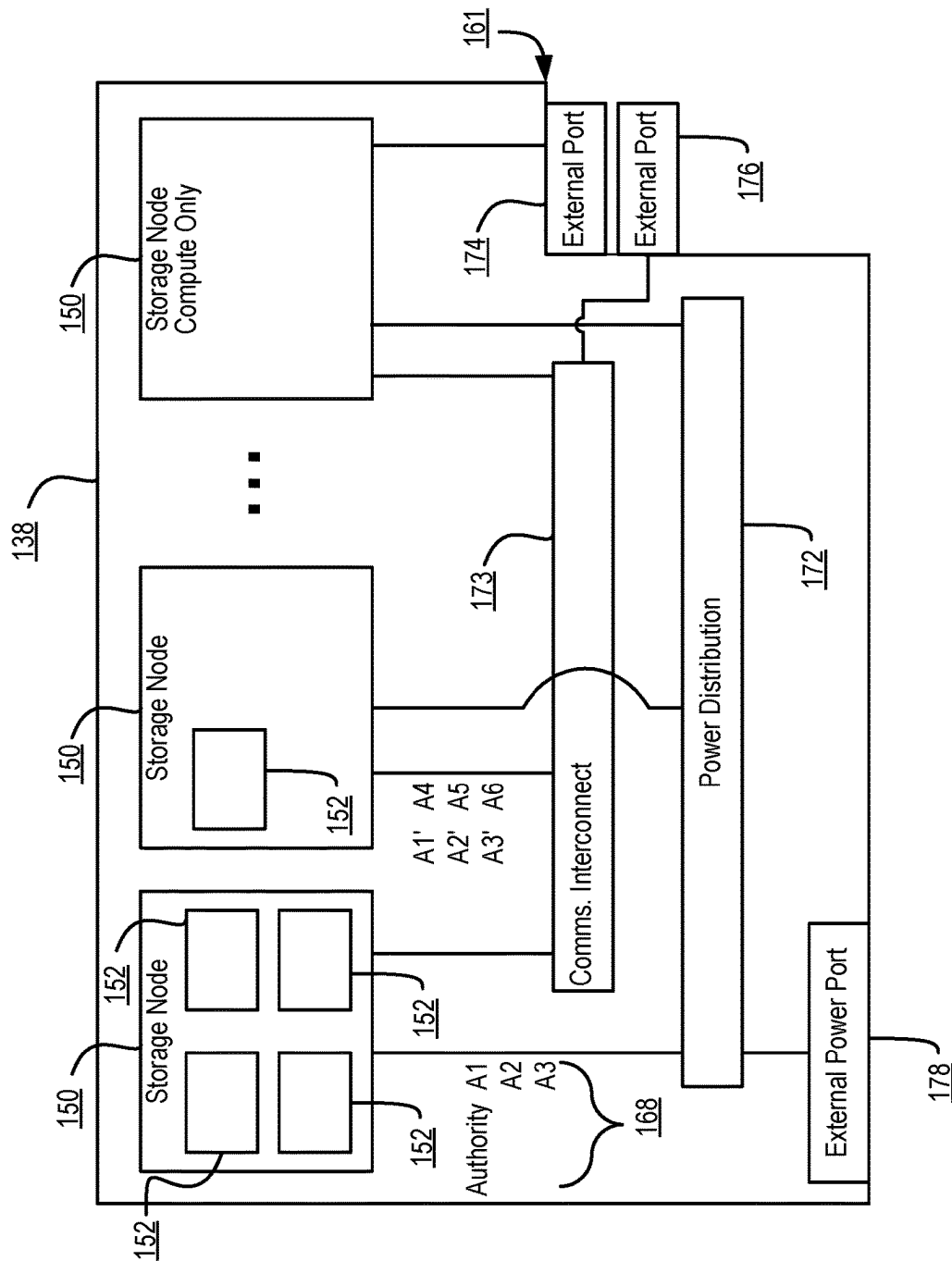


FIG. 2B

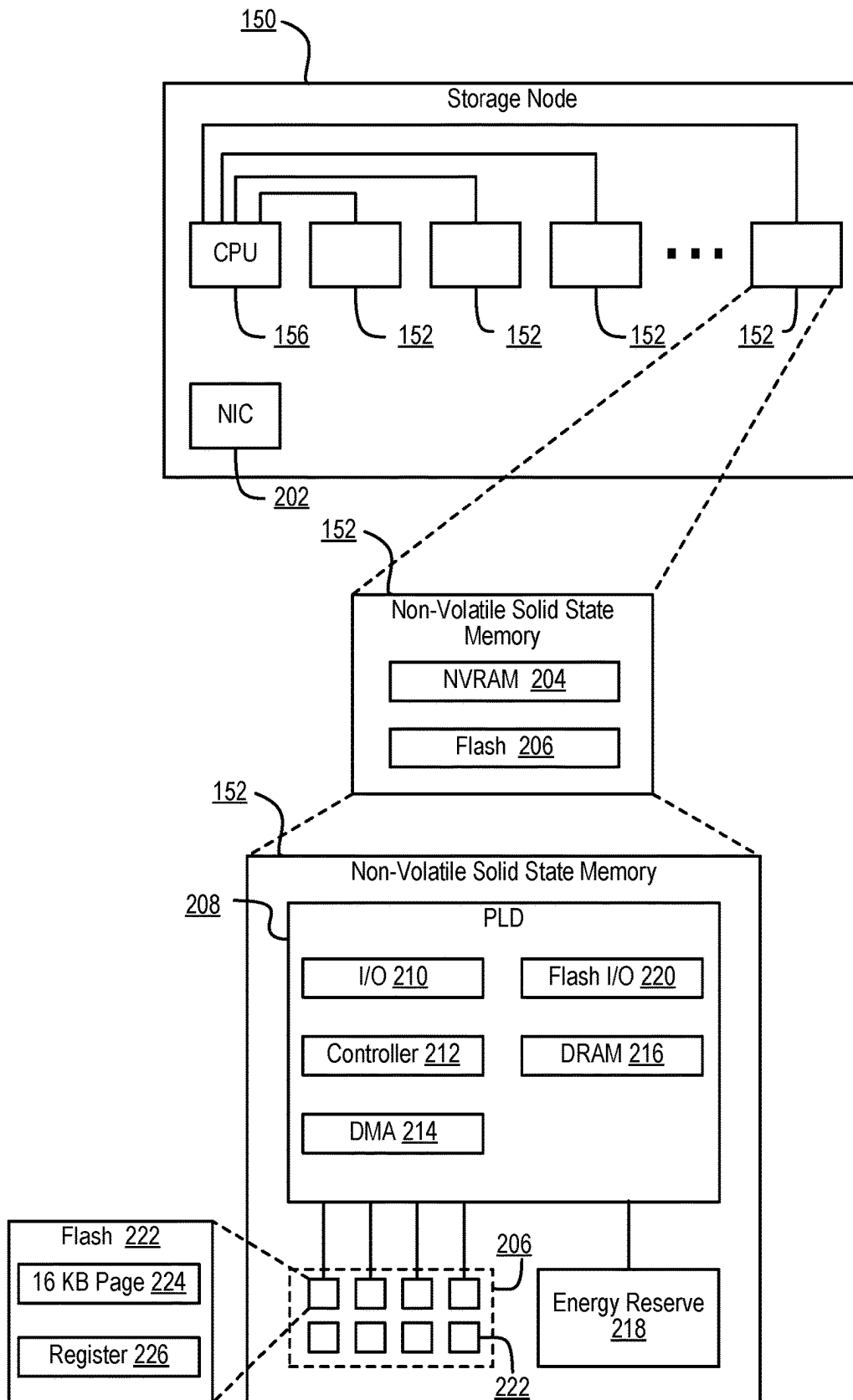


FIG. 2C

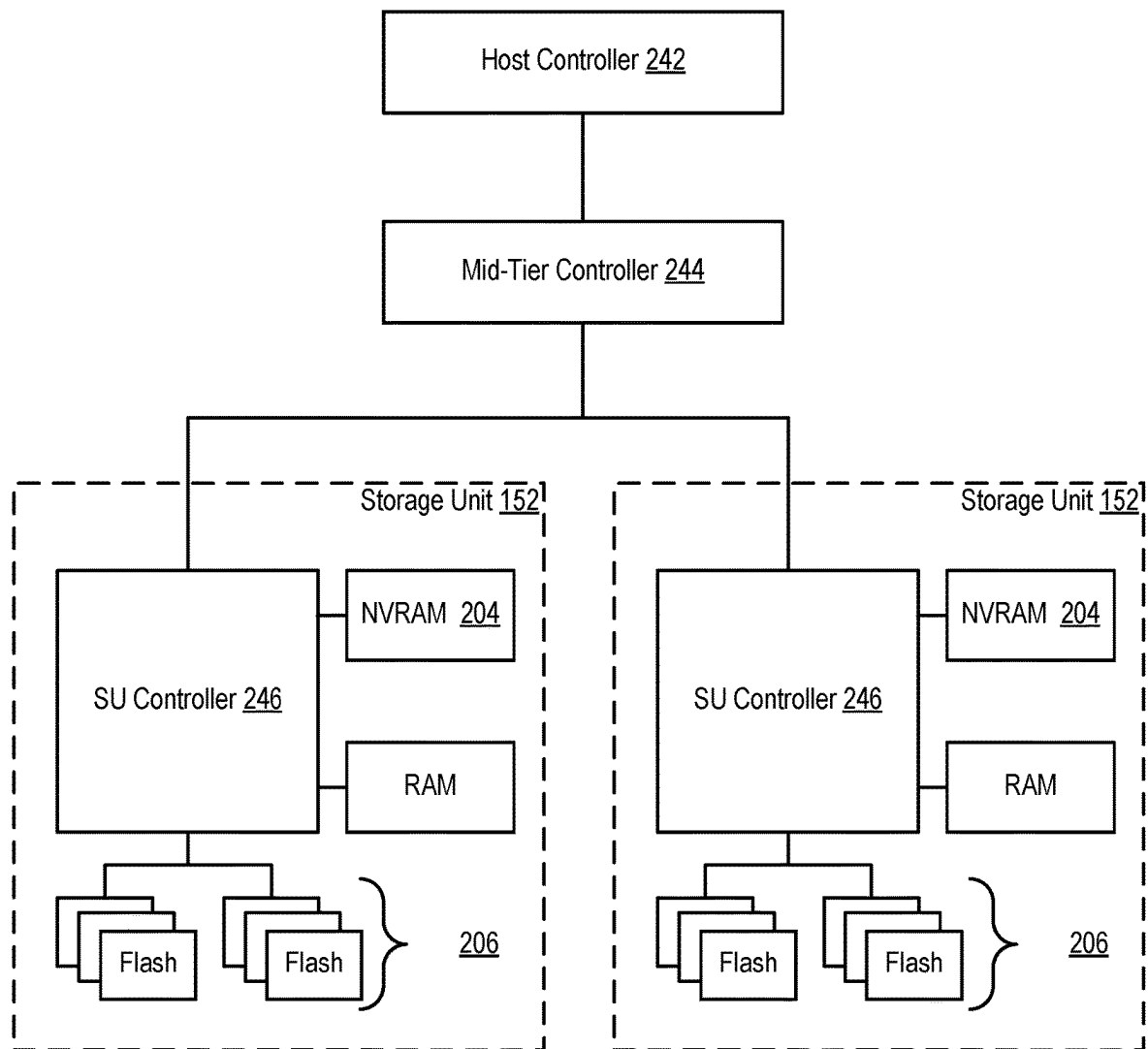


FIG. 2D

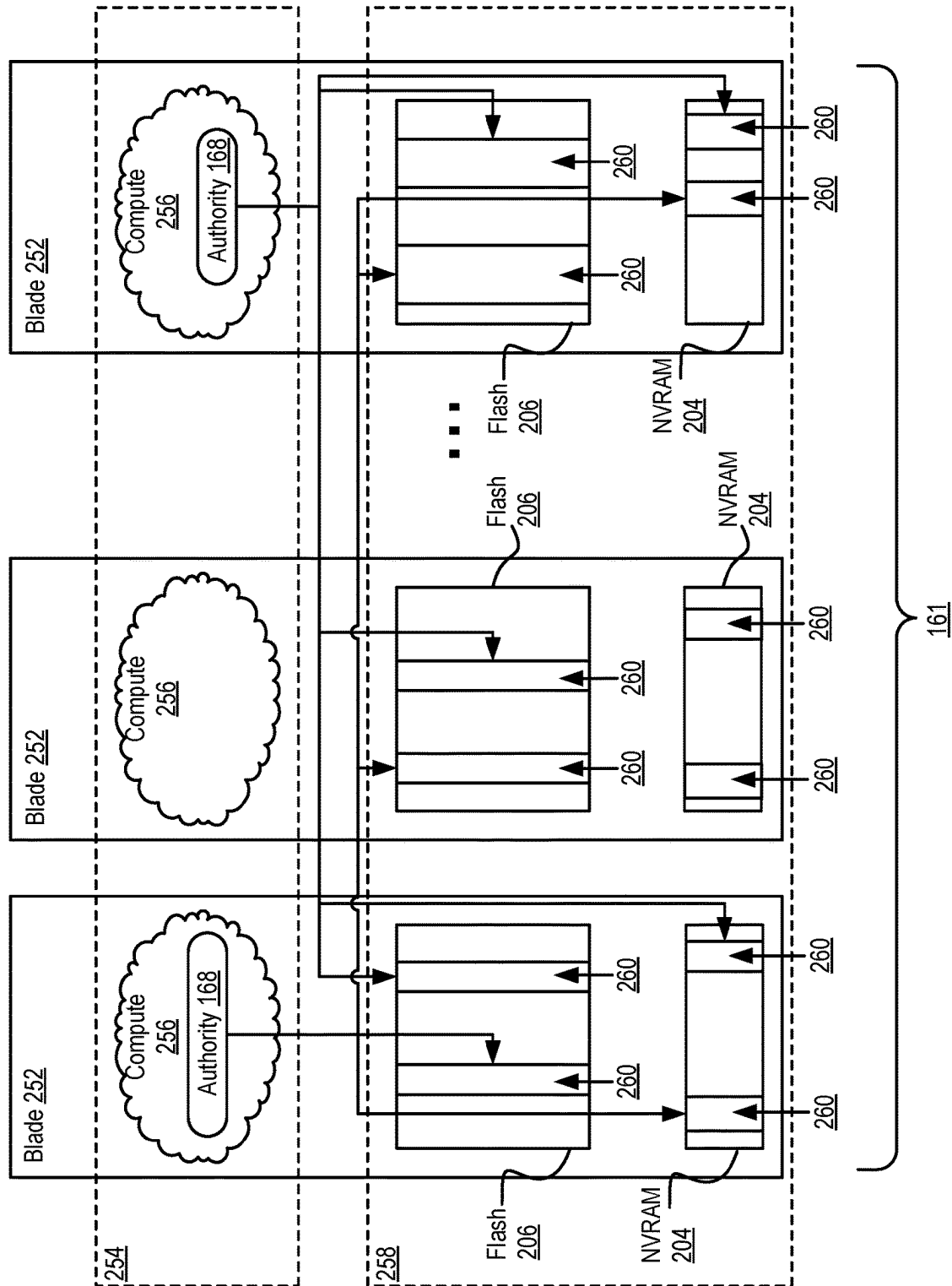


FIG. 2E

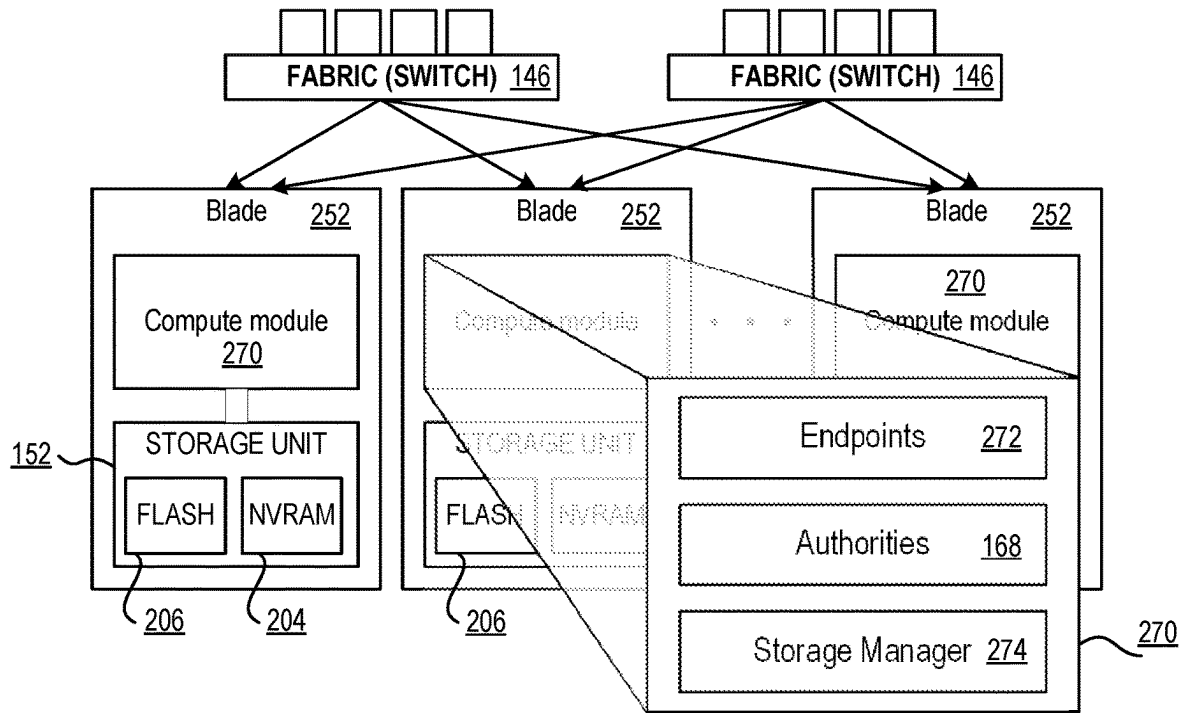


FIG. 2F

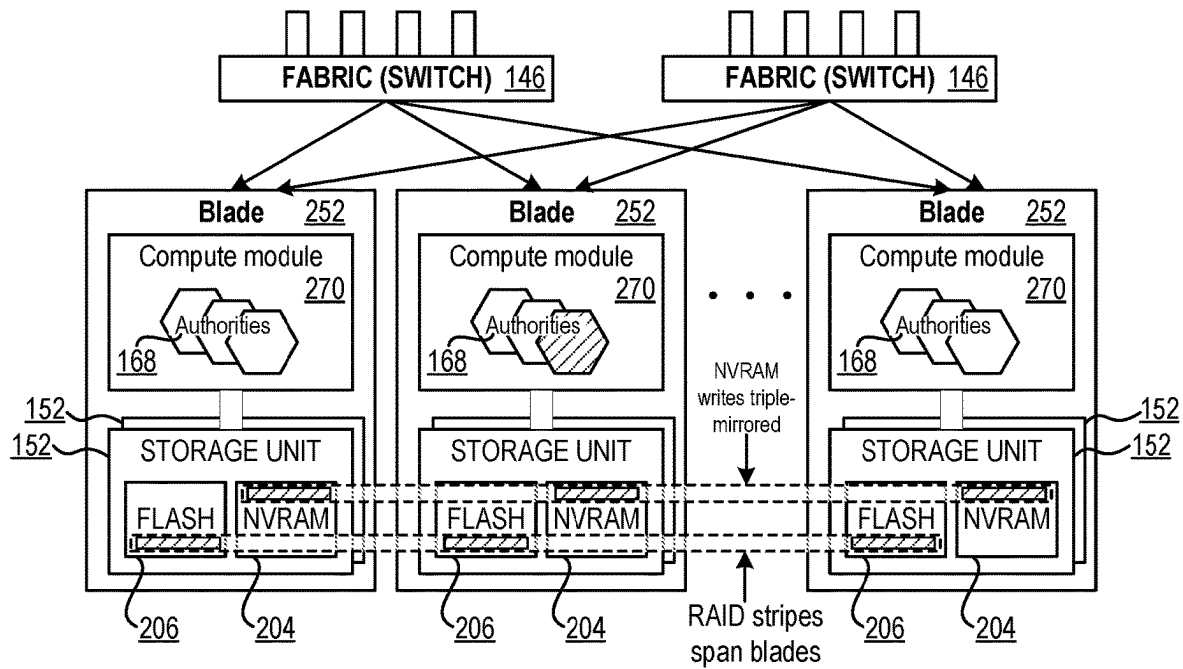


FIG. 2G

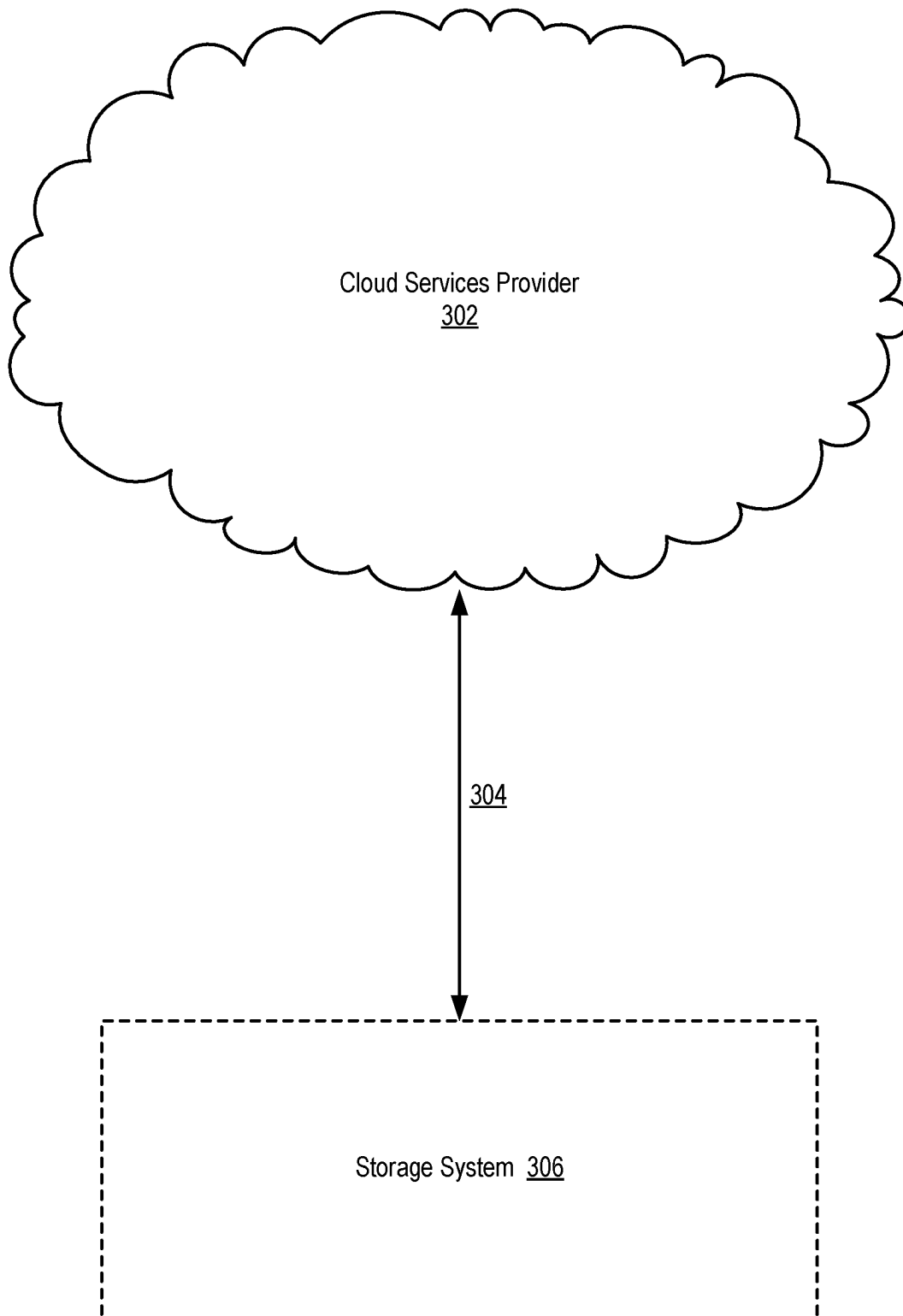


FIG. 3A

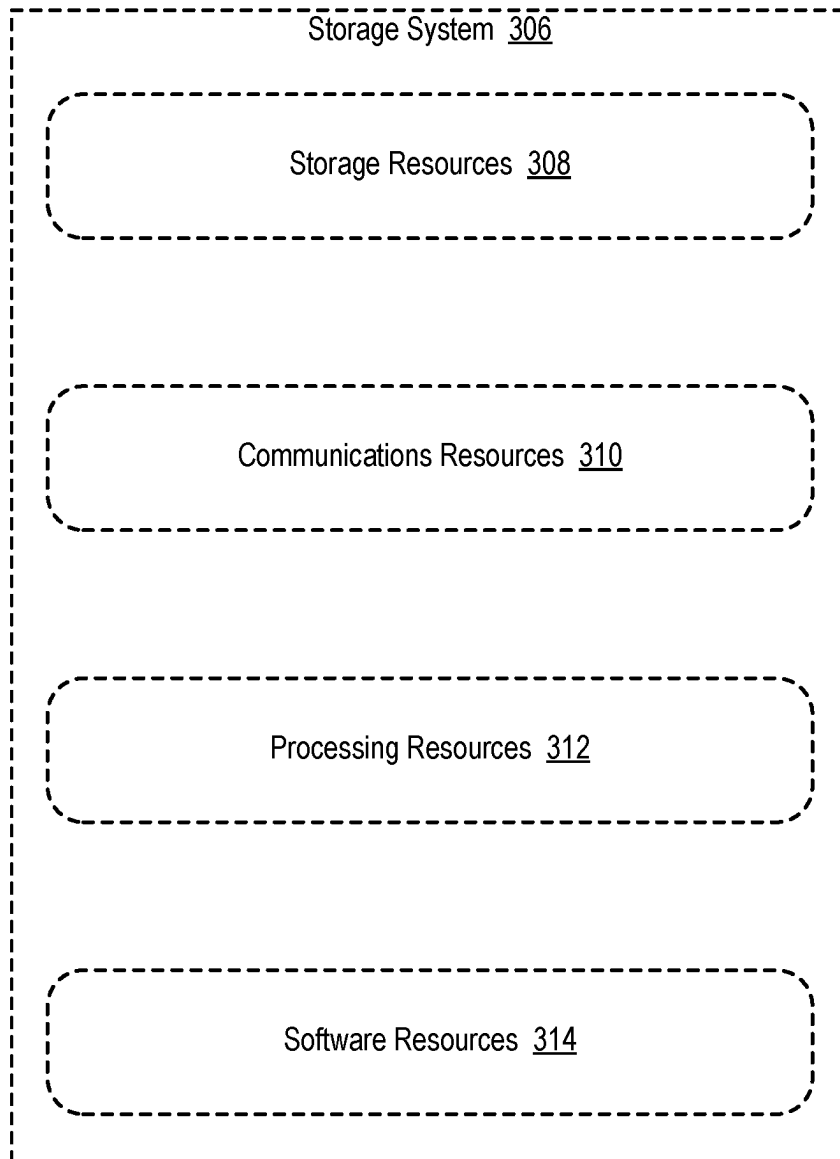


FIG. 3B

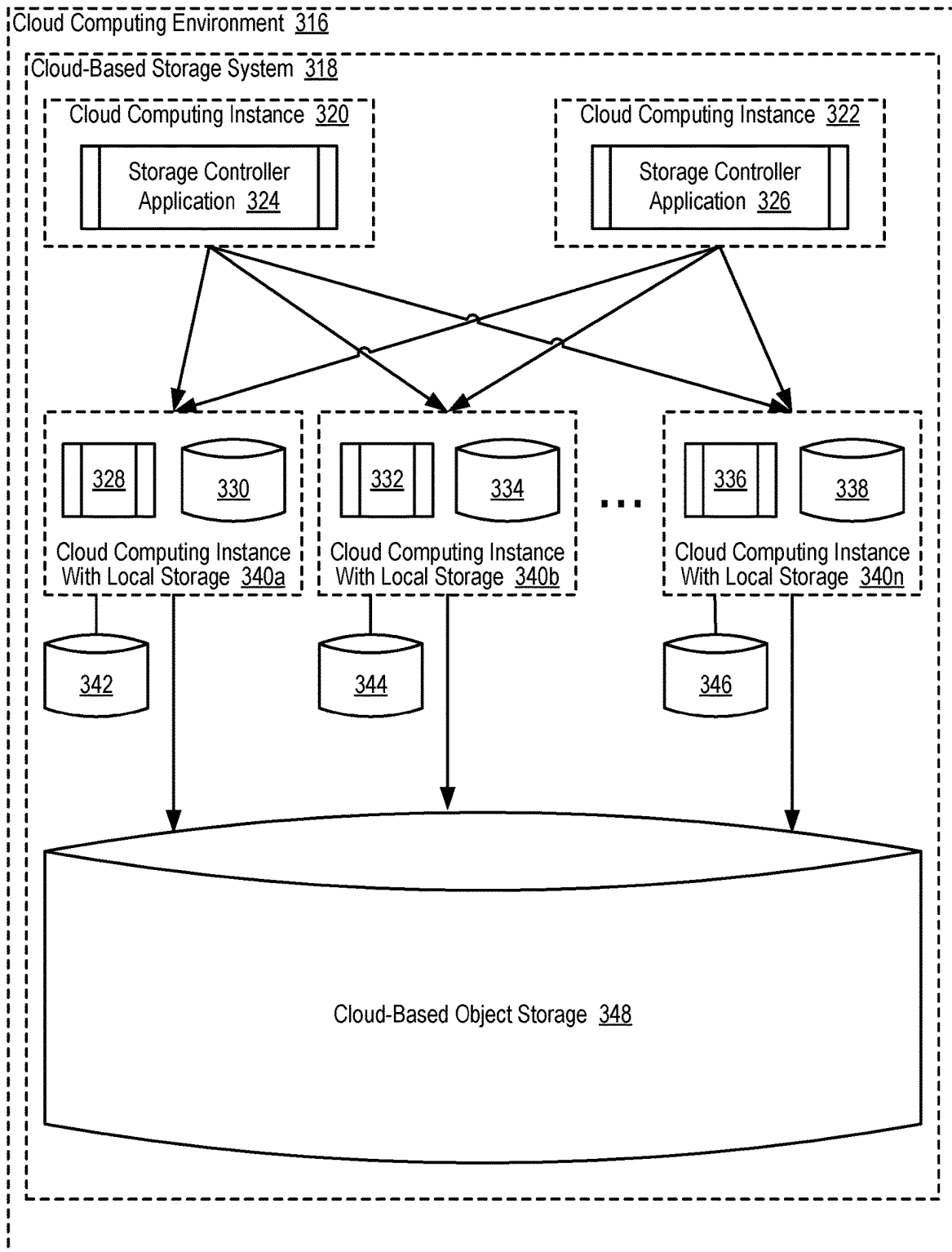


FIG. 3C

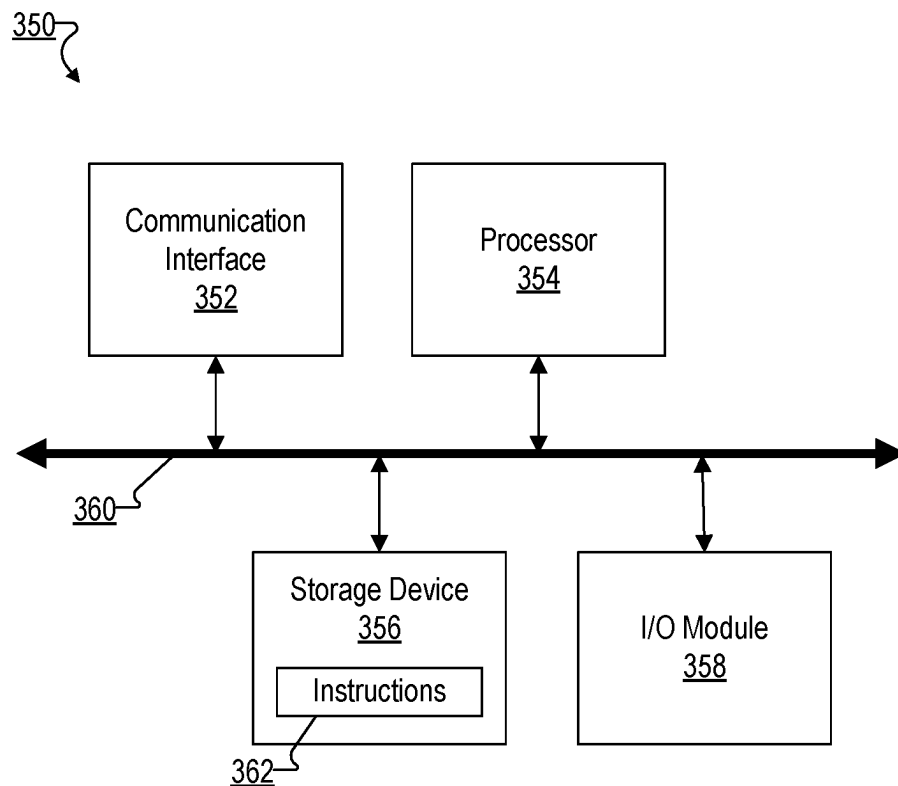


FIG. 3D

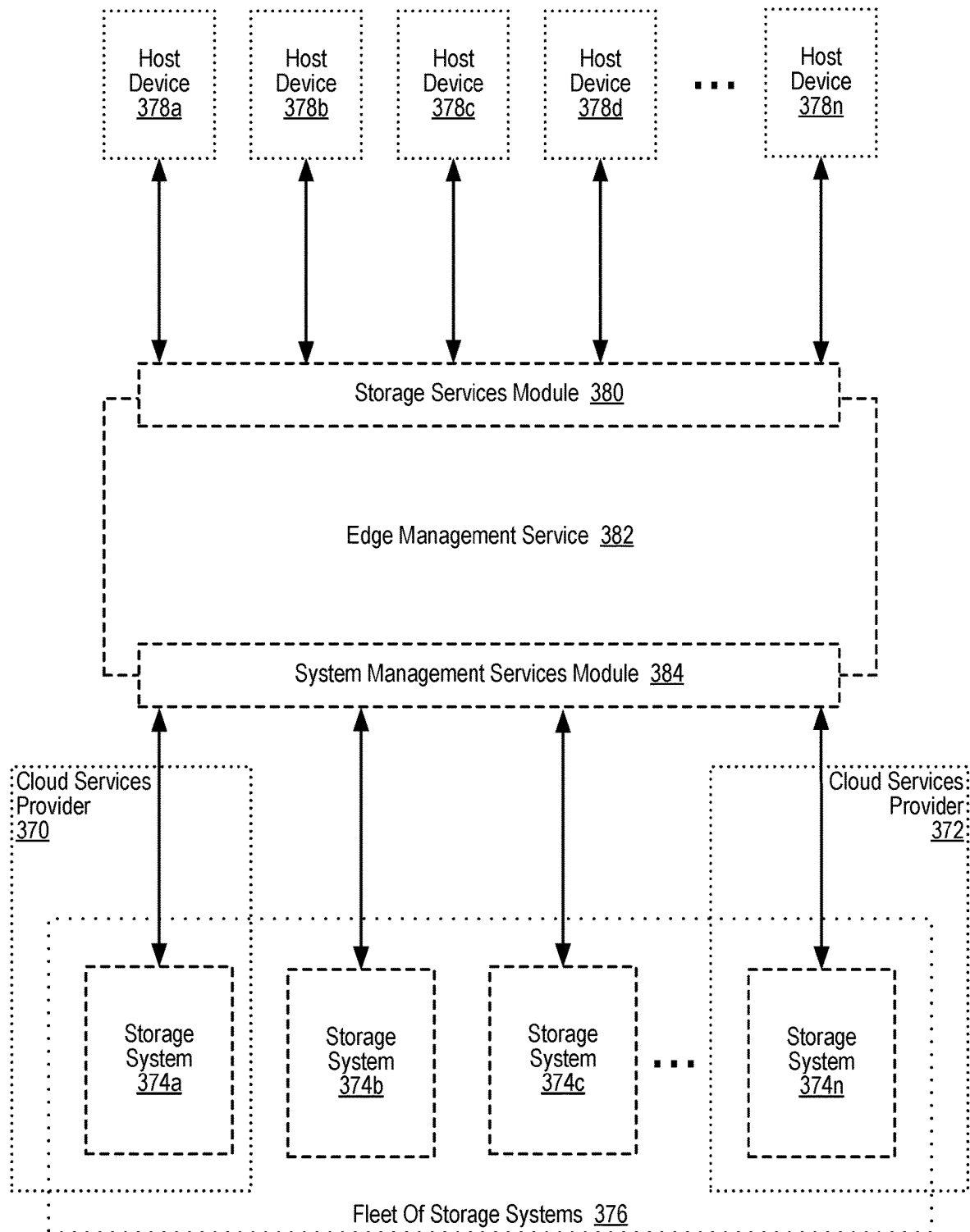


FIG. 3E

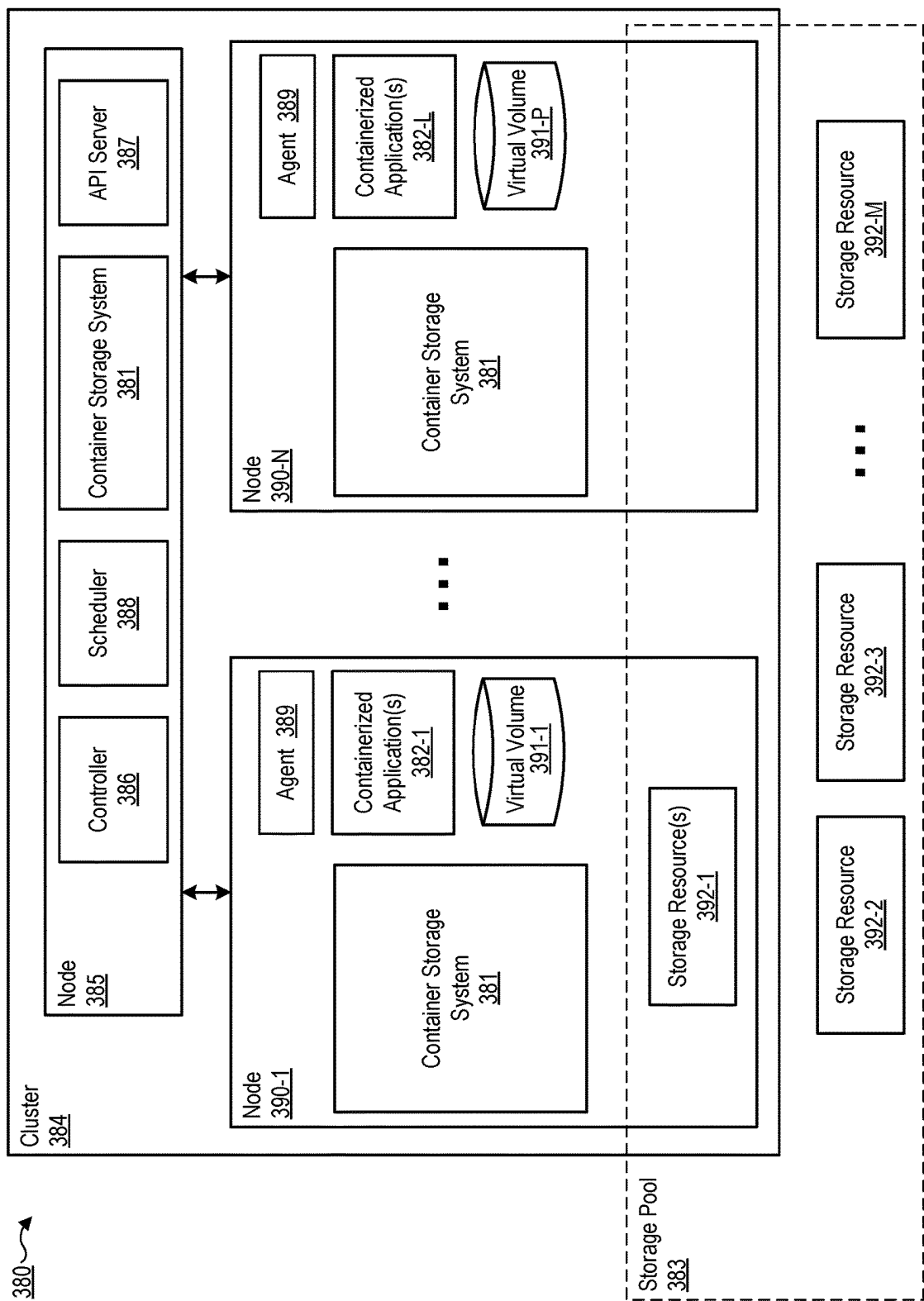


FIG. 3F

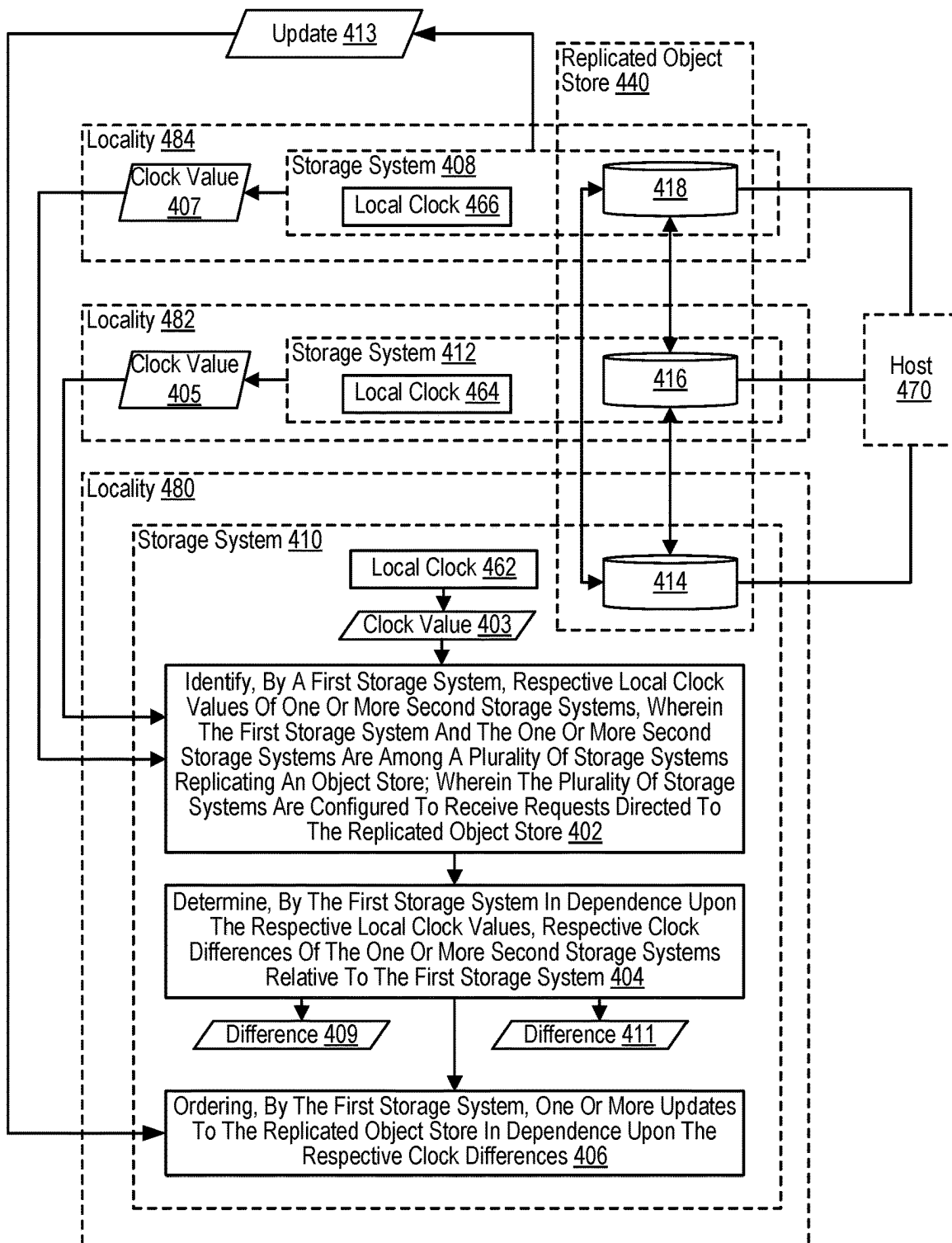


FIG. 4

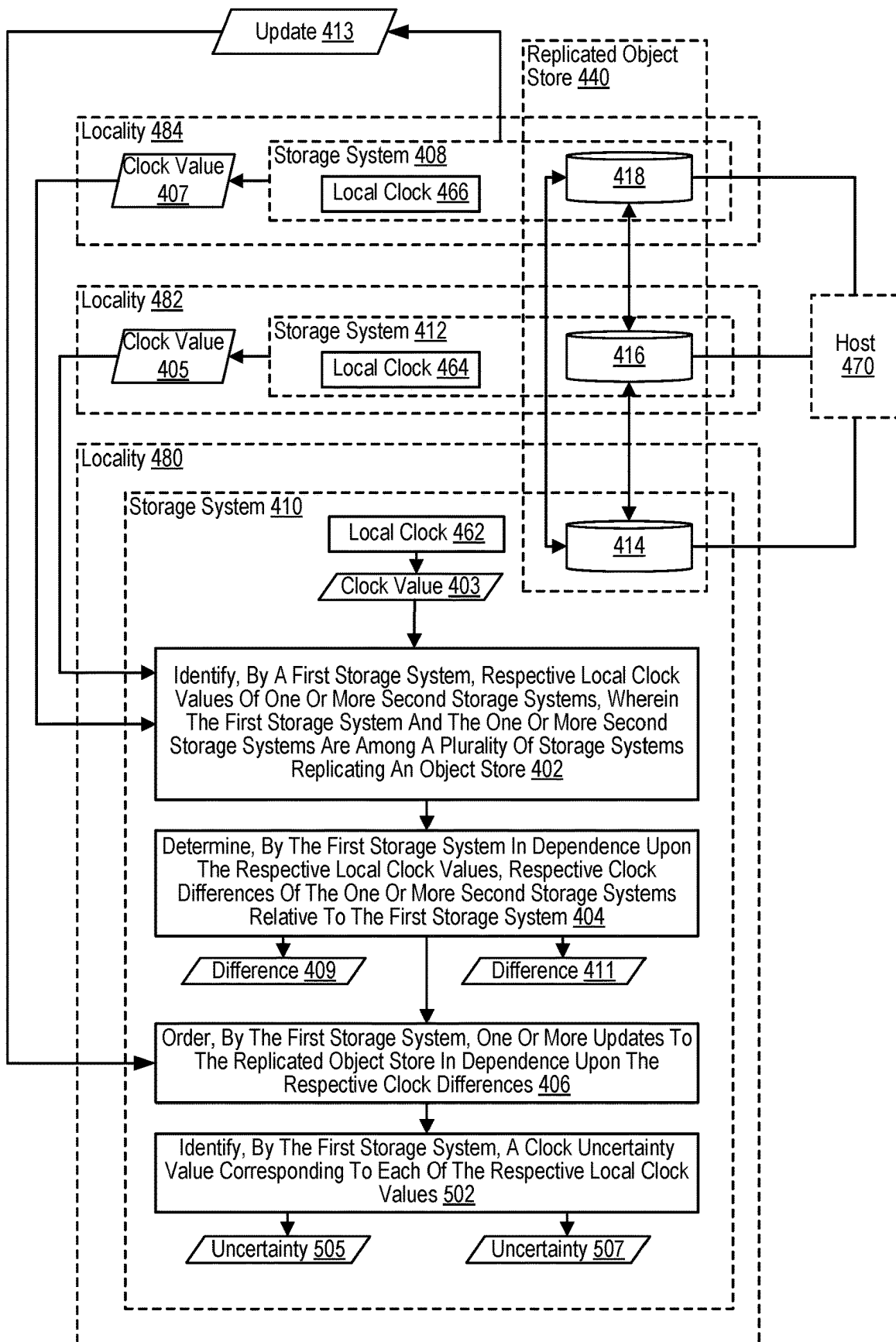


FIG. 5

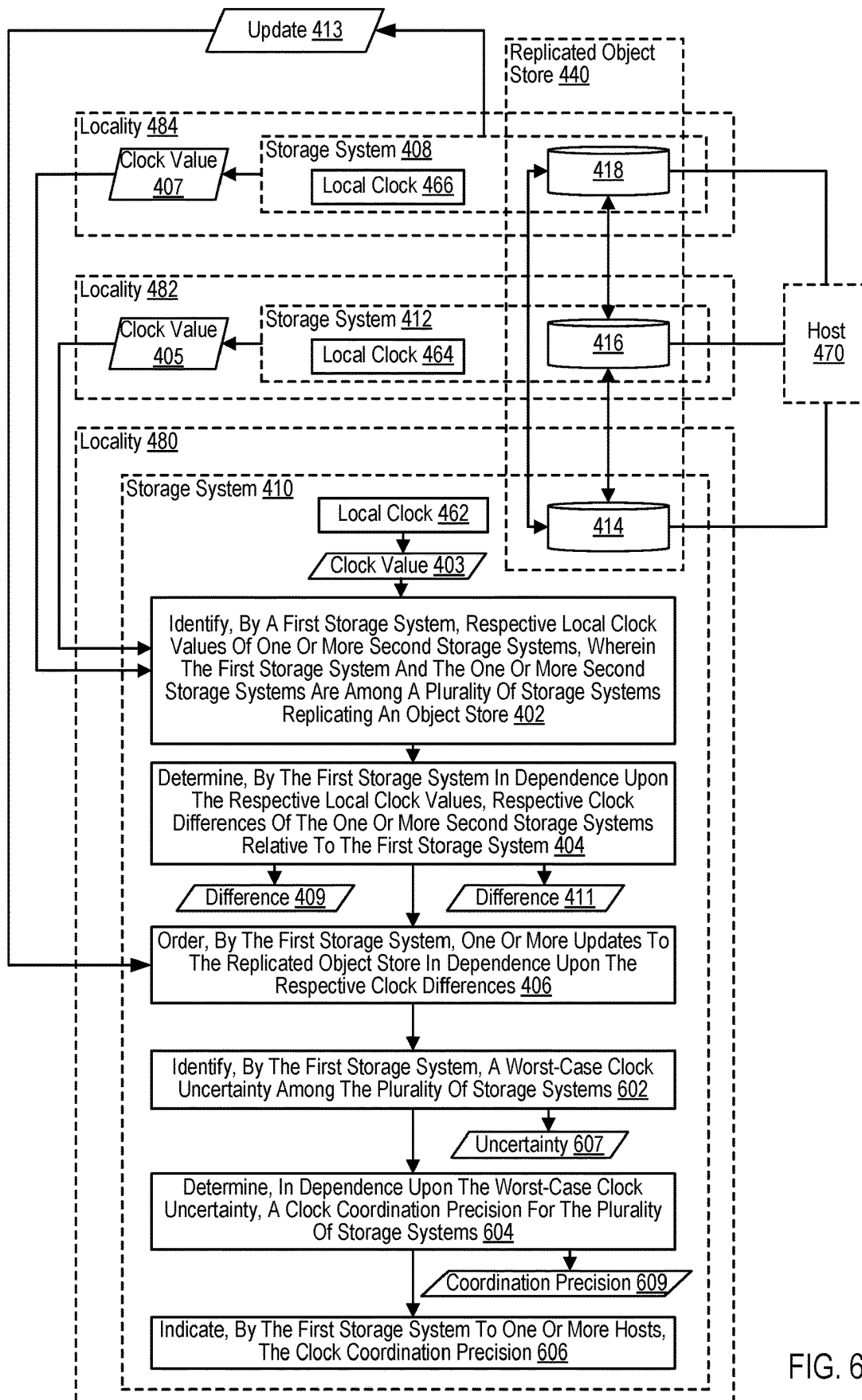
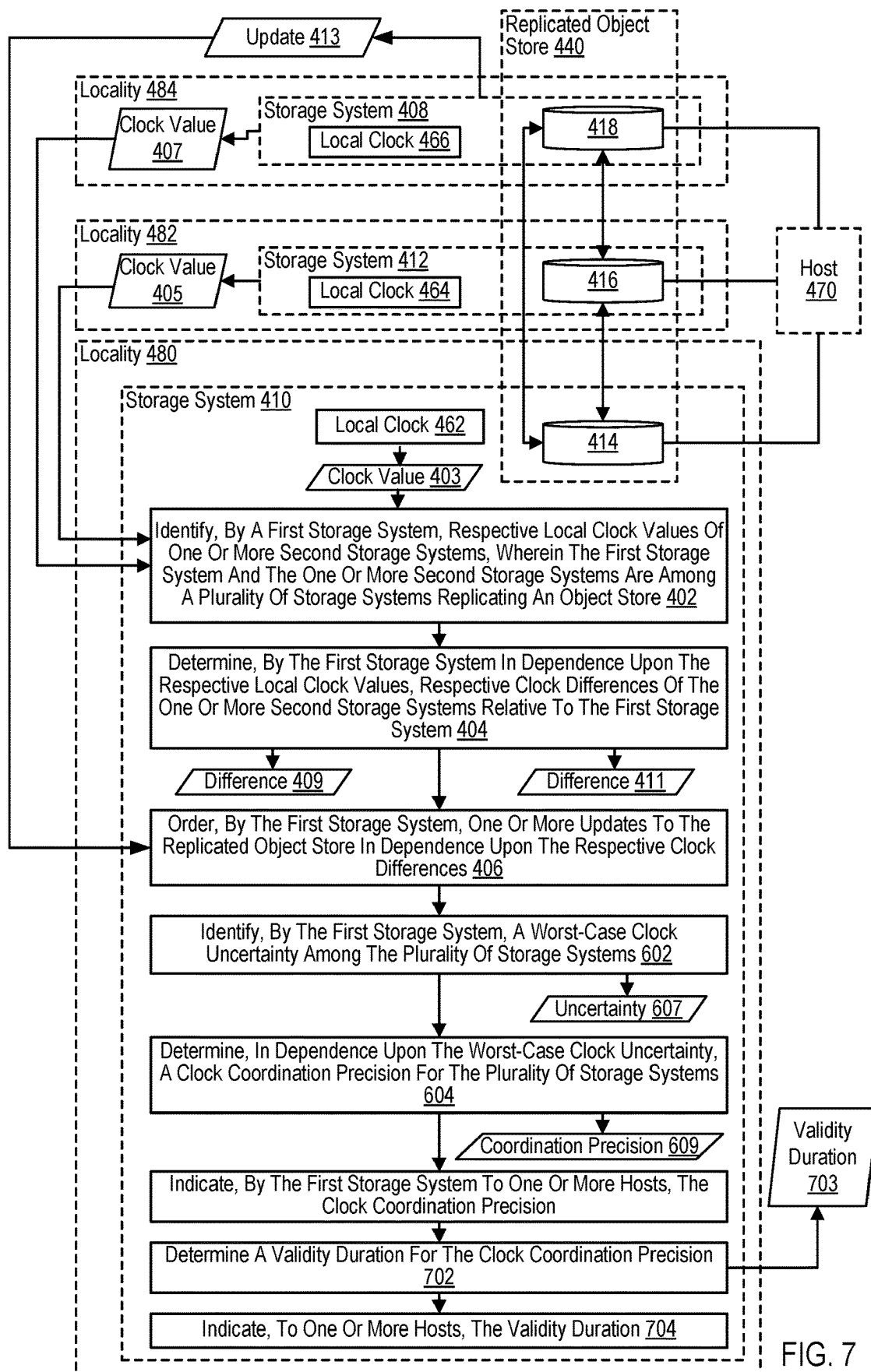


FIG. 6



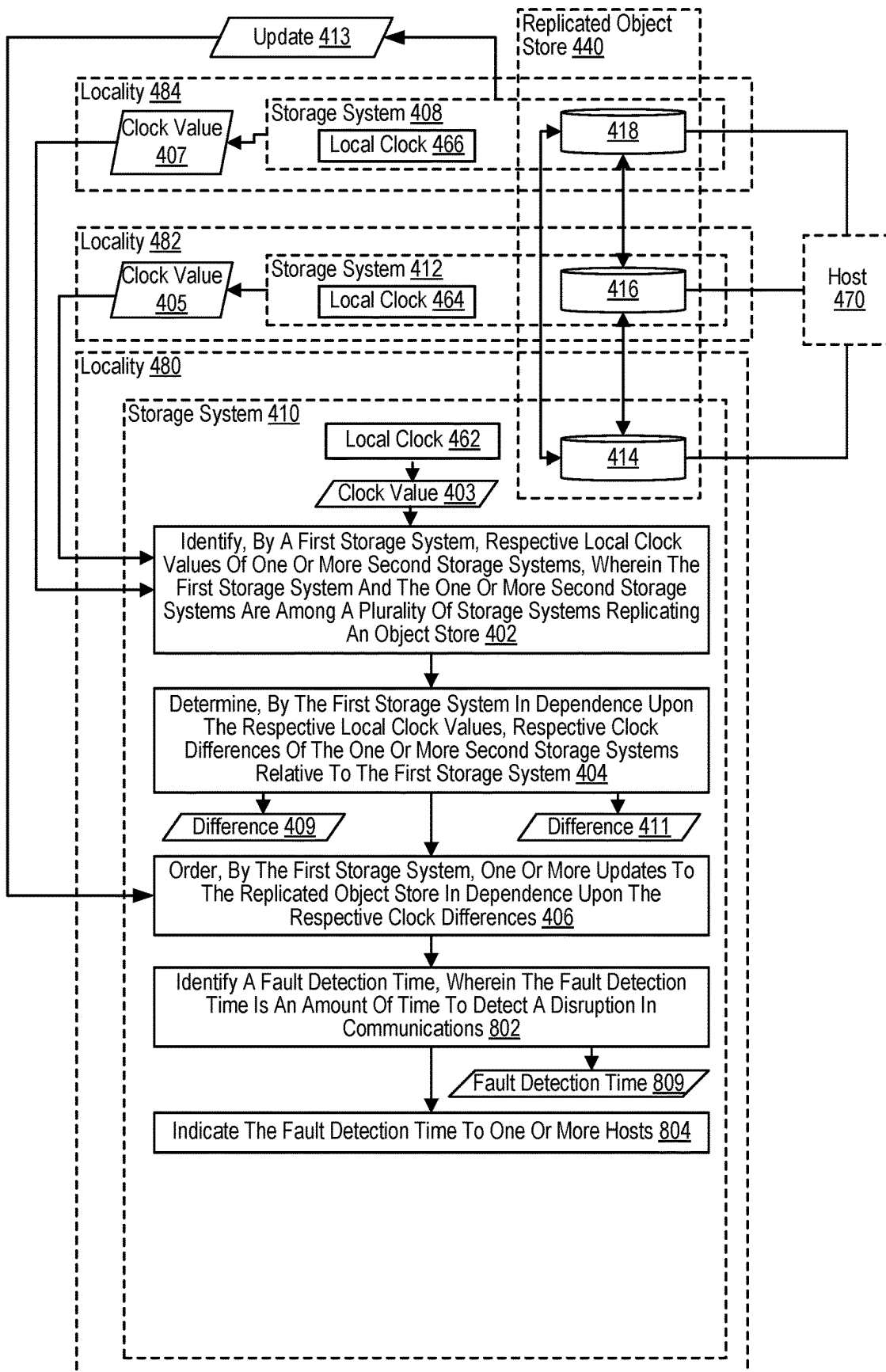


FIG. 8

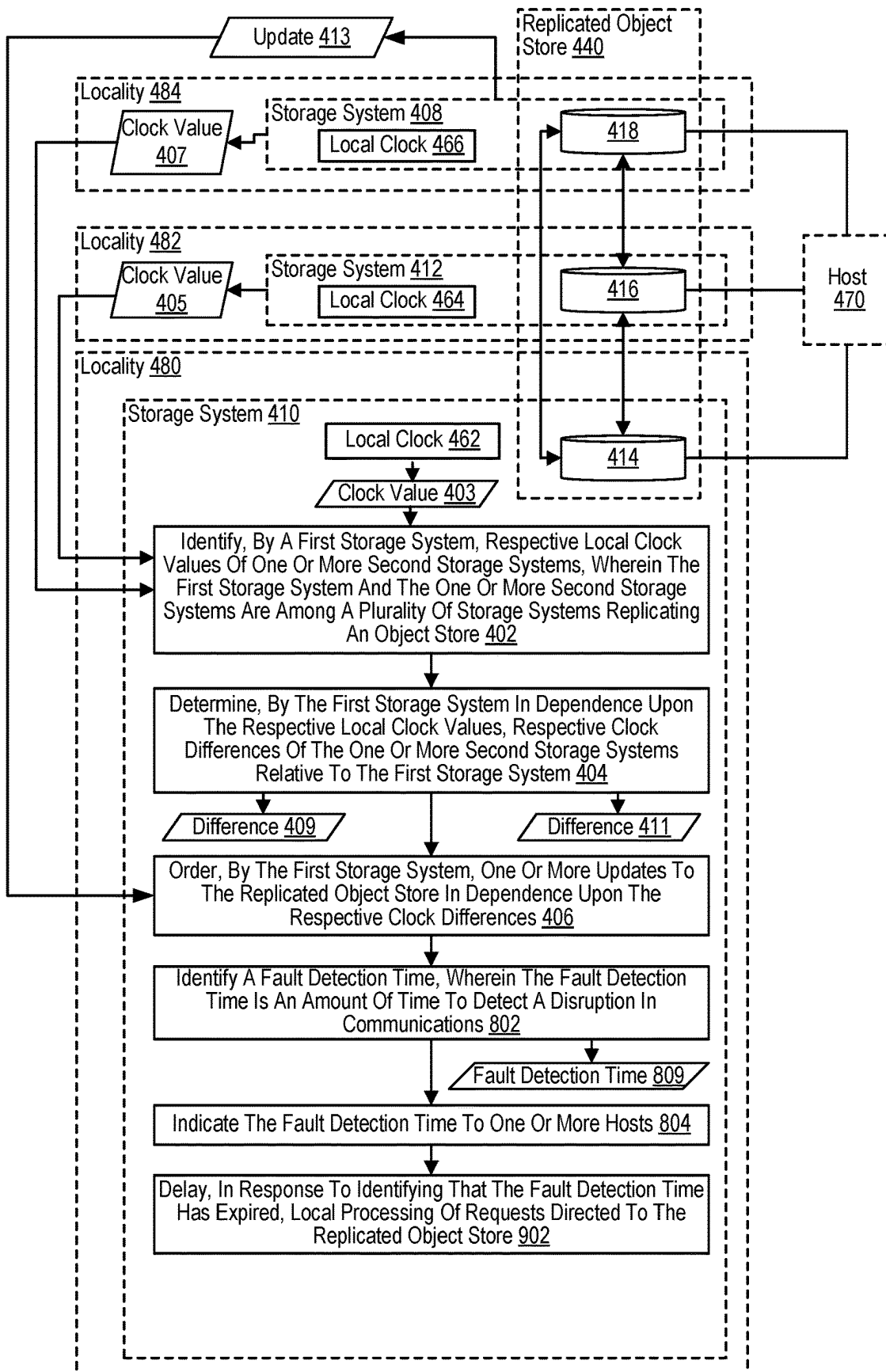


FIG. 9

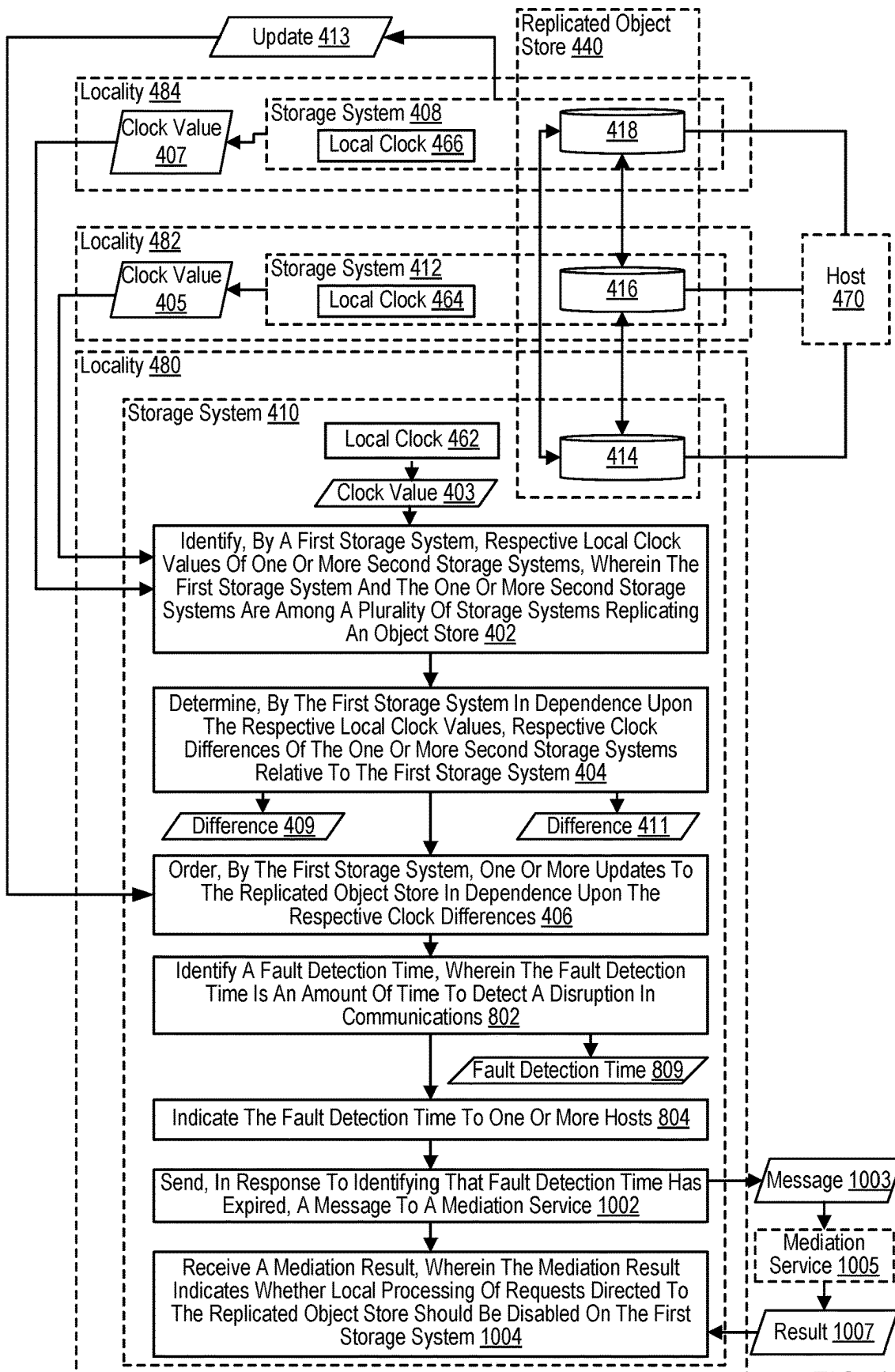


FIG. 10

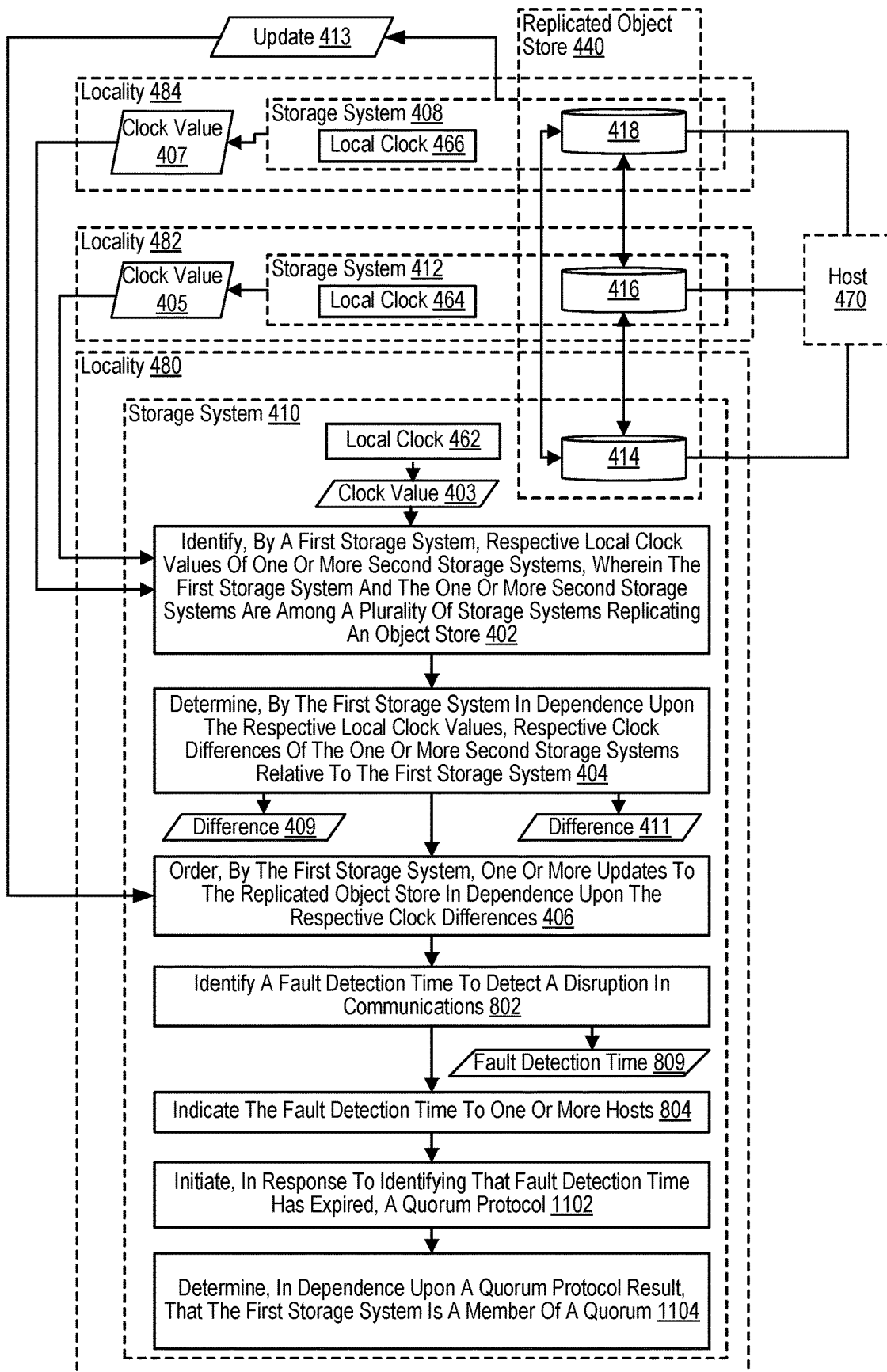


FIG. 11

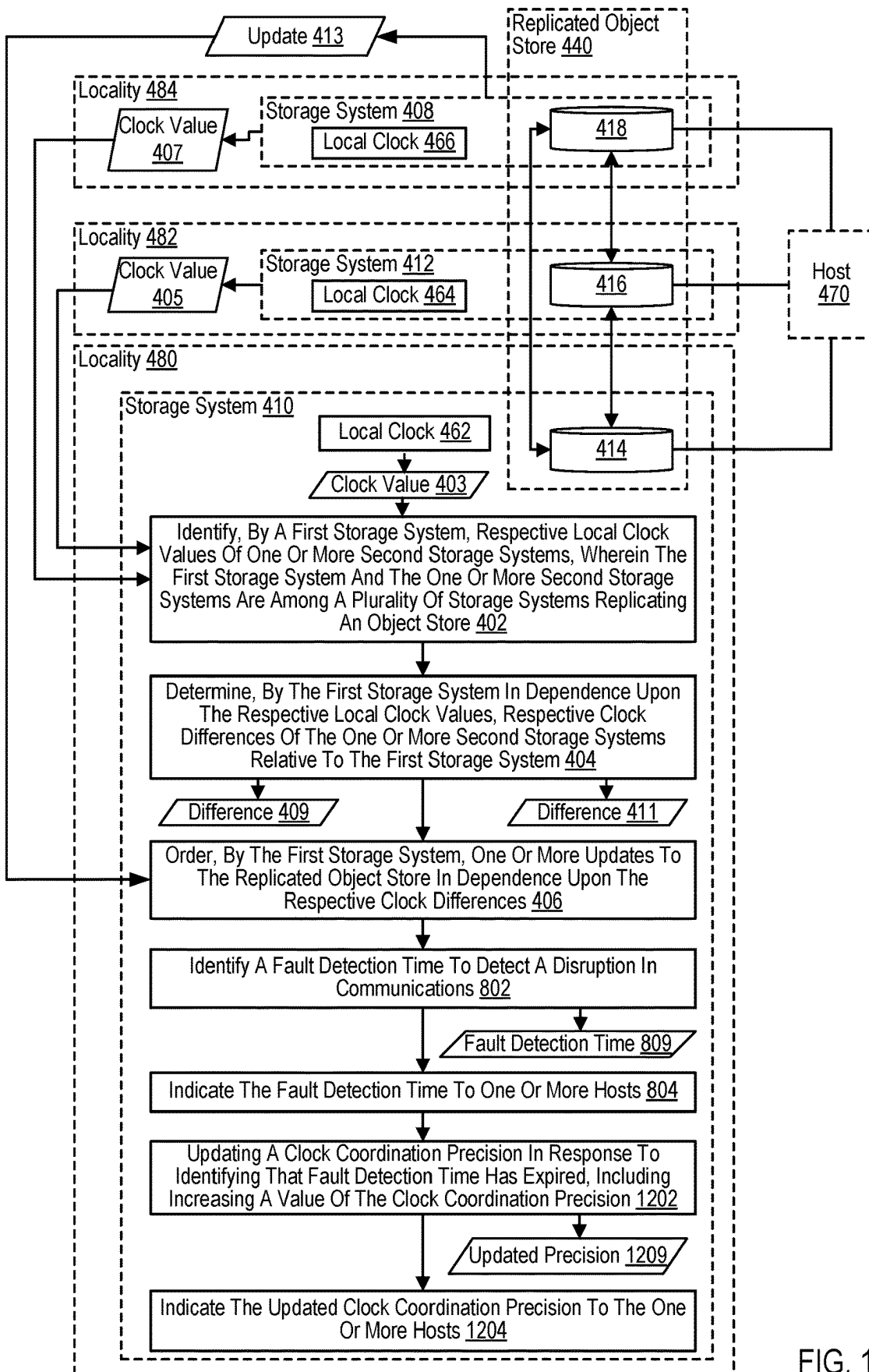


FIG. 12

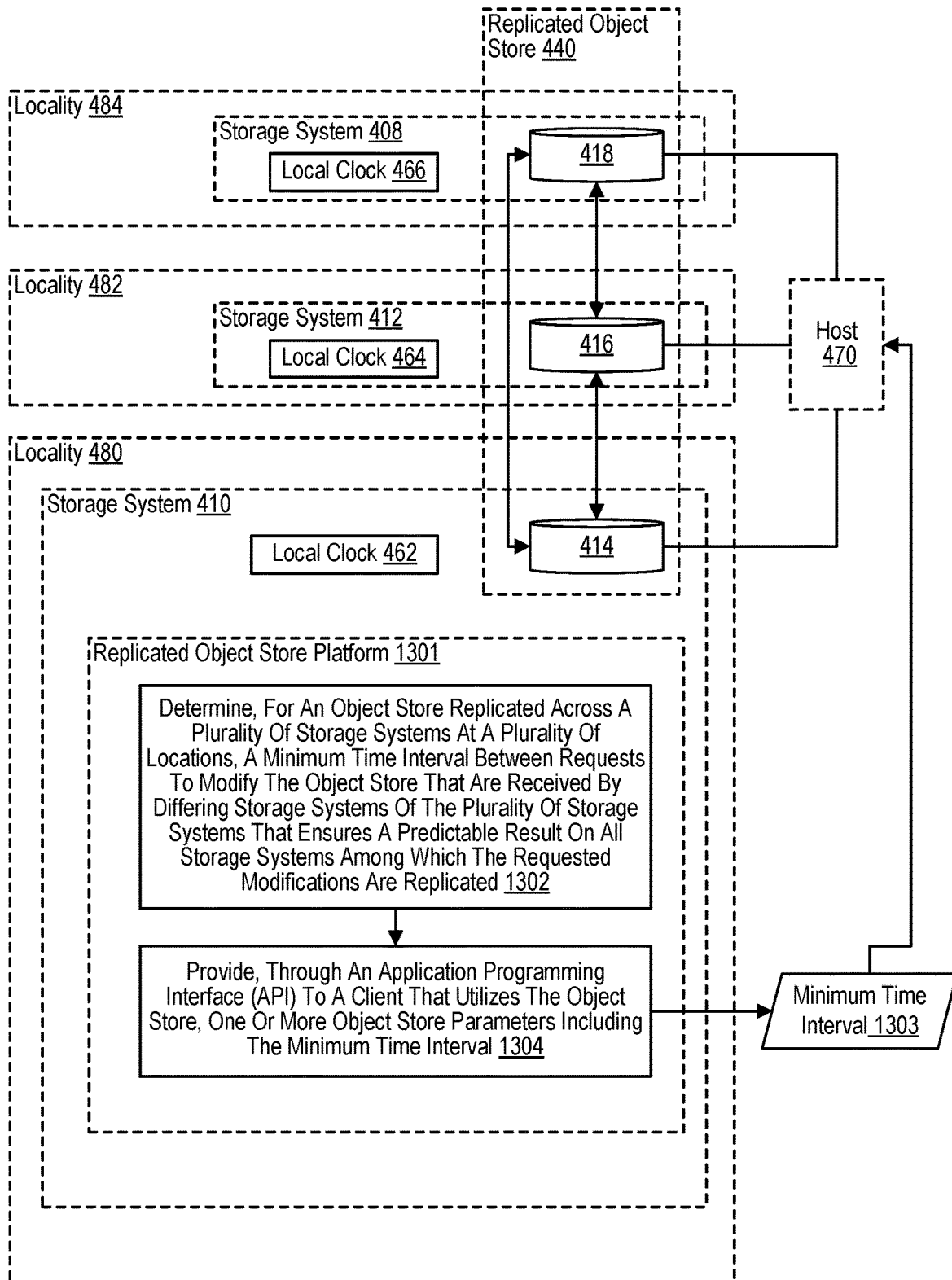


FIG. 13

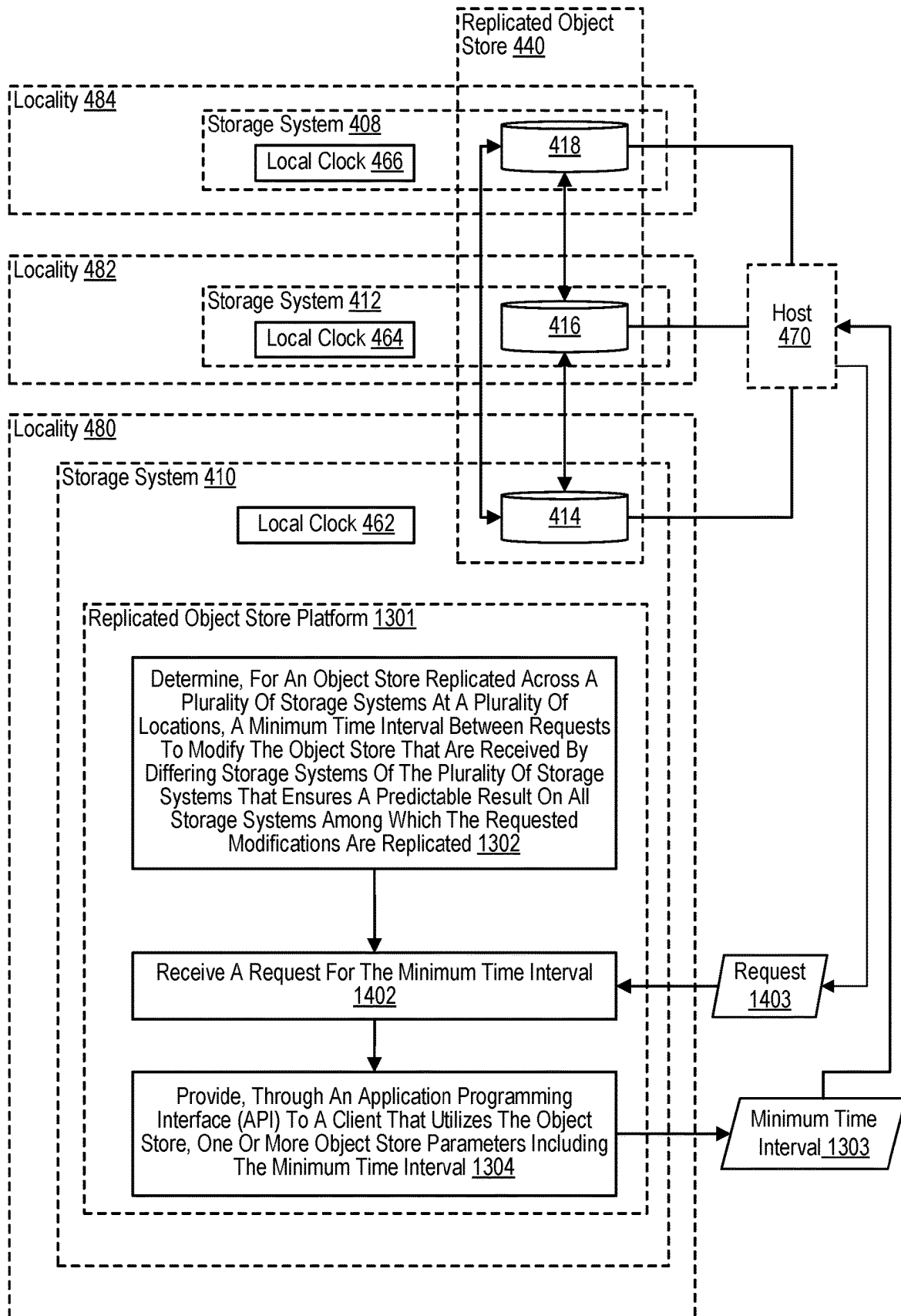


FIG. 14

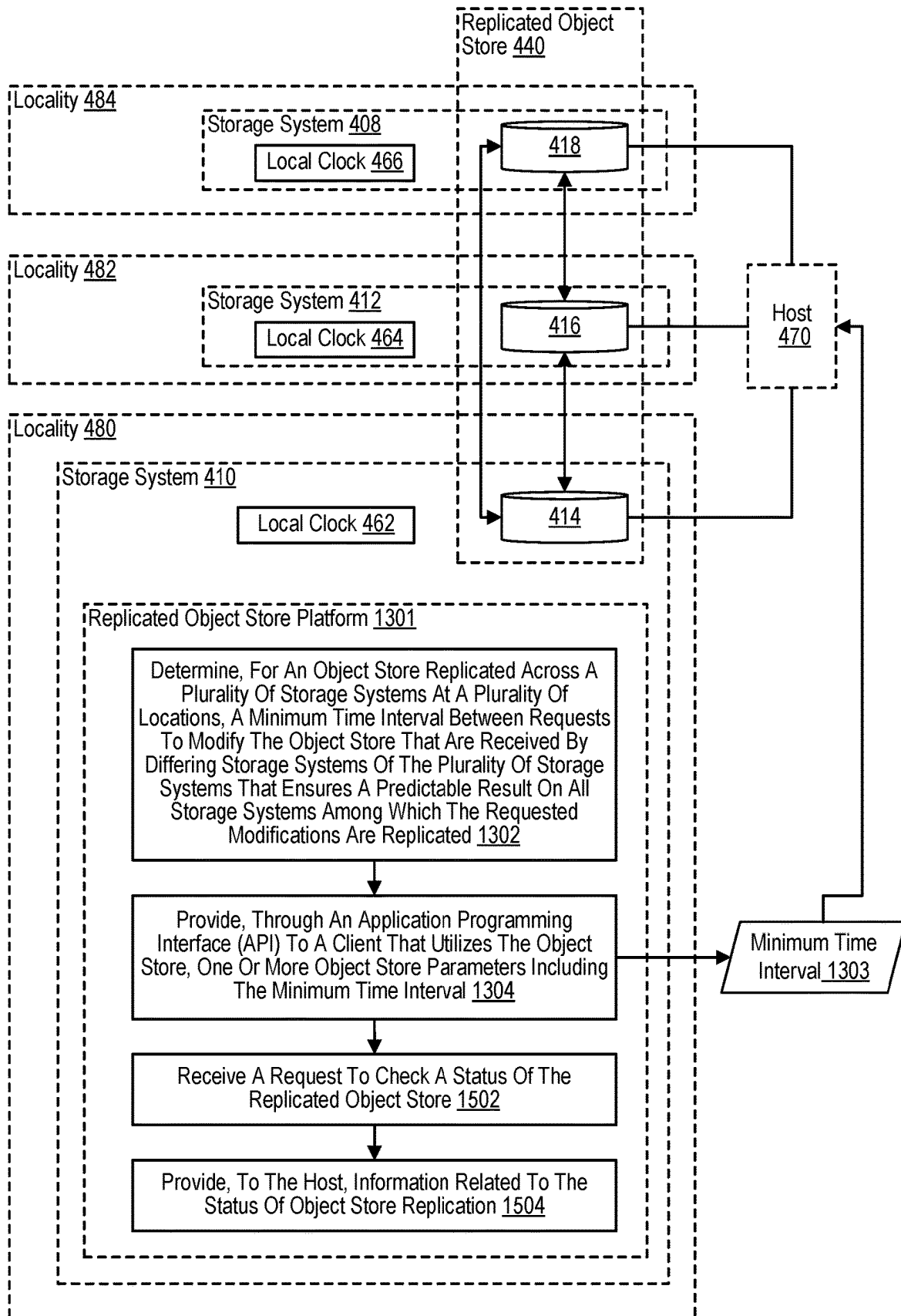


FIG. 15

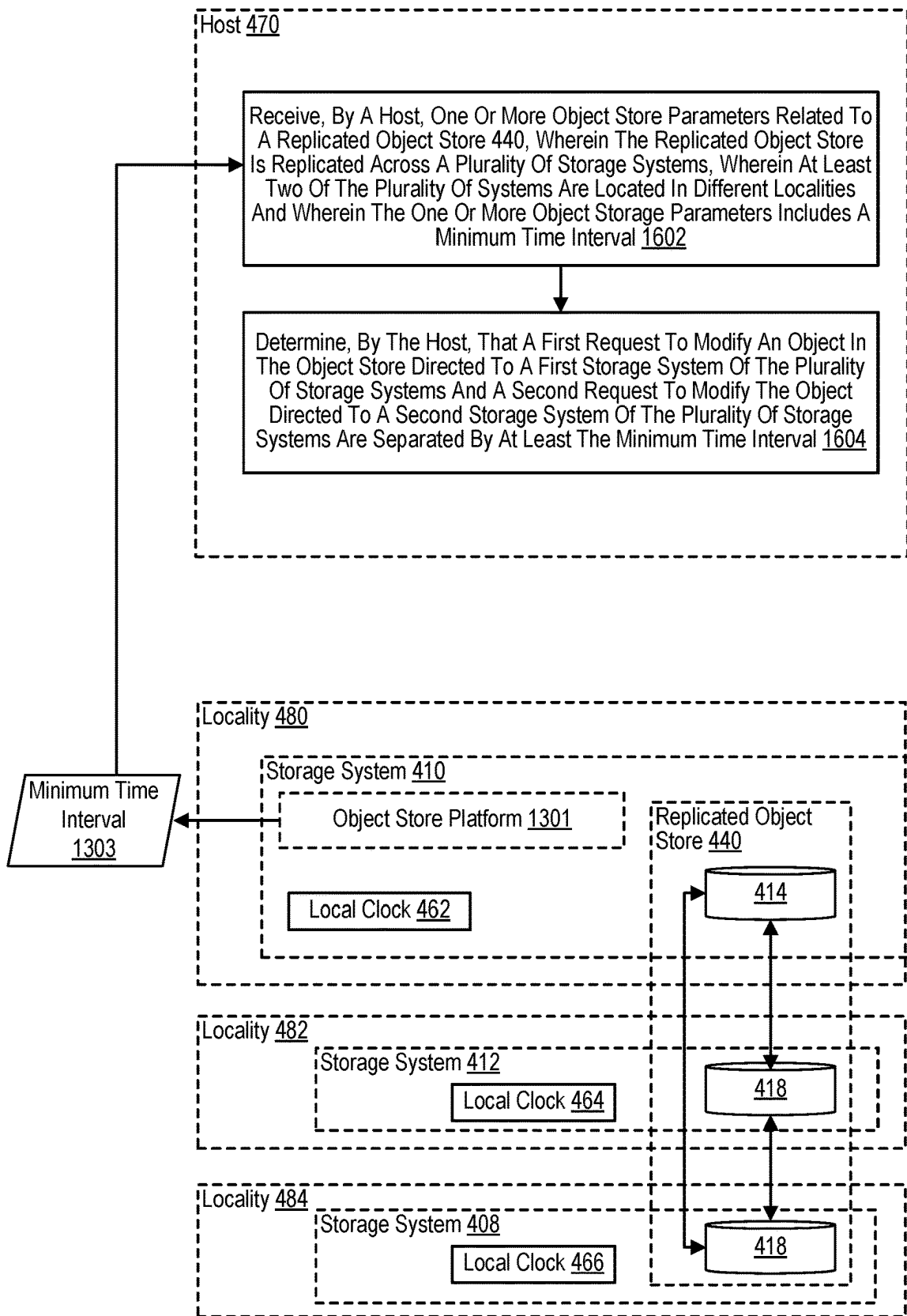


FIG. 16

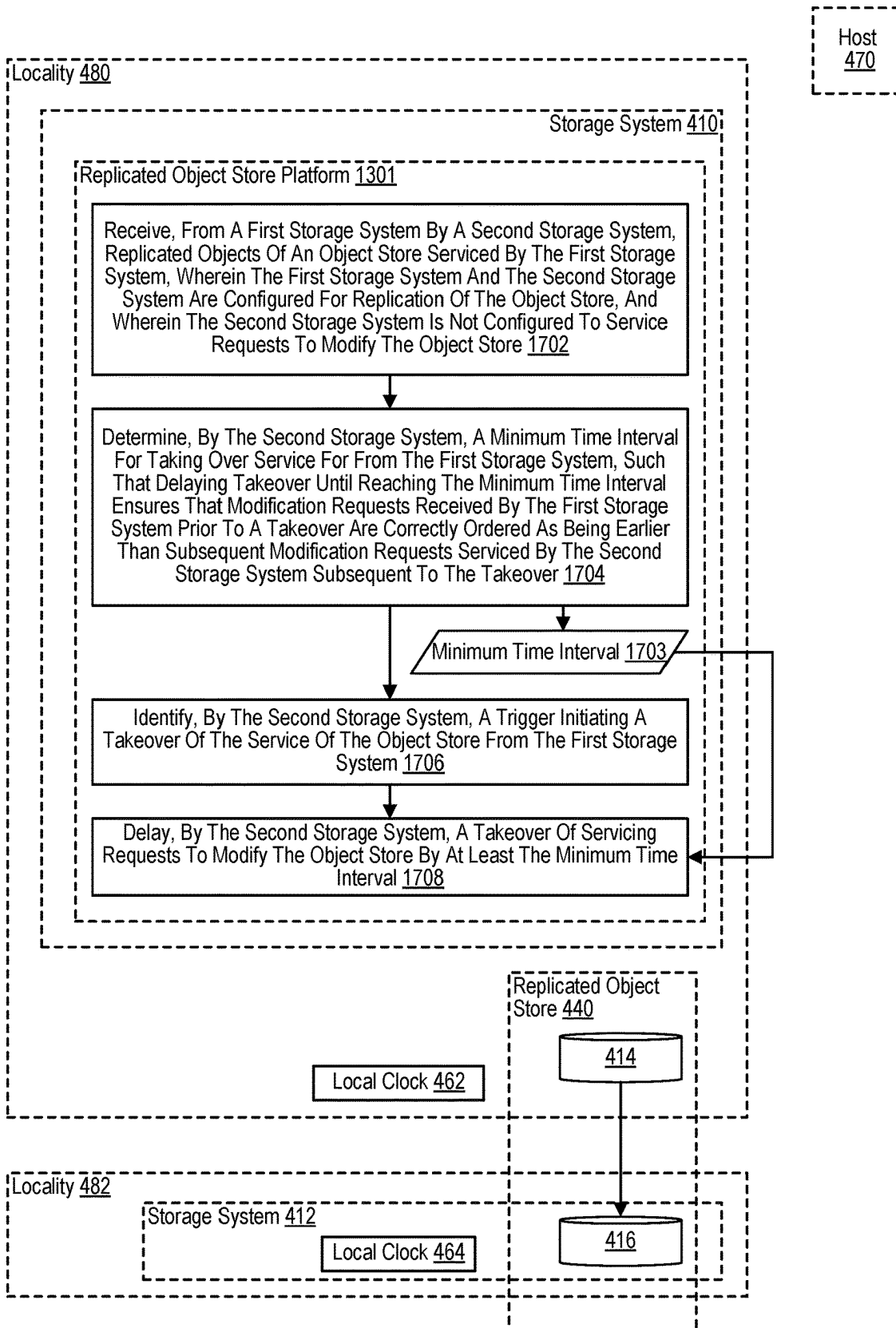


FIG. 17

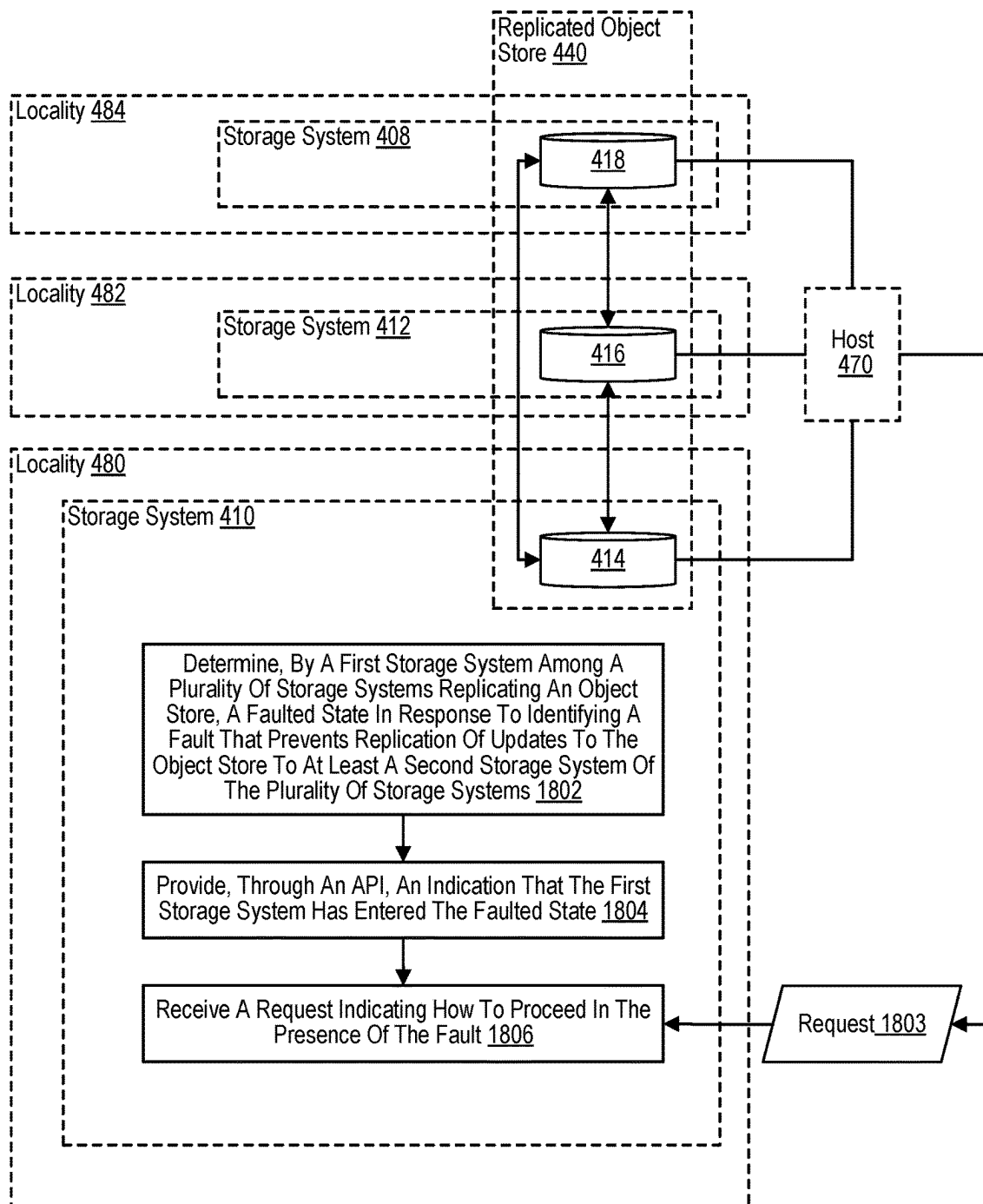


FIG. 18

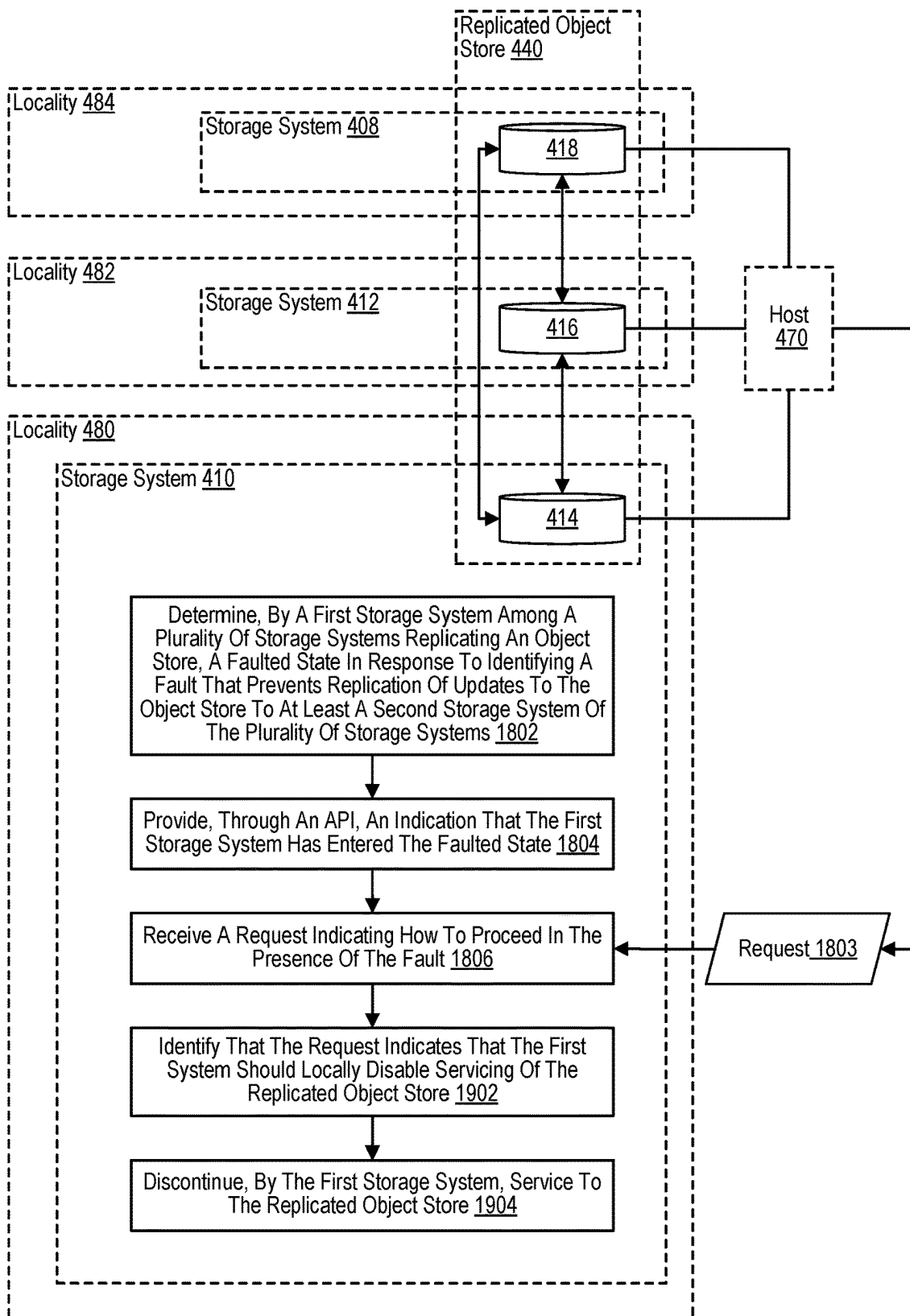


FIG. 19

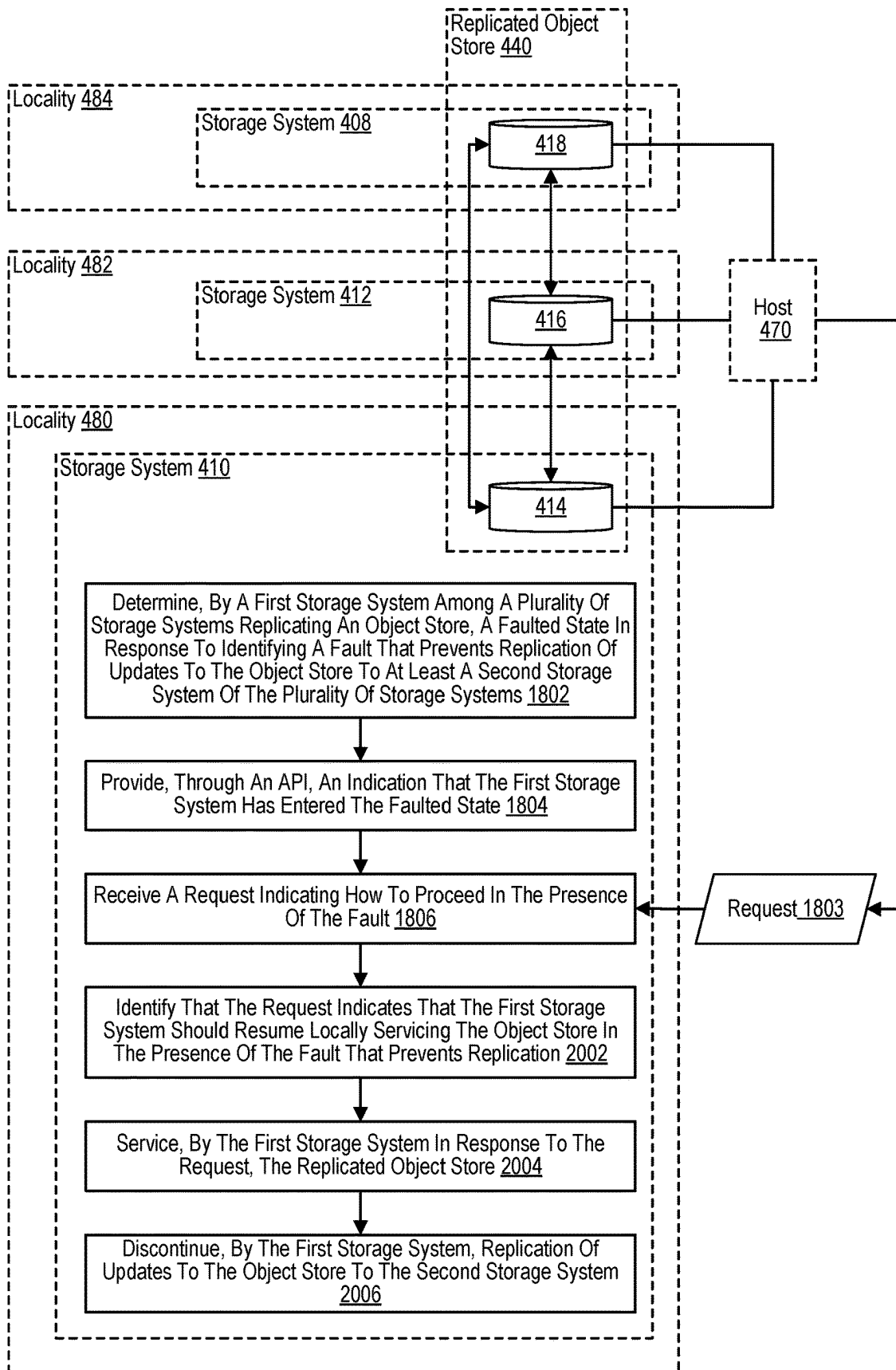


FIG. 20

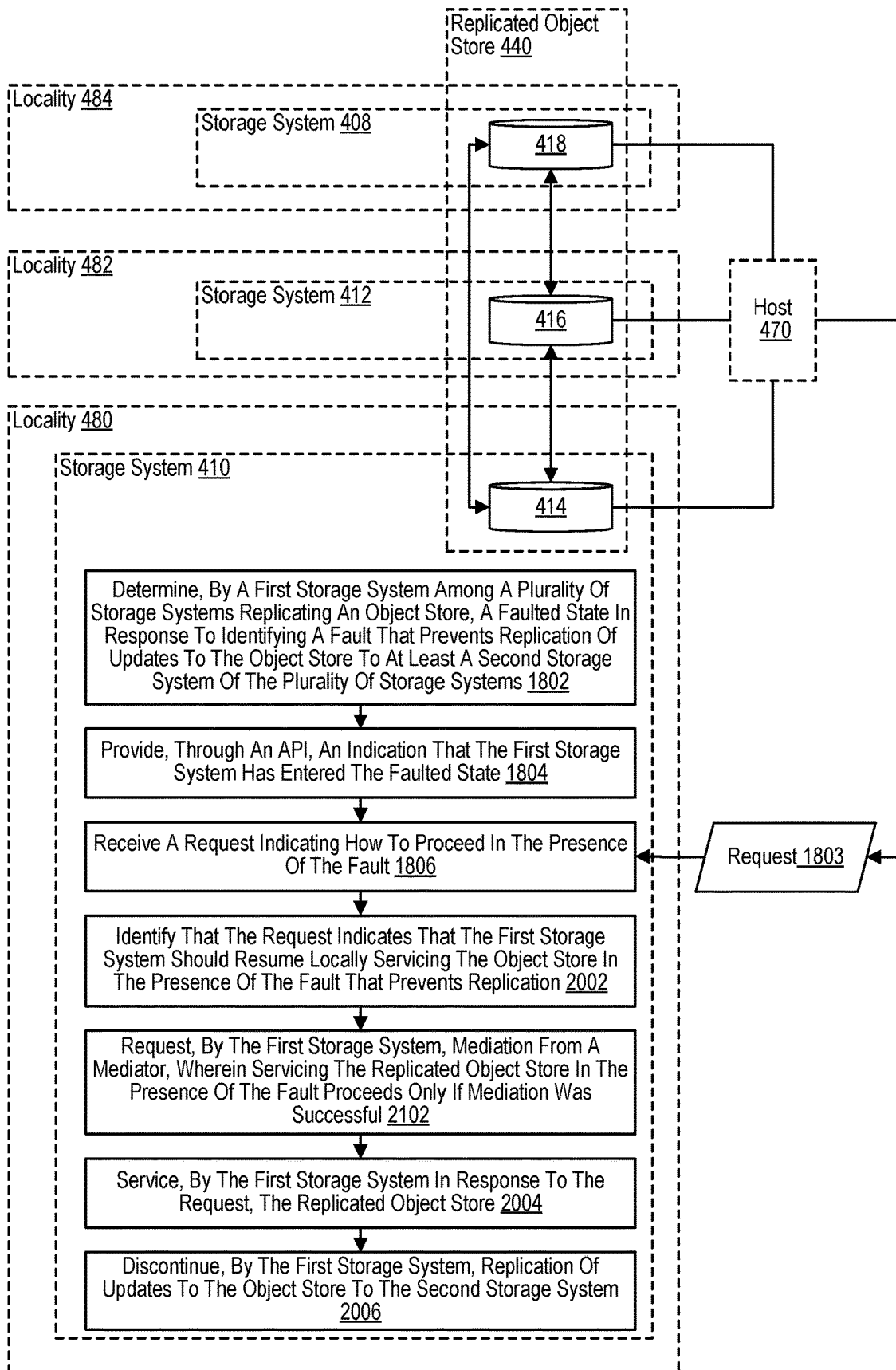


FIG. 21

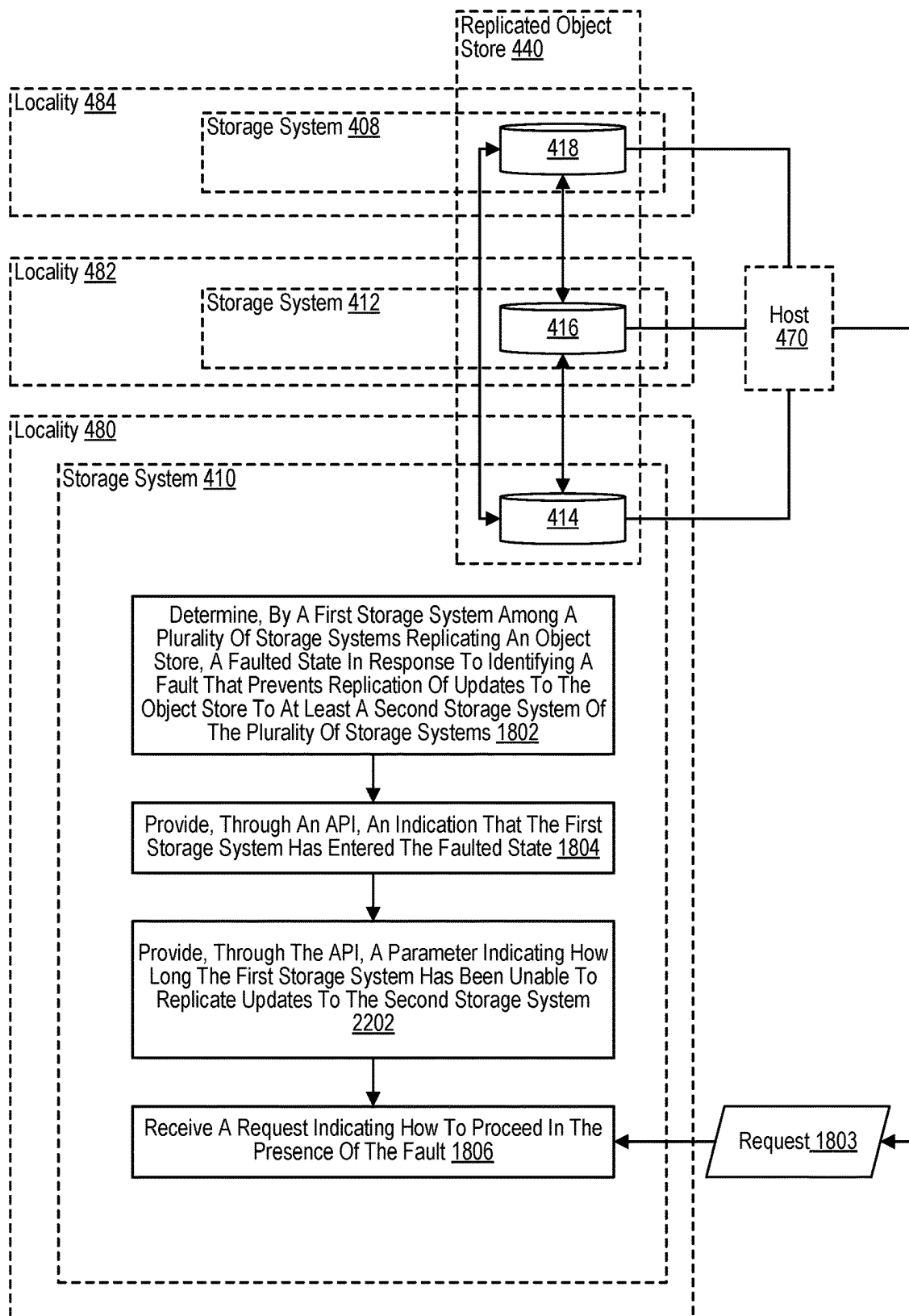


FIG. 22

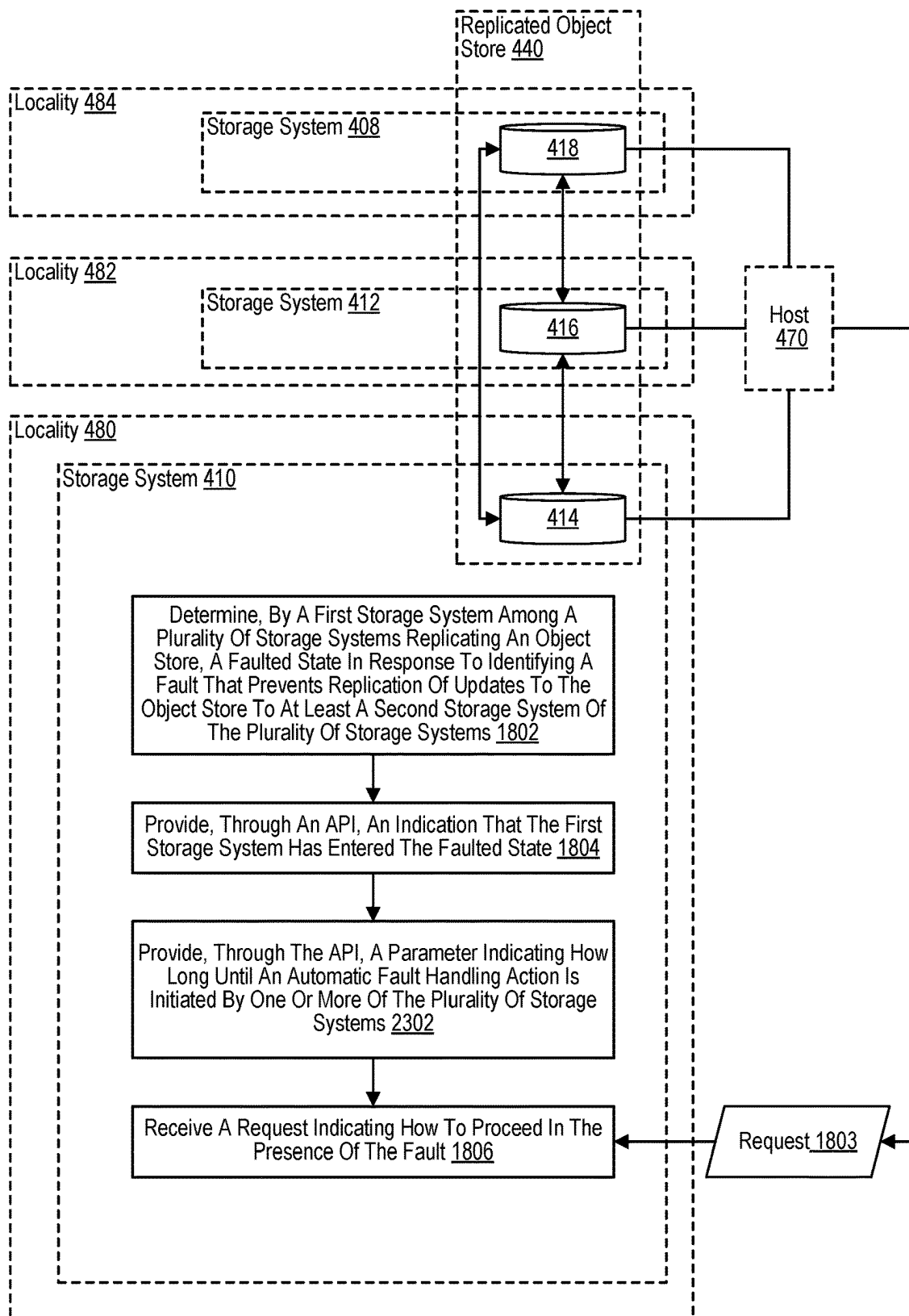


FIG. 23

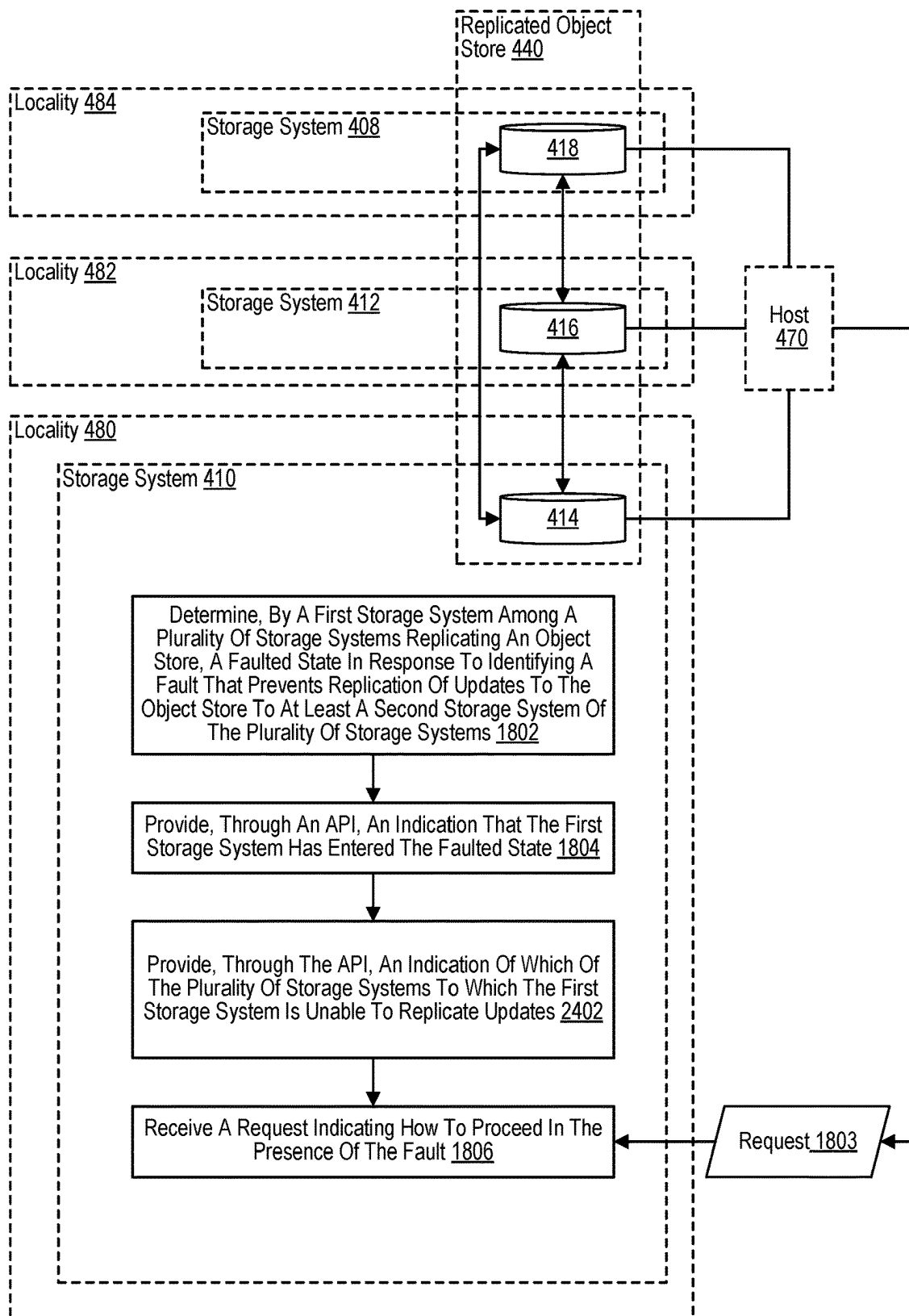


FIG. 24

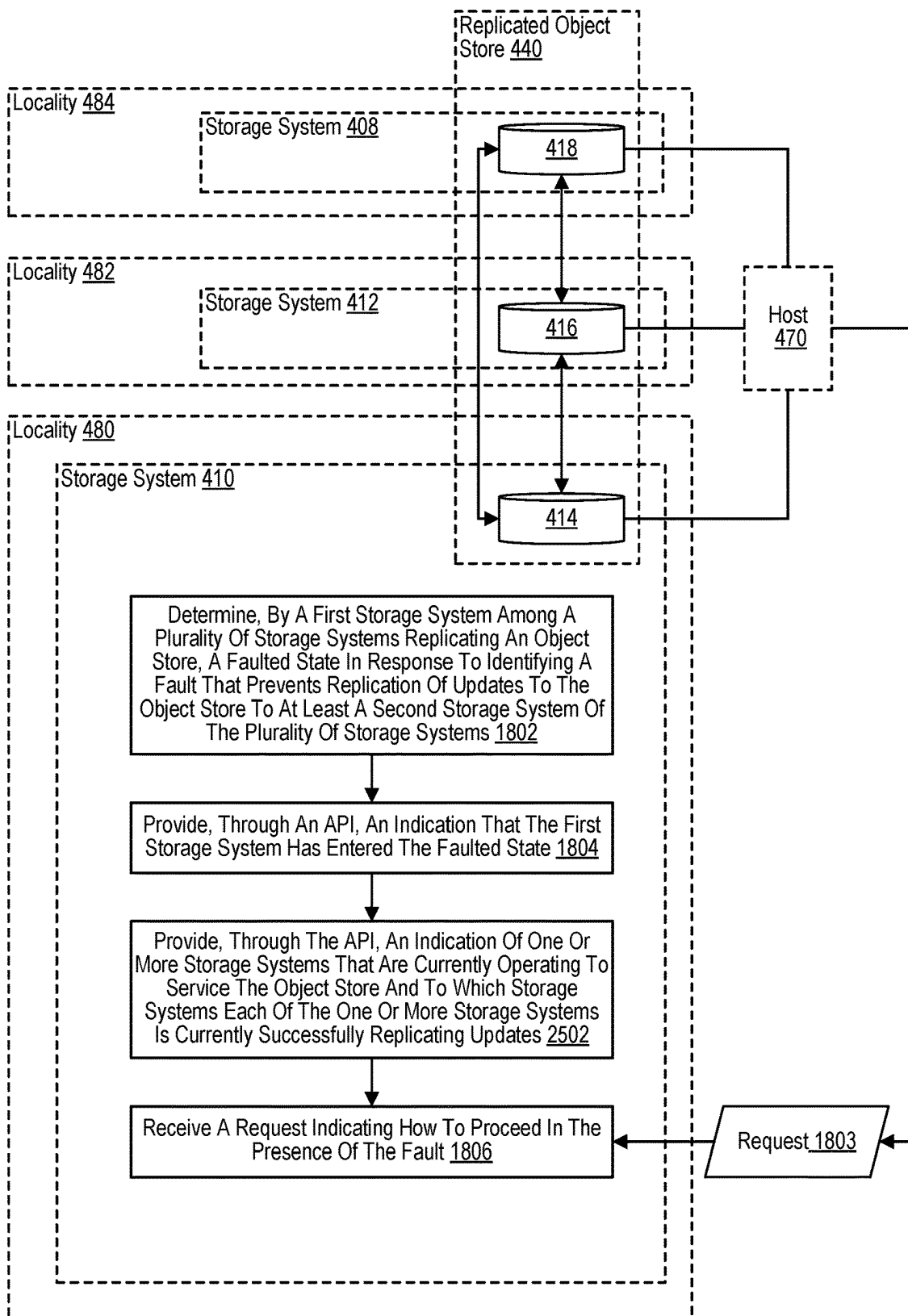


FIG. 25

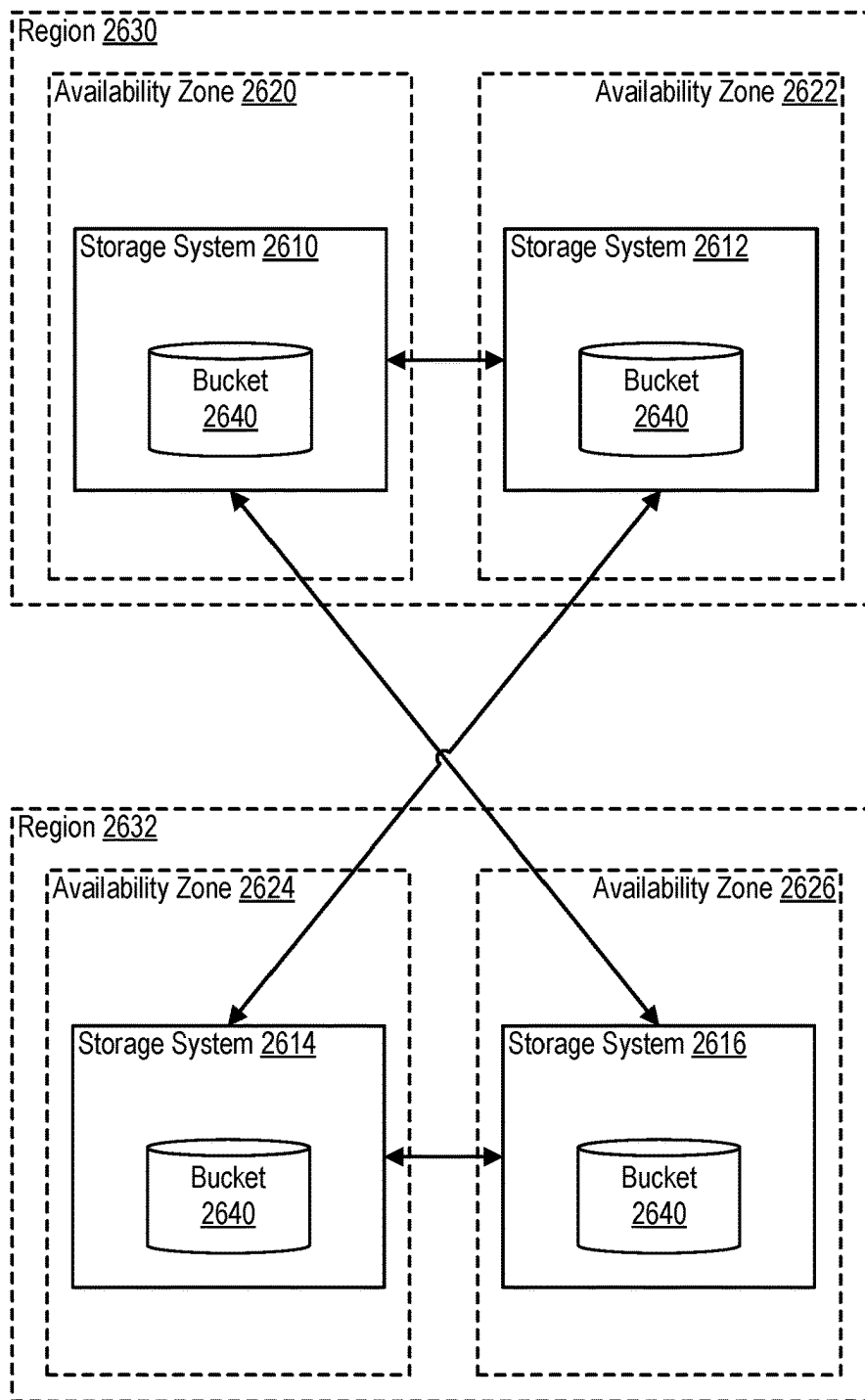


FIG. 26

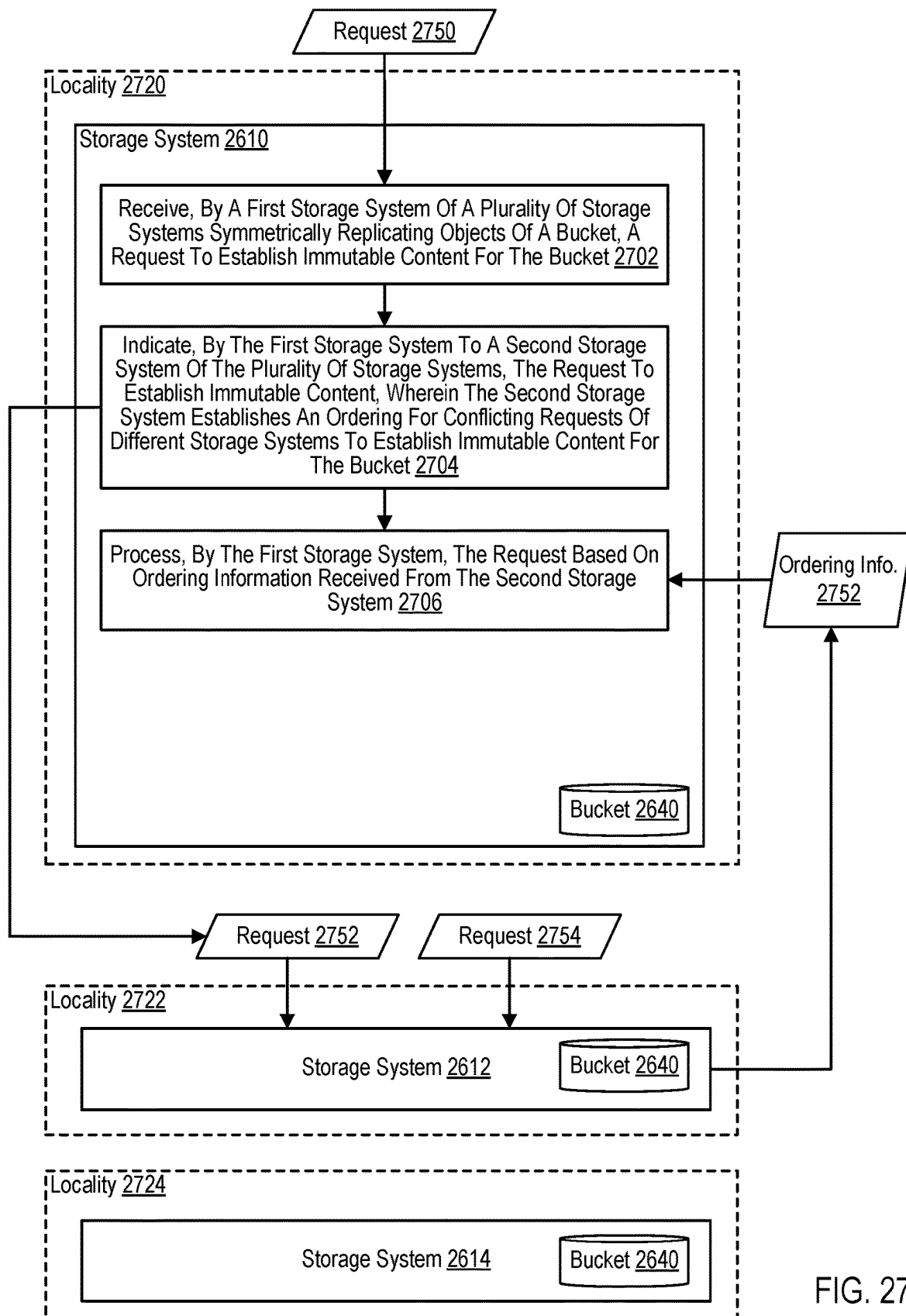


FIG. 27

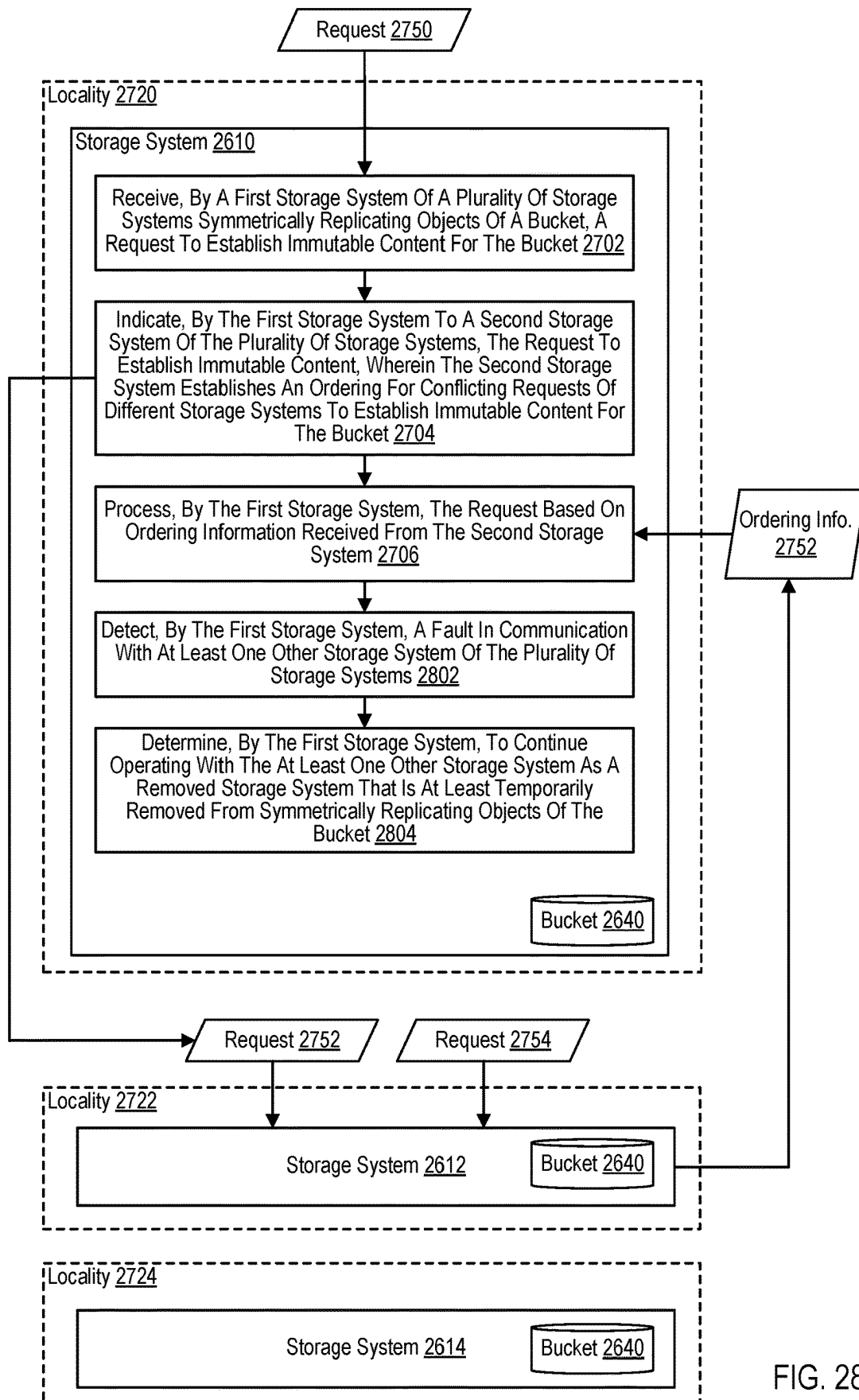


FIG. 28

1

**PROVIDING APPLICATION-SIDE
INFRASTRUCTURE TO CONTROL
CROSS-REGION REPLICATED OBJECT
STORES**

CROSS-REFERENCE TO RELATED
APPLICATIONS

This is a continuation in-part application for patent entitled to a filing date and claiming the benefit of earlier-filed U.S. patent application Ser. No. 18/152,148, filed Jan. 9, 2023, herein incorporated by reference in its entirety, which claims priority to U.S. Provisional Patent Application No. 63/298,161, filed Jan. 10, 2022.

BRIEF DESCRIPTION OF DRAWINGS

FIG. 1A illustrates a first example system for data storage. FIG. 1B illustrates a second example system for data storage.

FIG. 1C illustrates a third example system for data storage.

FIG. 1D illustrates a fourth example system for data storage.

FIG. 2A is a perspective view of a storage cluster with multiple storage nodes and internal storage coupled to each storage node to provide network attached storage.

FIG. 2B is a block diagram showing an interconnect switch coupling multiple storage nodes in accordance with some embodiments.

FIG. 2C is a multiple level block diagram, showing contents of a storage node and contents of one of the non-volatile solid state storage units.

FIG. 2D shows a storage server environment, which uses embodiments of the storage nodes and storage units of some previous figures in accordance with some embodiments.

FIG. 2E is a blade hardware block diagram, showing a control plane, compute and storage planes, and authorities interacting with underlying physical resources.

FIG. 2F depicts elasticity software layers in blades of a storage cluster.

FIG. 2G depicts authorities and storage resources in blades of a storage cluster.

FIG. 3A sets forth a diagram of a storage system that is coupled for data communications with a cloud services provider in accordance with some embodiments of the present disclosure.

FIG. 3B sets forth a diagram of a storage system.

FIG. 3C sets forth an example of a cloud-based storage system.

FIG. 3D illustrates an exemplary computing device that may be specifically configured to perform one or more of the processes described herein.

FIG. 3E illustrates an example of a fleet of storage systems for providing storage services (also referred to herein as 'data services') in accordance with some embodiments.

FIG. 3F illustrates an example container system.

FIG. 4 sets forth a flow chart of an example method for establishing a guarantee for maintaining a replication relationship between object stores during a communications outage in accordance with some embodiments of the present disclosure.

FIG. 5 sets forth a flow chart of another example method for establishing a guarantee for maintaining a replication

2

relationship between object stores during a communications outage in accordance with some embodiments of the present disclosure.

FIG. 6 sets forth a flow chart of another example method for establishing a guarantee for maintaining a replication relationship between object stores during a communications outage in accordance with some embodiments of the present disclosure.

FIG. 7 sets forth a flow chart of another example method for establishing a guarantee for maintaining a replication relationship between object stores during a communications outage in accordance with some embodiments of the present disclosure.

FIG. 8 sets forth a diagram of an example interaction for establishing a guarantee for maintaining a replication relationship between object stores during a communications outage in accordance with some embodiments of the present disclosure.

FIG. 9 sets forth a flow chart of another example method for establishing a guarantee for maintaining a replication relationship between object stores during a communications outage in accordance with some embodiments of the present disclosure.

FIG. 10 sets forth a flow chart of another example method for establishing a guarantee for maintaining a replication relationship between object stores during a communications outage in accordance with some embodiments of the present disclosure.

FIG. 11 sets forth a flow chart of another example method for establishing a guarantee for maintaining a replication relationship between object stores during a communications outage in accordance with some embodiments of the present disclosure.

FIG. 12 sets forth a flow chart of another example method for establishing a guarantee for maintaining a replication relationship between object stores during a communications outage in accordance with some embodiments of the present disclosure.

FIG. 13 sets forth a flow chart of an example method for providing application-side infrastructure to control cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 14 sets forth a flow chart of another example method for providing application-side infrastructure to control cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 15 sets forth a flow chart of another example method for providing application-side infrastructure to control cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 16 sets forth a flow chart of another example method for providing application-side infrastructure to control cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 17 sets forth a flow chart of an example method for controlling the direction of replication between cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 18 sets forth a flow chart of another example method for application-managed fault handling for cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 19 sets forth a flow chart of another example method for application-managed fault handling for cross-region replicated object stores in accordance with some embodiments of the present disclosure.

3

FIG. 20 sets forth a flow chart of another example method for application-managed fault handling for cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 21 sets forth a flow chart of another example method for application-managed fault handling for cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 22 sets forth a flow chart of another example method for application-managed fault handling for cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 23 sets forth a flow chart of another example method for application-managed fault handling for cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 24 sets forth a flow chart of another example method for application-managed fault handling for cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 25 sets forth a flow chart of another example method for application-managed fault handling for cross-region replicated object stores in accordance with some embodiments of the present disclosure.

FIG. 26 sets forth an example environment for high availability and disaster recovery for replication object stores in accordance with some embodiments of the present disclosure.

FIG. 27 sets forth a flow chart of an example method for high availability and disaster recovery for replication object stores in accordance with some embodiments of the present disclosure.

FIG. 28 sets forth a flow chart of another example method for high availability and disaster recovery for replication object stores in accordance with some embodiments of the present disclosure.

DESCRIPTION OF EMBODIMENTS

Example methods, apparatus, and products for providing application-side infrastructure to control cross-region replicated object stores in accordance with embodiments of the present disclosure are described with reference to the accompanying drawings, beginning with FIG. 1A. FIG. 1A illustrates an example system for data storage, in accordance with some implementations. System 100 (also referred to as “storage system” herein) includes numerous elements for purposes of illustration rather than limitation. It may be noted that system 100 may include the same, more, or fewer elements configured in the same or different manner in other implementations.

System 100 includes a number of computing devices 164A-B. Computing devices (also referred to as “client devices” herein) may be embodied, for example, a server in a data center, a workstation, a personal computer, a notebook, or the like. Computing devices 164A-B may be coupled for data communications to one or more storage arrays 102A-B through a storage area network (“SAN”) 158 or a local area network (“LAN”) 160.

The SAN 158 may be implemented with a variety of data communications fabrics, devices, and protocols. For example, the fabrics for SAN 158 may include Fibre Channel, Ethernet, Infiniband, Serial Attached Small Computer System Interface (“SAS”), or the like. Data communications protocols for use with SAN 158 may include Advanced Technology Attachment (“ATA”), Fibre Channel Protocol, Small Computer System Interface (“SCSI”), Internet Small

4

Computer System Interface (“iSCSI”), HyperSCSI, Non-Volatile Memory Express (“NVMe”) over Fabrics, or the like. It may be noted that SAN 158 is provided for illustration, rather than limitation. Other data communication couplings may be implemented between computing devices 164A-B and storage arrays 102A-B.

The LAN 160 may also be implemented with a variety of fabrics, devices, and protocols. For example, the fabrics for LAN 160 may include Ethernet (802.3), wireless (802.11), or the like. Data communication protocols for use in LAN 160 may include Transmission Control Protocol (“TCP”), User Datagram Protocol (“UDP”), Internet Protocol (“IP”), HyperText Transfer Protocol (“HTTP”), Wireless Access Protocol (“WAP”), Handheld Device Transport Protocol (“HDT”), Session Initiation Protocol (“SIP”), Real Time Protocol (“RTP”), or the like.

Storage arrays 102A-B may provide persistent data storage for the computing devices 164A-B. Storage array 102A may be contained in a chassis (not shown), and storage array 102B may be contained in another chassis (not shown), in some implementations. Storage array 102A and 102B may include one or more storage array controllers 110A-D (also referred to as “controller” herein). A storage array controller 110A-D may be embodied as a module of automated computing machinery comprising computer hardware, computer software, or a combination of computer hardware and software. In some implementations, the storage array controllers 110A-D may be configured to carry out various storage tasks. Storage tasks may include writing data received from the computing devices 164A-B to storage array 102A-B, erasing data from storage array 102A-B, retrieving data from storage array 102A-B and providing data to computing devices 164A-B, monitoring and reporting of storage device utilization and performance, performing redundancy operations, such as Redundant Array of Independent Drives (“RAID”) or RAID-like data redundancy operations, compressing data, encrypting data, and so forth.

Storage array controller 110A-D may be implemented in a variety of ways, including as a Field Programmable Gate Array (“FPGA”), a Programmable Logic Chip (“PLC”), an Application Specific Integrated Circuit (“ASIC”), System-on-Chip (“SOC”), or any computing device that includes discrete components such as a processing device, central processing unit, computer memory, or various adapters. Storage array controller 110A-D may include, for example, a data communications adapter configured to support communications via the SAN 158 or LAN 160. In some implementations, storage array controller 110A-D may be independently coupled to the LAN 160. In some implementations, storage array controller 110A-D may include an I/O controller or the like that couples the storage array controller 110A-D for data communications, through a midplane (not shown), to a persistent storage resource 170A-B (also referred to as a “storage resource” herein). The persistent storage resource 170A-B may include any number of storage drives 171A-F (also referred to as “storage devices” herein) and any number of non-volatile Random Access Memory (“NVRAM”) devices (not shown).

In some implementations, the NVRAM devices of a persistent storage resource 170A-B may be configured to receive, from the storage array controller 110A-D, data to be stored in the storage drives 171A-F. In some examples, the data may originate from computing devices 164A-B. In some examples, writing data to the NVRAM device may be carried out more quickly than directly writing data to the storage drive 171A-F. In some implementations, the storage array controller 110A-D may be configured to utilize the

5

NVRAM devices as a quickly accessible buffer for data destined to be written to the storage drives 171A-F. Latency for write requests using NVRAM devices as a buffer may be improved relative to a system in which a storage array controller 110A-D writes data directly to the storage drives 171A-F. In some implementations, the NVRAM devices may be implemented with computer memory in the form of high bandwidth, low latency RAM. The NVRAM device is referred to as “non-volatile” because the NVRAM device may receive or include a unique power source that maintains the state of the RAM after main power loss to the NVRAM device. Such a power source may be a battery, one or more capacitors, or the like. In response to a power loss, the NVRAM device may be configured to write the contents of the RAM to a persistent storage, such as the storage drives 171A-F.

In some implementations, storage drive 171A-F may refer to any device configured to record data persistently, where “persistently” or “persistent” refers as to a device’s ability to maintain recorded data after loss of power. In some implementations, storage drive 171A-F may correspond to non-disk storage media. For example, the storage drive 171A-F may be one or more solid-state drives (“SSDs”), flash memory based storage, any type of solid-state non-volatile memory, or any other type of non-mechanical storage device. In other implementations, storage drive 171A-F may include mechanical or spinning hard disk, such as hard-disk drives (“HDD”).

In some implementations, the storage array controllers 110A-D may be configured for offloading device management responsibilities from storage drive 171A-F in storage array 102A-B. For example, storage array controllers 110A-D may manage control information that may describe the state of one or more memory blocks in the storage drives 171A-F. The control information may indicate, for example, that a particular memory block has failed and should no longer be written to, that a particular memory block contains boot code for a storage array controller 110A-D, the number of program-erase (“PIE”) cycles that have been performed on a particular memory block, the age of data stored in a particular memory block, the type of data that is stored in a particular memory block, and so forth. In some implementations, the control information may be stored with an associated memory block as metadata. In other implementations, the control information for the storage drives 171A-F may be stored in one or more particular memory blocks of the storage drives 171A-F that are selected by the storage array controller 110A-D. The selected memory blocks may be tagged with an identifier indicating that the selected memory block contains control information. The identifier may be utilized by the storage array controllers 110A-D in conjunction with storage drives 171A-F to quickly identify the memory blocks that contain control information. For example, the storage controllers 110A-D may issue a command to locate memory blocks that contain control information. It may be noted that control information may be so large that parts of the control information may be stored in multiple locations, that the control information may be stored in multiple locations for purposes of redundancy, for example, or that the control information may otherwise be distributed across multiple memory blocks in the storage drives 171A-F.

In some implementations, storage array controllers 110A-D may offload device management responsibilities from storage drives 171A-F of storage array 102A-B by retrieving, from the storage drives 171A-F, control information describing the state of one or more memory blocks in

6

the storage drives 171A-F. Retrieving the control information from the storage drives 171A-F may be carried out, for example, by the storage array controller 110A-D querying the storage drives 171A-F for the location of control information for a particular storage drive 171A-F. The storage drives 171A-F may be configured to execute instructions that enable the storage drives 171A-F to identify the location of the control information. The instructions may be executed by a controller (not shown) associated with or otherwise located on the storage drive 171A-F and may cause the storage drive 171A-F to scan a portion of each memory block to identify the memory blocks that store control information for the storage drives 171A-F. The storage drives 171A-F may respond by sending a response message to the storage array controller 110A-D that includes the location of control information for the storage drive 171A-F. Responsive to receiving the response message, storage array controllers 110A-D may issue a request to read data stored at the address associated with the location of control information for the storage drives 171A-F.

In other implementations, the storage array controllers 110A-D may further offload device management responsibilities from storage drives 171A-F by performing, in response to receiving the control information, a storage drive management operation. A storage drive management operation may include, for example, an operation that is typically performed by the storage drive 171A-F (e.g., the controller (not shown) associated with a particular storage drive 171A-F). A storage drive management operation may include, for example, ensuring that data is not written to failed memory blocks within the storage drive 171A-F, ensuring that data is written to memory blocks within the storage drive 171A-F in such a way that adequate wear leveling is achieved, and so forth.

In some implementations, storage array 102A-B may implement two or more storage array controllers 110A-D. For example, storage array 102A may include storage array controllers 110A and storage array controllers 110B. At a given instant, a single storage array controller 110A-D (e.g., storage array controller 110A) of a storage system 100 may be designated with primary status (also referred to as “primary controller” herein), and other storage array controllers 110A-D (e.g., storage array controller 110B) may be designated with secondary status (also referred to as “secondary controller” herein). The primary controller may have particular rights, such as permission to alter data in persistent storage resource 170A-B (e.g., writing data to persistent storage resource 170A-B). At least some of the rights of the primary controller may supersede the rights of the secondary controller. For instance, the secondary controller may not have permission to alter data in persistent storage resource 170A-B when the primary controller has the right. The status of storage array controllers 110A-D may change. For example, storage array controller 110A may be designated with secondary status, and storage array controller 110B may be designated with primary status.

In some implementations, a primary controller, such as storage array controller 110A, may serve as the primary controller for one or more storage arrays 102A-B, and a second controller, such as storage array controller 110B, may serve as the secondary controller for the one or more storage arrays 102A-B. For example, storage array controller 110A may be the primary controller for storage array 102A and storage array 102B, and storage array controller 110B may be the secondary controller for storage array 102A and 102B. In some implementations, storage array controllers 110C and 110D (also referred to as “storage

processing modules”) may neither have primary or secondary status. Storage array controllers **110C** and **110D**, implemented as storage processing modules, may act as a communication interface between the primary and secondary controllers (e.g., storage array controllers **110A** and **110B**, respectively) and storage array **102B**. For example, storage array controller **110A** of storage array **102A** may send a write request, via SAN **158**, to storage array **102B**. The write request may be received by both storage array controllers **110C** and **110D** of storage array **102B**. Storage array controllers **110C** and **110D** facilitate the communication, e.g., send the write request to the appropriate storage drive **171A-F**. It may be noted that in some implementations storage processing modules may be used to increase the number of storage drives controlled by the primary and secondary controllers.

In some implementations, storage array controllers **110A-D** are communicatively coupled, via a midplane (not shown), to one or more storage drives **171A-F** and to one or more NVRAM devices (not shown) that are included as part of a storage array **102A-B**. The storage array controllers **110A-D** may be coupled to the midplane via one or more data communication links and the midplane may be coupled to the storage drives **171A-F** and the NVRAM devices via one or more data communications links. The data communications links described herein are collectively illustrated by data communications links **108A-D** and may include a Peripheral Component Interconnect Express (‘PCIe’) bus, for example.

FIG. 1B illustrates an example system for data storage, in accordance with some implementations. Storage array controller **101** illustrated in FIG. 1B may be similar to the storage array controllers **110A-D** described with respect to FIG. 1A. In one example, storage array controller **101** may be similar to storage array controller **110A** or storage array controller **110B**. Storage array controller **101** includes numerous elements for purposes of illustration rather than limitation. It may be noted that storage array controller **101** may include the same, more, or fewer elements configured in the same or different manner in other implementations. It may be noted that elements of FIG. 1A may be included below to help illustrate features of storage array controller **101**.

Storage array controller **101** may include one or more processing devices **104** and random access memory (‘RAM’) **111**. Processing device **104** (or controller **101**) represents one or more general-purpose processing devices such as a microprocessor, central processing unit, or the like. More particularly, the processing device **104** (or controller **101**) may be a complex instruction set computing (‘CISC’) microprocessor, reduced instruction set computing (‘RISC’) microprocessor, very long instruction word (‘VLIW’) microprocessor, or a processor implementing other instruction sets or processors implementing a combination of instruction sets. The processing device **104** (or controller **101**) may also be one or more special-purpose processing devices such as an ASIC, an FPGA, a digital signal processor (‘DSP’), network processor, or the like.

The processing device **104** may be connected to the RAM **111** via a data communications link **106**, which may be embodied as a high speed memory bus such as a Double-Data Rate 4 (‘DDR4’) bus. Stored in RAM **111** is an operating system **112**. In some implementations, instructions **113** are stored in RAM **111**. Instructions **113** may include computer program instructions for performing operations in a direct-mapped flash storage system. In one embodiment, a direct-mapped flash storage system is one that addresses

data blocks within flash drives directly and without an address translation performed by the storage controllers of the flash drives.

In some implementations, storage array controller **101** includes one or more host bus adapters **103A-C** that are coupled to the processing device **104** via a data communications link **105A-C**. In some implementations, host bus adapters **103A-C** may be computer hardware that connects a host system (e.g., the storage array controller) to other network and storage arrays. In some examples, host bus adapters **103A-C** may be a Fibre Channel adapter that enables the storage array controller **101** to connect to a SAN, an Ethernet adapter that enables the storage array controller **101** to connect to a LAN, or the like. Host bus adapters **103A-C** may be coupled to the processing device **104** via a data communications link **105A-C** such as, for example, a PCIe bus.

In some implementations, storage array controller **101** may include a host bus adapter **114** that is coupled to an expander **115**. The expander **115** may be used to attach a host system to a larger number of storage drives. The expander **115** may, for example, be a SAS expander utilized to enable the host bus adapter **114** to attach to storage drives in an implementation where the host bus adapter **114** is embodied as a SAS controller.

In some implementations, storage array controller **101** may include a switch **116** coupled to the processing device **104** via a data communications link **109**. The switch **116** may be a computer hardware device that can create multiple endpoints out of a single endpoint, thereby enabling multiple devices to share a single endpoint. The switch **116** may, for example, be a PCIe switch that is coupled to a PCIe bus (e.g., data communications link **109**) and presents multiple PCIe connection points to the midplane.

In some implementations, storage array controller **101** includes a data communications link **107** for coupling the storage array controller **101** to other storage array controllers. In some examples, data communications link **107** may be a QuickPath Interconnect (QPI) interconnect.

A traditional storage system that uses traditional flash drives may implement a process across the flash drives that are part of the traditional storage system. For example, a higher level process of the storage system may initiate and control a process across the flash drives. However, a flash drive of the traditional storage system may include its own storage controller that also performs the process. Thus, for the traditional storage system, a higher level process (e.g., initiated by the storage system) and a lower level process (e.g., initiated by a storage controller of the storage system) may both be performed.

To resolve various deficiencies of a traditional storage system, operations may be performed by higher level processes and not by the lower level processes. For example, the flash storage system may include flash drives that do not include storage controllers that provide the process. Thus, the operating system of the flash storage system itself may initiate and control the process. This may be accomplished by a direct-mapped flash storage system that addresses data blocks within the flash drives directly and without an address translation performed by the storage controllers of the flash drives.

In some implementations, storage drive **171A-F** may be one or more zoned storage devices. In some implementations, the one or more zoned storage devices may be a shingled HDD. In some implementations, the one or more storage devices may be a flash-based SSD. In a zoned storage device, a zoned namespace on the zoned storage

device can be addressed by groups of blocks that are grouped and aligned by a natural size, forming a number of addressable zones. In some implementations utilizing an SSD, the natural size may be based on the erase block size of the SSD. In some implementations, the zones of the zoned storage device may be defined during initialization of the zoned storage device. In some implementations, the zones may be defined dynamically as data is written to the zoned storage device.

In some implementations, zones may be heterogeneous, with some zones each being a page group and other zones being multiple page groups. In some implementations, some zones may correspond to an erase block and other zones may correspond to multiple erase blocks. In an implementation, zones may be any combination of differing numbers of pages in page groups and/or erase blocks, for heterogeneous mixes of programming modes, manufacturers, product types and/or product generations of storage devices, as applied to heterogeneous assemblies, upgrades, distributed storages, etc. In some implementations, zones may be defined as having usage characteristics, such as a property of supporting data with particular kinds of longevity (very short lived or very long lived, for example). These properties could be used by a zoned storage device to determine how the zone will be managed over the zone's expected lifetime.

It should be appreciated that a zone is a virtual construct. Any particular zone may not have a fixed location at a storage device. Until allocated, a zone may not have any location at a storage device. A zone may correspond to a number representing a chunk of virtually allocatable space that is the size of an erase block or other block size in various implementations. When the system allocates or opens a zone, zones get allocated to flash or other solid-state storage memory and, as the system writes to the zone, pages are written to that mapped flash or other solid-state storage memory of the zoned storage device. When the system closes the zone, the associated erase block(s) or other sized block(s) are completed. At some point in the future, the system may delete a zone which will free up the zone's allocated space. During its lifetime, a zone may be moved around to different locations of the zoned storage device, e.g., as the zoned storage device does internal maintenance.

In some implementations, the zones of the zoned storage device may be in different states. A zone may be in an empty state in which data has not been stored at the zone. An empty zone may be opened explicitly, or implicitly by writing data to the zone. This is the initial state for zones on a fresh zoned storage device, but may also be the result of a zone reset. In some implementations, an empty zone may have a designated location within the flash memory of the zoned storage device. In an implementation, the location of the empty zone may be chosen when the zone is first opened or first written to (or later if writes are buffered into memory). A zone may be in an open state either implicitly or explicitly, where a zone that is in an open state may be written to store data with write or append commands. In an implementation, a zone that is in an open state may also be written to using a copy command that copies data from a different zone. In some implementations, a zoned storage device may have a limit on the number of open zones at a particular time.

A zone in a closed state is a zone that has been partially written to, but has entered a closed state after issuing an explicit close operation. A zone in a closed state may be left available for future writes, but may reduce some of the run-time overhead consumed by keeping the zone in an open state. In some implementations, a zoned storage device may have a limit on the number of closed zones at a particular

time. A zone in a full state is a zone that is storing data and can no longer be written to. A zone may be in a full state either after writes have written data to the entirety of the zone or as a result of a zone finish operation. Prior to a finish operation, a zone may or may not have been completely written. After a finish operation, however, the zone may not be opened a written to further without first performing a zone reset operation.

The mapping from a zone to an erase block (or to a shingled track in an HDD) may be arbitrary, dynamic, and hidden from view. The process of opening a zone may be an operation that allows a new zone to be dynamically mapped to underlying storage of the zoned storage device, and then allows data to be written through appending writes into the zone until the zone reaches capacity. The zone can be finished at any point, after which further data may not be written into the zone. When the data stored at the zone is no longer needed, the zone can be reset which effectively deletes the zone's content from the zoned storage device, making the physical storage held by that zone available for the subsequent storage of data. Once a zone has been written and finished, the zoned storage device ensures that the data stored at the zone is not lost until the zone is reset. In the time between writing the data to the zone and the resetting of the zone, the zone may be moved around between shingle tracks or erase blocks as part of maintenance operations within the zoned storage device, such as by copying data to keep the data refreshed or to handle memory cell aging in an SSD.

In some implementations utilizing an HDD, the resetting of the zone may allow the shingle tracks to be allocated to a new, opened zone that may be opened at some point in the future. In some implementations utilizing an SSD, the resetting of the zone may cause the associated physical erase block(s) of the zone to be erased and subsequently reused for the storage of data. In some implementations, the zoned storage device may have a limit on the number of open zones at a point in time to reduce the amount of overhead dedicated to keeping zones open.

The operating system of the flash storage system may identify and maintain a list of allocation units across multiple flash drives of the flash storage system. The allocation units may be entire erase blocks or multiple erase blocks. The operating system may maintain a map or address range that directly maps addresses to erase blocks of the flash drives of the flash storage system.

Direct mapping to the erase blocks of the flash drives may be used to rewrite data and erase data. For example, the operations may be performed on one or more allocation units that include a first data and a second data where the first data is to be retained and the second data is no longer being used by the flash storage system. The operating system may initiate the process to write the first data to new locations within other allocation units and erasing the second data and marking the allocation units as being available for use for subsequent data. Thus, the process may only be performed by the higher level operating system of the flash storage system without an additional lower level process being performed by controllers of the flash drives.

Advantages of the process being performed only by the operating system of the flash storage system include increased reliability of the flash drives of the flash storage system as unnecessary or redundant write operations are not being performed during the process. One possible point of novelty here is the concept of initiating and controlling the process at the operating system of the flash storage system. In addition, the process can be controlled by the operating

11

system across multiple flash drives. This is in contrast to the process being performed by a storage controller of a flash drive.

A storage system can consist of two storage array controllers that share a set of drives for failover purposes, or it could consist of a single storage array controller that provides a storage service that utilizes multiple drives, or it could consist of a distributed network of storage array controllers each with some number of drives or some amount of Flash storage where the storage array controllers in the network collaborate to provide a complete storage service and collaborate on various aspects of a storage service including storage allocation and garbage collection.

FIG. 1C illustrates a third example system 117 for data storage in accordance with some implementations. System 117 (also referred to as “storage system” herein) includes numerous elements for purposes of illustration rather than limitation. It may be noted that system 117 may include the same, more, or fewer elements configured in the same or different manner in other implementations.

In one embodiment, system 117 includes a dual Peripheral Component Interconnect (‘PCI’) flash storage device 118 with separately addressable fast write storage. System 117 may include a storage device controller 119. In one embodiment, storage device controller 119A-D may be a CPU, ASIC, FPGA, or any other circuitry that may implement control structures necessary according to the present disclosure. In one embodiment, system 117 includes flash memory devices (e.g., including flash memory devices 120a-n), operatively coupled to various channels of the storage device controller 119. Flash memory devices 120a-n may be presented to the controller 119A-D as an addressable collection of Flash pages, erase blocks, and/or control elements sufficient to allow the storage device controller 119A-D to program and retrieve various aspects of the Flash. In one embodiment, storage device controller 119A-D may perform operations on flash memory devices 120a-n including storing and retrieving data content of pages, arranging and erasing any blocks, tracking statistics related to the use and reuse of Flash memory pages, erase blocks, and cells, tracking and predicting error codes and faults within the Flash memory, controlling voltage levels associated with programming and retrieving contents of Flash cells, etc.

In one embodiment, system 117 may include RAM 121 to store separately addressable fast-write data. In one embodiment, RAM 121 may be one or more separate discrete devices. In another embodiment, RAM 121 may be integrated into storage device controller 119A-D or multiple storage device controllers. The RAM 121 may be utilized for other purposes as well, such as temporary program memory for a processing device (e.g., a CPU) in the storage device controller 119.

In one embodiment, system 117 may include a stored energy device 122, such as a rechargeable battery or a capacitor. Stored energy device 122 may store energy sufficient to power the storage device controller 119, some amount of the RAM (e.g., RAM 121), and some amount of Flash memory (e.g., Flash memory 120a-120n) for sufficient time to write the contents of RAM to Flash memory. In one embodiment, storage device controller 119A-D may write the contents of RAM to Flash Memory if the storage device controller detects loss of external power.

In one embodiment, system 117 includes two data communications links 123a, 123b. In one embodiment, data communications links 123a, 123b may be PCI interfaces. In another embodiment, data communications links 123a, 123b may be based on other communications standards (e.g.,

12

HyperTransport, InfiniB and, etc.). Data communications links 123a, 123b may be based on non-volatile memory express (‘NVMe’) or NVMe over fabrics (‘NVMeF’) specifications that allow external connection to the storage device controller 119A-D from other components in the storage system 117. It should be noted that data communications links may be interchangeably referred to herein as PCI buses for convenience.

System 117 may also include an external power source (not shown), which may be provided over one or both data communications links 123a, 123b, or which may be provided separately. An alternative embodiment includes a separate Flash memory (not shown) dedicated for use in storing the content of RAM 121. The storage device controller 119A-D may present a logical device over a PCI bus which may include an addressable fast-write logical device, or a distinct part of the logical address space of the storage device 118, which may be presented as PCI memory or as persistent storage. In one embodiment, operations to store into the device are directed into the RAM 121. On power failure, the storage device controller 119A-D may write stored content associated with the addressable fast-write logical storage to Flash memory (e.g., Flash memory 120a-n) for long-term persistent storage.

In one embodiment, the logical device may include some presentation of some or all of the content of the Flash memory devices 120a-n, where that presentation allows a storage system including a storage device 118 (e.g., storage system 117) to directly address Flash memory pages and directly reprogram erase blocks from storage system components that are external to the storage device through the PCI bus. The presentation may also allow one or more of the external components to control and retrieve other aspects of the Flash memory including some or all of: tracking statistics related to use and reuse of Flash memory pages, erase blocks, and cells across all the Flash memory devices; tracking and predicting error codes and faults within and across the Flash memory devices; controlling voltage levels associated with programming and retrieving contents of Flash cells; etc.

In one embodiment, the stored energy device 122 may be sufficient to ensure completion of in-progress operations to the Flash memory devices 120a-120n. The stored energy device 122 may power storage device controller 119A-D and associated Flash memory devices (e.g., 120a-n) for those operations, as well as for the storing of fast-write RAM to Flash memory. Stored energy device 122 may be used to store accumulated statistics and other parameters kept and tracked by the Flash memory devices 120a-n and/or the storage device controller 119. Separate capacitors or stored energy devices (such as smaller capacitors near or embedded within the Flash memory devices themselves) may be used for some or all of the operations described herein.

Various schemes may be used to track and optimize the life span of the stored energy component, such as adjusting voltage levels over time, partially discharging the stored energy device 122 to measure corresponding discharge characteristics, etc. If the available energy decreases over time, the effective available capacity of the addressable fast-write storage may be decreased to ensure that it can be written safely based on the currently available stored energy.

FIG. 1D illustrates a third example storage system 124 for data storage in accordance with some implementations. In one embodiment, storage system 124 includes storage controllers 125a, 125b. In one embodiment, storage controllers 125a, 125b are operatively coupled to Dual PCI storage

devices. Storage controllers **125a**, **125b** may be operatively coupled (e.g., via a storage network **130**) to some number of host computers **127a-n**.

In one embodiment, two storage controllers (e.g., **125a** and **125b**) provide storage services, such as a SCSI block storage array, a file server, an object server, a database or data analytics service, etc. The storage controllers **125a**, **125b** may provide services through some number of network interfaces (e.g., **126a-d**) to host computers **127a-n** outside of the storage system **124**. Storage controllers **125a**, **125b** may provide integrated services or an application entirely within the storage system **124**, forming a converged storage and compute system. The storage controllers **125a**, **125b** may utilize the fast write memory within or across storage devices **119a-d** to journal in progress operations to ensure the operations are not lost on a power failure, storage controller removal, storage controller or storage system shutdown, or some fault of one or more software or hardware components within the storage system **124**.

In one embodiment, storage controllers **125a**, **125b** operate as PCI masters to one or the other PCI buses **128a**, **128b**. In another embodiment, **128a** and **128b** may be based on other communications standards (e.g., HyperTransport, InfiniBand, etc.). Other storage system embodiments may operate storage controllers **125a**, **125b** as multi-masters for both PCI buses **128a**, **128b**. Alternately, a PCI/NVMe/NVMf switching infrastructure or fabric may connect multiple storage controllers. Some storage system embodiments may allow storage devices to communicate with each other directly rather than communicating only with storage controllers. In one embodiment, a storage device controller **119a** may be operable under direction from a storage controller **125a** to synthesize and transfer data to be stored into Flash memory devices from data that has been stored in RAM (e.g., RAM **121** of FIG. 1C). For example, a recalculated version of RAM content may be transferred after a storage controller has determined that an operation has fully committed across the storage system, or when fast-write memory on the device has reached a certain used capacity, or after a certain amount of time, to ensure improve safety of the data or to release addressable fast-write capacity for reuse. This mechanism may be used, for example, to avoid a second transfer over a bus (e.g., **128a**, **128b**) from the storage controllers **125a**, **125b**. In one embodiment, a recalculation may include compressing data, attaching indexing or other metadata, combining multiple data segments together, performing erasure code calculations, etc.

In one embodiment, under direction from a storage controller **125a**, **125b**, a storage device controller **119a**, **119b** may be operable to calculate and transfer data to other storage devices from data stored in RAM (e.g., RAM **121** of FIG. 1C) without involvement of the storage controllers **125a**, **125b**. This operation may be used to mirror data stored in one storage controller **125a** to another storage controller **125b**, or it could be used to offload compression, data aggregation, and/or erasure coding calculations and transfers to storage devices to reduce load on storage controllers or the storage controller interface **129a**, **129b** to the PCI bus **128a**, **128b**.

A storage device controller **119A-D** may include mechanisms for implementing high availability primitives for use by other parts of a storage system external to the Dual PCI storage device **118**. For example, reservation or exclusion primitives may be provided so that, in a storage system with two storage controllers providing a highly available storage service, one storage controller may prevent the other storage controller from accessing or continuing to access the storage

device. This could be used, for example, in cases where one controller detects that the other controller is not functioning properly or where the interconnect between the two storage controllers may itself not be functioning properly.

In one embodiment, a storage system for use with Dual PCI direct mapped storage devices with separately addressable fast write storage includes systems that manage erase blocks or groups of erase blocks as allocation units for storing data on behalf of the storage service, or for storing metadata (e.g., indexes, logs, etc.) associated with the storage service, or for proper management of the storage system itself. Flash pages, which may be a few kilobytes in size, may be written as data arrives or as the storage system is to persist data for long intervals of time (e.g., above a defined threshold of time). To commit data more quickly, or to reduce the number of writes to the Flash memory devices, the storage controllers may first write data into the separately addressable fast write storage on one or more storage devices.

In one embodiment, the storage controllers **125a**, **125b** may initiate the use of erase blocks within and across storage devices (e.g., **118**) in accordance with an age and expected remaining lifespan of the storage devices, or based on other statistics. The storage controllers **125a**, **125b** may initiate garbage collection and data migration between storage devices in accordance with pages that are no longer needed as well as to manage Flash page and erase block lifespans and to manage overall system performance.

In one embodiment, the storage system **124** may utilize mirroring and/or erasure coding schemes as part of storing data into addressable fast write storage and/or as part of writing data into allocation units associated with erase blocks. Erasure codes may be used across storage devices, as well as within erase blocks or allocation units, or within and across Flash memory devices on a single storage device, to provide redundancy against single or multiple storage device failures or to protect against internal corruptions of Flash memory pages resulting from Flash memory operations or from degradation of Flash memory cells. Mirroring and erasure coding at various levels may be used to recover from multiple types of failures that occur separately or in combination.

The embodiments depicted with reference to FIGS. 2A-G illustrate a storage cluster that stores user data, such as user data originating from one or more user or client systems or other sources external to the storage cluster. The storage cluster distributes user data across storage nodes housed within a chassis, or across multiple chassis, using erasure coding and redundant copies of metadata. Erasure coding refers to a method of data protection or reconstruction in which data is stored across a set of different locations, such as disks, storage nodes or geographic locations. Flash memory is one type of solid-state memory that may be integrated with the embodiments, although the embodiments may be extended to other types of solid-state memory or other storage medium, including non-solid state memory. Control of storage locations and workloads are distributed across the storage locations in a clustered peer-to-peer system. Tasks such as mediating communications between the various storage nodes, detecting when a storage node has become unavailable, and balancing I/Os (inputs and outputs) across the various storage nodes, are all handled on a distributed basis. Data is laid out or distributed across multiple storage nodes in data fragments or stripes that support data recovery in some embodiments. Ownership of data can be reassigned within a cluster, independent of input and output patterns. This architecture described in more

15

detail below allows a storage node in the cluster to fail, with the system remaining operational, since the data can be reconstructed from other storage nodes and thus remain available for input and output operations. In various embodiments, a storage node may be referred to as a cluster node, a blade, or a server.

The storage cluster may be contained within a chassis, i.e., an enclosure housing one or more storage nodes. A mechanism to provide power to each storage node, such as a power distribution bus, and a communication mechanism, such as a communication bus that enables communication between the storage nodes are included within the chassis. The storage cluster can run as an independent system in one location according to some embodiments. In one embodiment, a chassis contains at least two instances of both the power distribution and the communication bus which may be enabled or disabled independently. The internal communication bus may be an Ethernet bus, however, other technologies such as PCIe, InfiniBand, and others, are equally suitable. The chassis provides a port for an external communication bus for enabling communication between multiple chassis, directly or through a switch, and with client systems. The external communication may use a technology such as Ethernet, InfiniBand, Fibre Channel, etc. In some embodiments, the external communication bus uses different communication bus technologies for inter-chassis and client communication. If a switch is deployed within or between chassis, the switch may act as a translation between multiple protocols or technologies. When multiple chassis are connected to define a storage cluster, the storage cluster may be accessed by a client using either proprietary interfaces or standard interfaces such as network file system ('NFS'), common internet file system ('CIFS'), small computer system interface ('SCSI') or hypertext transfer protocol ('HTTP'). Translation from the client protocol may occur at the switch, chassis external communication bus or within each storage node. In some embodiments, multiple chassis may be coupled or connected to each other through an aggregator switch. A portion and/or all of the coupled or connected chassis may be designated as a storage cluster. As discussed above, each chassis can have multiple blades, each blade has a media access control ('MAC') address, but the storage cluster is presented to an external network as having a single cluster IP address and a single MAC address in some embodiments.

Each storage node may be one or more storage servers and each storage server is connected to one or more non-volatile solid state memory units, which may be referred to as storage units or storage devices. One embodiment includes a single storage server in each storage node and between one to eight non-volatile solid state memory units, however this one example is not meant to be limiting. The storage server may include a processor, DRAM and interfaces for the internal communication bus and power distribution for each of the power buses. Inside the storage node, the interfaces and storage unit share a communication bus, e.g., PCI Express, in some embodiments. The non-volatile solid state memory units may directly access the internal communication bus interface through a storage node communication bus, or request the storage node to access the bus interface. The non-volatile solid state memory unit contains an embedded CPU, solid state storage controller, and a quantity of solid state mass storage, e.g., between 2-32 terabytes ('TB') in some embodiments. An embedded volatile storage medium, such as DRAM, and an energy reserve apparatus are included in the non-volatile solid state memory unit. In some embodiments, the energy reserve apparatus is a capaci-

16

tor, super-capacitor, or battery that enables transferring a subset of DRAM contents to a stable storage medium in the case of power loss. In some embodiments, the non-volatile solid state memory unit is constructed with a storage class memory, such as phase change or magnetoresistive random access memory ('MRAM') that substitutes for DRAM and enables a reduced power hold-up apparatus.

One of many features of the storage nodes and non-volatile solid state storage is the ability to proactively rebuild data in a storage cluster. The storage nodes and non-volatile solid state storage can determine when a storage node or non-volatile solid state storage in the storage cluster is unreachable, independent of whether there is an attempt to read data involving that storage node or non-volatile solid state storage. The storage nodes and non-volatile solid state storage then cooperate to recover and rebuild the data in at least partially new locations. This constitutes a proactive rebuild, in that the system rebuilds data without waiting until the data is needed for a read access initiated from a client system employing the storage cluster. These and further details of the storage memory and operation thereof are discussed below.

FIG. 2A is a perspective view of a storage cluster 161, with multiple storage nodes 150 and internal solid-state memory coupled to each storage node to provide network attached storage or storage area network, in accordance with some embodiments. A network attached storage, storage area network, or a storage cluster, or other storage memory, could include one or more storage clusters 161, each having one or more storage nodes 150, in a flexible and reconfigurable arrangement of both the physical components and the amount of storage memory provided thereby. The storage cluster 161 is designed to fit in a rack, and one or more racks can be set up and populated as desired for the storage memory. The storage cluster 161 has a chassis 138 having multiple slots 142. It should be appreciated that chassis 138 may be referred to as a housing, enclosure, or rack unit. In one embodiment, the chassis 138 has fourteen slots 142, although other numbers of slots are readily devised. For example, some embodiments have four slots, eight slots, sixteen slots, thirty-two slots, or other suitable number of slots. Each slot 142 can accommodate one storage node 150 in some embodiments. Chassis 138 includes flaps 148 that can be utilized to mount the chassis 138 on a rack. Fans 144 provide air circulation for cooling of the storage nodes 150 and components thereof, although other cooling components could be used, or an embodiment could be devised without cooling components. A switch fabric 146 couples storage nodes 150 within chassis 138 together and to a network for communication to the memory. In an embodiment depicted in herein, the slots 142 to the left of the switch fabric 146 and fans 144 are shown occupied by storage nodes 150, while the slots 142 to the right of the switch fabric 146 and fans 144 are empty and available for insertion of storage node 150 for illustrative purposes. This configuration is one example, and one or more storage nodes 150 could occupy the slots 142 in various further arrangements. The storage node arrangements need not be sequential or adjacent in some embodiments. Storage nodes 150 are hot pluggable, meaning that a storage node 150 can be inserted into a slot 142 in the chassis 138, or removed from a slot 142, without stopping or powering down the system. Upon insertion or removal of storage node 150 from slot 142, the system automatically reconfigures in order to recognize and adapt to the change. Reconfiguration, in some embodiments, includes restoring redundancy and/or rebalancing data or load.

17

Each storage node **150** can have multiple components. In the embodiment shown here, the storage node **150** includes a printed circuit board **159** populated by a CPU **156**, i.e., processor, a memory **154** coupled to the CPU **156**, and a non-volatile solid state storage **152** coupled to the CPU **156**, although other mountings and/or components could be used in further embodiments. The memory **154** has instructions which are executed by the CPU **156** and/or data operated on by the CPU **156**. As further explained below, the non-volatile solid state storage **152** includes flash or, in further embodiments, other types of solid-state memory.

Referring to FIG. 2A, storage cluster **161** is scalable, meaning that storage capacity with non-uniform storage sizes is readily added, as described above. One or more storage nodes **150** can be plugged into or removed from each chassis and the storage cluster self-configures in some embodiments. Plug-in storage nodes **150**, whether installed in a chassis as delivered or later added, can have different sizes. For example, in one embodiment a storage node **150** can have any multiple of 4 TB, e.g., 8 TB, 12 TB, 16 TB, 32 TB, etc. In further embodiments, a storage node **150** could have any multiple of other storage amounts or capacities. Storage capacity of each storage node **150** is broadcast, and influences decisions of how to stripe the data. For maximum storage efficiency, an embodiment can self-configure as wide as possible in the stripe, subject to a predetermined requirement of continued operation with loss of up to one, or up to two, non-volatile solid state storage **152** units or storage nodes **150** within the chassis.

FIG. 2B is a block diagram showing a communications interconnect **173** and power distribution bus **172** coupling multiple storage nodes **150**. Referring back to FIG. 2A, the communications interconnect **173** can be included in or implemented with the switch fabric **146** in some embodiments. Where multiple storage clusters **161** occupy a rack, the communications interconnect **173** can be included in or implemented with a top of rack switch, in some embodiments. As illustrated in FIG. 2B, storage cluster **161** is enclosed within a single chassis **138**. External port **176** is coupled to storage nodes **150** through communications interconnect **173**, while external port **174** is coupled directly to a storage node. External power port **178** is coupled to power distribution bus **172**. Storage nodes **150** may include varying amounts and differing capacities of non-volatile solid state storage **152** as described with reference to FIG. 2A. In addition, one or more storage nodes **150** may be a compute only storage node as illustrated in FIG. 2B. Authorities **168** are implemented on the non-volatile solid state storage **152**, for example as lists or other data structures stored in memory. In some embodiments the authorities are stored within the non-volatile solid state storage **152** and supported by software executing on a controller or other processor of the non-volatile solid state storage **152**. In a further embodiment, authorities **168** are implemented on the storage nodes **150**, for example as lists or other data structures stored in the memory **154** and supported by software executing on the CPU **156** of the storage node **150**. Authorities **168** control how and where data is stored in the non-volatile solid state storage **152** in some embodiments. This control assists in determining which type of erasure coding scheme is applied to the data, and which storage nodes **150** have which portions of the data. Each authority **168** may be assigned to a non-volatile solid state storage **152**. Each authority may control a range of inode numbers, segment numbers, or other data identifiers which are assigned to data by a file system, by the storage nodes **150**, or by the non-volatile solid state storage **152**, in various embodiments.

18

Every piece of data, and every piece of metadata, has redundancy in the system in some embodiments. In addition, every piece of data and every piece of metadata has an owner, which may be referred to as an authority. If that authority is unreachable, for example through failure of a storage node, there is a plan of succession for how to find that data or that metadata. In various embodiments, there are redundant copies of authorities **168**. Authorities **168** have a relationship to storage nodes **150** and non-volatile solid state storage **152** in some embodiments. Each authority **168**, covering a range of data segment numbers or other identifiers of the data, may be assigned to a specific non-volatile solid state storage **152**. In some embodiments the authorities **168** for all of such ranges are distributed over the non-volatile solid state storage **152** of a storage cluster. Each storage node **150** has a network port that provides access to the non-volatile solid state storage(s) **152** of that storage node **150**. Data can be stored in a segment, which is associated with a segment number and that segment number is an indirection for a configuration of a RAID (redundant array of independent disks) stripe in some embodiments. The assignment and use of the authorities **168** thus establishes an indirection to data. Indirection may be referred to as the ability to reference data indirectly, in this case via an authority **168**, in accordance with some embodiments. A segment identifies a set of non-volatile solid state storage **152** and a local identifier into the set of non-volatile solid state storage **152** that may contain data. In some embodiments, the local identifier is an offset into the device and may be reused sequentially by multiple segments. In other embodiments the local identifier is unique for a specific segment and never reused. The offsets in the non-volatile solid state storage **152** are applied to locating data for writing to or reading from the non-volatile solid state storage **152** (in the form of a RAID stripe). Data is striped across multiple units of non-volatile solid state storage **152**, which may include or be different from the non-volatile solid state storage **152** having the authority **168** for a particular data segment.

If there is a change in where a particular segment of data is located, e.g., during a data move or a data reconstruction, the authority **168** for that data segment should be consulted, at that non-volatile solid state storage **152** or storage node **150** having that authority **168**. In order to locate a particular piece of data, embodiments calculate a hash value for a data segment or apply an inode number or a data segment number. The output of this operation points to a non-volatile solid state storage **152** having the authority **168** for that particular piece of data. In some embodiments there are two stages to this operation. The first stage maps an entity identifier (ID), e.g., a segment number, inode number, or directory number to an authority identifier. This mapping may include a calculation such as a hash or a bit mask. The second stage is mapping the authority identifier to a particular non-volatile solid state storage **152**, which may be done through an explicit mapping. The operation is repeatable, so that when the calculation is performed, the result of the calculation repeatably and reliably points to a particular non-volatile solid state storage **152** having that authority **168**. The operation may include the set of reachable storage nodes as input. If the set of reachable non-volatile solid state storage units changes the optimal set changes. In some embodiments, the persisted value is the current assignment (which is always true) and the calculated value is the target assignment the cluster will attempt to reconfigure towards. This calculation may be used to determine the optimal non-volatile solid state storage **152** for an authority in the

presence of a set of non-volatile solid state storage **152** that are reachable and constitute the same cluster. The calculation also determines an ordered set of peer non-volatile solid state storage **152** that will also record the authority to non-volatile solid state storage mapping so that the authority may be determined even if the assigned non-volatile solid state storage is unreachable. A duplicate or substitute authority **168** may be consulted if a specific authority **168** is unavailable in some embodiments.

With reference to FIGS. 2A and 2B, two of the many tasks of the CPU **156** on a storage node **150** are to break up write data, and reassemble read data. When the system has determined that data is to be written, the authority **168** for that data is located as above. When the segment ID for data is already determined the request to write is forwarded to the non-volatile solid state storage **152** currently determined to be the host of the authority **168** determined from the segment. The host CPU **156** of the storage node **150**, on which the non-volatile solid state storage **152** and corresponding authority **168** reside, then breaks up or shards the data and transmits the data out to various non-volatile solid state storage **152**. The transmitted data is written as a data stripe in accordance with an erasure coding scheme. In some embodiments, data is requested to be pulled, and in other embodiments, data is pushed. In reverse, when data is read, the authority **168** for the segment ID containing the data is located as described above. The host CPU **156** of the storage node **150** on which the non-volatile solid state storage **152** and corresponding authority **168** reside requests the data from the non-volatile solid state storage and corresponding storage nodes pointed to by the authority. In some embodiments the data is read from flash storage as a data stripe. The host CPU **156** of storage node **150** then reassembles the read data, correcting any errors (if present) according to the appropriate erasure coding scheme, and forwards the reassembled data to the network. In further embodiments, some or all of these tasks can be handled in the non-volatile solid state storage **152**. In some embodiments, the segment host requests the data be sent to storage node **150** by requesting pages from storage and then sending the data to the storage node making the original request.

In embodiments, authorities **168** operate to determine how operations will proceed against particular logical elements. Each of the logical elements may be operated on through a particular authority across a plurality of storage controllers of a storage system. The authorities **168** may communicate with the plurality of storage controllers so that the plurality of storage controllers collectively perform operations against those particular logical elements.

In embodiments, logical elements could be, for example, files, directories, object buckets, individual objects, delineated parts of files or objects, other forms of key-value pair databases, or tables. In embodiments, performing an operation can involve, for example, ensuring consistency, structural integrity, and/or recoverability with other operations against the same logical element, reading metadata and data associated with that logical element, determining what data should be written durably into the storage system to persist any changes for the operation, or where metadata and data can be determined to be stored across modular storage devices attached to a plurality of the storage controllers in the storage system.

In some embodiments the operations are token based transactions to efficiently communicate within a distributed system. Each transaction may be accompanied by or associated with a token, which gives permission to execute the transaction. The authorities **168** are able to maintain a

pre-transaction state of the system until completion of the operation in some embodiments. The token based communication may be accomplished without a global lock across the system, and also enables restart of an operation in case of a disruption or other failure.

In some systems, for example in UNIX-style file systems, data is handled with an index node or inode, which specifies a data structure that represents an object in a file system. The object could be a file or a directory, for example. Metadata may accompany the object, as attributes such as permission data and a creation timestamp, among other attributes. A segment number could be assigned to all or a portion of such an object in a file system. In other systems, data segments are handled with a segment number assigned elsewhere. For purposes of discussion, the unit of distribution is an entity, and an entity can be a file, a directory or a segment. That is, entities are units of data or metadata stored by a storage system. Entities are grouped into sets called authorities. Each authority has an authority owner, which is a storage node that has the exclusive right to update the entities in the authority. In other words, a storage node contains the authority, and that the authority, in turn, contains entities.

A segment is a logical container of data in accordance with some embodiments. A segment is an address space between medium address space and physical flash locations, i.e., the data segment number, are in this address space. Segments may also contain meta-data, which enable data redundancy to be restored (rewritten to different flash locations or devices) without the involvement of higher level software. In one embodiment, an internal format of a segment contains client data and medium mappings to determine the position of that data. Each data segment is protected, e.g., from memory and other failures, by breaking the segment into a number of data and parity shards, where applicable. The data and parity shards are distributed, i.e., striped, across non-volatile solid state storage **152** coupled to the host CPUs **156** (See FIGS. 2E and 2G) in accordance with an erasure coding scheme. Usage of the term segments refers to the container and its place in the address space of segments in some embodiments. Usage of the term stripe refers to the same set of shards as a segment and includes how the shards are distributed along with redundancy or parity information in accordance with some embodiments.

A series of address-space transformations takes place across an entire storage system. At the top are the directory entries (file names) which link to an inode. Modes point into medium address space, where data is logically stored. Medium addresses may be mapped through a series of indirect mediums to spread the load of large files, or implement data services like deduplication or snapshots. Segment addresses are then translated into physical flash locations. Physical flash locations have an address range bounded by the amount of flash in the system in accordance with some embodiments. Medium addresses and segment addresses are logical containers, and in some embodiments use a 128 bit or larger identifier so as to be practically infinite, with a likelihood of reuse calculated as longer than the expected life of the system. Addresses from logical containers are allocated in a hierarchical fashion in some embodiments. Initially, each non-volatile solid state storage **152** unit may be assigned a range of address space. Within this assigned range, the non-volatile solid state storage **152** is able to allocate addresses without synchronization with other non-volatile solid state storage **152**.

Data and metadata is stored by a set of underlying storage layouts that are optimized for varying workload patterns and storage devices. These layouts incorporate multiple redun-

dancy schemes, compression formats and index algorithms. Some of these layouts store information about authorities and authority masters, while others store file metadata and file data. The redundancy schemes include error correction codes that tolerate corrupted bits within a single storage device (such as a NAND flash chip), erasure codes that tolerate the failure of multiple storage nodes, and replication schemes that tolerate data center or regional failures. In some embodiments, low density parity check ('LDPC') code is used within a single storage unit. Reed-Solomon encoding is used within a storage cluster, and mirroring is used within a storage grid in some embodiments. Metadata may be stored using an ordered log structured index (such as a Log Structured Merge Tree), and large data may not be stored in a log structured layout.

In order to maintain consistency across multiple copies of an entity, the storage nodes agree implicitly on two things through calculations: (1) the authority that contains the entity, and (2) the storage node that contains the authority. The assignment of entities to authorities can be done by pseudo randomly assigning entities to authorities, by splitting entities into ranges based upon an externally produced key, or by placing a single entity into each authority. Examples of pseudorandom schemes are linear hashing and the Replication Under Scalable Hashing ('RUSH') family of hashes, including Controlled Replication Under Scalable Hashing ('CRUSH'). In some embodiments, pseudo-random assignment is utilized only for assigning authorities to nodes because the set of nodes can change. The set of authorities cannot change so any subjective function may be applied in these embodiments. Some placement schemes automatically place authorities on storage nodes, while other placement schemes rely on an explicit mapping of authorities to storage nodes. In some embodiments, a pseudorandom scheme is utilized to map from each authority to a set of candidate authority owners. A pseudorandom data distribution function related to CRUSH may assign authorities to storage nodes and create a list of where the authorities are assigned. Each storage node has a copy of the pseudorandom data distribution function, and can arrive at the same calculation for distributing, and later finding or locating an authority. Each of the pseudorandom schemes requires the reachable set of storage nodes as input in some embodiments in order to conclude the same target nodes. Once an entity has been placed in an authority, the entity may be stored on physical devices so that no expected failure will lead to unexpected data loss. In some embodiments, rebalancing algorithms attempt to store the copies of all entities within an authority in the same layout and on the same set of machines.

Examples of expected failures include device failures, stolen machines, datacenter fires, and regional disasters, such as nuclear or geological events. Different failures lead to different levels of acceptable data loss. In some embodiments, a stolen storage node impacts neither the security nor the reliability of the system, while depending on system configuration, a regional event could lead to no loss of data, a few seconds or minutes of lost updates, or even complete data loss.

In the embodiments, the placement of data for storage redundancy is independent of the placement of authorities for data consistency. In some embodiments, storage nodes that contain authorities do not contain any persistent storage. Instead, the storage nodes are connected to non-volatile solid state storage units that do not contain authorities. The communications interconnect between storage nodes and non-volatile solid state storage units consists of multiple

communication technologies and has non-uniform performance and fault tolerance characteristics. In some embodiments, as mentioned above, non-volatile solid state storage units are connected to storage nodes via PCI express, storage nodes are connected together within a single chassis using Ethernet backplane, and chassis are connected together to form a storage cluster. Storage clusters are connected to clients using Ethernet or fiber channel in some embodiments. If multiple storage clusters are configured into a storage grid, the multiple storage clusters are connected using the Internet or other long-distance networking links, such as a "metro scale" link or private link that does not traverse the internet.

Authority owners have the exclusive right to modify entities, to migrate entities from one non-volatile solid state storage unit to another non-volatile solid state storage unit, and to add and remove copies of entities. This allows for maintaining the redundancy of the underlying data. When an authority owner fails, is going to be decommissioned, or is overloaded, the authority is transferred to a new storage node. Transient failures make it non-trivial to ensure that all non-faulty machines agree upon the new authority location. The ambiguity that arises due to transient failures can be achieved automatically by a consensus protocol such as Paxos, hot-warm failover schemes, via manual intervention by a remote system administrator, or by a local hardware administrator (such as by physically removing the failed machine from the cluster, or pressing a button on the failed machine). In some embodiments, a consensus protocol is used, and failover is automatic. If too many failures or replication events occur in too short a time period, the system goes into a self-preservation mode and halts replication and data movement activities until an administrator intervenes in accordance with some embodiments.

As authorities are transferred between storage nodes and authority owners update entities in their authorities, the system transfers messages between the storage nodes and non-volatile solid state storage units. With regard to persistent messages, messages that have different purposes are of different types. Depending on the type of the message, the system maintains different ordering and durability guarantees. As the persistent messages are being processed, the messages are temporarily stored in multiple durable and non-durable storage hardware technologies. In some embodiments, messages are stored in RAM, NVRAM and on NAND flash devices, and a variety of protocols are used in order to make efficient use of each storage medium. Latency-sensitive client requests may be persisted in replicated NVRAM, and then later NAND, while background rebalancing operations are persisted directly to NAND.

Persistent messages are persistently stored prior to being transmitted. This allows the system to continue to serve client requests despite failures and component replacement. Although many hardware components contain unique identifiers that are visible to system administrators, manufacturer, hardware supply chain and ongoing monitoring quality control infrastructure, applications running on top of the infrastructure address virtualized addresses. These virtualized addresses do not change over the lifetime of the storage system, regardless of component failures and replacements. This allows each component of the storage system to be replaced over time without reconfiguration or disruptions of client request processing, i.e., the system supports non-disruptive upgrades.

In some embodiments, the virtualized addresses are stored with sufficient redundancy. A continuous monitoring system correlates hardware and software status and the hardware

23

identifiers. This allows detection and prediction of failures due to faulty components and manufacturing details. The monitoring system also enables the proactive transfer of authorities and entities away from impacted devices before failure occurs by removing the component from the critical path in some embodiments.

FIG. 2C is a multiple level block diagram, showing contents of a storage node **150** and contents of a non-volatile solid state storage **152** of the storage node **150**. Data is communicated to and from the storage node **150** by a network interface controller ('NIC') **202** in some embodiments. Each storage node **150** has a CPU **156**, and one or more non-volatile solid state storage **152**, as discussed above. Moving down one level in FIG. 2C, each non-volatile solid state storage **152** has a relatively fast non-volatile solid state memory, such as nonvolatile random access memory ('NVRAM') **204**, and flash memory **206**. In some embodiments, NVRAM **204** may be a component that does not require program/erase cycles (DRAM, MRAM, PCM), and can be a memory that can support being written vastly more often than the memory is read from. Moving down another level in FIG. 2C, the NVRAM **204** is implemented in one embodiment as high speed volatile memory, such as dynamic random access memory (DRAM) **216**, backed up by energy reserve **218**. Energy reserve **218** provides sufficient electrical power to keep the DRAM **216** powered long enough for contents to be transferred to the flash memory **206** in the event of power failure. In some embodiments, energy reserve **218** is a capacitor, super-capacitor, battery, or other device, that supplies a suitable supply of energy sufficient to enable the transfer of the contents of DRAM **216** to a stable storage medium in the case of power loss. The flash memory **206** is implemented as multiple flash dies **222**, which may be referred to as packages of flash dies **222** or an array of flash dies **222**. It should be appreciated that the flash dies **222** could be packaged in any number of ways, with a single die per package, multiple dies per package (i.e., multichip packages), in hybrid packages, as bare dies on a printed circuit board or other substrate, as encapsulated dies, etc. In the embodiment shown, the non-volatile solid state storage **152** has a controller **212** or other processor, and an input output (I/O) port **210** coupled to the controller **212**. I/O port **210** is coupled to the CPU **156** and/or the network interface controller **202** of the flash storage node **150**. Flash input output (I/O) port **220** is coupled to the flash dies **222**, and a direct memory access unit (DMA) **214** is coupled to the controller **212**, the DRAM **216** and the flash dies **222**. In the embodiment shown, the I/O port **210**, controller **212**, DMA unit **214** and flash I/O port **220** are implemented on a programmable logic device ('PLD') **208**, e.g., an FPGA. In this embodiment, each flash die **222** has pages, organized as sixteen kB (kilobyte) pages **224**, and a register **226** through which data can be written to or read from the flash die **222**. In further embodiments, other types of solid-state memory are used in place of, or in addition to flash memory illustrated within flash die **222**.

Storage clusters **161**, in various embodiments as disclosed herein, can be contrasted with storage arrays in general. The storage nodes **150** are part of a collection that creates the storage cluster **161**. Each storage node **150** owns a slice of data and computing required to provide the data. Multiple storage nodes **150** cooperate to store and retrieve the data. Storage memory or storage devices, as used in storage arrays in general, are less involved with processing and manipulating the data. Storage memory or storage devices in a storage array receive commands to read, write, or erase data. The storage memory or storage devices in a storage array are

24

not aware of a larger system in which they are embedded, or what the data means. Storage memory or storage devices in storage arrays can include various types of storage memory, such as RAM, solid state drives, hard disk drives, etc. The non-volatile solid state storage **152** units described herein have multiple interfaces active simultaneously and serving multiple purposes. In some embodiments, some of the functionality of a storage node **150** is shifted into a storage unit **152**, transforming the storage unit **152** into a combination of storage unit **152** and storage node **150**. Placing computing (relative to storage data) into the storage unit **152** places this computing closer to the data itself. The various system embodiments have a hierarchy of storage node layers with different capabilities. By contrast, in a storage array, a controller owns and knows everything about all of the data that the controller manages in a shelf or storage devices. In a storage cluster **161**, as described herein, multiple controllers in multiple non-volatile solid state storage **152** units and/or storage nodes **150** cooperate in various ways (e.g., for erasure coding, data sharding, metadata communication and redundancy, storage capacity expansion or contraction, data recovery, and so on).

FIG. 2D shows a storage server environment, which uses embodiments of the storage nodes **150** and storage **152** units of FIGS. 2A-C. In this version, each non-volatile solid state storage **152** unit has a processor such as controller **212** (see FIG. 2C), an FPGA, flash memory **206**, and NVRAM **204** (which is super-capacitor backed DRAM **216**, see FIGS. 2B and 2C) on a PCIe (peripheral component interconnect express) board in a chassis **138** (see FIG. 2A). The non-volatile solid state storage **152** unit may be implemented as a single board containing storage, and may be the largest tolerable failure domain inside the chassis. In some embodiments, up to two non-volatile solid state storage **152** units may fail and the device will continue with no data loss.

The physical storage is divided into named regions based on application usage in some embodiments. The NVRAM **204** is a contiguous block of reserved memory in the non-volatile solid state storage **152** DRAM **216**, and is backed by NAND flash. NVRAM **204** is logically divided into multiple memory regions written for two as spool (e.g., spool_region). Space within the NVRAM **204** spools is managed by each authority **168** independently. Each device provides an amount of storage space to each authority **168**. That authority **168** further manages lifetimes and allocations within that space. Examples of a spool include distributed transactions or notions. When the primary power to a non-volatile solid state storage **152** unit fails, onboard super-capacitors provide a short duration of power hold up. During this holdup interval, the contents of the NVRAM **204** are flushed to flash memory **206**. On the next power-on, the contents of the NVRAM **204** are recovered from the flash memory **206**.

As for the storage unit controller, the responsibility of the logical "controller" is distributed across each of the blades containing authorities **168**. This distribution of logical control is shown in FIG. 2D as a host controller **242**, mid-tier controller **244** and storage unit controller(s) **246**. Management of the control plane and the storage plane are treated independently, although parts may be physically co-located on the same blade. Each authority **168** effectively serves as an independent controller. Each authority **168** provides its own data and metadata structures, its own background workers, and maintains its own lifecycle.

FIG. 2E is a blade **252** hardware block diagram, showing a control plane **254**, compute and storage planes **256**, **258**, and authorities **168** interacting with underlying physical

25

resources, using embodiments of the storage nodes **150** and storage units **152** of FIGS. 2A-C in the storage server environment of FIG. 2D. The control plane **254** is partitioned into a number of authorities **168** which can use the compute resources in the compute plane **256** to run on any of the blades **252**. The storage plane **258** is partitioned into a set of devices, each of which provides access to flash **206** and NVRAM **204** resources. In one embodiment, the compute plane **256** may perform the operations of a storage array controller, as described herein, on one or more devices of the storage plane **258** (e.g., a storage array).

In the compute and storage planes **256**, **258** of FIG. 2E, the authorities **168** interact with the underlying physical resources (i.e., devices). From the point of view of an authority **168**, its resources are striped over all of the physical devices. From the point of view of a device, it provides resources to all authorities **168**, irrespective of where the authorities happen to run. Each authority **168** has allocated or has been allocated one or more partitions **260** of storage memory in the storage units **152**, e.g., partitions **260** in flash memory **206** and NVRAM **204**. Each authority **168** uses those allocated partitions **260** that belong to it, for writing or reading user data. Authorities can be associated with differing amounts of physical storage of the system. For example, one authority **168** could have a larger number of partitions **260** or larger sized partitions **260** in one or more storage units **152** than one or more other authorities **168**.

FIG. 2F depicts elasticity software layers in blades **252** of a storage cluster, in accordance with some embodiments. In the elasticity structure, elasticity software is symmetric, i.e., each blade's compute module **270** runs the three identical layers of processes depicted in FIG. 2F. Storage managers **274** execute read and write requests from other blades **252** for data and metadata stored in local storage unit **152** NVRAM **204** and flash **206**. Authorities **168** fulfill client requests by issuing the necessary reads and writes to the blades **252** on whose storage units **152** the corresponding data or metadata resides. Endpoints **272** parse client connection requests received from switch fabric **146** supervisory software, relay the client connection requests to the authorities **168** responsible for fulfillment, and relay the authorities' **168** responses to clients. The symmetric three-layer structure enables the storage system's high degree of concurrency. Elasticity scales out efficiently and reliably in these embodiments. In addition, elasticity implements a unique scale-out technique that balances work evenly across all resources regardless of client access pattern, and maximizes concurrency by eliminating much of the need for inter-blade coordination that typically occurs with conventional distributed locking.

Still referring to FIG. 2F, authorities **168** running in the compute modules **270** of a blade **252** perform the internal operations required to fulfill client requests. One feature of elasticity is that authorities **168** are stateless, i.e., they cache active data and metadata in their own blades' **252** DRAMs for fast access, but the authorities store every update in their NVRAM **204** partitions on three separate blades **252** until the update has been written to flash **206**. All the storage system writes to NVRAM **204** are in triplicate to partitions on three separate blades **252** in some embodiments. With triple-mirrored NVRAM **204** and persistent storage protected by parity and Reed-Solomon RAID checksums, the storage system can survive concurrent failure of two blades **252** with no loss of data, metadata, or access to either.

Because authorities **168** are stateless, they can migrate between blades **252**. Each authority **168** has a unique identifier. NVRAM **204** and flash **206** partitions are associ-

26

ated with authorities' **168** identifiers, not with the blades **252** on which they are running in some embodiments. Thus, when an authority **168** migrates, the authority **168** continues to manage the same storage partitions from its new location.

When a new blade **252** is installed in an embodiment of the storage cluster, the system automatically rebalances load by: partitioning the new blade's **252** storage for use by the system's authorities **168**, migrating selected authorities **168** to the new blade **252**, starting endpoints **272** on the new blade **252** and including them in the switch fabric's **146** client connection distribution algorithm.

From their new locations, migrated authorities **168** persist the contents of their NVRAM **204** partitions on flash **206**, process read and write requests from other authorities **168**, and fulfill the client requests that endpoints **272** direct to them. Similarly, if a blade **252** fails or is removed, the system redistributes its authorities **168** among the system's remaining blades **252**. The redistributed authorities **168** continue to perform their original functions from their new locations.

FIG. 2G depicts authorities **168** and storage resources in blades **252** of a storage cluster, in accordance with some embodiments. Each authority **168** is exclusively responsible for a partition of the flash **206** and NVRAM **204** on each blade **252**. The authority **168** manages the content and integrity of its partitions independently of other authorities **168**. Authorities **168** compress incoming data and preserve it temporarily in their NVRAM **204** partitions, and then consolidate, RAID-protect, and persist the data in segments of the storage in their flash **206** partitions. As the authorities **168** write data to flash **206**, storage managers **274** perform the necessary flash translation to optimize write performance and maximize media longevity. In the background, authorities **168** "garbage collect," or reclaim space occupied by data that clients have made obsolete by overwriting the data. It should be appreciated that since authorities' **168** partitions are disjoint, there is no need for distributed locking to execute client and writes or to perform background functions.

The embodiments described herein may utilize various software, communication and/or networking protocols. In addition, the configuration of the hardware and/or software may be adjusted to accommodate various protocols. For example, the embodiments may utilize Active Directory, which is a database based system that provides authentication, directory, policy, and other services in a WINDOWS™ environment. In these embodiments, LDAP (Lightweight Directory Access Protocol) is one example application protocol for querying and modifying items in directory service providers such as Active Directory. In some embodiments, a network lock manager ('NLM') is utilized as a facility that works in cooperation with the Network File System ('NFS') to provide a System V style of advisory file and record locking over a network. The Server Message Block ('SMB') protocol, one version of which is also known as Common Internet File System ('CIFS'), may be integrated with the storage systems discussed herein. SMB operates as an application-layer network protocol typically used for providing shared access to files, printers, and serial ports and miscellaneous communications between nodes on a network. SMB also provides an authenticated inter-process communication mechanism. AMAZON™ S3 (Simple Storage Service) is a web service offered by Amazon Web Services, and the systems described herein may interface with Amazon S3 through web services interfaces (REST (representational state transfer), SOAP (simple object access protocol), and BitTorrent). A RESTful API (application

programming interface) breaks down a transaction to create a series of small modules. Each module addresses a particular underlying part of the transaction. The control or permissions provided with these embodiments, especially for object data, may include utilization of an access control list ('ACL'). The ACL is a list of permissions attached to an object and the ACL specifies which users or system processes are granted access to objects, as well as what operations are allowed on given objects. The systems may utilize Internet Protocol version 6 ('IPv6'), as well as IPv4, for the communications protocol that provides an identification and location system for computers on networks and routes traffic across the Internet. The routing of packets between networked systems may include Equal-cost multi-path routing ('ECMP'), which is a routing strategy where next-hop packet forwarding to a single destination can occur over multiple "best paths" which tie for top place in routing metric calculations. Multi-path routing can be used in conjunction with most routing protocols, because it is a per-hop decision limited to a single router. The software may support Multi-tenancy, which is an architecture in which a single instance of a software application serves multiple customers. Each customer may be referred to as a tenant. Tenants may be given the ability to customize some parts of the application, but may not customize the application's code, in some embodiments. The embodiments may maintain audit logs. An audit log is a document that records an event in a computing system. In addition to documenting what resources were accessed, audit log entries typically include destination and source addresses, a timestamp, and user login information for compliance with various regulations. The embodiments may support various key management policies, such as encryption key rotation. In addition, the system may support dynamic root passwords or some variation dynamically changing passwords.

FIG. 3A sets forth a diagram of a storage system 306 that is coupled for data communications with a cloud services provider 302 in accordance with some embodiments of the present disclosure. Although depicted in less detail, the storage system 306 depicted in FIG. 3A may be similar to the storage systems described above with reference to FIGS. 1A-1D and FIGS. 2A-2G. In some embodiments, the storage system 306 depicted in FIG. 3A may be embodied as a storage system that includes imbalanced active/active controllers, as a storage system that includes balanced active/active controllers, as a storage system that includes active/active controllers where less than all of each controller's resources are utilized such that each controller has reserve resources that may be used to support failover, as a storage system that includes fully active/active controllers, as a storage system that includes dataset-segregated controllers, as a storage system that includes dual-layer architectures with front-end controllers and back-end integrated storage controllers, as a storage system that includes scale-out clusters of dual-controller arrays, as well as combinations of such embodiments.

In the example depicted in FIG. 3A, the storage system 306 is coupled to the cloud services provider 302 via a data communications link 304. Such a data communications link 304 may be fully wired, fully wireless, or some aggregation of wired and wireless data communications pathways. In such an example, digital information may be exchanged between the storage system 306 and the cloud services provider 302 via the data communications link 304 using one or more data communications protocols. For example, digital information may be exchanged between the storage system 306 and the cloud services provider 302 via the data

communications link 304 using the handheld device transfer protocol ('HDTP'), hypertext transfer protocol ('HTTP'), internet protocol (IP), real-time transfer protocol ('RTP'), transmission control protocol ('TCP'), user datagram protocol ('UDP'), wireless application protocol ('WAP'), or other protocol.

The cloud services provider 302 depicted in FIG. 3A may be embodied, for example, as a system and computing environment that provides a vast array of services to users of the cloud services provider 302 through the sharing of computing resources via the data communications link 304. The cloud services provider 302 may provide on-demand access to a shared pool of configurable computing resources such as computer networks, servers, storage, applications and services, and so on.

In the example depicted in FIG. 3A, the cloud services provider 302 may be configured to provide a variety of services to the storage system 306 and users of the storage system 306 through the implementation of various service models. For example, the cloud services provider 302 may be configured to provide services through the implementation of an infrastructure as a service ('IaaS') service model, through the implementation of a platform as a service ('PaaS') service model, through the implementation of a software as a service ('SaaS') service model, through the implementation of an authentication as a service ('AaaS') service model, through the implementation of a storage as a service model where the cloud services provider 302 offers access to its storage infrastructure for use by the storage system 306 and users of the storage system 306, and so on.

In the example depicted in FIG. 3A, the cloud services provider 302 may be embodied, for example, as a private cloud, as a public cloud, or as a combination of a private cloud and public cloud. In an embodiment in which the cloud services provider 302 is embodied as a private cloud, the cloud services provider 302 may be dedicated to providing services to a single organization rather than providing services to multiple organizations. In an embodiment where the cloud services provider 302 is embodied as a public cloud, the cloud services provider 302 may provide services to multiple organizations. In still alternative embodiments, the cloud services provider 302 may be embodied as a mix of a private and public cloud services with a hybrid cloud deployment.

Although not explicitly depicted in FIG. 3A, readers will appreciate that a vast amount of additional hardware components and additional software components may be necessary to facilitate the delivery of cloud services to the storage system 306 and users of the storage system 306. For example, the storage system 306 may be coupled to (or even include) a cloud storage gateway. Such a cloud storage gateway may be embodied, for example, as hardware-based or software-based appliance that is located on-premises with the storage system 306. Such a cloud storage gateway may operate as a bridge between local applications that are executing on the storage system 306 and remote, cloud-based storage that is utilized by the storage system 306. Through the use of a cloud storage gateway, organizations may move primary iSCSI or NAS to the cloud services provider 302, thereby enabling the organization to save space on their on-premises storage systems. Such a cloud storage gateway may be configured to emulate a disk array, a block-based device, a file server, or other storage system that can translate the SCSI commands, file server commands, or other appropriate command into REST-space protocols that facilitate communications with the cloud services provider 302.

In order to enable the storage system 306 and users of the storage system 306 to make use of the services provided by the cloud services provider 302, a cloud migration process may take place during which data, applications, or other elements from an organization's local systems (or even from another cloud environment) are moved to the cloud services provider 302. In order to successfully migrate data, applications, or other elements to the cloud services provider's 302 environment, middleware such as a cloud migration tool may be utilized to bridge gaps between the cloud services provider's 302 environment and an organization's environment. In order to further enable the storage system 306 and users of the storage system 306 to make use of the services provided by the cloud services provider 302, a cloud orchestrator may also be used to arrange and coordinate automated tasks in pursuit of creating a consolidated process or workflow. Such a cloud orchestrator may perform tasks such as configuring various components, whether those components are cloud components or on-premises components, as well as managing the interconnections between such components.

In the example depicted in FIG. 3A, and as described briefly above, the cloud services provider 302 may be configured to provide services to the storage system 306 and users of the storage system 306 through the usage of a SaaS service model. For example, the cloud services provider 302 may be configured to provide access to data analytics applications to the storage system 306 and users of the storage system 306. Such data analytics applications may be configured, for example, to receive vast amounts of telemetry data phoned home by the storage system 306. Such telemetry data may describe various operating characteristics of the storage system 306 and may be analyzed for a vast array of purposes including, for example, to determine the health of the storage system 306, to identify workloads that are executing on the storage system 306, to predict when the storage system 306 will run out of various resources, to recommend configuration changes, hardware or software upgrades, workflow migrations, or other actions that may improve the operation of the storage system 306.

The cloud services provider 302 may also be configured to provide access to virtualized computing environments to the storage system 306 and users of the storage system 306. Examples of such virtualized environments can include virtual machines that are created to emulate an actual computer, virtualized desktop environments that separate a logical desktop from a physical machine, virtualized file systems that allow uniform access to different types of concrete file systems, and many others.

Although the example depicted in FIG. 3A illustrates the storage system 306 being coupled for data communications with the cloud services provider 302, in other embodiments the storage system 306 may be part of a hybrid cloud deployment in which private cloud elements (e.g., private cloud services, on-premises infrastructure, and so on) and public cloud elements (e.g., public cloud services, infrastructure, and so on) that may be provided by one or more cloud services providers are combined to form a single solution, with orchestration among the various platforms. Such a hybrid cloud deployment may leverage hybrid cloud management software such as, for example, Azure™ Arc from Microsoft™, that centralize the management of the hybrid cloud deployment to any infrastructure and enable the deployment of services anywhere. In such an example, the hybrid cloud management software may be configured to create, update, and delete resources (both physical and virtual) that form the hybrid cloud deployment, to allocate

compute and storage to specific workloads, to monitor workloads and resources for performance, policy compliance, updates and patches, security status, or to perform a variety of other tasks.

Readers will appreciate that by pairing the storage systems described herein with one or more cloud services providers, various offerings may be enabled. For example, disaster recovery as a service ('DRaaS') may be provided where cloud resources are utilized to protect applications and data from disruption caused by disaster, including in embodiments where the storage systems may serve as the primary data store. In such embodiments, a total system backup may be taken that allows for business continuity in the event of system failure. In such embodiments, cloud data backup techniques (by themselves or as part of a larger DRaaS solution) may also be integrated into an overall solution that includes the storage systems and cloud services providers described herein.

The storage systems described herein, as well as the cloud services providers, may be utilized to provide a wide array of security features. For example, the storage systems may encrypt data at rest (and data may be sent to and from the storage systems encrypted) and may make use of Key Management-as-a-Service ('KMaaS') to manage encryption keys, keys for locking and unlocking storage devices, and so on. Likewise, cloud data security gateways or similar mechanisms may be utilized to ensure that data stored within the storage systems does not improperly end up being stored in the cloud as part of a cloud data backup operation. Furthermore, microsegmentation or identity-based-segmentation may be utilized in a data center that includes the storage systems or within the cloud services provider, to create secure zones in data centers and cloud deployments that enables the isolation of workloads from one another.

For further explanation, FIG. 3B sets forth a diagram of a storage system 306 in accordance with some embodiments of the present disclosure. Although depicted in less detail, the storage system 306 depicted in FIG. 3B may be similar to the storage systems described above with reference to FIGS. 1A-1D and FIGS. 2A-2G as the storage system may include many of the components described above.

The storage system 306 depicted in FIG. 3B may include a vast amount of storage resources 308, which may be embodied in many forms. For example, the storage resources 308 can include nano-RAM or another form of nonvolatile random access memory that utilizes carbon nanotubes deposited on a substrate, 3D crosspoint non-volatile memory, flash memory including single-level cell ('SLC') NAND flash, multi-level cell ('MLC') NAND flash, triple-level cell ('TLC') NAND flash, quad-level cell ('QLC') NAND flash, or others. Likewise, the storage resources 308 may include non-volatile magnetoresistive random-access memory ('MRAM'), including spin transfer torque ('STT') MRAM. The example storage resources 308 may alternatively include non-volatile phase-change memory ('PCM'), quantum memory that allows for the storage and retrieval of photonic quantum information, resistive random-access memory ('ReRAM'), storage class memory ('SCM'), or other form of storage resources, including any combination of resources described herein. Readers will appreciate that other forms of computer memories and storage devices may be utilized by the storage systems described above, including DRAM, SRAM, EEPROM, universal memory, and many others. The storage resources 308 depicted in FIG. 3A may be embodied in a variety of form factors, including but not limited to, dual

31

in-line memory modules ('DIMMs'), non-volatile dual in-line memory modules ('NVDIMMs'), M.2, U.2, and others.

The storage resources **308** depicted in FIG. 3B may include various forms of SCM. SCM may effectively treat fast, non-volatile memory (e.g., NAND flash) as an extension of DRAM such that an entire dataset may be treated as an in-memory dataset that resides entirely in DRAM. SCM may include non-volatile media such as, for example, NAND flash. Such NAND flash may be accessed utilizing NVMe that can use the PCIe bus as its transport, providing for relatively low access latencies compared to older protocols. In fact, the network protocols used for SSDs in all-flash arrays can include NVMe using Ethernet (ROCE, NVMe TCP), Fibre Channel (NVMe FC), InfiniBand (iWARP), and others that make it possible to treat fast, non-volatile memory as an extension of DRAM. In view of the fact that DRAM is often byte-addressable and fast, non-volatile memory such as NAND flash is block-addressable, a controller software/hardware stack may be needed to convert the block data to the bytes that are stored in the media. Examples of media and software that may be used as SCM can include, for example, 3D XPoint, Intel Memory Drive Technology, Samsung's Z-SSD, and others.

The storage resources **308** depicted in FIG. 3B may also include racetrack memory (also referred to as domain-wall memory). Such racetrack memory may be embodied as a form of non-volatile, solid-state memory that relies on the intrinsic strength and orientation of the magnetic field created by an electron as it spins in addition to its electronic charge, in solid-state devices. Through the use of spin-coherent electric current to move magnetic domains along a nanoscopic permalloy wire, the domains may pass by magnetic read/write heads positioned near the wire as current is passed through the wire, which alter the domains to record patterns of bits. In order to create a racetrack memory device, many such wires and read/write elements may be packaged together.

The example storage system **306** depicted in FIG. 3B may implement a variety of storage architectures. For example, storage systems in accordance with some embodiments of the present disclosure may utilize block storage where data is stored in blocks, and each block essentially acts as an individual hard drive. Storage systems in accordance with some embodiments of the present disclosure may utilize object storage, where data is managed as objects. Each object may include the data itself, a variable amount of metadata, and a globally unique identifier, where object storage can be implemented at multiple levels (e.g., device level, system level, interface level). Storage systems in accordance with some embodiments of the present disclosure utilize file storage in which data is stored in a hierarchical structure. Such data may be saved in files and folders, and presented to both the system storing it and the system retrieving it in the same format.

The example storage system **306** depicted in FIG. 3B may be embodied as a storage system in which additional storage resources can be added through the use of a scale-up model, additional storage resources can be added through the use of a scale-out model, or through some combination thereof. In a scale-up model, additional storage may be added by adding additional storage devices. In a scale-out model, however, additional storage nodes may be added to a cluster of storage nodes, where such storage nodes can include additional processing resources, additional networking resources, and so on.

The example storage system **306** depicted in FIG. 3B may leverage the storage resources described above in a variety

32

of different ways. For example, some portion of the storage resources may be utilized to serve as a write cache, storage resources within the storage system may be utilized as a read cache, or tiering may be achieved within the storage systems by placing data within the storage system in accordance with one or more tiering policies.

The storage system **306** depicted in FIG. 3B also includes communications resources **310** that may be useful in facilitating data communications between components within the storage system **306**, as well as data communications between the storage system **306** and computing devices that are outside of the storage system **306**, including embodiments where those resources are separated by a relatively vast expanse. The communications resources **310** may be configured to utilize a variety of different protocols and data communication fabrics to facilitate data communications between components within the storage systems as well as computing devices that are outside of the storage system. For example, the communications resources **310** can include fibre channel ('FC') technologies such as FC fabrics and FC protocols that can transport SCSI commands over FC network, FC over ethernet ('FCoE') technologies through which FC frames are encapsulated and transmitted over Ethernet networks, InfiniB and ('IB') technologies in which a switched fabric topology is utilized to facilitate transmissions between channel adapters, NVM Express ('NVMe') technologies and NVMe over fabrics ('NVMeoF') technologies through which non-volatile storage media attached via a PCI express ('PCIe') bus may be accessed, and others. In fact, the storage systems described above may, directly or indirectly, make use of neutrino communication technologies and devices through which information (including binary information) is transmitted using a beam of neutrinos.

The communications resources **310** can also include mechanisms for accessing storage resources **308** within the storage system **306** utilizing serial attached SCSI ('SAS'), serial ATA ('SATA') bus interfaces for connecting storage resources **308** within the storage system **306** to host bus adapters within the storage system **306**, internet small computer systems interface ('iSCSI') technologies to provide block-level access to storage resources **308** within the storage system **306**, and other communications resources that may be useful in facilitating data communications between components within the storage system **306**, as well as data communications between the storage system **306** and computing devices that are outside of the storage system **306**.

The storage system **306** depicted in FIG. 3B also includes processing resources **312** that may be useful in executing computer program instructions and performing other computational tasks within the storage system **306**. The processing resources **312** may include one or more ASICs that are customized for some particular purpose as well as one or more CPUs. The processing resources **312** may also include one or more DSPs, one or more FPGAs, one or more systems on a chip ('SoCs'), or other form of processing resources **312**. The storage system **306** may utilize the storage resources **312** to perform a variety of tasks including, but not limited to, supporting the execution of software resources **314** that will be described in greater detail below.

The storage system **306** depicted in FIG. 3B also includes software resources **314** that, when executed by processing resources **312** within the storage system **306**, may perform a vast array of tasks. The software resources **314** may include, for example, one or more modules of computer program instructions that when executed by processing resources **312** within the storage system **306** are useful in

carrying out various data protection techniques. Such data protection techniques may be carried out, for example, by system software executing on computer hardware within the storage system, by a cloud services provider, or in other ways. Such data protection techniques can include data archiving, data backup, data replication, data snapshotting, data and database cloning, and other data protection techniques.

The software resources **314** may also include software that is useful in implementing software-defined storage ('SDS'). In such an example, the software resources **314** may include one or more modules of computer program instructions that, when executed, are useful in policy-based provisioning and management of data storage that is independent of the underlying hardware. Such software resources **314** may be useful in implementing storage virtualization to separate the storage hardware from the software that manages the storage hardware.

The software resources **314** may also include software that is useful in facilitating and optimizing I/O operations that are directed to the storage system **306**. For example, the software resources **314** may include software modules that perform various data reduction techniques such as, for example, data compression, data deduplication, and others. The software resources **314** may include software modules that intelligently group together I/O operations to facilitate better usage of the underlying storage resource **308**, software modules that perform data migration operations to migrate from within a storage system, as well as software modules that perform other functions. Such software resources **314** may be embodied as one or more software containers or in many other ways.

For further explanation, FIG. 3C sets forth an example of a cloud-based storage system **318** in accordance with some embodiments of the present disclosure. In the example depicted in FIG. 3C, the cloud-based storage system **318** is created entirely in a cloud computing environment **316** such as, for example, Amazon Web Services ('AWS')TM, Microsoft AzureTM, Google Cloud PlatformTM, IBM CloudTM, Oracle CloudTM, and others. The cloud-based storage system **318** may be used to provide services similar to the services that may be provided by the storage systems described above.

The cloud-based storage system **318** depicted in FIG. 3C includes two cloud computing instances **320**, **322** that each are used to support the execution of a storage controller application **324**, **326**. The cloud computing instances **320**, **322** may be embodied, for example, as instances of cloud computing resources (e.g., virtual machines) that may be provided by the cloud computing environment **316** to support the execution of software applications such as the storage controller application **324**, **326**. For example, each of the cloud computing instances **320**, **322** may execute on an Azure VM, where each Azure VM may include high speed temporary storage that may be leveraged as a cache (e.g., as a read cache). In one embodiment, the cloud computing instances **320**, **322** may be embodied as Amazon Elastic Compute Cloud ('EC2') instances. In such an example, an Amazon Machine Image ('AMI') that includes the storage controller application **324**, **326** may be booted to create and configure a virtual machine that may execute the storage controller application **324**, **326**.

In the example method depicted in FIG. 3C, the storage controller application **324**, **326** may be embodied as a module of computer program instructions that, when executed, carries out various storage tasks. For example, the storage controller application **324**, **326** may be embodied as

a module of computer program instructions that, when executed, carries out the same tasks as the controllers **110A**, **110B** in FIG. 1A described above such as writing data to the cloud-based storage system **318**, erasing data from the cloud-based storage system **318**, retrieving data from the cloud-based storage system **318**, monitoring and reporting of storage device utilization and performance, performing redundancy operations, such as RAID or RAID-like data redundancy operations, compressing data, encrypting data, deduplicating data, and so forth. Readers will appreciate that because there are two cloud computing instances **320**, **322** that each include the storage controller application **324**, **326**, in some embodiments one cloud computing instance **320** may operate as the primary controller as described above while the other cloud computing instance **322** may operate as the secondary controller as described above. Readers will appreciate that the storage controller application **324**, **326** depicted in FIG. 3C may include identical source code that is executed within different cloud computing instances **320**, **322** such as distinct EC2 instances.

Readers will appreciate that other embodiments that do not include a primary and secondary controller are within the scope of the present disclosure. For example, each cloud computing instance **320**, **322** may operate as a primary controller for some portion of the address space supported by the cloud-based storage system **318**, each cloud computing instance **320**, **322** may operate as a primary controller where the servicing of I/O operations directed to the cloud-based storage system **318** are divided in some other way, and so on. In fact, in other embodiments where costs savings may be prioritized over performance demands, only a single cloud computing instance may exist that contains the storage controller application.

The cloud-based storage system **318** depicted in FIG. 3C includes cloud computing instances **340a**, **340b**, **340n** with local storage **330**, **334**, **338**. The cloud computing instances **340a**, **340b**, **340n** may be embodied, for example, as instances of cloud computing resources that may be provided by the cloud computing environment **316** to support the execution of software applications. The cloud computing instances **340a**, **340b**, **340n** of FIG. 3C may differ from the cloud computing instances **320**, **322** described above as the cloud computing instances **340a**, **340b**, **340n** of FIG. 3C have local storage **330**, **334**, **338** resources whereas the cloud computing instances **320**, **322** that support the execution of the storage controller application **324**, **326** need not have local storage resources. The cloud computing instances **340a**, **340b**, **340n** with local storage **330**, **334**, **338** may be embodied, for example, as EC2 M5 instances that include one or more SSDs, as EC2 R5 instances that include one or more SSDs, as EC2 I3 instances that include one or more SSDs, and so on. In some embodiments, the local storage **330**, **334**, **338** must be embodied as solid-state storage (e.g., SSDs) rather than storage that makes use of hard disk drives.

In the example depicted in FIG. 3C, each of the cloud computing instances **340a**, **340b**, **340n** with local storage **330**, **334**, **338** can include a software daemon **328**, **332**, **336** that, when executed by a cloud computing instance **340a**, **340b**, **340n** can present itself to the storage controller applications **324**, **326** as if the cloud computing instance **340a**, **340b**, **340n** were a physical storage device (e.g., one or more SSDs). In such an example, the software daemon **328**, **332**, **336** may include computer program instructions similar to those that would normally be contained on a storage device such that the storage controller applications **324**, **326** can send and receive the same commands that a storage controller would send to storage devices. In such a

35

way, the storage controller applications 324, 326 may include code that is identical to (or substantially identical to) the code that would be executed by the controllers in the storage systems described above. In these and similar embodiments, communications between the storage controller applications 324, 326 and the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 may utilize iSCSI, NVMe over TCP, messaging, a custom protocol, or in some other mechanism.

In the example depicted in FIG. 3C, each of the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 may also be coupled to block storage 342, 344, 346 that is offered by the cloud computing environment 316 such as, for example, as Amazon Elastic Block Store ('EBS') volumes. In such an example, the block storage 342, 344, 346 that is offered by the cloud computing environment 316 may be utilized in a manner that is similar to how the NVRAM devices described above are utilized, as the software daemon 328, 332, 336 (or some other module) that is executing within a particular cloud computing instance 340a, 340b, 340n may, upon receiving a request to write data, initiate a write of the data to its attached EBS volume as well as a write of the data to its local storage 330, 334, 338 resources. In some alternative embodiments, data may only be written to the local storage 330, 334, 338 resources within a particular cloud computing instance 340a, 340b, 340n. In an alternative embodiment, rather than using the block storage 342, 344, 346 that is offered by the cloud computing environment 316 as NVRAM, actual RAM on each of the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 may be used as NVRAM, thereby decreasing network utilization costs that would be associated with using an EBS volume as the NVRAM. In yet another embodiment, high performance block storage resources such as one or more Azure Ultra Disks may be utilized as the NVRAM.

When a request to write data is received by a particular cloud computing instance 340a, 340b, 340n with local storage 330, 334, 338, the software daemon 328, 332, 336 may be configured to not only write the data to its own local storage 330, 334, 338 resources and any appropriate block storage 342, 344, 346 resources, but the software daemon 328, 332, 336 may also be configured to write the data to cloud-based object storage 348 that is attached to the particular cloud computing instance 340a, 340b, 340n. The cloud-based object storage 348 that is attached to the particular cloud computing instance 340a, 340b, 340n may be embodied, for example, as Amazon Simple Storage Service ('S3'). In other embodiments, the cloud computing instances 320, 322 that each include the storage controller application 324, 326 may initiate the storage of the data in the local storage 330, 334, 338 of the cloud computing instances 340a, 340b, 340n and the cloud-based object storage 348. In other embodiments, rather than using both the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 (also referred to herein as 'virtual drives') and the cloud-based object storage 348 to store data, a persistent storage layer may be implemented in other ways. For example, one or more Azure Ultra disks may be used to persistently store data (e.g., after the data has been written to the NVRAM layer). In an embodiment where one or more Azure Ultra disks may be used to persistently store data, the usage of a cloud-based object storage 348 may be eliminated such that data is only stored persistently in the Azure Ultra disks without also writing the data to an object storage layer.

While the local storage 330, 334, 338 resources and the block storage 342, 344, 346 resources that are utilized by the

36

cloud computing instances 340a, 340b, 340n may support block-level access, the cloud-based object storage 348 that is attached to the particular cloud computing instance 340a, 340b, 340n supports only object-based access. The software daemon 328, 332, 336 may therefore be configured to take blocks of data, package those blocks into objects, and write the objects to the cloud-based object storage 348 that is attached to the particular cloud computing instance 340a, 340b, 340n.

In some embodiments, all data that is stored by the cloud-based storage system 318 may be stored in both: 1) the cloud-based object storage 348, and 2) at least one of the local storage 330, 334, 338 resources or block storage 342, 344, 346 resources that are utilized by the cloud computing instances 340a, 340b, 340n. In such embodiments, the local storage 330, 334, 338 resources and block storage 342, 344, 346 resources that are utilized by the cloud computing instances 340a, 340b, 340n may effectively operate as cache that generally includes all data that is also stored in S3, such that all reads of data may be serviced by the cloud computing instances 340a, 340b, 340n without requiring the cloud computing instances 340a, 340b, 340n to access the cloud-based object storage 348. Readers will appreciate that in other embodiments, however, all data that is stored by the cloud-based storage system 318 may be stored in the cloud-based object storage 348, but less than all data that is stored by the cloud-based storage system 318 may be stored in at least one of the local storage 330, 334, 338 resources or block storage 342, 344, 346 resources that are utilized by the cloud computing instances 340a, 340b, 340n. In such an example, various policies may be utilized to determine which subset of the data that is stored by the cloud-based storage system 318 should reside in both: 1) the cloud-based object storage 348, and 2) at least one of the local storage 330, 334, 338 resources or block storage 342, 344, 346 resources that are utilized by the cloud computing instances 340a, 340b, 340n.

One or more modules of computer program instructions that are executing within the cloud-based storage system 318 (e.g., a monitoring module that is executing on its own EC2 instance) may be designed to handle the failure of one or more of the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338. In such an example, the monitoring module may handle the failure of one or more of the cloud computing instances 340a, 340b, 340n with local storage 330, 334, 338 by creating one or more new cloud computing instances with local storage, retrieving data that was stored on the failed cloud computing instances 340a, 340b, 340n from the cloud-based object storage 348, and storing the data retrieved from the cloud-based object storage 348 in local storage on the newly created cloud computing instances. Readers will appreciate that many variants of this process may be implemented.

Readers will appreciate that various performance aspects of the cloud-based storage system 318 may be monitored (e.g., by a monitoring module that is executing in an EC2 instance) such that the cloud-based storage system 318 can be scaled-up or scaled-out as needed. For example, if the cloud computing instances 320, 322 that are used to support the execution of a storage controller application 324, 326 are undersized and not sufficiently servicing the I/O requests that are issued by users of the cloud-based storage system 318, a monitoring module may create a new, more powerful cloud computing instance (e.g., a cloud computing instance of a type that includes more processing power, more memory, etc. . . .) that includes the storage controller application such that the new, more powerful cloud com-

puting instance can begin operating as the primary controller. Likewise, if the monitoring module determines that the cloud computing instances **320**, **322** that are used to support the execution of a storage controller application **324**, **326** are oversized and that cost savings could be gained by switching to a smaller, less powerful cloud computing instance, the monitoring module may create a new, less powerful (and less expensive) cloud computing instance that includes the storage controller application such that the new, less powerful cloud computing instance can begin operating as the primary controller.

The storage systems described above may carry out intelligent data backup techniques through which data stored in the storage system may be copied and stored in a distinct location to avoid data loss in the event of equipment failure or some other form of catastrophe. For example, the storage systems described above may be configured to examine each backup to avoid restoring the storage system to an undesirable state. Consider an example in which malware infects the storage system. In such an example, the storage system may include software resources **314** that can scan each backup to identify backups that were captured before the malware infected the storage system and those backups that were captured after the malware infected the storage system. In such an example, the storage system may restore itself from a backup that does not include the malware—or at least not restore the portions of a backup that contained the malware. In such an example, the storage system may include software resources **314** that can scan each backup to identify the presences of malware (or a virus, or some other undesirable), for example, by identifying write operations that were serviced by the storage system and originated from a network subnet that is suspected to have delivered the malware, by identifying write operations that were serviced by the storage system and originated from a user that is suspected to have delivered the malware, by identifying write operations that were serviced by the storage system and examining the content of the write operation against fingerprints of the malware, and in many other ways.

Readers will further appreciate that the backups (often in the form of one or more snapshots) may also be utilized to perform rapid recovery of the storage system. Consider an example in which the storage system is infected with ransomware that locks users out of the storage system. In such an example, software resources **314** within the storage system may be configured to detect the presence of ransomware and may be further configured to restore the storage system to a point-in-time, using the retained backups, prior to the point-in-time at which the ransomware infected the storage system. In such an example, the presence of ransomware may be explicitly detected through the use of software tools utilized by the system, through the use of a key (e.g., a USB drive) that is inserted into the storage system, or in a similar way. Likewise, the presence of ransomware may be inferred in response to system activity meeting a predetermined fingerprint such as, for example, no reads or writes coming into the system for a predetermined period of time.

Readers will appreciate that the various components described above may be grouped into one or more optimized computing packages as converged infrastructures. Such converged infrastructures may include pools of computers, storage and networking resources that can be shared by multiple applications and managed in a collective manner using policy-driven processes. Such converged infrastructures may be implemented with a converged infrastructure reference architecture, with standalone appliances, with a

software driven hyper-converged approach (e.g., hyper-converged infrastructures), or in other ways.

Readers will appreciate that the storage systems described in this disclosure may be useful for supporting various types of software applications. In fact, the storage systems may be ‘application aware’ in the sense that the storage systems may obtain, maintain, or otherwise have access to information describing connected applications (e.g., applications that utilize the storage systems) to optimize the operation of the storage system based on intelligence about the applications and their utilization patterns. For example, the storage system may optimize data layouts, optimize caching behaviors, optimize ‘QoS’ levels, or perform some other optimization that is designed to improve the storage performance that is experienced by the application.

As an example of one type of application that may be supported by the storage systems described herein, the storage system **306** may be useful in supporting artificial intelligence (‘AI’) applications, database applications, XOps projects (e.g., DevOps projects, DataOps projects, MLOps projects, ModelOps projects, PlatformOps projects), electronic design automation tools, event-driven software applications, high performance computing applications, simulation applications, high-speed data capture and analysis applications, machine learning applications, media production applications, media serving applications, picture archiving and communication systems (‘PACS’) applications, software development applications, virtual reality applications, augmented reality applications, and many other types of applications by providing storage resources to such applications.

In view of the fact that the storage systems include compute resources, storage resources, and a wide variety of other resources, the storage systems may be well suited to support applications that are resource intensive such as, for example, AI applications. AI applications may be deployed in a variety of fields, including: predictive maintenance in manufacturing and related fields, healthcare applications such as patient data & risk analytics, retail and marketing deployments (e.g., search advertising, social media advertising), supply chains solutions, fintech solutions such as business analytics & reporting tools, operational deployments such as real-time analytics tools, application performance management tools, IT infrastructure management tools, and many others.

Such AI applications may enable devices to perceive their environment and take actions that maximize their chance of success at some goal. Examples of such AI applications can include IBM Watson™, Microsoft Oxford™, Google DeepMind™, Baidu Minwa™, and others.

The storage systems described above may also be well suited to support other types of applications that are resource intensive such as, for example, machine learning applications. Machine learning applications may perform various types of data analysis to automate analytical model building. Using algorithms that iteratively learn from data, machine learning applications can enable computers to learn without being explicitly programmed. One particular area of machine learning is referred to as reinforcement learning, which involves taking suitable actions to maximize reward in a particular situation.

In addition to the resources already described, the storage systems described above may also include graphics processing units (‘GPUs’), occasionally referred to as visual processing unit (‘VPUs’). Such GPUs may be embodied as specialized electronic circuits that rapidly manipulate and alter memory to accelerate the creation of images in a frame

buffer intended for output to a display device. Such GPUs may be included within any of the computing devices that are part of the storage systems described above, including as one of many individually scalable components of a storage system, where other examples of individually scalable components of such storage system can include storage components, memory components, compute components (e.g., CPUs, FPGAs, ASICs), networking components, software components, and others. In addition to GPUs, the storage systems described above may also include neural network processors ('NNPs') for use in various aspects of neural network processing. Such NNPs may be used in place of (or in addition to) GPUs and may also be independently scalable.

As described above, the storage systems described herein may be configured to support artificial intelligence applications, machine learning applications, big data analytics applications, and many other types of applications. The rapid growth in these sort of applications is being driven by three technologies: deep learning (DL), GPU processors, and Big Data. Deep learning is a computing model that makes use of massively parallel neural networks inspired by the human brain. Instead of experts handcrafting software, a deep learning model writes its own software by learning from lots of examples. Such GPUs may include thousands of cores that are well-suited to run algorithms that loosely represent the parallel nature of the human brain.

Advances in deep neural networks, including the development of multi-layer neural networks, have ignited a new wave of algorithms and tools for data scientists to tap into their data with artificial intelligence (AI). With improved algorithms, larger data sets, and various frameworks (including open-source software libraries for machine learning across a range of tasks), data scientists are tackling new use cases like autonomous driving vehicles, natural language processing and understanding, computer vision, machine reasoning, strong AI, and many others. Applications of AI techniques have materialized in a wide array of products include, for example, Amazon Echo's speech recognition technology that allows users to talk to their machines, Google Translate™ which allows for machine-based language translation, Spotify's Discover Weekly that provides recommendations on new songs and artists that a user may like based on the user's usage and traffic analysis, Quill's text generation offering that takes structured data and turns it into narrative stories, Chatbots that provide real-time, contextually specific answers to questions in a dialog format, and many others.

Data is the heart of modern AI and deep learning algorithms. Before training can begin, one problem that must be addressed revolves around collecting the labeled data that is crucial for training an accurate AI model. A full scale AI deployment may be required to continuously collect, clean, transform, label, and store large amounts of data. Adding additional high quality data points directly translates to more accurate models and better insights. Data samples may undergo a series of processing steps including, but not limited to: 1) ingesting the data from an external source into the training system and storing the data in raw form, 2) cleaning and transforming the data in a format convenient for training, including linking data samples to the appropriate label, 3) exploring parameters and models, quickly testing with a smaller dataset, and iterating to converge on the most promising models to push into the production cluster, 4) executing training phases to select random batches of input data, including both new and older samples, and feeding those into production GPU servers for compu-

tation to update model parameters, and 5) evaluating including using a holdback portion of the data not used in training in order to evaluate model accuracy on the holdout data. This lifecycle may apply for any type of parallelized machine learning, not just neural networks or deep learning. For example, standard machine learning frameworks may rely on CPUs instead of GPUs but the data ingest and training workflows may be the same. Readers will appreciate that a single shared storage data hub creates a coordination point throughout the lifecycle without the need for extra data copies among the ingest, preprocessing, and training stages. Rarely is the ingested data used for only one purpose, and shared storage gives the flexibility to train multiple different models or apply traditional analytics to the data.

Readers will appreciate that each stage in the AI data pipeline may have varying requirements from the data hub (e.g., the storage system or collection of storage systems). Scale-out storage systems must deliver uncompromising performance for all manner of access types and patterns—from small, metadata-heavy to large files, from random to sequential access patterns, and from low to high concurrency. The storage systems described above may serve as an ideal AI data hub as the systems may service unstructured workloads. In the first stage, data is ideally ingested and stored on to the same data hub that following stages will use, in order to avoid excess data copying. The next two steps can be done on a standard compute server that optionally includes a GPU, and then in the fourth and last stage, full training production jobs are run on powerful GPU-accelerated servers. Often, there is a production pipeline alongside an experimental pipeline operating on the same dataset. Further, the GPU-accelerated servers can be used independently for different models or joined together to train on one larger model, even spanning multiple systems for distributed training. If the shared storage tier is slow, then data must be copied to local storage for each phase, resulting in wasted time staging data onto different servers. The ideal data hub for the AI training pipeline delivers performance similar to data stored locally on the server node while also having the simplicity and performance to enable all pipeline stages to operate concurrently.

In order for the storage systems described above to serve as a data hub or as part of an AI deployment, in some embodiments the storage systems may be configured to provide DMA between storage devices that are included in the storage systems and one or more GPUs that are used in an AI or big data analytics pipeline. The one or more GPUs may be coupled to the storage system, for example, via NVMe-over-Fabrics ('NVMe-oF') such that bottlenecks such as the host CPU can be bypassed and the storage system (or one of the components contained therein) can directly access GPU memory. In such an example, the storage systems may leverage API hooks to the GPUs to transfer data directly to the GPUs. For example, the GPUs may be embodied as Nvidia™ GPUs and the storage systems may support GPUDirect Storage ('GDS') software, or have similar proprietary software, that enables the storage system to transfer data to the GPUs via RDMA or similar mechanism.

Although the preceding paragraphs discuss deep learning applications, readers will appreciate that the storage systems described herein may also be part of a distributed deep learning ('DDL') platform to support the execution of DDL algorithms. The storage systems described above may also be paired with other technologies such as TensorFlow, an open-source software library for dataflow programming across a range of tasks that may be used for machine

learning applications such as neural networks, to facilitate the development of such machine learning models, applications, and so on.

The storage systems described above may also be used in a neuromorphic computing environment. Neuromorphic computing is a form of computing that mimics brain cells. To support neuromorphic computing, an architecture of interconnected “neurons” replace traditional computing models with low-powered signals that go directly between neurons for more efficient computation. Neuromorphic computing may make use of very-large-scale integration (VLSI) systems containing electronic analog circuits to mimic neuro-biological architectures present in the nervous system, as well as analog, digital, mixed-mode analog/digital VLSI, and software systems that implement models of neural systems for perception, motor control, or multisensory integration.

Readers will appreciate that the storage systems described above may be configured to support the storage or use of (among other types of data) blockchains and derivative items such as, for example, open source blockchains and related tools that are part of the IBM™ Hyperledger project, permissioned blockchains in which a certain number of trusted parties are allowed to access the block chain, blockchain products that enable developers to build their own distributed ledger projects, and others. Blockchains and the storage systems described herein may be leveraged to support on-chain storage of data as well as off-chain storage of data.

Off-chain storage of data can be implemented in a variety of ways and can occur when the data itself is not stored within the blockchain. For example, in one embodiment, a hash function may be utilized and the data itself may be fed into the hash function to generate a hash value. In such an example, the hashes of large pieces of data may be embedded within transactions, instead of the data itself. Readers will appreciate that, in other embodiments, alternatives to blockchains may be used to facilitate the decentralized storage of information. For example, one alternative to a blockchain that may be used is a blockweave. While conventional blockchains store every transaction to achieve validation, a blockweave permits secure decentralization without the usage of the entire chain, thereby enabling low cost on-chain storage of data. Such blockweaves may utilize a consensus mechanism that is based on proof of access (PoA) and proof of work (PoW).

The storage systems described above may, either alone or in combination with other computing devices, be used to support in-memory computing applications. In-memory computing involves the storage of information in RAM that is distributed across a cluster of computers. Readers will appreciate that the storage systems described above, especially those that are configurable with customizable amounts of processing resources, storage resources, and memory resources (e.g., those systems in which blades that contain configurable amounts of each type of resource), may be configured in a way so as to provide an infrastructure that can support in-memory computing. Likewise, the storage systems described above may include component parts (e.g., NVDIMMs, 3D crosspoint storage that provide fast random access memory that is persistent) that can actually provide for an improved in-memory computing environment as compared to in-memory computing environments that rely on RAM distributed across dedicated servers.

In some embodiments, the storage systems described above may be configured to operate as a hybrid in-memory computing environment that includes a universal interface to

all storage media (e.g., RAM, flash storage, 3D crosspoint storage). In such embodiments, users may have no knowledge regarding the details of where their data is stored but they can still use the same full, unified API to address data.

In such embodiments, the storage system may (in the background) move data to the fastest layer available—including intelligently placing the data in dependence upon various characteristics of the data or in dependence upon some other heuristic. In such an example, the storage systems may even make use of existing products such as Apache Ignite and GridGain to move data between the various storage layers, or the storage systems may make use of custom software to move data between the various storage layers. The storage systems described herein may implement various optimizations to improve the performance of in-memory computing such as, for example, having computations occur as close to the data as possible.

Readers will further appreciate that in some embodiments, the storage systems described above may be paired with other resources to support the applications described above. For example, one infrastructure could include primary compute in the form of servers and workstations which specialize in using General-purpose computing on graphics processing units (“GPGPU”) to accelerate deep learning applications that are interconnected into a computation engine to train parameters for deep neural networks. Each system may have Ethernet external connectivity, InfiniB and external connectivity, some other form of external connectivity, or some combination thereof. In such an example, the GPUs can be grouped for a single large training or used independently to train multiple models. The infrastructure could also include a storage system such as those described above to provide, for example, a scale-out all-flash file or object store through which data can be accessed via high-performance protocols such as NFS, S3, and so on. The infrastructure can also include, for example, redundant top-of-rack Ethernet switches connected to storage and compute via ports in MLAG port channels for redundancy. The infrastructure could also include additional compute in the form of whitebox servers, optionally with GPUs, for data ingestion, pre-processing, and model debugging. Readers will appreciate that additional infrastructures are also possible.

Readers will appreciate that the storage systems described above, either alone or in coordination with other computing machinery may be configured to support other AI related tools. For example, the storage systems may make use of tools like ONNX or other open neural network exchange formats that make it easier to transfer models written in different AI frameworks. Likewise, the storage systems may be configured to support tools like Amazon’s Glue that allow developers to prototype, build, and train deep learning models. In fact, the storage systems described above may be part of a larger platform, such as IBM™ Cloud Private for Data, that includes integrated data science, data engineering and application building services.

Readers will further appreciate that the storage systems described above may also be deployed as an edge solution. Such an edge solution may be in place to optimize cloud computing systems by performing data processing at the edge of the network, near the source of the data. Edge computing can push applications, data and computing power (i.e., services) away from centralized points to the logical extremes of a network. Through the use of edge solutions such as the storage systems described above, computational tasks may be performed using the compute resources provided by such storage systems, data may be storage using the

storage resources of the storage system, and cloud-based services may be accessed through the use of various resources of the storage system (including networking resources). By performing computational tasks on the edge solution, storing data on the edge solution, and generally making use of the edge solution, the consumption of expensive cloud-based resources may be avoided and, in fact, performance improvements may be experienced relative to a heavier reliance on cloud-based resources.

While many tasks may benefit from the utilization of an edge solution, some particular uses may be especially suited for deployment in such an environment. For example, devices like drones, autonomous cars, robots, and others may require extremely rapid processing—so fast, in fact, that sending data up to a cloud environment and back to receive data processing support may simply be too slow. As an additional example, some IoT devices such as connected video cameras may not be well-suited for the utilization of cloud-based resources as it may be impractical (not only from a privacy perspective, security perspective, or a financial perspective) to send the data to the cloud simply because of the pure volume of data that is involved. As such, many tasks that really on data processing, storage, or communications may be better suited by platforms that include edge solutions such as the storage systems described above.

The storage systems described above may alone, or in combination with other computing resources, serves as a network edge platform that combines compute resources, storage resources, networking resources, cloud technologies and network virtualization technologies, and so on. As part of the network, the edge may take on characteristics similar to other network facilities, from the customer premise and backhaul aggregation facilities to Points of Presence (PoPs) and regional data centers. Readers will appreciate that network workloads, such as Virtual Network Functions (VNFs) and others, will reside on the network edge platform. Enabled by a combination of containers and virtual machines, the network edge platform may rely on controllers and schedulers that are no longer geographically collocated with the data processing resources. The functions, as microservices, may split into control planes, user and data planes, or even state machines, allowing for independent optimization and scaling techniques to be applied. Such user and data planes may be enabled through increased accelerators, both those residing in server platforms, such as FPGAs and Smart NICs, and through SDN-enabled merchant silicon and programmable ASICs.

The storage systems described above may also be optimized for use in big data analytics, including being leveraged as part of a composable data analytics pipeline where containerized analytics architectures, for example, make analytics capabilities more composable. Big data analytics may be generally described as the process of examining large and varied data sets to uncover hidden patterns, unknown correlations, market trends, customer preferences and other useful information that can help organizations make more-informed business decisions. As part of that process, semi-structured and unstructured data such as, for example, internet clickstream data, web server logs, social media content, text from customer emails and survey responses, mobile-phone call-detail records, IoT sensor data, and other data may be converted to a structured form.

The storage systems described above may also support (including implementing as a system interface) applications that perform tasks in response to human speech. For example, the storage systems may support the execution of intelligent personal assistant applications such as, for

example, Amazon's Alexa™, Apple Siri™, Google Voice™, Samsung Bixby™, Microsoft Cortana™, and others. While the examples described in the previous sentence make use of voice as input, the storage systems described above may also support chatbots, talkbots, chatterbots, or artificial conversational entities or other applications that are configured to conduct a conversation via auditory or textual methods. Likewise, the storage system may actually execute such an application to enable a user such as a system administrator to interact with the storage system via speech. Such applications are generally capable of voice interaction, music playback, making to-do lists, setting alarms, streaming podcasts, playing audiobooks, and providing weather, traffic, and other real time information, such as news, although in embodiments in accordance with the present disclosure, such applications may be utilized as interfaces to various system management operations.

The storage systems described above may also implement AI platforms for delivering on the vision of self-driving storage. Such AI platforms may be configured to deliver global predictive intelligence by collecting and analyzing large amounts of storage system telemetry data points to enable effortless management, analytics and support. In fact, such storage systems may be capable of predicting both capacity and performance, as well as generating intelligent advice on workload deployment, interaction and optimization. Such AI platforms may be configured to scan all incoming storage system telemetry data against a library of issue fingerprints to predict and resolve incidents in real-time, before they impact customer environments, and captures hundreds of variables related to performance that are used to forecast performance load.

The storage systems described above may support the serialized or simultaneous execution of artificial intelligence applications, machine learning applications, data analytics applications, data transformations, and other tasks that collectively may form an AI ladder. Such an AI ladder may effectively be formed by combining such elements to form a complete data science pipeline, where exist dependencies between elements of the AI ladder. For example, AI may require that some form of machine learning has taken place, machine learning may require that some form of analytics has taken place, analytics may require that some form of data and information architecting has taken place, and so on. As such, each element may be viewed as a rung in an AI ladder that collectively can form a complete and sophisticated AI solution.

The storage systems described above may also, either alone or in combination with other computing environments, be used to deliver an AI everywhere experience where AI permeates wide and expansive aspects of business and life. For example, AI may play an important role in the delivery of deep learning solutions, deep reinforcement learning solutions, artificial general intelligence solutions, autonomous vehicles, cognitive computing solutions, commercial UAVs or drones, conversational user interfaces, enterprise taxonomies, ontology management solutions, machine learning solutions, smart dust, smart robots, smart workplaces, and many others.

The storage systems described above may also, either alone or in combination with other computing environments, be used to deliver a wide range of transparently immersive experiences (including those that use digital twins of various “things” such as people, places, processes, systems, and so on) where technology can introduce transparency between people, businesses, and things. Such transparently immersive experiences may be delivered as augmented reality

technologies, connected homes, virtual reality technologies, brain-computer interfaces, human augmentation technologies, nanotube electronics, volumetric displays, 4D printing technologies, or others.

The storage systems described above may also, either alone or in combination with other computing environments, be used to support a wide variety of digital platforms. Such digital platforms can include, for example, 5G wireless systems and platforms, digital twin platforms, edge computing platforms, IoT platforms, quantum computing platforms, serverless PaaS, software-defined security, neuro-morphic computing platforms, and so on.

The storage systems described above may also be part of a multi-cloud environment in which multiple cloud computing and storage services are deployed in a single heterogeneous architecture. In order to facilitate the operation of such a multi-cloud environment, DevOps tools may be deployed to enable orchestration across clouds. Likewise, continuous development and continuous integration tools may be deployed to standardize processes around continuous integration and delivery, new feature rollout and provisioning cloud workloads. By standardizing these processes, a multi-cloud strategy may be implemented that enables the utilization of the best provider for each workload.

The storage systems described above may be used as a part of a platform to enable the use of crypto-anchors that may be used to authenticate a product's origins and contents to ensure that it matches a blockchain record associated with the product. Similarly, as part of a suite of tools to secure data stored on the storage system, the storage systems described above may implement various encryption technologies and schemes, including lattice cryptography. Lattice cryptography can involve constructions of cryptographic primitives that involve lattices, either in the construction itself or in the security proof. Unlike public-key schemes such as the RSA, Diffie-Hellman or Elliptic-Curve cryptosystems, which are easily attacked by a quantum computer, some lattice-based constructions appear to be resistant to attack by both classical and quantum computers.

A quantum computer is a device that performs quantum computing. Quantum computing is computing using quantum-mechanical phenomena, such as superposition and entanglement. Quantum computers differ from traditional computers that are based on transistors, as such traditional computers require that data be encoded into binary digits (bits), each of which is always in one of two definite states (0 or 1). In contrast to traditional computers, quantum computers use quantum bits, which can be in superpositions of states. A quantum computer maintains a sequence of qubits, where a single qubit can represent a one, a zero, or any quantum superposition of those two qubit states. A pair of qubits can be in any quantum superposition of 4 states, and three qubits in any superposition of 8 states. A quantum computer with n qubits can generally be in an arbitrary superposition of up to 2^n different states simultaneously, whereas a traditional computer can only be in one of these states at any one time. A quantum Turing machine is a theoretical model of such a computer.

The storage systems described above may also be paired with FPGA-accelerated servers as part of a larger AI or ML infrastructure. Such FPGA-accelerated servers may reside near (e.g., in the same data center) the storage systems described above or even incorporated into an appliance that includes one or more storage systems, one or more FPGA-accelerated servers, networking infrastructure that supports communications between the one or more storage systems and the one or more FPGA-accelerated servers, as well as

other hardware and software components. Alternatively, FPGA-accelerated servers may reside within a cloud computing environment that may be used to perform computer-related tasks for AI and ML jobs. Any of the embodiments described above may be used to collectively serve as a FPGA-based AI or ML platform. Readers will appreciate that, in some embodiments of the FPGA-based AI or ML platform, the FPGAs that are contained within the FPGA-accelerated servers may be reconfigured for different types of ML models (e.g., LSTMs, CNNs, GRUs). The ability to reconfigure the FPGAs that are contained within the FPGA-accelerated servers may enable the acceleration of a ML or AI application based on the most optimal numerical precision and memory model being used. Readers will appreciate that by treating the collection of FPGA-accelerated servers as a pool of FPGAs, any CPU in the data center may utilize the pool of FPGAs as a shared hardware microservice, rather than limiting a server to dedicated accelerators plugged into it.

The FPGA-accelerated servers and the GPU-accelerated servers described above may implement a model of computing where, rather than keeping a small amount of data in a CPU and running a long stream of instructions over it as occurred in more traditional computing models, the machine learning model and parameters are pinned into the high-bandwidth on-chip memory with lots of data streaming through the high-bandwidth on-chip memory. FPGAs may even be more efficient than GPUs for this computing model, as the FPGAs can be programmed with only the instructions needed to run this kind of computing model.

The storage systems described above may be configured to provide parallel storage, for example, through the use of a parallel file system such as BeeGFS. Such parallel file systems may include a distributed metadata architecture. For example, the parallel file system may include a plurality of metadata servers across which metadata is distributed, as well as components that include services for clients and storage servers.

The systems described above can support the execution of a wide array of software applications. Such software applications can be deployed in a variety of ways, including container-based deployment models. Containerized applications may be managed using a variety of tools. For example, containerized applications may be managed using Docker Swarm, Kubernetes, and others. Containerized applications may be used to facilitate a serverless, cloud native computing deployment and management model for software applications. In support of a serverless, cloud native computing deployment and management model for software applications, containers may be used as part of an event handling mechanisms (e.g., AWS Lambdas) such that various events cause a containerized application to be spun up to operate as an event handler.

The systems described above may be deployed in a variety of ways, including being deployed in ways that support fifth generation ('5G') networks. 5G networks may support substantially faster data communications than previous generations of mobile communications networks and, as a consequence may lead to the disaggregation of data and computing resources as modern massive data centers may become less prominent and may be replaced, for example, by more-local, micro data centers that are close to the mobile-network towers. The systems described above may be included in such local, micro data centers and may be part of or paired to multi-access edge computing ('MEC') systems. Such MEC systems may enable cloud computing capabilities and an IT service environment at the edge of the

cellular network. By running applications and performing related processing tasks closer to the cellular customer, network congestion may be reduced and applications may perform better.

The storage systems described above may also be configured to implement NVMe Zoned Namespaces. Through the use of NVMe Zoned Namespaces, the logical address space of a namespace is divided into zones. Each zone provides a logical block address range that must be written sequentially and explicitly reset before rewriting, thereby enabling the creation of namespaces that expose the natural boundaries of the device and offload management of internal mapping tables to the host. In order to implement NVMe Zoned Name Spaces ('ZNS'), ZNS SSDs or some other form of zoned block devices may be utilized that expose a namespace logical address space using zones. With the zones aligned to the internal physical properties of the device, several inefficiencies in the placement of data can be eliminated. In such embodiments, each zone may be mapped, for example, to a separate application such that functions like wear levelling and garbage collection could be performed on a per-zone or per-application basis rather than across the entire device. In order to support ZNS, the storage controllers described herein may be configured with to interact with zoned block devices through the usage of, for example, the Linux™ kernel zoned block device interface or other tools.

The storage systems described above may also be configured to implement zoned storage in other ways such as, for example, through the usage of shingled magnetic recording (SMR) storage devices. In examples where zoned storage is used, device-managed embodiments may be deployed where the storage devices hide this complexity by managing it in the firmware, presenting an interface like any other storage device. Alternatively, zoned storage may be implemented via a host-managed embodiment that depends on the operating system to know how to handle the drive, and only write sequentially to certain regions of the drive. Zoned storage may similarly be implemented using a host-aware embodiment in which a combination of a drive managed and host managed implementation is deployed.

The storage systems described herein may be used to form a data lake. A data lake may operate as the first place that an organization's data flows to, where such data may be in a raw format. Metadata tagging may be implemented to facilitate searches of data elements in the data lake, especially in embodiments where the data lake contains multiple stores of data, in formats not easily accessible or readable (e.g., unstructured data, semi-structured data, structured data). From the data lake, data may go downstream to a data warehouse where data may be stored in a more processed, packaged, and consumable format. The storage systems described above may also be used to implement such a data warehouse. In addition, a data mart or data hub may allow for data that is even more easily consumed, where the storage systems described above may also be used to provide the underlying storage resources necessary for a data mart or data hub. In embodiments, queries the data lake may require a schema-on-read approach, where data is applied to a plan or schema as it is pulled out of a stored location, rather than as it goes into the stored location.

The storage systems described herein may also be configured to implement a recovery point objective ('RPO'), which may be established by a user, established by an administrator, established as a system default, established as part of a storage class or service that the storage system is participating in the delivery of, or in some other way. A

"recovery point objective" is a goal for the maximum time difference between the last update to a source dataset and the last recoverable replicated dataset update that would be correctly recoverable, given a reason to do so, from a continuously or frequently updated copy of the source dataset. An update is correctly recoverable if it properly takes into account all updates that were processed on the source dataset prior to the last recoverable replicated dataset update.

In synchronous replication, the RPO would be zero, meaning that under normal operation, all completed updates on the source dataset should be present and correctly recoverable on the copy dataset. In best effort nearly synchronous replication, the RPO can be as low as a few seconds. In snapshot-based replication, the RPO can be roughly calculated as the interval between snapshots plus the time to transfer the modifications between a previous already transferred snapshot and the most recent to-be-replicated snapshot.

If updates accumulate faster than they are replicated, then an RPO can be missed. If more data to be replicated accumulates between two snapshots, for snapshot-based replication, than can be replicated between taking the snapshot and replicating that snapshot's cumulative updates to the copy, then the RPO can be missed. If, again in snapshot-based replication, data to be replicated accumulates at a faster rate than could be transferred in the time between subsequent snapshots, then replication can start to fall further behind which can extend the miss between the expected recovery point objective and the actual recovery point that is represented by the last correctly replicated update.

The storage systems described above may also be part of a shared nothing storage cluster. In a shared nothing storage cluster, each node of the cluster has local storage and communicates with other nodes in the cluster through networks, where the storage used by the cluster is (in general) provided only by the storage connected to each individual node. A collection of nodes that are synchronously replicating a dataset may be one example of a shared nothing storage cluster, as each storage system has local storage and communicates to other storage systems through a network, where those storage systems do not (in general) use storage from somewhere else that they share access to through some kind of interconnect. In contrast, some of the storage systems described above are themselves built as a shared-storage cluster, since there are drive shelves that are shared by the paired controllers. Other storage systems described above, however, are built as a shared nothing storage cluster, as all storage is local to a particular node (e.g., a blade) and all communication is through networks that link the compute nodes together.

In other embodiments, other forms of a shared nothing storage cluster can include embodiments where any node in the cluster has a local copy of all storage they need, and where data is mirrored through a synchronous style of replication to other nodes in the cluster either to ensure that the data isn't lost or because other nodes are also using that storage. In such an embodiment, if a new cluster node needs some data, that data can be copied to the new node from other nodes that have copies of the data.

In some embodiments, mirror-copy-based shared storage clusters may store multiple copies of all the cluster's stored data, with each subset of data replicated to a particular set of nodes, and different subsets of data replicated to different sets of nodes. In some variations, embodiments may store all of the cluster's stored data in all nodes, whereas in other variations nodes may be divided up such that a first set of

nodes will all store the same set of data and a second, different set of nodes will all store a different set of data.

Readers will appreciate that RAFT-based databases (e.g., etcd) may operate like shared-nothing storage clusters where all RAFT nodes store all data. The amount of data stored in a RAFT cluster, however, may be limited so that extra copies don't consume too much storage. A container server cluster might also be able to replicate all data to all cluster nodes, presuming the containers don't tend to be too large and their bulk data (the data manipulated by the applications that run in the containers) is stored elsewhere such as in an S3 cluster or an external file server. In such an example, the container storage may be provided by the cluster directly through its shared-nothing storage model, with those containers providing the images that form the execution environment for parts of an application or service.

For further explanation, FIG. 3D illustrates an exemplary computing device 350 that may be specifically configured to perform one or more of the processes described herein. As shown in FIG. 3D, computing device 350 may include a communication interface 352, a processor 354, a storage device 356, and an input/output ("I/O") module 358 communicatively connected one to another via a communication infrastructure 360. While an exemplary computing device 350 is shown in FIG. 3D, the components illustrated in FIG. 3D are not intended to be limiting. Additional or alternative components may be used in other embodiments. Components of computing device 350 shown in FIG. 3D will now be described in additional detail.

Communication interface 352 may be configured to communicate with one or more computing devices. Examples of communication interface 352 include, without limitation, a wired network interface (such as a network interface card), a wireless network interface (such as a wireless network interface card), a modem, an audio/video connection, and any other suitable interface.

Processor 354 generally represents any type or form of processing unit capable of processing data and/or interpreting, executing, and/or directing execution of one or more of the instructions, processes, and/or operations described herein. Processor 354 may perform operations by executing computer-executable instructions 362 (e.g., an application, software, code, and/or other executable data instance) stored in storage device 356.

Storage device 356 may include one or more data storage media, devices, or configurations and may employ any type, form, and combination of data storage media and/or device. For example, storage device 356 may include, but is not limited to, any combination of the non-volatile media and/or volatile media described herein. Electronic data, including data described herein, may be temporarily and/or permanently stored in storage device 356. For example, data representative of computer-executable instructions 362 configured to direct processor 354 to perform any of the operations described herein may be stored within storage device 356. In some examples, data may be arranged in one or more databases residing within storage device 356.

I/O module 358 may include one or more I/O modules configured to receive user input and provide user output. I/O module 358 may include any hardware, firmware, software, or combination thereof supportive of input and output capabilities. For example, I/O module 358 may include hardware and/or software for capturing user input, including, but not limited to, a keyboard or keypad, a touchscreen component (e.g., touchscreen display), a receiver (e.g., an RF or infrared receiver), motion sensors, and/or one or more input buttons.

I/O module 358 may include one or more devices for presenting output to a user, including, but not limited to, a graphics engine, a display (e.g., a display screen), one or more output drivers (e.g., display drivers), one or more audio speakers, and one or more audio drivers. In certain embodiments, I/O module 358 is configured to provide graphical data to a display for presentation to a user. The graphical data may be representative of one or more graphical user interfaces and/or any other graphical content as may serve a particular implementation. In some examples, any of the systems, computing devices, and/or other components described herein may be implemented by computing device 350.

For further explanation, FIG. 3E illustrates an example of a fleet of storage systems 376 for providing storage services (also referred to herein as 'data services'). The fleet of storage systems 376 depicted in FIG. 3E includes a plurality of storage systems 374a, 374b, 374c, through 374n that may each be similar to the storage systems described herein. The storage systems 374a, 374b, 374c, through 374n in the fleet of storage systems 376 may be embodied as identical storage systems or as different types of storage systems. For example, two of the storage systems 374a, 374n depicted in FIG. 3E are depicted as being cloud-based storage systems, as the resources that collectively form each of the storage systems 374a, 374n are provided by distinct cloud services providers 370, 372. For example, the first cloud services provider 370 may be Amazon AWS™ whereas the second cloud services provider 372 is Microsoft Azure™, although in other embodiments one or more public clouds, private clouds, or combinations thereof may be used to provide the underlying resources that are used to form a particular storage system in the fleet of storage systems 376.

The example depicted in FIG. 3E includes an edge management service 366 for delivering storage services in accordance with some embodiments of the present disclosure. The storage services (also referred to herein as 'data services') that are delivered may include, for example, services to provide a certain amount of storage to a consumer, services to provide storage to a consumer in accordance with a predetermined service level agreement, services to provide storage to a consumer in accordance with predetermined regulatory requirements, and many others.

The edge management service 366 depicted in FIG. 3E may be embodied, for example, as one or more modules of computer program instructions executing on computer hardware such as one or more computer processors. Alternatively, the edge management service 366 may be embodied as one or more modules of computer program instructions executing on a virtualized execution environment such as one or more virtual machines, in one or more containers, or in some other way. In other embodiments, the edge management service 366 may be embodied as a combination of the embodiments described above, including embodiments where the one or more modules of computer program instructions that are included in the edge management service 366 are distributed across multiple physical or virtual execution environments.

The edge management service 366 may operate as a gateway for providing storage services to storage consumers, where the storage services leverage storage offered by one or more storage systems 374a, 374b, 374c, through 374n. For example, the edge management service 366 may be configured to provide storage services to host devices 378a, 378b, 378c, 378d, 378n that are executing one or more applications that consume the storage services. In such an example, the edge management service 366 may operate as

a gateway between the host devices **378a**, **378b**, **378c**, **378d**, **378n** and the storage systems **374a**, **374b**, **374c**, through **374n**, rather than requiring that the host devices **378a**, **378b**, **378c**, **378d**, **378n** directly access the storage systems **374a**, **374b**, **374c**, through **374n**.

The edge management service **366** of FIG. 3E exposes a storage services module **364** to the host devices **378a**, **378b**, **378c**, **378d**, **378n** of FIG. 3E, although in other embodiments the edge management service **366** may expose the storage services module **364** to other consumers of the various storage services. The various storage services may be presented to consumers via one or more user interfaces, via one or more APIs, or through some other mechanism provided by the storage services module **364**. As such, the storage services module **364** depicted in FIG. 3E may be embodied as one or more modules of computer program instructions executing on physical hardware, on a virtualized execution environment, or combinations thereof, where executing such modules causes enables a consumer of storage services to be offered, select, and access the various storage services.

The edge management service **366** of FIG. 3E also includes a system management services module **368**. The system management services module **368** of FIG. 3E includes one or more modules of computer program instructions that, when executed, perform various operations in coordination with the storage systems **374a**, **374b**, **374c**, through **374n** to provide storage services to the host devices **378a**, **378b**, **378c**, **378d**, **378n**. The system management services module **368** may be configured, for example, to perform tasks such as provisioning storage resources from the storage systems **374a**, **374b**, **374c**, through **374n** via one or more APIs exposed by the storage systems **374a**, **374b**, **374c**, through **374n**, migrating datasets or workloads amongst the storage systems **374a**, **374b**, **374c**, through **374n** via one or more APIs exposed by the storage systems **374a**, **374b**, **374c**, through **374n**, setting one or more tunable parameters (i.e., one or more configurable settings) on the storage systems **374a**, **374b**, **374c**, through **374n** via one or more APIs exposed by the storage systems **374a**, **374b**, **374c**, through **374n**, and so on. For example, many of the services described below relate to embodiments where the storage systems **374a**, **374b**, **374c**, through **374n** are configured to operate in some way. In such examples, the system management services module **368** may be responsible for using APIs (or some other mechanism) provided by the storage systems **374a**, **374b**, **374c**, through **374n** to configure the storage systems **374a**, **374b**, **374c**, through **374n** to operate in the ways described below.

In addition to configuring the storage systems **374a**, **374b**, **374c**, through **374n**, the edge management service **366** itself may be configured to perform various tasks required to provide the various storage services. Consider an example in which the storage service includes a service that, when selected and applied, causes personally identifiable information ('PII') contained in a dataset to be obfuscated when the dataset is accessed. In such an example, the storage systems **374a**, **374b**, **374c**, through **374n** may be configured to obfuscate PII when servicing read requests directed to the dataset. Alternatively, the storage systems **374a**, **374b**, **374c**, through **374n** may service reads by returning data that includes the PII, but the edge management service **366** itself may obfuscate the PII as the data is passed through the edge management service **366** on its way from the storage systems **374a**, **374b**, **374c**, through **374n** to the host devices **378a**, **378b**, **378c**, **378d**, **378n**.

The storage systems **374a**, **374b**, **374c**, through **374n** depicted in FIG. 3E may be embodied as one or more of the storage systems described above with reference to FIGS. 1A-3D, including variations thereof. In fact, the storage systems **374a**, **374b**, **374c**, through **374n** may serve as a pool of storage resources where the individual components in that pool have different performance characteristics, different storage characteristics, and so on. For example, one of the storage systems **374a** may be a cloud-based storage system, another storage system **374b** may be a storage system that provides block storage, another storage system **374c** may be a storage system that provides file storage, another storage system **374d** may be a relatively high-performance storage system while another storage system **374n** may be a relatively low-performance storage system, and so on. In alternative embodiments, only a single storage system may be present.

The storage systems **374a**, **374b**, **374c**, through **374n** depicted in FIG. 3E may also be organized into different failure domains so that the failure of one storage system **374a** should be totally unrelated to the failure of another storage system **374b**. For example, each of the storage systems may receive power from independent power systems, each of the storage systems may be coupled for data communications over independent data communications networks, and so on. Furthermore, the storage systems in a first failure domain may be accessed via a first gateway whereas storage systems in a second failure domain may be accessed via a second gateway. For example, the first gateway may be a first instance of the edge management service **366** and the second gateway may be a second instance of the edge management service **366**, including embodiments where each instance is distinct, or each instance is part of a distributed edge management service **366**.

As an illustrative example of available storage services, storage services may be presented to a user that are associated with different levels of data protection. For example, storage services may be presented to the user that, when selected and enforced, guarantee the user that data associated with that user will be protected such that various recovery point objectives ('RPO') can be guaranteed. A first available storage service may ensure, for example, that some dataset associated with the user will be protected such that any data that is more than 5 seconds old can be recovered in the event of a failure of the primary data store whereas a second available storage service may ensure that the dataset that is associated with the user will be protected such that any data that is more than 5 minutes old can be recovered in the event of a failure of the primary data store.

An additional example of storage services that may be presented to a user, selected by a user, and ultimately applied to a dataset associated with the user can include one or more data compliance services. Such data compliance services may be embodied, for example, as services that may be provided to consumers (i.e., a user) the data compliance services to ensure that the user's datasets are managed in a way to adhere to various regulatory requirements. For example, one or more data compliance services may be offered to a user to ensure that the user's datasets are managed in a way so as to adhere to the General Data Protection Regulation ('GDPR'), one or data compliance services may be offered to a user to ensure that the user's datasets are managed in a way so as to adhere to the Sarbanes-Oxley Act of 2002 ('SOX'), or one or more data compliance services may be offered to a user to ensure that the user's datasets are managed in a way so as to adhere to

some other regulatory act. In addition, the one or more data compliance services may be offered to a user to ensure that the user's datasets are managed in a way so as to adhere to some non-governmental guidance (e.g., to adhere to best practices for auditing purposes), the one or more data compliance services may be offered to a user to ensure that the user's datasets are managed in a way so as to adhere to a particular clients or organizations requirements, and so on.

In order to provide this particular data compliance service, the data compliance service may be presented to a user (e.g., via a GUI) and selected by the user. In response to receiving the selection of the particular data compliance service, one or more storage services policies may be applied to a dataset associated with the user to carry out the particular data compliance service. For example, a storage services policy may be applied requiring that the dataset be encrypted prior to being stored in a storage system, prior to being stored in a cloud environment, or prior to being stored elsewhere. In order to enforce this policy, a requirement may be enforced not only requiring that the dataset be encrypted when stored, but a requirement may be put in place requiring that the dataset be encrypted prior to transmitting the dataset (e.g., sending the dataset to another party). In such an example, a storage services policy may also be put in place requiring that any encryption keys used to encrypt the dataset are not stored on the same system that stores the dataset itself. Readers will appreciate that many other forms of data compliance services may be offered and implemented in accordance with embodiments of the present disclosure.

The storage systems **374a**, **374b**, **374c**, through **374n** in the fleet of storage systems **376** may be managed collectively, for example, by one or more fleet management modules. The fleet management modules may be part of or separate from the system management services module **368** depicted in FIG. 3E. The fleet management modules may perform tasks such as monitoring the health of each storage system in the fleet, initiating updates or upgrades on one or more storage systems in the fleet, migrating workloads for loading balancing or other performance purposes, and many other tasks. As such, and for many other reasons, the storage systems **374a**, **374b**, **374c**, through **374n** may be coupled to each other via one or more data communications links in order to exchange data between the storage systems **374a**, **374b**, **374c**, through **374n**.

In some embodiments, one or more storage systems or one or more elements of storage systems (e.g., features, services, operations, components, etc. of storage systems), such as any of the illustrative storage systems or storage system elements described herein may be implemented in one or more container systems. A container system may include any system that supports execution of one or more containerized applications or services. Such a service may be software deployed as infrastructure for building applications, for operating a run-time environment, and/or as infrastructure for other services. In the discussion that follows, descriptions of containerized applications generally apply to containerized services as well.

A container may combine one or more elements of a containerized software application together with a runtime environment for operating those elements of the software application bundled into a single image. For example, each such container of a containerized application may include executable code of the software application and various dependencies, libraries, and/or other components, together with network configurations and configured access to additional resources, used by the elements of the software application within the particular container in order to enable

operation of those elements. A containerized application can be represented as a collection of such containers that together represent all the elements of the application combined with the various run-time environments needed for all those elements to run. As a result, the containerized application may be abstracted away from host operating systems as a combined collection of lightweight and portable packages and configurations, where the containerized application may be uniformly deployed and consistently executed in different computing environments that use different container-compatible operating systems or different infrastructures. In some embodiments, a containerized application shares a kernel with a host computer system and executes as an isolated environment (an isolated collection of files and directories, processes, system and network resources, and configured access to additional resources and capabilities) that is isolated by an operating system of a host system in conjunction with a container management framework. When executed, a containerized application may provide one or more containerized workloads and/or services.

The container system may include and/or utilize a cluster of nodes. For example, the container system may be configured to manage deployment and execution of containerized applications on one or more nodes in a cluster. The containerized applications may utilize resources of the nodes, such as memory, processing and/or storage resources provided and/or accessed by the nodes. The storage resources may include any of the illustrative storage resources described herein and may include on-node resources such as a local tree of files and directories, off-node resources such as external networked file systems, databases or object stores, or both on-node and off-node resources. Access to additional resources and capabilities that could be configured for containers of a containerized application could include specialized computation capabilities such as GPUs and AI/ML engines, or specialized hardware such as sensors and cameras.

In some embodiments, the container system may include a container orchestration system (which may also be referred to as a container orchestrator, a container orchestration platform, etc.) designed to make it reasonably simple and for many use cases automated to deploy, scale, and manage containerized applications. In some embodiments, the container system may include a storage management system configured to provision and manage storage resources (e.g., virtual volumes) for private or shared use by cluster nodes and/or containers of containerized applications.

FIG. 3F illustrates an example container system **380**. In this example, the container system **380** includes a container storage system **381** that may be configured to perform one or more storage management operations to organize, provision, and manage storage resources for use by one or more containerized applications **382-1** through **382-L** of container system **380**. In particular, the container storage system **381** may organize storage resources into one or more storage pools **383** of storage resources for use by containerized applications **382-1** through **382-L**. The container storage system may itself be implemented as a containerized service.

The container system **380** may include or be implemented by one or more container orchestration systems, including Kubernetes™, Mesos™, Docker Swarm™, among others. The container orchestration system may manage the container system **380** running on a cluster **384** through services implemented by a control node, depicted as **385**, and may further manage the container storage system or the relationship between individual containers and their storage,

memory and CPU limits, networking, and their access to additional resources or services.

A control plane of the container system **380** may implement services that include: deploying applications via a controller **386**, monitoring applications via the controller **386**, providing an interface via an API server **387**, and scheduling deployments via scheduler **388**. In this example, controller **386**, scheduler **388**, API server **387**, and container storage system **381** are implemented on a single node, node **385**. In other examples, for resiliency, the control plane may be implemented by multiple, redundant nodes, where if a node that is providing management services for the container system **380** fails, then another, redundant node may provide management services for the cluster **384**.

A data plane of the container system **380** may include a set of nodes that provides container runtimes for executing containerized applications. An individual node within the cluster **384** may execute a container runtime, such as Docker™, and execute a container manager, or node agent, such as a kubelet in Kubernetes (not depicted) that communicates with the control plane via a local network-connected agent (sometimes called a proxy), such as an agent **389**. The agent **389** may route network traffic to and from containers using, for example, Internet Protocol (IP) port numbers. For example, a containerized application may request a storage class from the control plane, where the request is handled by the container manager, and the container manager communicates the request to the control plane using the agent **389**.

Cluster **384** may include a set of nodes that run containers for managed containerized applications. A node may be a virtual or physical machine. A node may be a host system.

The container storage system **381** may orchestrate storage resources to provide storage to the container system **380**. For example, the container storage system **381** may provide persistent storage to containerized applications **382-1-382-L** using the storage pool **383**. The container storage system **381** may itself be deployed as a containerized application by a container orchestration system.

For example, the container storage system **381** application may be deployed within cluster **384** and perform management functions for providing storage to the containerized applications **382**. Management functions may include determining one or more storage pools from available storage resources, provisioning virtual volumes on one or more nodes, replicating data, responding to and recovering from host and network faults, or handling storage operations. The storage pool **383** may include storage resources from one or more local or remote sources, where the storage resources may be different types of storage, including, as examples, block storage, file storage, and object storage.

The container storage system **381** may also be deployed on a set of nodes for which persistent storage may be provided by the container orchestration system. In some examples, the container storage system **381** may be deployed on all nodes in a cluster **384** using, for example, a Kubernetes DaemonSet. In this example, nodes **390-1** through **390-N** provide a container runtime where container storage system **381** executes. In other examples, some, but not all nodes in a cluster may execute the container storage system **381**.

The container storage system **381** may handle storage on a node and communicate with the control plane of container system **380**, to provide dynamic volumes, including persistent volumes. A persistent volume may be mounted on a node as a virtual volume, such as virtual volumes **391-1** and **391-P**. After a virtual volume **391** is mounted, containerized applications may request and use, or be otherwise config-

ured to use, storage provided by the virtual volume **391**. In this example, the container storage system **381** may install a driver on a kernel of a node, where the driver handles storage operations directed to the virtual volume. In this example, the driver may receive a storage operation directed to a virtual volume, and in response, the driver may perform the storage operation on one or more storage resources within the storage pool **383**, possibly under direction from or using additional logic within containers that implement the container storage system **381** as a containerized service.

The container storage system **381** may, in response to being deployed as a containerized service, determine available storage resources. For example, storage resources **392-1** through **392-M** may include local storage, remote storage (storage on a separate node in a cluster), or both local and remote storage. Storage resources may also include storage from external sources such as various combinations of block storage systems, file storage systems, and object storage systems. The storage resources **392-1** through **392-M** may include any type(s) and/or configuration(s) of storage resources (e.g., any of the illustrative storage resources described above), and the container storage system **381** may be configured to determine the available storage resources in any suitable way, including based on a configuration file. For example, a configuration file may specify account and authentication information for cloud-based object storage **348** or for a cloud-based storage system **318**. The container storage system **381** may also determine availability of one or more storage devices **356** or one or more storage systems. An aggregate amount of storage from one or more of storage device(s) **356**, storage system(s), cloud-based storage system(s) **318**, edge management services **366**, cloud-based object storage **348**, or any other storage resources, or any combination or sub-combination of such storage resources may be used to provide the storage pool **383**. The storage pool **383** is used to provision storage for the one or more virtual volumes mounted on one or more of the nodes **390** within cluster **384**.

In some implementations, the container storage system **381** may create multiple storage pools. For example, the container storage system **381** may aggregate storage resources of a same type into an individual storage pool. In this example, a storage type may be one of: a storage device **356**, a storage array **102**, a cloud-based storage system **318**, storage via an edge management service **366**, or a cloud-based object storage **348**. Or it could be storage configured with a certain level or type of redundancy or distribution, such as a particular combination of striping, mirroring, or erasure coding.

The container storage system **381** may execute within the cluster **384** as a containerized container storage system service, where instances of containers that implement elements of the containerized container storage system service may operate on different nodes within the cluster **384**. In this example, the containerized container storage system service may operate in conjunction with the container orchestration system of the container system **380** to handle storage operations, mount virtual volumes to provide storage to a node, aggregate available storage into a storage pool **383**, provision storage for a virtual volume from a storage pool **383**, generate backup data, replicate data between nodes, clusters, environments, among other storage system operations. In some examples, the containerized container storage system service may provide storage services across multiple clusters operating in distinct computing environments. For example, other storage system operations may include storage system operations described herein. Persistent storage provided by

the containerized container storage system service may be used to implement stateful and/or resilient containerized applications.

The container storage system **381** may be configured to perform any suitable storage operations of a storage system. For example, the container storage system **381** may be configured to perform one or more of the illustrative storage management operations described herein to manage storage resources used by the container system.

In some embodiments, one or more storage operations, including one or more of the illustrative storage management operations described herein, may be containerized. For example, one or more storage operations may be implemented as one or more containerized applications configured to be executed to perform the storage operation(s). Such containerized storage operations may be executed in any suitable runtime environment to manage any storage system(s), including any of the illustrative storage systems described herein.

The storage systems described herein may support various forms of data replication. For example, two or more of the storage systems may synchronously replicate a dataset between each other. In synchronous replication, distinct copies of a particular dataset may be maintained by multiple storage systems, but all accesses (e.g., a read) of the dataset should yield consistent results regardless of which storage system the access was directed to. For example, a read directed to any of the storage systems that are synchronously replicating the dataset should return identical results. As such, while updates to the version of the dataset need not occur at exactly the same time, precautions must be taken to ensure consistent accesses to the dataset. For example, if an update (e.g., a write) that is directed to the dataset is received by a first storage system, the update may only be acknowledged as being completed if all storage systems that are synchronously replicating the dataset have applied the update to their copies of the dataset. In such an example, synchronous replication may be carried out through the use of I/O forwarding (e.g., a write received at a first storage system is forwarded to a second storage system), communications between the storage systems (e.g., each storage system indicating that it has completed the update), or in other ways.

In other embodiments, a dataset may be replicated through the use of checkpoints. In checkpoint-based replication (also referred to as 'nearly synchronous replication'), a set of updates to a dataset (e.g., one or more write operations directed to the dataset) may occur between different checkpoints, such that a dataset has been updated to a specific checkpoint only if all updates to the dataset prior to the specific checkpoint have been completed. Consider an example in which a first storage system stores a live copy of a dataset that is being accessed by users of the dataset. In this example, assume that the dataset is being replicated from the first storage system to a second storage system using checkpoint-based replication. For example, the first storage system may send a first checkpoint (at time $t=0$) to the second storage system, followed by a first set of updates to the dataset, followed by a second checkpoint (at time $t=1$), followed by a second set of updates to the dataset, followed by a third checkpoint (at time $t=2$). In such an example, if the second storage system has performed all updates in the first set of updates but has not yet performed all updates in the second set of updates, the copy of the dataset that is stored on the second storage system may be up-to-date until the second checkpoint. Alternatively, if the second storage system has performed all updates in both the first set of updates

and the second set of updates, the copy of the dataset that is stored on the second storage system may be up-to-date until the third checkpoint. Readers will appreciate that various types of checkpoints may be used (e.g., metadata only checkpoints), checkpoints may be spread out based on a variety of factors (e.g., time, number of operations, an RPO setting), and so on.

In other embodiments, a dataset may be replicated through snapshot-based replication (also referred to as 'asynchronous replication'). In snapshot-based replication, snapshots of a dataset may be sent from a replication source such as a first storage system to a replication target such as a second storage system. In such an embodiment, each snapshot may include the entire dataset or a subset of the dataset such as, for example, only the portions of the dataset that have changed since the last snapshot was sent from the replication source to the replication target. Readers will appreciate that snapshots may be sent on-demand, based on a policy that takes a variety of factors into consideration (e.g., time, number of operations, an RPO setting), or in some other way.

The storage systems described above may, either alone or in combination, be configured to serve as a continuous data protection store. A continuous data protection store is a feature of a storage system that records updates to a dataset in such a way that consistent images of prior contents of the dataset can be accessed with a low time granularity (often on the order of seconds, or even less), and stretching back for a reasonable period of time (often hours or days). These allow access to very recent consistent points in time for the dataset, and also allow access to points in time for a dataset that might have just preceded some event that, for example, caused parts of the dataset to be corrupted or otherwise lost, while retaining close to the maximum number of updates that preceded that event. Conceptually, they are like a sequence of snapshots of a dataset taken very frequently and kept for a long period of time, though continuous data protection stores are often implemented quite differently from snapshots. A storage system implementing a data continuous data protection store may further provide a means of accessing these points in time, accessing one or more of these points in time as snapshots or as cloned copies, or reverting the dataset back to one of those recorded points in time.

Over time, to reduce overhead, some points in the time held in a continuous data protection store can be merged with other nearby points in time, essentially deleting some of these points in time from the store. This can reduce the capacity needed to store updates. It may also be possible to convert a limited number of these points in time into longer duration snapshots. For example, such a store might keep a low granularity sequence of points in time stretching back a few hours from the present, with some points in time merged or deleted to reduce overhead for up to an additional day. Stretching back in the past further than that, some of these points in time could be converted to snapshots representing consistent point-in-time images from only every few hours.

Although some embodiments are described largely in the context of a storage system, readers of skill in the art will recognize that embodiments of the present disclosure may also take the form of a computer program product disposed upon computer readable storage media for use with any suitable processing system. Such computer readable storage media may be any storage medium for machine-readable information, including magnetic media, optical media, solid-state media, or other suitable media. Examples of such media include magnetic disks in hard drives or diskettes,

compact disks for optical drives, magnetic tape, and others as will occur to those of skill in the art. Persons skilled in the art will immediately recognize that any computer system having suitable programming means will be capable of executing the steps described herein as embodied in a computer program product. Persons skilled in the art will recognize also that, although some of the embodiments described in this specification are oriented to software installed and executing on computer hardware, nevertheless, alternative embodiments implemented as firmware or as hardware are well within the scope of the present disclosure.

In some examples, a non-transitory computer-readable medium storing computer-readable instructions may be provided in accordance with the principles described herein. The instructions, when executed by a processor of a computing device, may direct the processor and/or computing device to perform one or more operations, including one or more of the operations described herein. Such instructions may be stored and/or transmitted using any of a variety of known computer-readable media.

A non-transitory computer-readable medium as referred to herein may include any non-transitory storage medium that participates in providing data (e.g., instructions) that may be read and/or executed by a computing device (e.g., by a processor of a computing device). For example, a non-transitory computer-readable medium may include, but is not limited to, any combination of non-volatile storage media and/or volatile storage media. Exemplary non-volatile storage media include, but are not limited to, read-only memory, flash memory, a solid-state drive, a magnetic storage device (e.g., a hard disk, a floppy disk, magnetic tape, etc.), ferroelectric random-access memory ("RAM"), and an optical disc (e.g., a compact disc, a digital video disc, a Blu-ray disc, etc.). Exemplary volatile storage media include, but are not limited to, RAM (e.g., dynamic RAM).

The storage systems described herein may support various forms of data replication. For example, two or more of the storage systems may synchronously replicate a dataset between each other. In synchronous replication, distinct copies of a particular dataset may be maintained by multiple storage systems, but all accesses (e.g., a read) of the dataset should yield consistent results regardless of which storage system the access was directed to. For example, a read directed to any of the storage systems that are synchronously replicating the dataset should return identical results. As such, while updates to the version of the dataset need not occur at exactly the same time, precautions must be taken to ensure consistent accesses to the dataset. For example, if an update (e.g., a write) that is directed to the dataset is received by a first storage system, the update may only be acknowledged as being completed if all storage systems that are synchronously replicating the dataset have applied the update to their copies of the dataset. In such an example, synchronous replication may be carried out through the use of I/O forwarding (e.g., a write received at a first storage system is forwarded to a second storage system), communications between the storage systems (e.g., each storage system indicating that it has completed the update), or in other ways.

In other embodiments, a dataset may be replicated through the use of checkpoints. In checkpoint-based replication (also referred to as 'nearly synchronous replication'), a set of updates to a dataset (e.g., one or more write operations directed to the dataset) may occur between different checkpoints, such that a dataset has been updated to a specific checkpoint only if all updates to the dataset prior to the specific checkpoint have been completed. Consider an

example in which a first storage system stores a live copy of a dataset that is being accessed by users of the dataset. In this example, assume that the dataset is being replicated from the first storage system to a second storage system using checkpoint-based replication. For example, the first storage system may send a first checkpoint (at time $t=0$) to the second storage system, followed by a first set of updates to the dataset, followed by a second checkpoint (at time $t=1$), followed by a second set of updates to the dataset, followed by a third checkpoint (at time $t=2$). In such an example, if the second storage system has performed all updates in the first set of updates but has not yet performed all updates in the second set of updates, the copy of the dataset that is stored on the second storage system may be up-to-date until the second checkpoint. Alternatively, if the second storage system has performed all updates in both the first set of updates and the second set of updates, the copy of the dataset that is stored on the second storage system may be up-to-date until the third checkpoint. Readers will appreciate that various types of checkpoints may be used (e.g., metadata only checkpoints), checkpoints may be spread out based on a variety of factors (e.g., time, number of operations, an RPO setting), and so on.

In other embodiments, a dataset may be replicated through snapshot-based replication (also referred to as 'asynchronous replication'). In snapshot-based replication, snapshots of a dataset may be sent from a replication source such as a first storage system to a replication target such as a second storage system. In such an embodiment, each snapshot may include the entire dataset or a subset of the dataset such as, for example, only the portions of the dataset that have changed since the last snapshot was sent from the replication source to the replication target. Readers will appreciate that snapshots may be sent on-demand, based on a policy that takes a variety of factors into consideration (e.g., time, number of operations, an RPO setting), or in some other way.

The storage systems described above may, either alone or in combination, be configured to serve as a continuous data protection store. A continuous data protection store is a feature of a storage system that records updates to a dataset in such a way that consistent images of prior contents of the dataset can be accessed with a low time granularity (often on the order of seconds, or even less), and stretching back for a reasonable period of time (often hours or days). These allow access to very recent consistent points in time for the dataset, and also allow access to points in time for a dataset that might have just preceded some event that, for example, caused parts of the dataset to be corrupted or otherwise lost, while retaining close to the maximum number of updates that preceded that event. Conceptually, they are like a sequence of snapshots of a dataset taken very frequently and kept for a long period of time, though continuous data protection stores are often implemented quite differently from snapshots. A storage system implementing a data continuous data protection store may further provide a means of accessing these points in time, accessing one or more of these points in time as snapshots or as cloned copies, or reverting the dataset back to one of those recorded points in time.

Over time, to reduce overhead, some points in the time held in a continuous data protection store can be merged with other nearby points in time, essentially deleting some of these points in time from the store. This can reduce the capacity needed to store updates. It may also be possible to convert a limited number of these points in time into longer duration snapshots. For example, such a store might keep a

61

low granularity sequence of points in time stretching back a few hours from the present, with some points in time merged or deleted to reduce overhead for up to an additional day. Stretching back in the past further than that, some of these points in time could be converted to snapshots representing consistent point-in-time images from only every few hours.

As discussed above, storage systems may employ an object storage architecture where data is managed as objects. An object can be a file, a data chunk, a directory, unstructured data, structured data, portions of structured or unstructured data, and so on. The object may include, for example, the data itself, metadata for the object, and an object key that is a unique identifier for the object within an object store. An object store may be a logical container, also referred to as a 'bucket,' for storing objects within a namespace. The bucket may be implemented on underlying physical storage resources such as flash or solid-state storage. In some implementations, an update to an object causes a new instance of the object to be written to storage resources at a new physical location, thus leaving the original instance of the object intact. These multiple instances of the same object are managed as 'versions' of that object and are associated with the same object key of the object. Metadata for the object may indicate the physical location in the storage resources where each version of the object is stored.

A version stack for the object represents an ordering of the versions of the object. In one example, the version stack is ordered based on a timestamp indicating the time that the version was created on the storage system. However, timestamp ordering may lead to inconsistencies where a bucket is synchronized across multiple storage systems. For example, a multisite bucket allows objects to be written to the same bucket, or paired replicated buckets, through storage systems at multiple locations. These storage systems may employ bidirectional replication to ensure that each storage system maintains a consistent copy of the contents of the bucket or the paired replicated buckets. The phrases "paired replicated bucket" and "multi-site bucket" and "replicated bucket" are generally used interchangeably in this specification, unless a distinction is specifically called out. In some cases, without safeguards in place it is possible that when an older version of an object is replicated from a first storage system to a second storage system, the version stack of the second storage system may place the older version on top of a newer version already present on the second storage system. This could result in inconsistent version stacks between the two storage systems. Consider an example where a first storage system receives a request to store version 1 of an object and soon after, but before version 1 is successfully replicated, a second storage system receives a request to store a version 2 of the object and where storage systems simply order versions in the order they were received. With bidirectional replication, each storage system replicates its versions to the other, so version 1 of the object is replicated from the first storage system to the second storage system and version 2 of the object is replicated from the second storage system to the first storage system. The first storage system orders version 2 after version 1 because it received version 2 later. However, the second storage system orders version 1 after version 2 because it received version 1 later. As such, the version stacks are inconsistent, even though both storage systems are consistent in their inclusion of all versions of the object.

To address this issue, storage systems implementing a multisite bucket may use the original creation time of the version to order their version stacks. In this case, the creation time is the timestamp assigned to the version by the storage

62

system that first stored the version. Thus, the version stack for the object represents an ordering of the versions of the object based on the creation time of each version, where the most-recently created version is at the 'top' of the stack.

Accordingly, a GET request for the object will return the most-recent version of that object in bucket. Consider an example, like above, where the first storage system creates version 1 at time T and the second storage system creates version 2 at time T+5, such that version 1 has a creation timestamp T and version 2 has a creation timestamp T+5. Upon replication at time T+10, each storage system reads the creation timestamp provided for the object, for example, from metadata of the object. This creation timestamp is utilized in the version stacks of both storage systems to order the version stack. In this case, version 2 is placed on top of version 1 in the version stacks of both storage systems, resulting in consistent version stacks.

However, using simple creation times to order version stacks may not be enough to preserve the logical write order of the versions in the presence of clock skew between the storage systems. That is, the creation timestamps are reflective of the locally observed time on each storage system. Thus, when the local clock on one storage system is significantly behind the local clock of the other storage system, the creation timestamps of those versions may not reflect the actual order in which the versions were created. Consider an example the second storage system writes version 2 N milliseconds seconds after the first storage system creates version 1. If the second storage system's local clock C_2 is more than N milliseconds behind the first storage system's local clock C_1 , the creation timestamp for version 2 will be earlier than that of version 1 even though version 2 was created logically after version 1. Thus, logical write ordering is not preserved in the version stacks for the object. Even if the local clocks were synchronized or otherwise coordinated, a clock drift between the storage systems could lead to inconsistencies in the face of a communications disruption that prevents clock coordination. To address the foregoing, in accordance with some embodiments of the present disclosure, the storage systems may agree on an amount of time that should separate successive writes to a replicated object store to ensure that any timestamp given to an object update will be later than any logically preceding object update. This amount of time should be large enough to account for plausible sources of differences between the two storage systems' clocks.

Ensuring that two clocks are nearly identical is in general a hard problem. Getting extremely close (such as within a few microseconds) often requires special hardware, such as GPS receivers, and can even require accounting for time dilation due to the placement of separate data centers and the orbital locations of various GPS satellites. More typical is to use protocols such as NTP (Network Time Protocol) to synchronize clocks with specific time sources, though such synchronization is only accurate to within a few tens to a few hundred milliseconds and is potentially subject to configuration errors as well as clock drift if an NTP server is unreachable for an extended time period. Further, NTP can speed up or slow down clocks temporarily to bring a particular system's clock back into proper synchronization.

For further explanation, FIG. 4 sets forth a flowchart illustrating an example method of establishing a guarantee for maintaining a replication relationship between object stores during a communications outage according to some embodiments of the present disclosure. The storage systems 408, 410, 412 depicted in FIG. 4 may be similar to the storage systems described in the previous figures, as each

storage system **408**, **410**, **412** may include any combination of the components as described with reference to the other figures described herein.

In some examples, the storage systems **408**, **410**, **412** are provided by computing and storage resources of different datacenters (e.g., private enterprise datacenters or cloud service provider datacenters) in different localities **480**, **482**, **484**. Thus, the computing and storage resources are isolated and separated by some physical distance. In some scenarios, the different localities **480**, **482**, **484** can correspond to different availability zones. For example, the different availability zones may be AWS availability zones or those of another cloud services provider. The localities **480**, **482**, **484** may be organized according to a geographic region. For example, the different geographic regions may be AWS regions or those of another cloud services provider. Thus, various examples, the storage systems **408**, **410**, **412** may be distributed across availability zones and/or regions. To aid illustration, one storage system **408** may be provided in a North American region while another storage system **410** may be provided in a European region, whereas storage system **408** and storage system **412** may be provided in different availability zones of the same region. As will be made apparent in the following description, the physical distance separating two replicating storage systems may influence the mechanisms utilized to achieve consistency among the replicated storage resources. Availability zones within the same region tend to be relatively near to each other, often just across the street or perhaps within a few kilometers, which can facilitate synchronous replication, where the storing of an object is replicated before the request to store the object is signaled as completed.

In the example of FIG. 4, each storage system **408**, **410**, **412** includes object-based storage resources such as object stores **414**, **416**, **418**. For example, the object stores **414**, **416** may be an Amazon S3™ bucket or an object store using a protocol derived from or compatible with S3, or that has some similar characteristics to S3 (for example, Microsoft's Azure Blob Storage™). The local object stores **414**, **416**, **418** are local copies of objects of a replicated object store **440** that is replicated among storage systems **408**, **410**, **412** at separate locations. In some examples, the local object stores **414**, **416**, **418** are distributed across different availability zones and/or different regions of a cloud services provider. Thus, the replicated object store **440** may be a multisite bucket that is distributed across two or more availability zones and/or two or more regions, where the storage systems in each location are configured for bidirectional replication with storage systems in other locations of local updates to their local object stores. Although FIG. 4 and the following examples are described in the context of object stores, it is not a requirement of this disclosure that the storage resources are object stores. It will be understood, for example, that principles of the present disclosure may be applied equivalently to file systems or to future storage system technologies that inherit traits from object stores, file stores, or other types of storage.

An object store primarily stores immutable content. Content is immutable if it cannot be modified except by deletion or replacement. In object stores, the content of an object is immutable in that once created it can only be deleted, or replaced with a new version of the object where that new version is identifiably different from the prior version. Immutable content in an object store can be formed by an operation to store specific content (such as through an operation to PUT an object) or by copying other immutable content (such as through an operation to copy another

object). Two PUTs of objects of the same name result in different versions of the object which are identifiably different. In a versioned bucket, two PUTs of objects of the same name generally result in two versions being stored for one object of that name, where one of the two objects is considered current. In a non-versioned bucket, two PUTs of objects of the same name will result in the content from one of the two PUTs being discarded due to the first object being replaced by the second.

Immutable content simplifies the operation of object stores, and the replication of object stores, because once a source of content has been identified and tied to a particular variant of that content, modifications to that content will not complicate further operations related to completing an operation, or replication or tiering or fault recovery or any other clustering or administrative tasks related to the operating the dataset. The remaining complexities relate to ensuring that versions or replacements of objects are handled consistently by the storage systems storing copies of the object store, and ensuring that existing content used to establish new content (such as by copying or by inclusion in a new composite object) uses a consistent source of immutable content for establishing the new immutable content.

A variety of replication mechanisms may be utilized to provide the replicated object store **440**. In some implementations, the storage systems **408**, **410**, **412** are configured for directional replication of the replicated object store **440**. For example, the storage systems **408**, **410**, **412** are configured to replicate updates made to one local object store **414** to the other separately located local object store **416**. In such implementations, one storage system **410** may be a primary storage system that provides an application layer with an active data path to the replicated object store **440** while the other storage systems **408**, **412** may be a secondary storage system that does not provide an active data path to the replicated object store **440**. The application layer and/or the storage systems themselves may initiate a reversal of the replication direction, for example, in response to a communications disruption. In other implementations, the storage systems **408**, **410**, **412** are configured to provide symmetric access to the replicated object store **440**, in that all storage systems **408**, **410**, **412** provide an active data path to the replicated object store **440**. In such implementations, the storage systems **408**, **410**, **412** are configured for bidirectional replication of the replicated object store **440**. For example, updates made to any local object store **414**, **416**, **418** are replicated to the other object stores **414**, **416**, **418** that are replication targets. The application layer and/or the storage systems themselves may initiate a switch to directional replication, for example, in response to a communications disruption. Further, the storage systems **408**, **410**, **412** may employ synchronous or asynchronous replication of updates to the replicated object store **440**. Whether synchronous or asynchronous replication is used may depend on whether updates are replicated within an availability zone, across availability zones, or across regions. For example, storage systems replicating between separate availability zones of the same region might employ synchronous replication while storage systems replicating between regions might employ asynchronous replication. To aid illustration, storage systems **408**, **410** replicating across regions might employ asynchronous replication for the replicated object store **440** while storage systems **408**, **412** replicating across availability zones within the same region might employ synchronous replication.

In the example of FIG. 4, the local object stores **414**, **416**, **418** include local instances of an object that is uniquely

identified by an object key across the storage systems **408**, **410**, **412**. As described above, an object store may be implemented on physical storage such that an update to an object causes a new version of the object to be written to a new set of storage locations, thus leaving the original version of the object intact. Thus, each local instance of the object can embody multiple versions of the object. With symmetric access, each storage system **408**, **410**, **412** may receive requests to modify the object, and may modify its local instance in response to requests. Such modification of the local instance of the object may result in a new version of the object that is initially stored locally on the storage system that received the modification request. In some examples, the storage systems **408**, **410**, **412** achieve consistency with respect to the object through bidirectional replication of local versions of the local instances of the object. That is, a version that is created on one storage system **410** is replicated to the other storage system **412**, and vice versa.

When the storage systems **408**, **410**, **412** are symmetrically replicating storage systems, conflicts and ordering inconsistencies may arise when applications are modifying the same object through two different storage systems **408**, **410**, **412**. For example, each storage system **408**, **410**, **412** may receive a request to store a new object having the same object key. In another example, each storage system **408**, **410**, **412** may receive a request to update the same object, resulting in different versions of the same object being written to different storage systems **408**, **410**, **412**. In some examples, the storage systems **408**, **410**, **412** each maintain a version stack for an object. The version stack can be ordered based on the timestamp for the version of the object, such that the most-recent version of the object is on the top of the object's version stack. The version stack may be embodied in metadata that associates each version of the object on the storage system with the underlying physical storage locations. In a particular example, a simple command to read an object (e.g., GET obj A) will return the most-recent version of the object (e.g., version 4 of obj A) at the top of the stack, whereas a more nuanced command (e.g., GET obj A, v3) will return the requested version of the object (e.g., version 3 of obj A). It is therefore important that the storage systems **408**, **410**, **412** maintain consistent version stacks so that a request to read an object will read the same version of the object regardless of which storage system receives the request, at least once all versions are exchanged with their paired replicated storage systems. Thus, where versioning among the storage systems **408**, **410**, **412** relies on timestamps for those versions, a mechanism for timestamp or clock coordination is necessary to provide consistent ordering for those versions across the storage systems **408**, **410**, **412**.

The method of FIG. 4 includes identifying **402**, by a first storage system **410**, respective local clock values **405**, **407** of one or more second storage systems **408**, **412**, wherein the first storage system **410** and the one or more second storage systems **408**, **412** are among a plurality of storage systems **408**, **410**, **412** replicating objects of an object store **440**. The ordering of updates can depend on the timestamps accorded to those updates by the storage system that receives the write request. Respective local clocks **462**, **464**, **466** of the storage system **408**, **410**, **412** are used to establish timestamps for objects or versions of objects written to the object stores **414**, **416**, **418**, where each new object or new version of an object receives a timestamp when it is stored on the storage system **408**, **410**, **412**. In some implementations, the local clocks **462**, **464**, **466** are monotonic clocks that advance at

the same rate. Such clocks are abstract in that they are independent of a system clock or 'wall clock'. In other examples, the local clock **462**, **464**, **466** is the storage system's system clock. For example, each storage system's system clock may be synchronized with an external clock source in accordance with network time protocol (NTP). However, it should be noted that the use of a monotonic clock avoids potential problems relating to storage systems resynchronizing their local clocks based on NTP, where the local clock of a storage system may jump to a new time or advance at a faster or slower rate in order to resynchronize its clock. Readers will appreciate that a variety of other mechanisms not discussed here may be used to implement the local clock **462**, **464**, **466** useful in establishing time-stamps of objects in the object stores **414**, **416**, **418**.

In some examples, a storage controller of the storage system **410** identifies **402** the respective local clock values **405**, **407** of the other storage systems **408**, **412** through messaging with those storage systems to determine the values of their local clocks (e.g., local monotonic clocks). For example, the storage system **410** may poll other storage systems **412** by sending a request for a local clock value to other storage systems **412**. In such an example, the storage system **410** may identify its own local clock value **403** and include that in the request message to the other storage systems **412**. In response to the request, the other storage systems **408**, **412** may respond with their own local clock values **405**, **407**. In some examples, local clock values **403**, **405**, **407** of the storage systems **408**, **410**, **412** are exchanged periodically. In some examples, all of the storage systems **408**, **410**, **412** replicating the object store **440** exchange respective local clock values **403**, **405**, **407** with one another. In other examples, one storage system **410** is designated as a reference system and local clock values are exchanged only between the reference system and the other storage systems **408**, **412**.

The method of FIG. 4 also includes determining **404**, by the first storage system **410** in dependence upon the respective local clock values **405**, **407**, respective clock differences **409**, **411** of the one or more second storage systems **408**, **412** relative to the first storage system **410**. In some examples, a storage system **410** compares its local clock value **403** to the received local clock values **405**, **407** to identify the differences between its local clock value **403** and the received clock values **405**, **407**. The storage system **410** then 'coordinates' its local clock **462** with the local clocks **464**, **466** of the other storage system by storing a difference **409**, **411** corresponding to each storage system **408**, **412** and then adjusting for that difference **409**, **411** when receiving updates replicated from the other storage systems **408**, **412**.

The method of FIG. 4 also includes ordering **406**, by the first storage system **410**, one or more updates **413** to the replicated object store **440** in dependence upon the respective clock differences **409**, **411**. As discussed above, a storage system **410** orders updates to an object stored in its local object store **414** based on timestamps of those updates. When the storage system **410** receives an update **413** replicated from another storage system **408**, the storage system adjusts the timestamp of included in the update **413** based on the determined difference **409** between its local clock **462** and the local clock **466** of the sending storage system. The update **413** is then stored in the local object store **414** with the adjusted timestamp. When the update **413** is a new version of an existing object, the update **413** will be correctly ordered with respect to other versions of that object that may be received by the storage system **410** from the host application layer and timestamped based on the local

67

clock 462 of the storage system 410. Thus, the coordination of clock values based on differences among the local clocks facilitates a consistent ordering of updates to the replicated object store 440 across the participating storage systems 408, 410, 412.

When using monotonic local clocks, which may be initialized at storage system startup time, the local clocks of the various storage systems may be wildly different. To aid illustration, consider an example where the local clock value of a first storage system is '100' and at the same universal time the local clock value of a second storage system is '5000'. Based on exchanged clock values, the first storage system determines that there is a clock difference of '+4900.' When the first storage system receives an update from the second storage system, the first storage system subtracts '4900' from the timestamp of the update when storing the update in its local object store with the adjusted timestamp. Thus, updates directed to the same object through both the first and second storage system will be ordered consistently.

However, clock coordination for update ordering carries an embedded imprecision. That is, when a storage system sends a clock request message and receives a clock request response including the local clock value of another storage system, the time that the storage system receives the local clock value is necessarily later than when the local clock value was actually captured due to messaging latency. To aid illustration, consider an example where storage system A sends a clock request to storage system B, and storage system B responds to that request by reading its local clock and then sending that local clock in its response to storage system A. A latency is present in that when storage system A receives the response (e.g., 20 milliseconds later), all that storage system A can determine is that storage system B's clock was the reported value sometime between when storage system A sent out the message and when storage system A received the response. Storage system A cannot know if storage system B received the request almost immediately and obtained its clock value almost immediately but the response transmission took the bulk of the latency (e.g., 20 milliseconds), or if most of the latency was storage system B receiving the request, with capturing the clock value and responding being almost immediate, or somewhere in-between with the messages in either direction taking significant time and/or time consumed by the scheduling of the CPU to capture the clock value. Thus, this latency period represents an imprecision or uncertainty with respect to a received clock value.

For further explanation, FIG. 5 sets forth a flowchart illustrating an additional example method of establishing a guarantee for maintaining a replication relationship between object stores during a communications outage according to some embodiments of the present disclosure. The example method depicted in FIG. 5 is similar to the example methods described above, as the example method depicted in FIG. 5 also includes many of the steps and elements referenced in FIG. 4.

The method of FIG. 5 includes identifying 502, by the first storage system 410, a clock uncertainty value 505, 507 corresponding to each of the respective local clock values 405, 407. As mentioned above, a clock uncertainty value 505, 507 corresponding to a local clock of another storage system is based on messaging latency and the inability to know with certainty when a received local clock value of another storage system was captured. Thus, the value of a remote local clock is a fuzzy number that carries with it a degree of uncertainty. By the time the first storage system

68

receives a time value t of a remote local clock, the actual time of that remote local clock may be anywhere between t and t minus n , where n is the round trip messaging time from the first storage system to the remote storage system and back, with the remote clock value, from the remote storage system to the first storage system.

In some examples, a storage system 410 identifies 502 a clock uncertainty value 505, 507 by measuring messaging latency between the storage system 410 and the other storage systems 408, 412. For example, a first storage system 410 may send a clock request message, at time t_0 on its local clock, to a second storage system 408 and receive a clock response message, at time t_1 on its local clock, from the second storage system 408, where the clock response message indicates a local clock value of t_c recorded by the second storage system 408. Thus, the time t_c may have been potentially captured at any point between t_0 and t_1 . As such, $t_{\text{delta}} = t_1 - t_0$ represents the uncertainty with which one storage system 410 knows the local clock of another storage system 408, where objectively t_c may be anywhere from t_0 to t_1 . The first storage system then records t_{delta} as the imprecision with which the local clock of the second storage system is known.

In this example, the storage systems 408, 410, 412 exchange clock request and clock response messages among them to determine respective messaging latencies between each pair of storage systems. The respective messaging latencies of these clock coordination messages are then used to establish the uncertainty values for the respective local clocks. For example, a first storage system 410 sends a clock request to and receives a clock response from a second storage system 408 to establish the uncertainty value for the second storage system's local clock, and repeats the process for a third storage system 412. Meanwhile, the second storage system 408 sends a clock request to and receives a clock response from the first storage system 410 to establish its knowledge of the value and the uncertainty of the first storage system's local clock, and repeats the process for the third storage system, and so on. In some examples, the storage systems 408, 410, 412 exchange multiple rounds of clock request and clock response messages among them to determine respective messaging latencies between each pair of storage systems. In measuring messaging latency over multiple rounds of clock coordination messages, the lowest measured messaging latency between a pair of storage systems may be used as the clock uncertainty value for that pair of storage systems. It might be appropriate to add a small additional delta to allow for some amount of clock jitter (small variations in clock ticks over short time intervals). For example, exchanges within a short period of time can use the lowest latency as the uncertainty. Multiple exchanges over longer periods of time can be used to track relative drift and clock jitter patterns.

When a storage system receives a local update to an object in its local object store from the application layer and also receives a replicated update for that object from a remote storage system, the correct ordering of the updates can be ensured when the timestamps of those updates are separated by at least the clock uncertainty value associated with the respective local clocks. To aid illustration, consider an example where the timestamp of the local update recorded at storage system A is t_A and the adjusted timestamp (i.e., adjusted for relative clock difference) of a received replicated update from storage system B is t_B . In this example, the correct ordering of the updates is ensured if t_A and t_B are separated by at least the clock imprecision t_{delta} determined for storage system A and storage system B. If the timestamps

are not separated by at least the clock imprecision t_{delta} determined for storage system A and storage system B, storage system A may flag the local update and received replicated update for later reconciliation. Eventual consistency for that object can be achieved through additional messaging to agree upon an ordering.

Because the precision associated with a local clock value of a storage system is based in part on messaging latency, the coordination of respective local clock values may depend on the physical distance between the storage systems and messaging delays attached to the messages carrying the reported clock values. For example, the storage systems 408, 410, 412 may be storage blades that are collocated in the same storage blade system. As another example, the storage systems 408, 410, 412 may be storage blades that are located in different, remote storage blade systems. As yet another example, the storage systems 408, 410, 412 may be storage arrays that are remote from one another. Still further, one storage system may be a physical on-premises storage system while another storage system is a cloud-based storage system. Further, messaging latency between storage systems in different availability zones in the same region will be lower than messaging latency between two storage systems in different regions. The increased messaging latency across regions decreases the precision to which clocks may be coordinated, thus increasing the opportunity for versions written through different storage systems to become inverted. These different configurations of storage systems may affect messaging latency between the storage controllers of these storage systems and thus the precision to which any local clock value may be known.

If the distance between storage systems is known, the uncertainty value can be compensated for the transmission medium. For example, 3 microsecond per kilometer may be subtracted from the uncertainty for radio transmission or 5 microseconds per kilometer for fibre optic transmission. That is, physics ensures that however switching and scheduling delays work in one direction vs. the other, at least that amount of time is spent with a message in transit between systems, so twice that physics-based delay could be subtracted from the latency-derived uncertainty from requesting and receiving a clock value from a remote system. The physical distance can be estimated based on availability zone or region.

In some examples, one or more of the storage systems is a storage cluster including two or more storage cluster nodes that together serve a local dataset. These storage cluster nodes could be built as 'blades' (e.g., as discussed above with reference to FIGS. 2A-2G), although 'blades' are a particular physical form factor for the more general concept of clusters. A cluster could also be regular rack-mounted servers stacked in one or more racks. In some examples, the term 'blade' refers to a server architecture where individual servers can be pulled out of a rack and pushed back in, to be connected in the back simply through pushing them in and to be disconnected simply through pulling them out. The storage cluster nodes may be collocated in the same rack or chassis, or placed within relatively close proximity in a data center, such that the communications latency resulting from physical distance and network switching delays between the storage cluster nodes is negligible. In such an example, when the storage cluster comes online, one of the storage cluster nodes may be selected as a reference for that storage system's clock and all other blades in the storage system may compare their clock with the reference clock as they boot up. In such examples, communication latencies between blades will be low (e.g., on the order of tens of

microseconds), and may be limited as much by thread scheduling as by communications latencies. Thus, storage cluster nodes in such a storage cluster may be able to synchronize their clock offsets to within a few tens of microseconds of the reference clock. If the reference storage cluster node fails, some other storage cluster node may be selected to be the reference while continuing to use the offset from the original reference clock. The round-trip time for messaging to achieve node-to-node clock synchronization may be used to determine a clock precision across the storage cluster. Thus, the round-trip time (e.g., 60 microseconds) between the reference storage cluster node and a particular other storage cluster node can be stored as the precision of that particular storage cluster node's clock. The node-to-node precisions on sender storage cluster node from one cluster and receiver storage cluster node from a second cluster and the node-to-node precision may be added together to get an overall precision. In one example, the worst-case node-to-node precision could simply be considered that storage cluster's precision.

When an object update is replicated from one storage system to another storage system, the receiving storage system can know within the calculated relative clock imprecisions when that object update happened. The receiving storage system can also know the order in which two updates received from the sending storage system were received by the sending system, because the received timestamps will differ in the correct direction. Further, if one storage system receives a local update and also receives an update from a remote storage system where the timestamp values differ by at least the imprecision value, then that ordering is also known. There may be ambiguity, however, when a local and a remote update differ by less than the imprecision value.

If two monotonic timestamps of two operations differ by less than the system-to-system precision yet the time duration of those two operations, adjusted for the uncertainty value, can be shown to overlap then they can be considered effectively concurrent. Two operations that overlap in time can be applied in any order, though it is important for the order to be consistent. For example, if the round trip latency between two storage systems is 2 milliseconds such that the paired clocks are coordinated within 2 milliseconds of each other, but the time between receiving an operation and completing an operation is more than 2 milliseconds, then if each of two symmetrically replicating systems receive an operation either system can order them as long as they agree on the ordering based on their comparison of clocks (the coordinated monotonic clock of a sender or the local monotonic clock of a receiver) before a query operation is received. This overlapping order flexibility is particularly useful in conjunction with synchronous replication, where round trip delays are expected as part of completing an update request.

In some instances, there may be a possibility that timestamps will be inverted compared to which update is considered to have happened first. Where this is a concern, there may need to be some way to fix time ordering that is reversed from update ordering. One way to address this is to ensure that the duration of an operation, from being received to being completed, is at least as long as the uncertainty value. If that happens, then two operations could only have an ordering issue if they were genuinely concurrent, in which case they can happen in any order as long as the order is made consistent before any query operations are received that depend on the order. This can be accomplished by artificially delaying completion indications for requests in

cases where the request would otherwise complete more quickly than the uncertainty interval.

For further explanation, FIG. 6 sets forth a flowchart illustrating an additional example method of establishing a guarantee for maintaining a replication relationship between object stores during a communications outage according to some embodiments of the present disclosure. The example method depicted in FIG. 6 is similar to the example methods described above, as the example method depicted in FIG. 6 also includes many of the steps and elements referenced in FIG. 4.

The method of FIG. 6 also includes identifying 602, by the first storage system 410, a worst-case clock uncertainty 607 among the plurality of storage systems 408, 410, 412. The worst-case clock uncertainty is the largest uncertainty value among all pairs of storage systems. Each storage system determines an uncertainty value with respect to the local clock of each other storage system replicating the object store 440. Thus, for each pairwise combination of storage systems, the uncertainty value for that combination is determined and the largest determined uncertainty value is the worst-case uncertainty among the storage systems. In some examples, each storage system sends a message to the other storage systems that includes the uncertainty values it has identified. For example, storage system 410 determines an uncertainty value with respect to the local clocks of storage system 408 and storage system 412, and sends a message to storage system 408 and storage system 412 indicating those uncertainty values. Similarly, storage system 412 determines an uncertainty value with respect to the local clocks of storage system 408 and storage system 410, and sends a message to storage system 408 and storage system 410 indicating those uncertainty values, and so on. Alternatively, in such messages, each storage system only indicates the largest uncertainty value it has identified. Knowing the uncertainty values that it has measured and the uncertainty values (or largest uncertainty values) measured by all other storage systems, the first storage system 410 can identify the largest of all known uncertainty values as the worst-case uncertainty among the storage systems. In other examples, one storage system is selected as a coordinating storage system. Rather than messaging identified uncertainties to all other storage systems, each storage system indicates its identified uncertainties to the coordinating storage system, which can identify the worst-case uncertainty and send a message that indicates the worst-case uncertainty to the other storage systems. Thus, each storage system exchanges clocks and determines pairwise uncertainty with all other storage systems, and then the uncertainties are communicated either to all storage systems, where the highest uncertainty is used, or to a single storage system which determines the highest uncertainty from the list and communicates that back to the other storage systems.

The method of FIG. 6 also includes determining 604, in dependence upon the worst-case clock uncertainty 607, a clock coordination precision 609 for the plurality of storage systems 408, 410, 412. The clock coordination precision 609 for the plurality of storage systems 408, 410, 412 indicates a minimum separation time between two update requests for a particular object when the two update requests are directed to different storage systems among the plurality of storage systems. In other words, the clock coordination precision 609 specifies, to the application layer, a time period that should be used by the application layer to separate the completion of a first write or PUT request and the sending of a request to perform a second write or PUT to the replicated object store 440 to ensure that updates to the same

object on two different storage systems receive timestamps that reflect the application's intended order of the write or PUT operations. In some examples, the clock coordination precision 609 is simply the value of the worst-case uncertainty 607. In other examples, the clock coordination precision 609 accounts for an amount of time in addition to the value of the worst-case uncertainty 607. In one example, the clock coordination precision 609 is the value of the worst-case uncertainty 607 plus an amount of time that compensates for clock drift. For example, it may be expected that clocks may drift 1 millisecond every 4 hours. Thus, the clock coordination precision 609 may include the worst-case uncertainty plus a worst case estimate for the amount of time a clock may be expected to drift since the last exchange of clock coordination messages.

Note that the application layer could itself be distributed, and could operate on servers running in various locations, including in separate availability zones or in separate regions, with those servers writing to their local or to any other storage system storing and replicating objects for the object store. The uncertainty should, where reasonably possible, be used to separate updates or the storing of new versions to the same object from any combination of the servers in any combination of locations running parts of the application (or applications) utilizing the object store and storing versions or other updates to the same object.

The method of FIG. 6 also includes indicating 606, by the first storage system 410 to one or more hosts 470 the clock coordination precision 609. In some examples, the storage controller of a storage system communicates the time boundary to a host 470. For example, the clock coordination precision 609 may be communicated via a message, alert, or other notification to one or more hosts 470. The clock coordination precision 609 may also be communicated as a response to an API call, such as an API call by a host 470 to request the clock coordination precision 609. In some examples, a coordinating storage system indicates the clock coordination precision 609 to the one or more hosts 470, while in other examples each storage system 408, 410, 412 individually communicates the clock coordination precision 609 to connected hosts 470. In some implementations, the clock coordination precision 609 is communicated when a host 470 has indicated that it intends to write to the replicated object store 440 through multiple storage systems 408, 412. More particularly, these multiple storage systems 408, 412 may span multiple regions. For example, a replicated object store 440 may be configured to include a local object store 414 within one region replicating with another local object store 416 within another region, and a host 470 may indicate a configuration to write object updates to any local object store 414, 416, 418. In such an example, the storage controller of a storage system 410 may indicate a clock coordination precision 609 for symmetric access to the replicated object store 440 through the different storage systems 408, 410, 412.

To aid illustration, consider an example where storage system A, storage system B, and storage system C are symmetrically replicating an object store. Assume that it has been determined, based on messaging latency among the storage systems, that there is a 4 second uncertainty between storage system A and storage system B, a 2 second uncertainty between storage system A and storage system C, and a 3 second uncertainty between storage system B and storage system A. Thus, a maximum uncertainty of 4 seconds may be used as the time boundary. At some absolute time, the local clock of storage system A reads 100 seconds, the local clock of storage system B reads 85 seconds, and the

local clock of storage system C clock reads 350 seconds. Thus, after a clock exchange, storage system A might think storage system B's clock is 89 given an uncertainty of 4 seconds, and thus records a difference of -11 relative to storage system A's own clock. Storage system A might think that storage system C's clock is 352 given an uncertainty of 2 seconds, and thus records a difference of +252 relative to storage system A's own clock. Storage system B might think storage system A's clock is 104 given an uncertainty of 4 seconds, and thus records a difference of +19 relative to storage system B's own clock. Storage system B might think storage system C's clock is 347 given an uncertainty of 3 seconds, and thus records a difference of +262 relative to storage system B's own clock. Storage system C might think that storage system A's clock is 98 given an uncertainty of 2 seconds, and thus records a difference of -252 relative to storage system C's own clock. Storage system C might think that storage system B's clock is 87 given an uncertainty of 3 seconds, and records a difference of -263 relative to C's own clock). Thus, each storage system knows the local clock of each other storage system, with some degree of imprecision, and uses those known local clocks to order updates.

Continuing the example, at a time that is 100 seconds later than the clock exchanges that established the clock differences above, the storage systems each receive a PUT for different versions of the same object in the replicated object store. The first version PUT is received by storage system A at local time **200** and is stored along with its local time value of **200**, and the first version is forwarded with a timestamp of **200** to storage systems B and C. Storage system B receives the first version, subtracts storage system B's clock difference (+19 seconds) relative to storage system A from the timestamp of **200** and so locally stores the first version with a local time value of **181**. Storage system C receives the first version, subtracts storage system C's clock difference (-252 seconds) relative to storage system A from the timestamp of **200** (this ends up adding 252 seconds) and thus locally stores the first version with a local time value of **452**.

In this example, the second version PUT is received by storage system B five seconds after the first version PUT, which is at storage system B's local time **190**, and is stored along with a local time value of **190** (which correctly sorts relative to the first version whose local time value was **181**), and the second version is forwarded with a timestamp of **190** to storage systems A and C. Storage system A receives the second version, subtracts storage system A's clock difference (-11 seconds) relative to storage system B from the timestamp of **190** (this ends up adding 11 seconds) and so locally stores the second version with a local time value of **201**, thus correctly sorting the second version after the first version whose local time value was **200**. Storage system C receives the second version, subtracts storage system C's clock difference (-263 seconds) relative to storage system B from the timestamp of **190** (this ends up adding 263 seconds) and so locally stores the second version with a local time of **453**, thus correctly sorting the second version after the first version whose local time value was **452**.

In this example, the third version PUT is received by storage system C five seconds after the second PUT, which is at storage system C's local time **460**, and is stored along with a local time value of **460** (which correctly sorts relative to the second version whose local time value was **452**), and the third version is forwarded with a timestamp of **460** to storage systems A and B. Storage system A receives the third version, subtracts storage system A's clock difference (+252 seconds) relative to storage system C from the timestamp of **460** and so locally stores the third version with a local time

value of **208**, thus correctly sorting the third version after the second version whose local time value was **201**. Storage system B receives the third version, subtracts storage system B's clock difference (+262 seconds) relative to storage system C from the timestamp of **460** and so locally stores the third version with a local time value of **198**, thus correctly sorting the third version after the second version whose local time value was **190**.

Thus, all storage system systems have their own local clocks, know each other's clocks only approximately, and store all versions based on their own local clocks, and as long as at least 4 seconds separates version PUTs to any of the storage systems, the versions will be correctly ordered. Accordingly, a clock coordination precision of 4 seconds is the minimum delay that the application layer should use to avoid version inversions, which is the worst case uncertainty across between any pairs.

In the above example, the correct ordering is maintained because each PUT is separated by five seconds, which is greater than the maximum uncertainty. However, if the first version PUT and the second version PUT were separated by only three seconds, the first version and the second version could be inverted on storage system A and storage system B.

To further aid illustration, an alternative approach may use one storage system's clock as the reference clock. In such an example, storage system B and storage system C may copy storage system A's local clock. Using this approach, each storage system will accept the timestamp of the PUT that is forwarded from another storage system because this timestamp is purportedly reflects the value of the shared reference clock. However, the value of the copied reference clock still includes the uncertainty due to the messaging latency in the messages used to copy those values. Assume that storage system A's clock reads 10:00:00 at some absolute time. Storage system copies storage system A's reference clock, but only within an accuracy of 4 seconds. Thus, at the same absolute time, storage system B's copying of storage system A's clock might be anywhere between 9:59:56 and 10:00:04. Likewise, storage system C has also copied storage system A's reference clock, but only within an accuracy of 2 seconds. Thus, at the same absolute time, storage system C's copy of storage system A's clock might be anywhere between 9:59:58 and 10:00:02. Accordingly, storage system C's copy of the reference clock could be anywhere in the range of 6 seconds ahead to 6 seconds behind storage system B's copy of the reference clock. Accordingly, the imprecisions of storage system B and storage system C must be added to determine the worst-case uncertainty. Thus, the reference clock approach will typically require a greater amount of separation time between updates.

For further explanation, FIG. 7 sets forth a flowchart illustrating an additional example method of establishing a guarantee for maintaining a replication relationship between object stores during a communications outage according to some embodiments of the present disclosure. The example method depicted in FIG. 7 is similar to the example methods described above, as the example method depicted in FIG. 7 also includes many of the steps and elements referenced in FIG. 6.

The example method of FIG. 7 further includes determining **702** a validity duration **703** for the clock coordination precision **609**. In some examples, a storage system **410** determines a validity duration **703** for the clock coordination precision **609** by identifying an amount of time that the clock coordination precision **609** can be guaranteed. For example, the amount of time may be the amount of time between a

75

small number of scheduled exchanges of clock coordination messages. In some examples, the validity duration **703** guarantees the validity of the clock coordination precision **609** even if the storage systems **408**, **410**, **412** are unable to carry out a clock coordination exchange. In such examples, the validity duration **703** may be based on an estimate of how much the various clocks **462**, **464**, **466** could drift relative to each other in a particular amount of time, or by how much exchanged clock measurements drift relative to each other over extended time periods. For example, it might be estimated that two or more of the local clocks **462**, **464**, **466** will drift apart by as much as N milliseconds every M hours. If the clock coordination precision **609** is compensated with the estimated rate of clock drift for a particular amount of time, the validity duration **703** is the amount of time the clock coordination precision **609** can be relied upon before update ordering with updates separated by that interval can no longer be assured. For example, if a communications disruption prevents a clock exchange between at least two of the storage system **408**, **410**, **412** and the clock coordination precision includes 1 millisecond of padding for potential clock drift, and the estimated rate of clock drift is 1 millisecond every 4 hours, the validity duration **703** for the clock coordination precision **609** in this example can be 4 hours. Beyond the validity duration, assuming communications have still not been resumed, it should be presumed that the respective local clocks have potentially drifted by such an amount that increases the likelihood of two object updates being inverted with respect to their logical write order. For example, a replicated remote update may have a timestamp that is later than that of a local update even though the remote update logically precedes the local update.

The example method of FIG. 7 also includes indicating **704** the validity duration **703** to one or more hosts **470**. For example, the validity duration **703** may be communicated via a message, alert, or other notification to one or more hosts **470**. The validity duration **703** may also be communicated as a response to an API call, such as an API call by a host **470** to request the clock coordination precision **609**. In some implementations, the validity duration **703** is communicated along with the clock coordination precision **609**. For example, the validity duration **703** may be specified in a lease on the clock coordination precision **609**. In some examples, a coordinating storage system indicates the validity duration **703** to the one or more hosts **470**, while in other examples each storage system **408**, **410**, **412** individually communicates the validity duration **703** to a connected host **470**.

Although the clock coordination precision may be sufficient to ensure consistency of the replicated object store **440** when the storage systems **408**, **410**, **412** are communicating normally, the application layer may wait a longer period of time to ensure faults or communication delays are recognized and reacted to if other forms of transient inconsistencies are to be avoided. For example, by itself the clock coordination precision may not be enough to handle an exclusive PUT with symmetric replication, where a PUT is only allowed to a name that does not already exist, and where it is guaranteed to fail if it already does. The clock coordination precision may not avoid other complex issues, such as communication delays affecting replication and fault and recovery handling, which can still lead to inconsistencies. An additional amount of time may be necessary to accommodate the time distance between two locations where conflicts cannot be reliably detected (for an exclusive

76

PUT) or where version conflicts are not resolved before a response but may be resolved at a later time.

For further explanation, FIG. 8 sets forth a flowchart illustrating an additional example method of establishing a guarantee for maintaining a replication relationship between object stores during a communications outage according to some embodiments of the present disclosure. The example method depicted in FIG. 8 is similar to the example methods described above, as the example method depicted in FIG. 8 also includes many of the steps and elements referenced in FIG. 4.

The method of FIG. 8 also includes identifying **802** a fault detection time **809** to detect a disruption in communications among the storage systems **408**, **410**, **412**. The fault detection time **809** indicates an amount of time before the storage system determines that there is a disruption in communications with another storage system. In some examples, the storage system **410** identifies that messages are not being exchanged with another storage system **408**, **412**. These kinds of messages that test for operating network communication with paired systems (testing that both the network and the paired system are functioning, or that one or the other is not functioning) are common features of distributed systems. These could be built on top of clock exchange messages, or clock exchange could be built on top of the communication testing messages, or the two types of messages could be distinct from each other. These kinds of uptime test messages are generally exchanged at some interval, such as every few seconds or a few times per minute. And, if messages fail to be exchanged within some period of time, then systems will use various clustering tricks to decide how to move forward. In such implementations, the storage system **410** may determine that a communications disruption is occurring when it has not exchanged a message with a particular storage system **412** within a fault detection time **809**. For example, if clock exchanges or other protocol exchanges occur every 5 seconds, the storage system may determine that a communications disruption with a particular storage system is occurring if it has not exchanged a message confirming a connection in the last 30 seconds. During a communications disruption, the storage system is unable to determine (perhaps unless some other network monitoring system or service informs it) whether a communications connection isn't functioning or the paired storage system has itself faulted. Thus, communications disruptions may be temporary if the communications connection is restored. The fault detection time **809** may be preconfigured in the storage systems **408**, **410**, **412** as a storage system parameter or otherwise agreed upon by the storage systems **408**, **410**, **412**, for example, through an exchange of configuration information.

The method of FIG. 8 also includes indicating **804** the fault detection time **809** to one or more hosts **470**. For example, the fault detection time **809** may be communicated via a message, alert, or other notification to one or more hosts **470**. The fault detection time **809** may also be communicated as a response to an API call, such as an API call by a host **470** to request the fault detection time **809**. In some implementations, the validity duration **703** is communicated with a lease on the fault detection time **809**. In some examples, a coordinating storage system indicates the fault detection time **809** to the one or more hosts **470**, while in other examples each storage system **408**, **410**, **412** individually communicates the fault detection time **809** to a connected host **470**.

To aid illustration, if a set of storage systems are replicating a collection of objects between each other (e.g., a

replicated object store), then if one of the storage systems goes down, or a network partition makes a subset of the storage systems unreachable, or perhaps splits the storage systems into subsets that may still be communicating with each other within the subsets, the fault detection time may be an amount of time by which processing of requests will be delayed until a further determination can be made about how to handle the fault. Before processing can be resumed, the fault must be handled in some way. For example, this may include having a subset of storage systems determine that they are still connected with each other and are a large enough set to remain online for the dataset, or with simple two-way paired storage systems they can race for a mediator (presuming both are up and it is the network that is not functioning) and whichever one wins remains online. Any storage system that fails to become part of a winning set that remains online for the replicated object store will go offline and will start rejecting requests for the dataset. When faults are repaired (for example, a storage system recovers from a fault or reboots or a network is repaired) the prior offline (or rebooted) storage systems can reconnect to the online storage systems, exchange any missing updates, and continue as before. Thus, in some examples, the fault detection time may provide additional guarantees beyond clock coordination where fault detection and handling could be used by a host in deciding when it is safe to switch over from using one storage system to another if one storage system is unresponsive. For example, if host waits for the fault detection time (or a fault detection time lease) to make requests to another storage system that is then accepting requests, then the host knows that any fault handling has been completed and if that storage system is accepting and completing requests then it managed to remain online after whatever internal faults were handled.

For further explanation, FIG. 9 sets forth a flowchart illustrating an additional example method of establishing a guarantee for maintaining a replication relationship between object stores during a communications outage according to some embodiments of the present disclosure. The example method depicted in FIG. 9 is similar to the example methods described above, as the example method depicted in FIG. 9 also includes many of the steps and elements referenced in FIG. 8.

The method of FIG. 9 also includes discontinuing 902, in response to identifying that the fault detection time 809 has expired, local processing of requests directed to the replicated object store 440. In the event of a communications failure among the replicating storage systems 408, 410, 412, either due to network partition or a storage system failure, a subset of the storage systems that are still running and that are still communicating may be designated through some means to continue servicing requests directed to the replicated object store 440 while any remaining non-failed storage systems discontinue servicing local requests directed to the replicated object store. When a particular storage system 410 identifies that it cannot communicate with at least one other replicating storage system 408 and identifies that the fault detection time 809 has expired, that storage system 410 determines whether it has been designated to continue servicing host access requests to the replicated object store 440 in the event of a failure. If the storage system has been designated, the servicing of host access requests continues while replication to any storage systems outside of the designated subset storage systems 408, 412 is discontinued. If a storage system has not been designated to continue servicing requests, that storage system discontinues servicing local requests directed to the replicated object

store 440 and may terminate normal host access to its copy of objects and other services of the replicated object store 440.

For further explanation, FIG. 10 sets forth a flowchart illustrating an additional example method of establishing a guarantee for maintaining a replication relationship between object stores during a communications outage according to some embodiments of the present disclosure. The example method depicted in FIG. 10 is similar to the example methods described above, as the example method depicted in FIG. 10 also includes many of the steps and elements referenced in FIG. 8.

To better identify which storage systems should continue servicing the replicated object store 440, a mediation service can be employed to select which storage system should remain online for the replicated object store in the face of a communications disruption among the storage systems. Thus, the method of FIG. 8 also includes sending 1002, in response to identifying that the fault detection time 809 has expired, a message 1003 to a mediation service 1005. In some examples, the storage systems 408, 410, 412 replicating the object store 440 are in communication with a mediation service 1005, where a mediation service may resolve which storage system continues to service the replicated object store 440 in the event of a communication fault between storage systems, in the event of a storage system going offline, or due to some other triggering event. The mediation service 1005 may be external to the storage systems 408, 410, 412. Specifically, if a first storage system 410 has detected a triggering event, such as loss of a communication link to a second storage system 412, the first storage system 410 may contact an external mediation service 1005 to determine whether it can safely take over the task of removing the non-communicating storage system from a replication group with respect to the replicated object store 440. In other cases, the first storage system 410 may contact the external mediation service 1005 and determine that it may have been removed from the replication group by a second storage system 412. In these examples, the storage systems 408, 410, 412 need not be in continuous communication with the external mediation service 1005 because under normal conditions the storage systems 408, 410, 412 do not need any information from the mediation service 1005 to operate normally. In other words, in this example, the mediation service 1005 may not have an active role in membership management of a replication group, and further, the mediation service 1005 may not even be aware of the normal operation of the storage systems 408, 410, 412 in the replication group. Instead, the mediation service 1005 may simply provide persistent information that is used by the storage systems 408, 410, 412 to determine membership in replication group, or to determine whether a storage system can act to remove another storage system.

In some examples, a mediation service 1005 may be contacted by one or more storage systems 408, 410, 412 in response to a triggering event such as a communication link failure preventing the storage systems 408, 410, 412 from communication with each other; however, each storage system 408, 410, 412 may be able to communicate with the mediation service 1005 over a communication channel that is different from the communication channel used between the storage systems 408, 410, 412. Consequently, while the storage systems 408, 410, 412 may be unable to communicate with each other, each of the storage systems 408, 410, 412 may still be in communication with the mediation service 1005, where the storage systems 408, 410, 412 may use the mediation service 1005 to resolve which storage

system may proceed to service requests directed to the replicated object store 440. Further, the storage system that wins mediation from the mediation service 1005 may remove another non-communicating storage system and update a replication group list indicating the storage systems that may continue to participate in replication and servicing of requests directed to the replicated object store 440.

The method of FIG. 10 also includes receiving 1004 a mediation result 1007, wherein the mediation result 1007 indicates whether local processing of requests directed to the replicated object store 440 should be disabled on the first storage system 410. In some examples, a particular storage system 410 that remains online but cannot communicate with at least one other storage system 408, 412 receives a mediation result in which that storage system loses mediation. In these examples, losing mediation indicates that the storage system will no longer participate in replication of the object store 440 and should therefore discontinue servicing requests directed to the replicated object store 440. In such examples, the storage system 410 may terminate a connection to one or more hosts 470.

For further explanation, FIG. 11 sets forth a flowchart illustrating an additional example method of establishing a guarantee for maintaining a replication relationship between object stores during a communications outage according to some embodiments of the present disclosure. The example method depicted in FIG. 11 is similar to the example methods described above, as the example method depicted in FIG. 11 also includes many of the steps and elements referenced in FIG. 8.

The method of FIG. 11 also includes initiating 1102, in response identifying that the fault detection time 809 has expired, a quorum protocol. In some examples, resolving a set of one or more storage systems 408, 410, 412 to continue servicing requests directed to a replicated object store 440 may be implemented by using a quorum protocol. A quorum protocol may be used unless a particular storage system 410 is able to determine that use of a quorum protocol would be unable to establish a quorum for determining the set of one or more storage systems 408, 410, 412 to continue servicing requests. For example, quorum could not be established if there are only two storage systems replicating the object store 440. In other words, in response to an error such as a communication fault between storage systems 408, 410, 412, a storage system 410 may determine whether or not a quorum can be established, where if a quorum is able to be established under a particular quorum protocol, then the quorum protocol is used for determining active membership in a replication group for the replicated object store 440; otherwise, if a quorum is not able to be established under a particular quorum protocol, then the storage system 410 may engage in mediation for determining active membership in a replication group for the replicated object store 440. If a storage system determines that a quorum protocol may be used, that storage system communicates a 'vote' to remove a non-communicating storage system from the replication group to all other storage systems with which it is communicating.

The method of FIG. 9 also includes determining 1104, in dependence upon a quorum protocol result, that the first storage system 410 is a member of a quorum. In the event of communications disruption, two or more storage systems may be unable to communicate with at least one other storage system. Storage systems that are still able to communicate with one another determine whether they represent enough storage systems to form a quorum and thus remain on-line for the replicated object store 440. Any storage

system that determines that it is not part of a quorum takes itself offline for the replicated object store 440.

In some alternative examples, the storage systems that can communicate with one another each provide a vote on which storage systems remain in the replication group or whether any storage system, if any, should be removed from the replication group for the replicated object store 440. If a majority of communicating storage systems agree on a quorum, any storage system that is not part of the quorum is removed from the replication group. In some implementations, one or more of the remaining storage systems may indicate to a connected host that the non-communicating storage system has been removed, such that requests directed to the replicated object store 440 should not be made on the removed storage system. If no storage system receives a majority of votes, the storage systems 408, 410, 412 may employ mediation. In some examples, other storage systems that are not replicating the object store 440 may be solicited for votes during a quorum protocol. That is, the storage systems 408, 410, 412 may be communicatively coupled to other storage systems that are not in the replication group and for which there are not replication links configured for the replicated object store 440. Yet, these storage systems can be relied upon to identify whether communication has failed with respect to one of the storage systems 408, 410, 412 in the replication group. These voting storage systems that are not members of the replication group may be selected from disparate regions.

To aid explanation, consider an alternative example where two storage systems 408, 410 are unable to communicate with a third storage system 412. A storage system 410 may initiate a quorum protocol by sending a message to the other communicating storage system 412 indicating a vote to remove the non-communicating storage system 412. That storage system may also send a message to the initiating storage system 410 indicating a vote to remove the non-communicating storage system 412. Having agreed upon a majority vote to remove the non-communicating storage systems, the remaining storage systems 408, 410 may discontinue replication to the non-communicating storage system 412 and report to one or more hosts that the storage system 412 no longer maintains a consistent copy of the replicated object store 440. If there were only two storage systems 410, 412 in the replication group, the initiating storage system might send a quorum protocol message to another storage system that is in the replication group and is located in an altogether different region. Votes from this storage system and the initiating storage system 410 may be sufficient to remove the non-communicating storage system 412 from the replication group.

For further explanation, FIG. 12 sets forth a flowchart illustrating an additional example method of establishing a guarantee for maintaining a replication relationship between object stores during a communications outage according to some embodiments of the present disclosure. The example method depicted in FIG. 12 is similar to the example methods described above, as the example method depicted in FIG. 12 also includes many of the steps and elements referenced in FIG. 8.

To avoid or delay the shutdown of one or more storage systems, the clock coordination precision can be increased to further compensate for potential clock drift. Thus, the method of FIG. 12 also includes determining 1202 an updated clock coordination precision 1209, in response to identifying that the fault detection time 809 has expired. When a particular storage system 410 identifies that it cannot communicate with at least one other replicating

81

storage system **408**, that storage system **410** can modify the clock coordination precision that is communicated to a host **470** to account for potential clock drift. In some examples, a particular storage system determines **1002** the updated clock coordination precision by increasing a value of the amount of time specified by clock coordination precision) by a predetermined amount based on estimated clock drift. For example, padding may be added to most-recently determined worst-case uncertainty, or padding that was already included may be increased. Increasing the value of the value of the clock coordination precision increases the separation between object updates and thus the separation in the timestamps assigned to those updates, thus accounting protecting against some amount of clock drift. Although the increase in separation between object updates may impact performance, it may also provide additional time for communications to resume, thus avoiding a failure mode.

The method of FIG. **12** also includes indicating **1204** the updated clock coordination precision **1209** to the one or more hosts **470**. For example, the updated clock coordination precision **1209** may be communicated via a message, alert, or other notification to one or more hosts **470**. The updated clock coordination precision **1209** may also be communicated as a response to an API call, such as an API call by a host **470** to request a clock coordination precision **1209**. In some examples, a coordinating storage system indicates the updated clock coordination precision **1209** to the one or more hosts **470**, while in other examples each storage system **408**, **410**, **412** individually communicates the clock coordination precision **1209** to a connected host **470**.

As discussed above, configuring a bidirectionally replicated object store such that a host may symmetrically write object updates through multiple storage systems requires some awareness on the part of the application issuing those writes. This is especially true where the local copies of the replicated object store are physically separated by large distances, where messaging latency can contribute significantly to problems related to clock disparity. Thus, in some embodiments, one or more of the storage systems includes a management interface that allows a host to configure and manage the replicated object store.

For further explanation, FIG. **13** sets forth a flowchart illustrating an example method of providing application-side infrastructure to control cross-region replicated object stores according to some embodiments of the present disclosure. In the example of FIG. **13** like numerals correspond to like elements with respect to the storage environment discussed above with reference to FIG. **4**. Like the example of FIG. **4**, in the example of FIG. **13** the storage systems **408**, **410**, **412** bidirectionally replicate the object store **440** and provide symmetric access to the replicated object store **440** through any storage system **408**, **410**, **412** in the replication group. That is, any storage system **408**, **410**, **412** in the replication group can service host requests directed to the replicated object store **440**. Here, servicing host requests directed to the object store can include receiving a request to modify the object store (an update request) by creating a new object or modifying an existing object in the object store, processing those requests, performing the modifications to the local copy of the object store, and replicating the modification to other storage systems that are replicating the object store **440**.

Thus, in symmetric replication, storage systems at multiple locations can serve clients of the object store at the same time, with updates to storage systems that are at one location being replicated to storage systems at the other locations as they are received from clients and processed. At

82

least one pair of storage systems among the plurality of storage systems **408**, **410**, **412** utilizes an eventual consistency model for bidirectional replication, where each storage system storing an object makes independent decisions as to ordering updates and applying modifications, with those decisions being updated as more data is received, yet all storage systems are expected to make the same decisions once they have the same data. As an example, a storage system may eventually receive two versions of an object, and if so that storage system should order those two versions identically with any other storage system that receives those two versions. Thus, the eventual consistency model for consistent ordering is based on non-synchronous replication. However, in some cases, replication between a particular pair of storage systems may be synchronous, in that completion responses to requests are delayed until replication between synchronous locations has been performed.

In some examples, the storage systems **408**, **410**, **412** are at different localities **480**, **482**, **484**. For example, the different localities can correspond to different availability zones, similar or identical to AWS availability zones or availability zones of another cloud services provider, or an equivalent within a set of private data center deployments. In other examples, the different localities can correspond to different regions, similar or identical to AWS regions or regions of another cloud services provider. Some storage systems may be located in different availability zones of the same region while another storage system is located in a different region. In yet other examples, the different localities correspond to different data centers that are physically separated by a substantial distance (e.g., 1000 kilometers or more).

As discussed above, each storage system includes a local clock that may be different than the local clock of the other storage systems. Regular system clocks can be coordinated to some degree through protocols like NTP (Network Time Protocol), but such protocols only work so well, may encounter temporary outages, and can cause relatively sudden jumps in clock values (or short periods of time where clocks are sped up or slowed down to get back in sync). Each storage system may further include a monotonic clock that begins at boot time and that is not subject to jumps or rate changes to get back into sync with an external source, but each system may boot at different times, leading to a variety of local clocks across the storage systems that, at any given point in universal time, have completely different clock values. Further, each system's clock, even a monotonic clock, might advance at slightly different rates, causing relative drift over relatively long time intervals vs clocks on other storage systems. Thus, using local clock values to order updates presents a problem. Accordingly, the storage systems **408**, **410**, **412** exchange respective local clock values with one another, where each storage system uses the exchanged clock values to track the local clocks of other storage systems and to order updates made to the replicated object store based on relative clock values. That is, each storage system uses the exchanged clock values to track the respective local clocks of the other storage systems and identify an offset of those clocks from its own local clock. Each storage system then orders updates to the object store using its own local clock in relation to respective local clocks of other storage systems based on the tracked offsets.

When ordering updates, the updates include a storage system's own modifications to the object store along with modifications that are replicated to it. For example, a storage system may receive a request (from a client) for a first update to an object, which it processes and stores in its local

copy of the replicated object store. This update is stored with an associated clock value of the storage system's own local clock. The storage system may also receive a second update to the same object that is replicated from another storage system. When received, this second update is associated with a clock value of the replication source storage system's local clock. The receiving storage system determines a relative clock value for the replicated update using its own local clock value and the tracked offset corresponding to the replication source storage system. The storage system then orders the first update and the second update according to the relative clock value of the second update. When carried out in this way by all of the storage systems symmetrically replicating the object store, the result should be a consistent and predictable ordering.

However, there is an uncertainty or imprecision with which any storage system can track the local clock of any other storage system. That tracking has an uncertainty due to non-zero delays in messages for sending and receiving clock values coupled with uncertainty in how the contributors to these delays, such as transmission and message processing delays, stack up, including whether messages might be faster in one direction versus another, or whether processing delays affect one system more than another, or any other source of uncertainty in the causes of transmission times in each direction. Uncertainties can also build up if the storage systems have not communicated clock values sufficiently recently because clocks can drift. Even monotonic clocks, which generally count up time from when a system booted, can tick at slightly different rates between systems. Thus, clock values should be exchanged frequently enough to avoid excessive build-up of differences. If communication links are down for extended time periods, a fault handling measure may be necessary in view of the inability to coordinate clocks.

As discussed above, one or more storage systems identifies a clock coordination precision that reflects the uncertainty with which any storage system can track the local clock of any other storage system, where that uncertainty is due to message latency and other considerations. This clock coordination precision represents a worst-case uncertainty with which one storage system in the plurality of storage system can track the local clock value of any other storage system using the exchanged clock values. In addition to uncertainty based on message latency, the clock coordination precision can account for potential clock drift and/or timing for detecting and handling a temporary loss of communications or other fault among the storage systems **408, 410, 412**. If two updates to the same object are written to the object store through different storage systems and those update requests are not separated by at least the clock coordination precision, inconsistencies in the order that those updates are applied may arise. Storage systems may employ internal mechanisms for resolving inconsistencies, but the result may not be predictable in that the ordering of the applied modifications may not reflect the actual ordering of those modifications by the applications making those modifications.

For example, where two versions of an object are created through PUT operation through two different storage systems replicating the object store, and those requests are received more closely in time than the clock coordination precision, those versions should still be consistently ordered through the storage systems detecting and resolving potential ordering conflicts that cannot be resolved purely by clock value comparison. However, the result may not have the same order as the order in which the PUT requests were

actually issued. In cases where version PUT operations are too close together and requested to separate storage systems, storage systems may have to eventually recognize that comparison by clock values may not produce a consistent result. Instead, inconsistencies can be resolved, for example, by exchanging the proposed orderings (or the clock values assigned to specific versions by each storage system), recognizing that the orderings would be inconsistent, and then using some method of for agreeing on an order (for example, the ordering by one system can take precedence over and replace the ordering by another). However, the agreed upon order may not be the order intended by the issuer. To ensure a predictable result that reflects the order of the modifications that would be expected by the issuer of those modification requests, a client utilizing the object store should be aware of the clock coordination precision. With this information, the client can separate dependent operations that are sent to different storage systems by an amount of time that avoids storage systems attempting to resolve inconsistencies in a non-predictable way.

In some implementations, putting of an object, or of a part of an object, can reference another object, or a part of an object, or a version of an object, or a part of a version of an object, rather than supplying the content of the object (or of the part) directly. This results in copying that source into the object. If the source is an object, or a part of an object, then content to be copied can depend on whether the copy is processed before or after an operation to replace the content of the object (or to add a newer version to an object). This can result in a potential dependency when replicating objects between locations. Specifically, if the copy is replicated as a copy operation then the separate locations might perform the copy from a different version of the content of the same named object because a first location might process the copy before a new put, whereas the second location might process the new put before the copy.

Implementations can and should provide an accommodation for such copy operations to ensure a consistent result. A simple implementation that can fix this is to perform the copy on the storage system that received the request, and then replicate the resulting content to paired storage systems rather than replicating the copy operation. Another implementation, which can avoid transmitting all of that "copied" content over the replication interconnect, ties the replication of a copy to a version even if the original request copied from an object rather than from a specific version. This works well for a versioned bucket. In the case of a non-versioned bucket, a version identifier of some kind supported by the implementation, for example the commonly supported E-tag, can still be associated with the request, and if the paired version does not have that version, then it could request a retransmit of the content from the storage system that originally performed the copy. In a particular example, a source version is attached to a copy operation or a source version is decided on by a leader or arbiter, such that a copy operation on an object is applied consistently by all storage systems replicating an object store by using the same source version of the object. The replacing of objects with new versions of the object is generally an infrequent operation, so requiring that the copied content be transmitted over the replication interconnect due to a version mismatch should be a rare occurrence.

To provide application-side infrastructure for ensuring such predictable and consistent results across the replicated object store, the method of FIG. 13 includes determining **1302**, for an object store **440** replicated across a plurality of storage systems **408, 410, 412** at a plurality of locations, a

minimum time interval **1303** between requests to modify the object store **440** that are received by differing storage systems of the plurality of storage systems which will ensure a predictable result on all storage systems among which the requested modifications are replicated. In some examples, the minimum time interval is the clock coordination precision discussed above. For example, for each pair of storage systems replicating the object store, an uncertainty in the exchanged clock values is determined, and the worst (i.e., the largest) uncertainty among all of these pairs is selected as the uncertainty in the coordination of local clocks across all storage systems replicating the object store. In some examples, the worst-case uncertainty is modified to account for an additional amount of time to compensate for clock drift, the time since the last clock exchange, or may subtract an amount of time that accounts for physical distance as maximum speed of travel for a packet may not be a source of uncertainty given basic physics, or may account for other variables as discussed above. In some cases, the amount of time specified by the minimum time interval **1303** is equal to the worst-case uncertainty, although in other cases the minimum time interval may account for additional delay factors.

The uncertainty for coordinating clocks between a pair of storage systems may be based on the message latency between those storage systems. Thus, in some examples, the minimum time interval is based, at least in part, on a round-trip time for messages between pairs of storage systems. Further, the uncertainty may increase over time due to potential clock drift, and thus the time since the last exchange of message may factor into the uncertainty in coordinating clocks between a pair of storage systems. Thus, in some examples, the minimum time interval may be based, at least in part, on the recency since a last completed round-trip between pairs of storage system. Still further, the physical distance between a pair of storage systems may factor into the uncertainty for coordinating clocks between those storage systems. Thus, in some examples the minimum time interval is based, at least in part, on known physical distances between pairs of storage systems. In some implementations, the minimum time interval is specific to a particular pair of storage systems. Thus, there may be multiple time intervals based on respective uncertainties among pairs of storage systems.

When used to separate dependent requests to modify the object store through differing storage systems, the minimum time interval will ensure a predictable result in the ordering of those modifications in each replica of the object store. A modification from an earlier request will always be applied before a modification from a later request by all storage systems because the requests are received separately by at least the minimum time interval needed so that the storage systems can reliably resolve inconsistencies through a comparison of relative clock values alone. To aid illustration consider an example where a first request to store a first version of an object is received by a first storage system of the symmetrically replicating storage systems and a second request to store a second version of the object is received by a second storage system of the symmetrically replicating storage systems. The second version is ordered consistently after the first version by all storage systems that store replicas of the first version and the second version when the first request and the second request are separated by at least the minimum time interval.

In some examples, each storage system identifies pairwise uncertainties with respect to every other storage system in the replication group and reports each of those uncertainties

to every other storage system in the replication group. Thus, any storage system in the group can identify the worst-case uncertainty among the group and determine, based on this value, the minimum time interval for ensuring a predictable result when dependent updates are received by differing storage systems and then replicated. In other examples, one storage system may be selected as the coordinating storage system, which receives the pairwise uncertainties reported from all storage systems in the group. In such examples, the coordinating storage system may identify the worst-case uncertainty among the group and determine, based on this value, the minimum time interval and report this minimum time interval to the other storage systems in the group. In still further examples, a management server or other administrative system associated replicating storage systems may receive the pairwise uncertainties identified by each storage system in the group. In such examples, the management server may identify the worst-case uncertainty among the group and determine, based on this value, the minimum time interval and report this minimum time interval.

Thus, in various examples, determining **1302** the minimum time interval **1303** is carried out by a replicated object store platform **1301**, which may be a component of some or all of the storage systems **408**, **410**, **412** replicating the object store **440** or a component of a separate management server or administrative system that is associated with the storage system **408**, **410**, **412**. For illustrative purposes, the replicated object store platform **1301** is shown as a component of storage system **410** in FIG. 13. However, it should be appreciated that, when the replicated object store platform **1301** is a component of a separate system associated with the replicating storage systems **408**, **410**, **412**, the object store platform **1301** determines **1302** the minimum time interval based on reported uncertainties or a worst-case uncertainty that are provided by one or more storage systems.

The method of FIG. 13 also includes providing **1304**, through an application programming interface (API) to a client that utilizes the object store, one or more object store parameters including the minimum time interval **1303**. In some examples, the object store platform **1301** exposes an API to clients, applications, hosts, or other consumers of storage (e.g., host **470**) of the replicated object store **440**. The API may be a request-based API used by the client to request object store parameters through polling, a callback API, a subscription-based API, and so on. In these examples, the object store platform **1301** provides **1304** one or more object store parameters for utilizing the object store to the client through this API. One such object store parameter is the minimum time interval.

To ensure a predictable and consistent result, the client (e.g., host **470**) may enforce a rule for itself and its various local or distributed components that dependent requests to modify the object store, which are sent to different storage systems, must be separated by at least the minimum time interval **1303** provided through the API. For example, the client may determine, when making a request to update an object, whether the minimum time interval has elapsed since the last request to update the same object. If these dependent requests are within the minimum time interval, the client may further determine whether the later request is being sent to a different storage system than the earlier request. If the later request is being sent to a different storage system, the client may delay the later request until the minimum time interval has elapsed before transmitting that request to the different storage system. Alternatively, the client may flag transactions that violated the rule for later analysis. For

example, the client may examine time stamps and actions listed in various logs to determine whether any dependent requests to update the object store that were sent to different storage systems had violated the minimum time interval.

Another object store parameter that may be associated with the minimum time interval **1303** is a parameter indicating that the minimum time interval **1303** should increase over time if it is not refreshed through subsequent API calls. The clock uncertainty between any two storage systems is dynamic and beyond simple networking delays can be further influenced by message processing latency (e.g., due to a storage system being busy) as well as by clock drift. Thus, the client should refresh the minimum time interval through periodic API calls. If the client does not refresh the minimum time interval through an API call, or otherwise does not receive a refreshed minimum time interval due to a communications disruption, the last known minimum time interval should be increased to account for a potential increase in the uncertainty of clock coordination. In some examples, an object store parameter indicates how often the minimum time interval should be refreshed. In some examples, an object store parameter indicates how much the minimum time interval should increase for a given amount of time since the last minimum time interval was acquired. For example, the parameter may indicate a rate or schedule for increasing the minimum time interval. In some examples, the rate of increase for the minimum time interval is based on an estimated rate of clock drift among the storage systems, which can be measured by tracking how often and by how much the tracked clock values are modified, and in one direction, between subsequent clock exchanges.

Another object store parameter that may be associated with the minimum time interval **1303** is a timed lease parameter. The timed lease parameter indicates a duration of time that the minimum time interval remains valid. For example, a minimum time interval may be guaranteed to be effective for ensuring the predictable and consistent result only for a specified amount of time; after that, the minimum time interval cannot be relied upon for such guarantees. The length of the lease may be based on how often the minimum time interval is calculated from message or clock value exchanges. The length of the lease may also be based on an amount of time required by a storage system to detect a communications disruption with other storage systems. The length of the lease may also be based on how long the clock of another storage system can be tracked without an exchange of clock values before potential clock drift exceeds a threshold. For example, as discussed above, a clock exchange is necessary to continuously update the clock coordination precision; however, during a communications disruption that prevents such a clock exchange, the respective local clocks may drift apart at an estimated rate. A client may use the timed lease parameter to determine when a minimum time interval has effectively expired, thus indicating that the client should attempt to refresh the minimum time interval.

In some examples, where the storage system pushes the minimum time interval through the API, the minimum time interval may be updated by the storage system in response to detecting a fault in communication with one or more other storage systems. The updated minimum time interval may reflect a greater degree of uncertainty to which the respective local clocks can be coordinated. For example, if a storage system does not receive a clock value from another storage system or if the storage system detects a fault that prevents communication, one or more storage systems may increase the minimum time interval and push that minimum

time interval to a client. This allows the client to continue to rely on the minimum time interval while the storage systems recover from the fault, at least for some period of time before a fault handling action is taken by the storage systems.

In some examples, one or more object store parameters includes timing parameters related to fault handling and failure recovery in an event where one or more of the plurality of storage systems takes over servicing of objects in the object store in response to a fault. The fault can be related to a failure to receive an exchanged clock value from another storage system or a failure to receive a response to some other message. This failure can be due to a communications disruption in the network, due to the other storage system faulting, and so on. When the fault is detected, the storage system will enter a faulted state in which it pauses the servicing of requests directed to the object store **440** until the fault resolves (e.g., the storage systems begin communicating again), the storage system takes itself offline for the object store, or a fault recovery action is taken. In one example, an object store parameter indicates an amount of time that can elapse before a storage system enters a faulted state. For example, the parameter may indicate how a storage system will wait without receiving communication from a storage system before it enters the faulted state. As another example, an object store parameter may indicate an amount of time the storage system will wait between entering the faulted state and taking itself offline for servicing the object store. As another example, an object store parameter may indicate an amount of time that a storage system will wait before taking a fault recovery action. The fault recovery action may include a quorum-based protocol or mediation to determine which storage system(s) will continue to service the object store and which will be taken offline for the object store.

In various examples, the object store platform **1301** can also expose an API for other administrative functions, such as enabling symmetric access to the replicated object store **440** across the plurality of storage systems, disabling the automatic shutdown of one or more of the storage systems during a communications disruption, requesting a current status of the replication links for the replicated object store **440**, and other administrative actions. It will be appreciated that the object store platform **1301** may provide other management and administrative functions related to the replicated object store **440** beyond those which are specifically identified in this disclosure. The object store platform **1301** may be a component of some or all of the storage systems **408**, **410**, **412** replicating the object store **440** or a component of a separate management server or other object store platform that administers the object store **440**.

In some examples, a software development kit (SDK) or other development framework includes a library of API calls for the replicated object store platform **1301**. The SDK may also include testing and analytics tools to aid the development of applications that can safely write object updates to the replicated object store through multiple storage systems. For example, such testing and analytics tools may test the implementation of a clock coordination precision, where the application employs the clock coordination precision to write object updates to different local copies of the replicated object store. The SDK may also provide documentation for configuring the replicated object store as well as configuring the application to safely write object updates to the replicated object store through multiple storage systems.

For further explanation, FIG. **14** sets forth a flowchart illustrating an additional example method of providing application-side infrastructure to control cross-region repli-

cated object stores according to some embodiments of the present disclosure. The example method depicted in FIG. 14 is similar to the example methods described above, as the example method depicted in FIG. 14 also includes many of the steps and elements referenced in FIG. 13. The example of FIG. 14 includes receiving 1402 a request 1403 for the minimum time interval 1303. In some examples, a storage system receives 1402 a request for the minimum time interval 1303 by identifying an API call to a storage system method that provides a value for the minimum time interval 1303 to one or more hosts.

For further explanation, FIG. 15 sets forth a flowchart illustrating an additional example method of providing application-side infrastructure to control cross-region replicated object stores according to some embodiments of the present disclosure. The example method depicted in FIG. 15 is similar to the example methods described above, as the example method depicted in FIG. 15 also includes many of the steps and elements referenced in FIG. 13.

In some examples, the object store platform 1301 includes an interface to query a status check of the replicated object store 440. Thus, the example of FIG. 15 includes receiving 1502 a request to check a status of the replicated object store 440. In some examples, a storage system receives 1502 a request to check the status of the replicated object store 440 by identifying an API call to a storage system method that identifies the status of the replication links among the storage systems 408, 410, 412. In some examples, a host 470 periodically performs a 'health check' on the replicated object store 440 by sending requests to each storage system to verify that the communications link between the host 470 and the storage system is intact and to verify that the communications links (for clock exchange and replication) among the storage systems 408, 410, 412 are working properly. A host 470 may initiate a health check when, for example, the host 470 identifies that one of the storage systems is not communicating with the host 470 (e.g., not acknowledging write requests).

The example of FIG. 15 also includes providing 1504, to the host 470, information related to the status of object store replication. In some examples, a storage system 410 provides 1504 information related to the status of the replicated object store 440 by indicating whether the communications links to the other storage systems 408, 412 in the replication group are intact. For example, a storage system may acknowledge that a communications link is intact when it has received a clock exchange message within a clock exchange interval. In some examples, a storage system 410 provides information related to the status of the replicated object store 440 by indicating that a replication link between two storage systems has failed. In some examples, the object store platform 1301 provides an indication of which of the plurality of storage systems to which the first storage system is able or unable to replicate updates. In some examples, the object store platform 1301 information detailing which of the storage systems are currently operating to service the object store 440 and to which storage systems each of the servicing storage systems is currently successfully replicating updates. The host 470 may continue to query a storage system for a health check even after the storage system has failed in order to determine whether the storage system has returned to operation.

In some examples, the host 470 provides status information to one or more storage systems 408, 410, 412. For example, when the host 470 identifies a communications disruption between the host 470 and a particular storage

system 408, the host 470 may notify the remaining storage systems 410, 412 of this communications disruption.

For further explanation, FIG. 16 sets forth a flowchart illustrating an example method of providing application-side infrastructure to control cross-region replicated object stores according to some embodiments of the present disclosure. In the example of FIG. 16 like numerals correspond to like elements with respect to the storage environment discussed above with reference to FIG. 4.

The method of FIG. 16 is directed to host-side operations to utilize the minimum time interval 1303. The method of FIG. 16 includes receiving 1602, by a host, one or more object store parameters related to a replicated object store 440, wherein the replicated object store 440 is replicated across a plurality of storage systems 408, 410, 412, wherein at least two of the plurality of systems 408, 410, 412 are located in different localities 480, 482, 484, and wherein the one or more object storage parameters includes a minimum time interval. For example, the minimum time interval and other object store parameters may be received through the API discussed above.

The method of FIG. 16 also includes determining 1604, by the host, that a first request to modify an object in the object store directed to a first storage system of the plurality of storage systems and a second request to modify the object directed to a second storage system of the plurality of storage systems are separated by at least the minimum time interval. In some examples, the host 470 determines that the second request is separated from the first request by the minimum time interval before sending the second request. If not, the host 470 may delay the second request. In other examples, a host of the client application which generally writes to a storage system in one location may exchange application-level locks with a host of the client application which generally writes to a storage system at another location, where the obtaining of these locks can include a delay intended to account for the minimum delay to ensure deterministic ordering. In yet other examples, the host 470 may determine, after both requests have been issued, that the first request and the second request were not separated by the minimum time interval. In such an example, the host 470 can flag the requests or flag the object to indicate a potential that the modifications made by those requests will have inverted ordering.

For further explanation, FIG. 17 sets forth a flowchart illustrating an example method of controlling the direction of replication between cross-region replicated object stores according to some embodiments of the present disclosure. In the example of FIG. 17 like numerals correspond to like elements with respect to the storage environment discussed above with reference to FIG. 4. The example storage topology of FIG. 17 is different from the example storage topology of FIG. 4 in that, in FIG. 17, a first storage system 410 and a second storage system 412 are configured for unidirectional replication of a replicated object store 440.

While symmetric replication of an object store is discussed above, replication may also be based on unidirectional, but reversible, replication of an object store from one location to another. In such examples, replication is directional and clients should interact with one side of the replication until there is some event that directs clients to interact with the other side of the replication. This event could be a fault which brings down the original servicing side, causing the other side to become the servicing side, or it could be a command that reverses the direction of replication. It could also be that there is a configured "preference" for which side services object store requests, so that

after a fault temporarily brings down the “preferred” side and enables object store request servicing from the other side, when the fault is remedied the servicing side reverts back to the preferred side.

The ordering of requests can be assured, similar to the above-described time constraints, by introducing a delay within the storage system itself when taking over service to the object store. Such delays ensure that any requests received and processed by the new servicing source storage system are processed at a time far enough after any requests that might have been received and processed (and perhaps not yet transmitted) by the original servicing storage system such that the new requests are given a time stamp that is assured to be higher than time stamps assigned by requests that may have been processed by the original servicing storage system. Thus, such delays ensure eventual consistency when communication is restored.

The example method of FIG. 17 includes receiving 1702, from a first storage system 410 system by a second storage system 412, replicated objects of an object store 440 serviced by the first storage system, wherein the first storage system 410 and the second storage system 412 are configured for replication of the object store, and wherein the second storage system is not configured to service requests to modify the object store 440. In the example of FIG. 17, the second storage system 412 is initially the replication target and the first storage system 410 is initially the replication source and services host requests directed to the object store. Updates to the replicated object store 440 are made through the first storage system 410, which replicates those updates to the second storage system 412. That is, updates are written to the local copy of the replicated object store 440 on the first storage system 410 and those updates are replicated to the local copy of the replicated object store 440 on the second storage system 412. The host 470 is configured for communication with both storage systems 410, 412. However, the replication target (in this case, storage system 412) does not service requests directed to the replicated object store 440. Although, in the example of FIG. 17, the second storage system is initially the replication target and the first storage system is initially the replication source, in other examples the second storage system may be the initial replication source and the first storage system may be the initial replication target.

In some examples, the storage systems 408, 410, 412 are at different localities 480, 482. For example, the different localities can correspond to different availability zones, similar to or identical to AWS availability zones or availability zones of another cloud services provider. In other examples, the different localities can correspond to different regions, similar or identical to AWS regions or regions of another cloud services provider. Some storage systems may be located in different availability zones of the same region while another storage system is located in a different region. In yet other examples, the different localities correspond to different data centers that are physically separated by a substantial distance (e.g., 1000 kilometers or more).

The method of FIG. 17 also includes determining 1704, by the second storage system 412, a minimum time interval 1703 for taking over service for the object store 440 from the first storage system 410, such that delaying takeover until reaching the minimum time interval 1703 ensures that modification requests received by the first storage system 410 prior to a takeover are correctly ordered as being earlier than subsequent modification requests serviced by the second storage system 412 subsequent to the takeover. In some examples, the storage systems 410, 412 determine the mini-

imum time interval 1703 by calculating the uncertainties in tracking each other’s local clocks using, for example, techniques discussed above. A storage system may measure clock uncertainty by sending a clock request message and receiving a clock request response that includes the value of the responding storage system’s local clock. The first storage system may identify the uncertainty of the second storage system’s local clock based on a round-trip messaging time, that is, the time difference between sending the clock request message and receiving the clock request response relative to the first storage system’s local clock. In some cases, the amount of time specified by the minimum time interval 1703 is equal to the worst-case uncertainty, although in other cases the minimum time interval 1703 may account for an additional amount of time to compensate for clock drift, the time since the last clock exchange, or may subtract an amount of time that accounts for physical distance as maximum speed of travel for a packet may not be a source of uncertainty given basic physics, or may account for other variables as discussed above.

The minimum time interval 1703 is useful to ensure consistency and logical write ordering when reversing the direction of replication. The timestamp given to an object update immediately preceding the reversal should be guaranteed to occur before the timestamp given to an object update immediately after the reversal. That is, the last object update processed by the old replication source should have a timestamp that is older than the first object update processed by the new replication source once the replication direction is reversed. To aid explanation, consider an example where the second storage system 412 is the replication target and the first storage system 410 is the replication source. Assume, for the sake of example, that the local clock 464 of the second storage system 412 is 50 milliseconds slower than the local clock 462 of the first storage system 410. If the direction of replication is then reversed and the second storage system 412 immediately begins processing new object store updates, a new object store update received by the second storage system 412 (the new replication source) within 50 milliseconds of the reversal could receive a timestamp that is earlier than an object store update received and processed by the first storage system 410 before the reversal. If a minimum time interval of 51 milliseconds or more is used to separate the processing of the object updates before and after the reversal of the replication direction, the ordering problems may be avoided.

In some examples, the second storage system 412 determines a minimum time interval 1703 that accounts for a fault detection time by both the first storage system 412 and the second storage system 412 where a fault is related to an inability to communicate between the first storage system and the second storage system. When an inability to communicate is discovered, the second storage system cannot know if it is due to a fault in the first storage system or a network failure, thus it is possible that the first storage system may still be operational and servicing requests for the object store 440. For example, the inability to communicate may be detected based on an expectation of some kind of periodic message exchange among the replication storage systems. In one example, the replicating storage systems 408, 410, 412 may operate on communications leases, where the lease must be periodically renewed. If a lease expires and is not renewed with respect to a particular storage system, it can be determined that there is an inability to communicate with that storage system. As another example, a quorum-based protocol may require that the replicating storage systems, from time to time, reestablish a quorum. Storage

systems that did not manage to establish themselves as part of a quorum may go into a fault handling state, while storage systems that did establish themselves as part of a quorum may recognize that some of the storage systems replicating the object store are no longer part of the quorum set.

As such, in some examples, the minimum time interval 1703 accounts for the time it will take for the first storage system 410 to identify that it is unable to communicate updates for the object store 440 to the second storage system and take itself offline for the object store by rejecting new requests. This amount of time can include the time to detect the fault as well as the time the first storage system will wait before it takes itself offline after detecting the fault. In such examples, the minimum time interval 1703 includes the uncertainty arising from clock coordination as well as this fault discovery and handling time. The minimum time interval 1703 may also account for the amount of time the storage systems will wait between detecting the fault and taking a failure recovery action such as requesting mediation and/or enacting quorum protocols. To aid illustration consider an example where the uncertainty in clock coordination is 50 milliseconds between a first storage system and a second storage system and the second storage system's estimated time that the first storage system will take to detect a fault (e.g., due to a lapsed message exchange with the second storage system) and start delaying operations is N milliseconds. N may be estimated based on, for example, a maximum potential messaging latency and processing latency for the first storage system to determine that there is a lapse in message or lease exchange and enter a faulted state, and, in some cases, an estimated amount of time for a mediation or quorum protocol to complete so that the first storage system takes itself offline for the replicated object store. In such an example, the second storage system 412 may determine the minimum time interval 1703 to be N+50 milliseconds.

The method of FIG. 17 also includes identifying 1706, by the second storage system 412, a trigger initiating a takeover of the service of the object store 440 from the first storage system 410. In some examples, the second storage system 412 identifies 1704 the trigger initiating the takeover by receiving an administrative command instructing the second storage system to take over service to the object store 440. The administrative command may also instruct the second storage system to reverse replication such that updates to the object store are subsequently replicated to the first storage system. In other examples, the second storage system 412 identifies 1704 the trigger initiating the takeover by detecting a fault in the ability to communicate with the first storage system 410. In such examples, the second storage system 412 utilizes the minimum time interval 1703 that accounts for the fault detection and handling time.

The method of FIG. 17 also includes delaying 1708, by the second storage system 412, a takeover of servicing requests to modify the object store 440 by at least the minimum time interval 1703. Upon identifying the trigger to take over service of the object store, the second storage system 412 waits until at least the minimum time interval has elapsed before it begins accepting and processing requests directed to the object store 440. Once the second storage system has taken over it will also attempt to replicate those updates to the first storage system 410. If still unable to communicate with the first storage system 410, the second storage system 412 will replicate its updates once/if the fault is resolved. The second storage system 412 may also receive updates to the object store from the first storage system 410 that the first storage system 410 received and processed, but

was unable to communicate, prior to the takeover. As a result of delaying the takeover by at least the minimum time delay, any modification that was processed by the first storage system 410 before the delay is guaranteed to have an earlier time stamp than any modification received and processed by the second storage system 412 after the takeover.

To aid illustration, consider an example where a first request to store a first version of an object is serviced by the first storage system 410 prior to the trigger. The first version is associated with a clock value of the first storage system 410. A second request to store a second version of the object is serviced by the second storage system 412 subsequent to the takeover by the second storage system 412. The second version is associated with a clock value of the second object storage system. When the replication reverses immediately after the takeover (when the trigger is an administrative command) or subsequently after communication is restored (when the trigger is a communications fault), the second version will be ordered after the first version. That is, wherein the second version is ordered after the first version by both storage systems after replication of the first version to the second storage system and replication of the second version to the first storage system.

For further explanation, FIG. 18 sets forth a flowchart illustrating an example method of application-managed fault handling for cross-region replicated object stores according to some embodiments of the present disclosure. In the example of FIG. 18 like numerals correspond to like elements with respect to the storage environment discussed above with reference to FIG. 4. Like FIG. 4, FIG. 18 includes a plurality of storage systems 408, 410, 412 that replicate an object store 440. In some examples, the plurality of storage systems 408, 410, 412 have a symmetrical replication relationship with respect to the object store 440, in that each storage system services requests directed to the object store, updates its local copy of the object store in response to requests, and replicates those updates to the other storage systems replicating the object store 440. In some examples, the storage systems 408, 410, 412 utilize an eventual consistency model for replication. In other examples, the storage systems 408, 410, 412 employ synchronous replication. In still further examples, one pair of the storage systems 408, 410, 412 may use synchronous replication while another pair uses eventual consistency. One such example is where one pair is close enough in proximity to rely on synchronous replication but another pair are too remote from one another to rely on synchronous replication.

In some examples, the storage systems 408, 410, 412 are at different localities. For example, the different localities can correspond to different availability zones such as AWS availability zones or availability zones of another cloud services provider. In other examples, the different localities can correspond to different regions such as AWS regions or regions of another cloud services provider. Some storage systems may be located in different availability zones of the same region while another storage system is located in a different region. In yet other examples, the different localities correspond to different data centers that are physically separated by a substantial distance (e.g., 1000 kilometers or more). For example, any two storage systems among the plurality of storage systems 408, 410, 412 may be in the same availability zone, different availability zones of the same region, or different regions.

Continuing the APIs discussed above, an API can provide clients with information about faults that have not yet been acted upon, particularly communication faults where two

sides that are replicating between each other may still be running but are not currently communicating. In such cases, application infrastructure that is outside of the storage systems themselves might be able to choose one side to continue running preferentially, which could involve simply temporarily allowing requests to operate to the existing servicing side but with the servicing no longer concerned with replicating to the other side, or which could involve switching which side is servicing the object store requests. Such an API can be used by application-side infrastructure to control fault handling when communication is down between replicating object storage systems. The API can be used by the application-side infrastructure to learn that the object storage systems are not currently communicating, how long it has been since they last communicated, or how long it will be before the object storage systems themselves might make a decision concerning how to proceed (such as by shutting down replication and enabling only one side to continue servicing requests). The API can further be used by application-side infrastructure to instruct the non-communicating storage systems how to proceed, such as by accepting that replication is no longer operating such that the storage systems are now independent of each other. At that point, it is up to the application to avoid making incompatible requests to both storage systems as the storage systems themselves no longer have any means of ordering or of ensuring that when they resume communication that conflicting requests will be resolved predictably.

The method of FIG. 18 includes determining 1802, by a first storage system 410 among a plurality of storage systems 408, 410, 412 replicating an object store 440, a faulted state in response to identifying a fault that prevents replication of updates to the object store 440 to at least a second storage system 412 of the plurality of storage systems 408, 410, 412. In some examples, the first storage system 410 determines 1802 the faulted state by detecting an inability to communicate with the second storage system 412. For example, a network partition or other communications disruption may lead to an inability to exchange messages between the storage systems, or the second storage system may have faulted and is in the process of rebooting. When the first storage system 410 (and the second storage system if still operating) detects the faulted state, the storage system pauses the servicing of requests directed to the object store 440. In some examples, during the I/O pause, new requests are accepted but not processed. In other examples, during the I/O pause, new requests are rejected.

The method of FIG. 18 also includes providing 1804, through an API, an indication that the first storage system 410 has entered the faulted state. In some examples, an object store platform exposes an API through which the object store platform provides 1804 the indication of the faulted state to a host 470 that utilizes the object store 440. The object store platform may be a component of some or all of the storage systems 408, 410, 412 replicating the object store 440 or a component of a separate management server associated with the storage systems. In some examples, the API is a subscription-based API through which the indication of the faulted state is pushed to the host 470. The indication may be a message or other notification that identifies the first storage system.

The method of FIG. 18 also includes receiving 1806 a request 1803 indicating how to proceed in the presence of the faulted state. In some examples, the object store platform receives 1806 the request 1803 through a call to the API by the host 470. The request indicates whether the first storage system 410 system should take itself offline for the object

store by rejecting future requests while the faulted condition remains present, or whether the first storage system 410 should continue to service the object store 440 in the presence of the fault condition. Thus, the plurality of storage systems 408, 410, 412 rely on the host 470 as an external fault handler. If the fault condition is removed, either through restoration of communication between the storage systems or the failed storage system coming back online, the storage systems can return to normal operation where all storage systems are servicing the object store 440 and replicating updates to the other storage systems, including the replication of any missed updates during the fault. On the other hand, the plurality of storage systems may be reconfigured to replace the faulted storage system.

For further explanation, FIG. 19 sets forth a flowchart illustrating another example method of application-managed fault handling for cross-region replicated object stores according to some embodiments of the present disclosure. The method of FIG. 19 extends the method of FIG. 18 in that the method of FIG. 19 also includes identifying 1902 that the request 1803 indicates that the first storage system 410 should locally disable servicing of the replicated object store 440. In some examples, the host 470 issues a request to the first storage system 410 to disable its servicing of the replicated object store in response to the indication of the faulted state. For example, the host 470 may have elected a different storage system to service the replicated object store and thus, in the presence of the fault condition, the first storage system 410 should take itself offline for the object store.

The method of FIG. 19 also includes discontinuing 1904, by the first storage system 410, service to the replicated object store 440. In some examples, the first storage system 410 discontinues servicing of the replicated object store 440 by rejecting all new requests to the object store in the presence of the fault condition.

For further explanation, FIG. 20 sets forth a flowchart illustrating another example method of application-managed fault handling for cross-region replicated object stores according to some embodiments of the present disclosure. The method of FIG. 20 extends the method of FIG. 18 in that the method of FIG. 20 also includes identifying 2002 that the request 1803 indicates that the first storage system 410 should resume locally servicing the object store 440 in the presence of the fault that prevents replication. In some examples, the host 470 issues a request that the first storage system continue to service the replicate object store in the presence of the fault condition without concern for replicating updates to the second storage system 412.

The method of FIG. 20 also includes servicing 2004, by the first storage system 410 in response to the request 1803, the replicated object store 440. When the request 1803 indicates that the first storage system 410 should continue to service the object store, the first storage system 410 resumes or 'un-pauses' its servicing of the object store 440 and begins accepting requests for modifications to the object store.

The method of FIG. 20 also includes discontinuing 2006, by the first storage system 410, replication of updates to the object store 440 to the second storage system 412. While the fault condition persists, the first storage system 410 will service the object store 440 without replicating updates to the second storage system 412.

For further explanation, FIG. 21 sets forth a flowchart illustrating another example method of application-managed fault handling for cross-region replicated object stores according to some embodiments of the present disclosure.

The method of FIG. 21 extends the method of FIG. 20 in that the method of FIG. 21 also includes requesting 2102, by the first storage system 410, mediation from a mediator, wherein servicing the replicated object store in the presence of the fault condition proceeds only if mediation was successful. In some examples, before resuming service to the replicated object store, the first storage system 410 first requests mediation from a mediator. If the first storage system 410 wins mediation, in that the mediator receives the first storage system's request first, the first storage system will resume servicing the replicated object store. However, if the first storage system 410 loses mediation, then the second storage system 412 is also operational and may be continuing to service the replicated object store 440. Given that the two storage systems are unable to communicate updates, this can lead to a split-brain scenario. One or both of the storage systems can inform the host 470 that the second storage system 412 won mediation and let the host 470 decide on how to proceed, or the first storage system 410 can remain offline for the object store. It could be the case that the host 470 failed to request the second storage system 412 to disable servicing of the object store 440.

For further explanation, FIG. 22 sets forth a flowchart illustrating another example method of application-managed fault handling for cross-region replicated object stores according to some embodiments of the present disclosure. The method of FIG. 22 extends the method of FIG. 18 in that the method of FIG. 22 also includes providing 2202, through the API, a parameter indicating how long the first storage system 410 has been unable to replicate updates to the second storage system 412. In some examples, in addition to the indication of the faulted state, the API provides a parameter indicating how much time has elapsed since the first storage system 410 last successfully communicated with the second storage system 412. For example, the parameter may indicate how much time since the first storage system received an update, message, message acknowledgement, or any other communication from the second storage system 412, or how long it has been since completing a such a bi-directional message exchange (for example, how long it has been since a message was sent that resulted in a received acknowledgement).

For further explanation, FIG. 23 sets forth a flowchart illustrating another example method of application-managed fault handling for cross-region replicated object stores according to some embodiments of the present disclosure. The method of FIG. 23 extends the method of FIG. 18 in that the method of FIG. 23 also includes providing 2302, through the API, a parameter indicating how long until an automatic fault handling action is initiated by one or more of the plurality of storage systems. In some examples, in addition to the indication of the faulted state, the API provides a parameter indicating a duration of time before the first storage system 410 will initiate an automatic fault handling action such as mediation of use of a quorum-based protocol. Absent any indication from the host 470, the first storage system 410 may initiate mediation, which may result in the first storage system 410 winning mediation and remaining online for the object store or losing mediation and taking itself offline for the object store, despite any preference the host might have. Similarly, absent any indication from the host 470, the first storage system 410 may initiate a quorum-based protocol, which may result in the first storage system 410 being part of a quorum of still-communicating storage systems and thus remaining online for the object store, or not being part of a quorum and taking itself offline for the object store, despite any preference the host might have. Thus, the

parameter indicating how long until the first storage system 410 initiates automatic fault handling tells the host 470 approximately how much time it has to indicate to the first storage system 410 whether it should continue to service the object store or disable servicing.

For further explanation, FIG. 24 sets forth a flowchart illustrating another example method of application-managed fault handling for cross-region replicated object stores according to some embodiments of the present disclosure. The method of FIG. 24 extends the method of FIG. 18 in that the method of FIG. 24 also includes providing 2402, through the API, an indication of which of the plurality of storage systems to which the first storage system 410 is unable to replicate updates. In some examples, when indicating the faulted state through the API, the indication also identifies one or more storage systems with which the first storage system 410 is unable to communicate. For example, the faulted state may be indicated with respect to a particular storage system among other storage systems replicating the object store 440.

For further explanation, FIG. 25 sets forth a flowchart illustrating another example method of application-managed fault handling for cross-region replicated object stores according to some embodiments of the present disclosure. The method of FIG. 25 extends the method of FIG. 18 in that the method of FIG. 25 also includes providing 2502, through the API, an indication of one or more storage systems that are currently operating to service the object store 440 and to which storage systems each of the one or more storage systems is currently successfully replicating updates. In some examples, the object store platform is a component of a management server or other object store platform associated with the storage systems that has global information on which storage systems are operating normally. In other examples, the object store platform is a component of the first storage system, where the first storage system has a global awareness of which storage systems are operating normally through an exchange of messages and status updates. In either case, the API exposed by the object store platform may provide information that maps which storage systems are currently servicing the object store 440. This information may also map, for each servicing storage system, to which of the other storage systems updates are being successfully replicated. This information provides the host 470 with a picture of which storage systems may be servicing the object store but that are unable to replicate updates to all of the other systems.

For further explanation, FIG. 26 sets forth an example environment for implementations of high availability and disaster recovery for replicated object stores according to some embodiments of the present disclosure. The example of FIG. 26 includes storage systems 2610, 2612, 2614, 2616 that replicate objects in a bucket 2640 in a replicated object store (e.g., the object store 440 described above). In some examples, the storage systems 2610, 2612, 2614, 2616 are configured the same as, or similar to, any of the storage systems described above. For example, the storage systems 2610, 2612, 2614, 2616 may implement the same functionalities as the storage systems 408, 410, 412 described above with reference to FIGS. 4-25. In some examples, the bucket 2640 includes the same properties as buckets of the object store 440 as described above. For example, the bucket 2640 stores objects that are replicated among the storage systems 2610, 2612, 2614, 2616 and each storage system 2610, 2612, 2614, 2616 may store a local copy of the bucket 2640. An object store implementation that includes bucket 2640 primarily stores immutable content. Content is immutable if it

cannot be modified except by deletion or replacement. In typical object stores, the content of an object is immutable in that once created it can only be deleted, or replaced with a new version of the object where that new version is identifiably different from the prior version. Immutable content in an object store can be formed by an operation to store specific content (such as through an operation to PUT an object) or by copying other immutable content (such as through an operation to copy another object). Two PUTs of objects of the same name result in different versions of the object which are identifiably different. In a versioned bucket, two PUTs of objects of the same name generally result in two versions being stored for one object of that name, where one of the two objects is considered current. In a non-versioned bucket, two PUTs of objects of the same name will generally result in the content from one of the two PUTs being effectively discarded due to the first object being replaced by the second.

Immutable content simplifies the operation of object stores, and the replication of object stores, because once a source of content has been identified and tied to a particular variant of that content, modifications to that content will not complicate further operations related to completing an operation, or replication or tiering or fault recovery or any other clustering or administrative tasks related to the operating the dataset. The remaining complexities relate to ensuring that versions or replacements of objects are handled consistently by any additional storage systems storing separate copies of the object, and ensuring that existing content used to establish new content (such as by copying or by inclusion in a new composite object) uses a consistent source of immutable content for establishing the new immutable content.

The storage systems **2610, 2612, 2614, 2616** may employ a variety of replication models for replicating objects in the bucket **2640**. In some examples, the two or more of the storage systems **2610, 2612, 2614, 2616** have a symmetrical replication relationship with respect to the bucket **2640**, in that each of those storage systems services requests directed to the object store, updates its local copy of the object store in response to requests, and replicates those updates to the other storage systems replicating the object store **440**. In some examples, two or more of the storage systems **2610, 2612, 2614, 2616** employ synchronous replication. In other examples, two or more of the storage systems **2610, 2612, 2614, 2616** utilize a more relaxed mode of replication such as non-synchronous replication based on an eventual consistency model. In yet other examples, one pair of the storage systems **2610, 2612** may use synchronous replication while another pair of storage systems **2610, 2614** uses eventual consistency. One such example is where one pair is close enough in proximity to rely on synchronous replication but another pair are too remote from one another to utilize synchronous replication with acceptably low operation latency.

In some examples, the storage systems **2610, 2612, 2614, 2616** are at different localities. For example, the different localities can correspond to different availability zones of a cloud services platform (e.g., AWS availability zones or availability). The different localities can also correspond to different regions of a cloud services platform (e.g., AWS regions). Some storage systems may be located in different availability zones of the same region while another storage system is located in a different region. The different localities can also correspond to different data centers that are physically separated by a substantial distance (e.g., 1000 kilometers or more). Thus, any two storage systems among

the storage systems **2610, 2612, 2614, 2616** may be in the same availability zone, different availability zones of the same region, or different regions. To aid illustration of the following discussion, FIG. **26** depicts an example where some storage systems **2610, 2612** are located in respective availability zones **2620, 2622** of a first region **2630** while storage systems **2614, 2616** are located in respective availability zones **2624, 2626** of a second region **2632**. However, it will be understood that implementations in accordance with the present disclosure are not limited to the arrangement shown in FIG. **26**. Further, it will be understood that implementations in accordance with the present disclosure are not limited to the number of storage systems shown, the number of availability zones shown, or the number of regions shown. It will also be understood that implementations in accordance with the present disclosure do not require a cloud services platform or availability zones and regions.

In some examples, two or more of the storage systems **2610, 2612, 2614, 2616** (e.g., storage systems **2610, 2612**) employ a symmetrical and synchronous replication model. These two or more storage systems may form a synchronous replication group, also referred to herein as a ‘cluster.’ The storage systems in the symmetrically and synchronously replicating cluster may be located in the same region, such that communication latencies between the storage systems is low enough to facilitate the messaging required for synchronous replication; however, it is not a requirement that the storage systems be in the same region as synchronous replication of updates to a bucket can be achieved across regions, or certain operations can be synchronously replicated while other types of operations are not synchronously replicated. The replication model of the cluster is symmetrical in that each storage system can receive and service host requests for operations on the bucket and replicate those operations to other storage systems in the cluster. The replication model of the cluster is synchronous in that a storage system does not acknowledge to a host that a request is complete until all of the other storage systems in the cluster have acknowledged that they have also applied the replicated operation to their local copy of the bucket.

In some examples, the cluster employs a leader/follower model to order dependent operations for a consistent result. For example, the leader/follower model can be used for the ordering of operations for the putting of objects, of parts of objects, of virtual or physical copying of objects or parts of objects to form new objects or new parts of objects, of versions, or parts of versions, of buckets, and of modifications to objects or buckets. Dependent operation ordering can then ensure that operations, such as creating a bucket of a name and then putting of an object into a bucket of a name and an operation to delete a bucket with the same name, even if they come in to different storage systems, are ordered consistently. For example, such that a bucket delete happens before the putting of the object, so that the put of the object consistently fails because the bucket does not exist, or such that the delete of the bucket happens before the create of the bucket, so that the bucket delete consistently fails (because no such bucket exists) and the other two operations succeed, or the object put happens first, resulting in consistent failure because no such bucket yet exists. A clock lease exchange can be used to determine whether a storage system is potentially entering, or has entered, a fault condition, and the same or a similar message exchange model can reduce the synchronous rebound trip message exchanges to one.

In some implementations, when cluster communication leases are used, the processing of operations can delay as

soon as a lease has expired and operations can be failed if subsequent attempts to remain online (such as through use of quorum, mediation, or an agreed upon survivorship) are unsuccessful. If a subsequent attempt to remain online is unsuccessful, operations can continue with adjustments in replication targets based on which storage systems remain online as part of the cluster.

In some implementations, the cluster employs mediation for fault handling, for example, when two storage systems are unable to communicate. A partition identifier can be used to identify a cluster prior to one member being taken offline, and when a first storage system can no longer communicate with a second storage system in the cluster, that storage system can attempt to establish a new partition identifier through an exchange with a mediator which either succeeds, in which case the storage system continues running under a new partition identifier and with the second storage system now excluded from the cluster, or the exchange fails (presumably because another storage system exchanged a different partition identifier with the mediator first) and the first storage system stops operating for the object store, or at least the part of the object store that is subject to the mediation. The removed storage system may attempt to rejoin the cluster when communication resumes.

In some implementations, the cluster employs a quorum protocol for fault handling. In such examples, the online storage systems that are still in communication determine whether their number is sufficient for a quorum. If there is a sufficient number to form a quorum, those storage systems continue to service the bucket while discontinuing replication to the non-communicating storage systems. If there is not a sufficient number to form a quorum, those storage systems take themselves offline for the bucket and may attempt to rejoin the cluster when communication resumes.

In some implementations, when a temporarily offline storage system rejoins a synchronously replicated cluster, the rejoining storage system can be caught up to match the currently online state of the cluster, which may involve backing out operations that had not successfully made it into the online cluster prior to failure, and then receiving all modified state from the online cluster. This can be based on differencing from some checkpoint that predated the rejoining storage system having gone temporarily offline, or it can be based simply on knowledge of what buckets, objects, versions, and modifications had been created, modified, or deleted recently enough that they might not have been stored on (or removed from) the rejoining storage system.

Catching up a joining storage system so that it has all the same content as the online storage systems, while the online storage systems are serving the object store and receiving new updates, does require some care. For example, in some implementations, the cluster may go into a mode where the joining storage system receives new updates even though it may not yet have some of the objects or buckets (or modifications) which those updates rely on. To handle this, there may be a transitional state where updates are received by the joining storage system but stored for later application once the transfer of prior buckets and objects have completed.

Implementations may handle this resynchronization associated with rejoining by tagging buckets, objects, and versions with some unique identity that is not the bucket, object, or version name and that is not reused. Then, replication of updates associated with new requests can indicate that they depend on some prior existing bucket, object, or version as identified by those unique identifiers. If an update to a bucket, object, or version with a unique identifier is not yet

present on the joining storage system, then that update can be left to wait for that identifier to be received. In the meantime, a background task can replicate all buckets, objects, and versions that may have been added while the joining storage system was out of the cluster. Bucket, object, and version deletions can be handled in a variety of ways. In one example, the online storage systems keeps a list of deleted buckets, objects, and versions (with the list recording at least those deleted since the online storage systems had removed the now joining storage system from the cluster). In another example, the online storage systems use snapshots and snapshot differencing to notice deleted buckets, objects, and versions. In yet another example, the joining storage system determines which buckets, objects, and versions it has (with their unique identifiers), sends that list of unique identifiers to the online storage systems of the cluster, and the online storage systems respond by sending the buckets, objects, and versions that the joining storage system does not have, and also by sending back the list of buckets, objects, and versions that no longer exist for the online storage systems so that the joining storage system can delete them as part of joining the cluster. Implementations may also include a unique update identifier associated with metadata updates to a bucket, object, or version (such as changes to properties like tags, policies, or authorizations) where, for example, each bucket or object (or version of an object) can have an associated unique update identifier that is changed whenever the bucket, object, or version is modified. Then, the joining storage system's list of its buckets, objects, and versions can also indicate the identifiers for their last updates, so that the metadata can be copied to the joining storage system if the joining storage system does not have the most up-to-date metadata for a bucket, object, or version.

Once caught up, the joining storage system can then become an online member of the cluster servicing the object store, and can then receive requests or can take over in case of future faults. Further, the now-joined storage system can participate with mediator or quorum models depending on how the cluster operates and how many storage systems are replicating the object store. Note that if there was a multipart upload in progress at the time a storage system was removed from the cluster, or that is in-progress during the join, those (and the parts that have so far been uploaded) can also be included in the join model. They can represent another type of entity managed similarly to the descriptions above. In some implementations, adding a new storage system that synchronously replicates an existing object store is basically the same as rejoining except that the joining storage system starts out with nothing, so all buckets, objects, and versions will be transferred from existing online cluster members.

Note that in the above description, the term 'storage system' can refer to a set of virtual or physical storage systems, for example at a single location, that together form a scale-out object store system at that location. This can operate, in a simple example, by having each of the participating virtual or physical storage systems being updated with the list of buckets for the object store, but with objects being stored on a single storage system, or perhaps erasure coded across a subset of the storage systems that form the scale-out object store. This scale-out object store can have its own internal models for determining whether parts of the scale-out object store have failed, and that cooperate in some way to communicate their state to other members of the synchronous replication cluster described above.

A variant model for symmetric synchronous replication establishes a bucket-to-bucket relationship or a property of

a particular bucket being that it is synchronously replicated between a particular set of storage systems. In a bucket-to-bucket relationship, each storage system (or scale-out object store) has its own list of buckets where some of those buckets have the property that they are symmetrically and synchronously replicated with buckets on one or more other storage systems (or other scale-out object stores). In such implementations, each storage system (or local scale-out object store) has a distinct bucket, and some of the discussions above relating to replication and recovery for the creating and deleting of buckets may not apply, while the ordering of object, version, and object/version metadata updates may apply.

The above description relates to example implementations for synchronous replication of object stores. One of ordinary skill in the art will recognize that these techniques can be used in combination with alternate implementations that yield similar results.

As mentioned above, the storage system **2610**, **2612**, **2614**, **2616** may employ more relaxed modes of replication having more relaxed semantics than the synchronous replication model described above. In some implementations, two or more of the storage systems may employ non-synchronous unidirectional replication, where a bucket stored on one storage system, such as at one location, is replicated to the same or a different bucket on another storage system, such as at another location. For example, storage system **2610** may unidirectionally replicate a bucket to storage system **2614**, while synchronously replicating the bucket with storage system **2612**. Variants where the separate locations have separate buckets may support some form of best effort semantics for replicating updates to the remote location, and may support reversibility of replication which might involve configuring a new replication mode which runs in the reverse direction with potentially relaxed guarantees for updates that are in flight at the time of reversal. Such models can be used to distribute objects or a bucket to alternate locations, such as for test/dev or for applications like media streaming where copies of data at alternate locations can reduce cross-region network traffic. Such models can also be used for disaster recovery where some loss of service at one location can result in takeover of the service at another location, and where there is some acceptance of data loss in exchange for getting the service back up and running quickly.

In some implementations, two or more of the storage systems may employ eventually-consistent bidirectional replication, where a bucket on one storage system is bidirectionally replicated with the same bucket, or a different bucket, on another storage system, such that modifications are eventually consistent (and are reflected in the results of read or query requests) across the replicating storage systems, but where there can be periods of time where the separate storage systems do not reflect the same set of modifications or the same order of modifications.

This is not a common classic storage semantic model as it traditionally could not be made particularly usable for real storage systems and real applications. However, object stores have evolved to have the relatively loose consistency semantics needed, and so many applications have adapted to this loose model. The immutable nature of most object store content and a typical object store's semantics for simply replacing (or adding versions) to objects for requests to store multiple objects of the same name also enable looser semantics than would typically work for a block or file store.

In some implementations, two or more of the storage systems **2610**, **2612**, **2614**, **2616** may employ consistent

read-after-write bidirectional replication, where an object store (i.e., a bucket, a set of buckets, or a bucket namespace) is replicated between storage systems and attempts are made to ensure read-after-write semantics, in that a read or query request that follows after a modification request is assured to reflect the modification if that modification is relevant to the read or query request, but where a modification request does not have to be completely replicated before it is signaled as completed. This is a compromise between synchronous, perfectly ordered bi-directional replication and the loose semantics of eventual consistency. This replication model bears some similarity to models implemented in some scale-out or distributed systems that depended on distributed locks where, in order to write to some storage structure (e.g., a block, a file, an Mode, a directory), a system needs to obtain an exclusive lock, and where in order to read a storage structure, a system needed at least a shared lock. Multiple storage systems could read using the same shared lock, but writing from one system requires that all other exclusive or shared locks covering the storage structure be revoked from any other systems. In some examples, distributed locks can be used to implement consistent read-after-write replication, but object models are simple enough that there can be reasonable alternatives. For example, the existence of a modification could be replicated prior to the modification itself being distributed. Basic distributed locks and transmitting of the existence, but not the content, of a modification can provide suitable read-after-write semantics, but may lose those semantics (and even lose some modifications altogether) in the presence of faults. Quorum-style models can also be used to fix the problem of data loss, including but not limited to distributed consensus algorithms such as PAXOS or RAFT for the storing of buckets, objects, and versions. Prior to signaling completion of an operation, a modification is distributed to a sufficient number of systems to ensure that any modifications signaled as completed will survive any allowed sequences of faults, coupled with reads and queries being copied to a sufficient number of systems to ensure that the read or query will encounter at least one system that received a completed modification.

In some cases, cross-region latency may be too high to rely strictly on synchronous replication models, and thus some relaxing of semantics may be necessary to ensure acceptable performance. However, this can depend on the particular operation. In some implementations, a mixture of the above-described replication models can be combined in accordance with various operations. For example, operations to create, modify, or delete buckets, all of which are generally infrequent, could be performed synchronously; modifications of objects properties, or putting of small objects or deleting of objects could use distributed locks; putting of large objects could notify remote storage systems that the object is coming along with its basic properties (so that queries can return the new object in results but reads of the object can wait for the object to be received); and versions could operate using eventual consistency. Some factors for mixing replication models can include traits of particular objects or of buckets. For example, if latency is not an issue for some application, or for certain key parts of an application, then buckets for that application, or objects for those parts of the application, could use synchronous replication even for replication between distant regions, while other objects or buckets could use another replication model.

Relaxed modes of replication can use fault detection and resolution models similar to those described above with respect to synchronous replication. However, in some cases

modifications that had been signaled as completed successfully could be lost, or might appear only eventually, potentially long after the modification was posted and signaled as completed. That may be the case in many fault handling models used with non-synchronous replication that allow at least one storage system to continue running in the presence of a fault that prevents replication from another storage system that could have received, processed, and signaled completion of requests prior to the fault.

With mediation, for example, the same model described previously can still apply. The same partition identifier can be used to identify a cluster prior to one member of the relaxed mode cluster being taken offline, and when a first storage system can no longer communicate with a second storage system in the relaxed mode cluster, that storage system can attempt to establish a new partition identifier through an exchange with a mediator which either succeeds, in which case the storage system continues running under a new partition identifier and with the second storage system now excluded from the cluster, or the exchange fails (presumably because another storage system exchanged a different partition identifier with the mediator first) and the first storage system stops operating for the object store, or at least the part of the object store that is subject to the mediation. The same can apply with quorums, with a preferred survivor, or with backoff from quorums to mediators as the size of the cluster reduces. Readers will recognize that there are a variety of techniques for switching the mediator or mediator service used by a cluster of storage systems, for example, based on the unavailability of a mediator or for administrative reasons. A preferred survivor can be selected at least temporarily between two replicating storage systems if the mediator is unavailable. The differences compared to synchronous replication pertain to the semantics between modifications on the various storage systems that had been signaled as completed, and replication, visibility, and potential loss or delay of those modifications, as well as details for how updates are tracked and conflicts are resolved, as well as how joining or rejoining the cluster operates.

In some implementations, replication modes can be combined based on the different locations the replicated bucket. For example, it can be advantageous to use synchronous replication, and even symmetric synchronous replication, between data centers that are nearby each other, such as those that operate in separate availability zones within a same geographic region. Availability zones are typically constructed as separate data centers that are relatively near each other but are separate enough that they can be on separate power grids, or have separate network connections to the outside world, or so that if there is a fire or natural disaster both are not likely to be affected simultaneously. As long two storage systems are close enough to have reasonably low latencies with each other and have reasonably high network bandwidth between each other using dedicated networks (to ensure good reliability and little network sharing that could result in congestion), then those locations can operate as availability zones within a region.

Regions, by contrast, are expected to be reasonably far from each other, and are generally located to be nearby some service that uses them or nearby customers that use the services provided within them. Replicating between regions can be used for disaster recovery as well, but either at the expense of either high latency for synchronously replicated modifications or with the potential to temporarily or permanently lose some written data that had been signaled as completed for non-synchronously replicated modifications. It is a common feature of cloud platforms that the platform

supports multiple regions, potentially scattered within large countries, between countries within a continent, or even across the world. It is also common for each region to have at least two availability zones. The same division of regions and availability zones can be configured for non-cloud platforms by, for example, configuring equipment in several owned data centers and/or through leasing equipment or racks to hold equipment at shared data centers.

For an object store, replicating between availability zones ensures that localized failures can be handled seamlessly with no data loss, while replicating between geographic regions ensures that the same data is available with low latency at multiple geographic locations and that sufficiently important services can continue running even after a larger scale fault that affects an entire region. Thus, in some implementations, replication between availability zones within a region is treated differently from replication between geographic regions, even if it is the same object store, or the same parts of an object store, that are being replicated both between availability zones within a region and between regions. This can extend to the fault handling models.

In one example, a mixed replication model provides for synchronous (and potentially symmetrical) replication between availability zones of a region, and treats that as an in-region cluster which operates effectively as a single virtual storage system for purposes of cross-region replication. The in-region cluster can use mediation, or a purely in-region quorum protocol, involving only storage systems within the region to manage faults of the storage systems of the various availability zones within the region. This can be used to create a reliable virtual storage system that can interact as a whole with paired virtual storage systems managed through mediation or quorum in other regions. Since in-region storage systems are reasonably close to each other, they should be able to coordinate clocks with a very low uncertainty. This uncertainty, no matter how low, can then be added to the uncertainty computed from clock exchanges between the in-region clusters, for use with object store replication implementations where clocks and clock exchanges, coupled with measurements of clock uncertainty are used in various ways as described previously.

To aid illustration, storage systems **2610**, **2612** in one region **2630** form an in-region cluster and storage systems **2614**, **2616** in another region **2632** form another in-region cluster. In one example, storage systems within each in-region cluster employ a synchronous replication model with respect to paired systems in the cluster, whereas a non-synchronous (e.g., eventually consistent) replication model is used for paired storage systems replicating across regions. For example, storage system **2610** and storage system **2612** synchronously replicate the bucket **2640**, whereas storage system **2610** and storage system **2614** non-synchronously replicate the bucket **2640**. In some implementations, a variant of the leader/follower model is used, in which one of the storage systems acts as an arbiter for eventual consistency results for all storage systems. In a particular example, one storage system **2610** acts as the leader of an in-region cluster of a first region, while another storage system **2614** acts as the leader of an in-region cluster of a second region, and where one storage system **2610** acts as the arbiter of eventual consistency results for cross-region non-synchronous replication for at least a subset of objects of the storage system.

From a state-model standpoint, further, since the storage systems of the in-region cluster synchronously replicate between each other, they can provide a uniform state model

to in-region clusters of other regions. For example, however eventual consistency works in a particular implementation, all the in-region storage systems should be able to continue implementing it with another region, and even if one storage system of an in-region cluster is faulted and at least temporarily removed from the in-region cluster, the other storage systems should have the necessary information to continue operating the eventual consistency implementation operating between the regions.

In some implementations, a particular modification is replicated only once between regions, and a single receiving storage system in a region delivers that modification to the other online storage systems of the in-region cluster. In such implementations, that modification is replicated to the other online storage systems of the in-region cluster (or at least to a suitable plurality of them such as a majority) before responding to the sending storage system in the originating region that the modification has been received and processed by the receiving in-region cluster.

All of this can complicate distributed lock-based consistent read-after-write non-synchronous replication implementations, as the lock represents state that should not be lost if a single node holding the lock on behalf of an in-region cluster is at least temporarily removed from the in-region cluster. To address this problem, the lock state itself can be stored in a reliable clustered database or in a distributed consensus database such as one based on PAXOS or RAFT.

In some implementations, non-synchronous modes of replication can signal as completed a modification request received by one of the storage systems and (locally) processed before the modification is successfully replicated to a non-synchronous replication target, and that modification can be temporarily or permanently lost if the storage system is removed from the cluster due to a fault. If the source is a member of an in-region cluster, there may be another storage system that can continue where the original storage system left off, but even in that case a sufficiently widespread fault (e.g. due to a regional outage such as a wide area network outage) can result in the modification being lost as storage systems in another region take over service from the removed storage system (or the removed in-region cluster).

An additional issue can arise with implementations of consistent read-after-write semantics with non-synchronous replication. In a distributed lock implementation, locks can be held by a storage system at the time that it faults. For the remaining online storage systems to continue, those locks will have to be broken. In an implementation where one storage system announces the existence of an object to paired storage systems, so that reads or queries can know to wait for the object or modification to be received but the originating storage system does not deliver the object itself (or the complete object) to the paired storage systems prior to the fault, then fault handling will likely result in the reads or queries ending their waits and proceeding with the missing modifications.

If the original storage system (or in-region cluster) eventually comes back online, the lost modification can be recovered and replicated. However, if the recovery is hours, days, or even weeks later, that may confuse things as much as helping, so whether or not those modifications should be applied may be an aspect of recovery to consider (for example, this could depend on the duration of the outage). Technically, both keeping and discarding the modifications are reasonably plausible actions. It is also possible that some kinds of modifications could be kept in a side namespace (for example, a clone of a rejoining storage system's, or

in-region cluster's, pre-rejoin bucket or object store, or a preservation of just objects or object versions that were not successfully replicated prior to the fault). In the case of object versions, an implementation can preserve a specially tagged version that is not the "top" version for an object even if it is technically newer than a version of the object that was older but had been successfully replicated prior to the fault. This inverted version stack could be implemented, for example, by having a cluster partition be part of the "age" of a version with technically older, but preserved, versions inheriting a newer partition identifier while the temporarily lost version preserves the older partition identifier. If the original storage system (or in-region cluster) never comes back online, then the modification may be lost anyway.

An aspect of some object stores is that they can have loose models for deletion, where "delete" operations often do little more than attach a "deleted" version to an object, then utilizing a "lifecycle" policy to actually delete objects (usually after some delay). Lifecycle policies can delete objects, or versions of objects, either sometime after they were deleted, or even just based on age, without there ever having been a deletion request. For example, non-current versions can be kept for 30 days. A lifecycle policy is metadata that can be attached to an object, and that can also be attached to a bucket to supply a default lifecycle policy for objects in that bucket. Lifecycle policies generally include a filter which restricts the objects or versions the filter applies to based on combinations of tags or name prefixes (or other object name patterns) and can be applied to objects, non-current versions, or objects marked as having been deleted (or whose current version indicates a deletion). Lifecycle policies can result in objects or versions being deleted, but can also result in objects or versions being moved to a different storage class. These storage class migrations might be performed separately by the separate storage systems of the replicated object store, but are generally unlikely to have a strong semantic effect related to replication so getting them right across replicated object stores is likely not that much of an issue. Deletions are more complicated. Lifecycle policies can delete objects without an explicit delete request. For example, objects with a particular prefix can be deleted 30 days after they were created, though there may be no guarantees on how timely these deletions will be.

These lifecycle policies can present some level of confusion between the various replicated object stores. A further confusion is the time element associated with them. If clocks are not perfectly coordinated, for example, then two separate object stores might not delete objects (or object versions) consistently. This may be acceptable as the deletes are eventually consistent. However, this could result in an object disappearing from one local object store accessed at one point in time for an application and then coming from another local object store after a replication or recovery action, or in the object being accessed by an application at a later point in time through another object store where no lifecycle policy deleted it. In some implementations, for directional object store replication or any implementation which uses a leader/follower model, the lifecycle policies may be controlled by the source object store system (or the leader object store system), but that does not prevent events, such as fault handling or replication direction reversals, from bringing back objects that had disappeared. Applications might have to be prepared for objects to disappear and come back if the application can make requests to separate object stores of a replicated object store. Another issue with lifecycle policies for synchronous replication implementations

is that the actions they take are not based on requests, and as a result, there is no response that can be delayed until some degree of consistency can be achieved.

In some implementations, object stores can have loose semantics for putting a new object in the presence (or lack of presence) of an object of the same name. For example, object stores may not generally support an exclusive object create operation. Thus, two requests can store objects of the same name, and if the object already exists whichever one is processed second becomes the object, overwriting the first. In a versioned bucket, the two PUT objects can instead become separate versions. In a non-versioned bucket, the second PUT replaces the first. In the case of objects for which there was a deletion request, but where the object has not yet been deleted by the object lifecycle policy, a put of an object will generally recreate the object (undoing the deletion) or (for versioned buckets) will store a more recent version than the version marked as “deleted”. These loose semantics do present some potential confusion in the handling of deletions, in particular as it pertains to high availability.

Bucket creations, however, often do require unique names and may fail if the bucket already exists. In AWS, for example, bucket names are even global. A bucket name can be reused if the bucket has been completely deleted, but deleting a bucket is a potentially complex task which cannot necessarily be completed in a timely and predictable manner as it depends on the triggering of lifecycle policies to follow through on the deletion of all versions of all objects of the bucket. If a bucket is replicated between cooperative object store implementations, the separate object stores can coordinate on this, and can use internal unique identifiers to track buckets, bucket deletions, and successful bucket creations of the same name that should replace prior versions (which could imply that a replica of an object store might have missed a bucket deletion and the replica object store should proceed by implicitly deleting the bucket and all objects even if it is not empty).

In some implementations, putting of an object, or of a part of an object, can reference another object, or a part of an object, or a version of an object, or a part of a version of an object, as content rather than supplying the content of the object (or of the part) directly. This results in copying that source into the object. If the source names an object, or a part of an object, then content to be copied can depend on whether the copy is processed before or after a separate operation to replace the content of the source object (or that adds a newer version to the named object). This can result in a potential dependency when replicating objects between locations. Specifically, if the copy is replicated as a copy then the separate locations might copy from different content of the same named object because a first location might process the copy before a new put, whereas the second location might process the new put before the copy.

Implementations should provide an accommodation for such copy operations to ensure a consistent result. A simple implementation that can fix this is to perform the copy on the storage system that received the request, and then replicate the resulting content to paired storage systems rather than replicating the operation as a copy. Another implementation, which can avoid transmitting all of that “copied” content over the replication interconnect, ties the replication of a copy to a version even if the original request copied from a named source object rather than from a specific version. This works well for a versioned bucket. In the case of a non-versioned bucket, a version identifier can still be associated with the request, and if the paired version does not have that

version (or in the case of a versioned bucket where the version had been deleted such as by a lifecycle policy), then the implementation could transmit the missing content from the storage system that originally performed the copy.

Much of the discussion above pertains to basic object stores, where objects (or versions of objects) are stored in one PUT operation, or in one sequence of multi-part upload operations to form a single object, and where, once an object (or a version of an object) has been fully created, that object (or version) remains with the same immutable content until it is deleted. At most, metadata about the object (lifecycle policy, tags, storage class, etc.) can be changed, while the content remains untouched until it is eventually deleted. This is the basic behavior in both AWS and Google Cloud Platform. Google Cloud Platform does provide for compound objects, which can be formed by incorporating the content of existing objects (or parts of existing objects) into new “compound” objects which have the concatenated content of the source objects (or object parts). Google Cloud Platform even supports including other compound objects in a new compound object. Compound objects would have similar issues to object copy operations. An operation to create a compound object includes the content of other objects or other parts of objects at the time the operation to create the compound object is processed. If an object PUT results in an object being replaced, or a new version added, at around the same time as the creation of a compound object, then the compound object will incorporate either the previous version of the object or the new version of the object depending on which is processed first. In a replication scenario, as with object copy operations, this can result in inconsistencies if the object to create the compound object is naïvely replicated. As with the copy operation, this can be remedied by the storage system which first processed the compound object creation associating with the operation the version which it used as the source for the new compound object. As with the copy operation described previously, if that version is not available on the target, such as because the bucket is not a versioned bucket, the content might have to be transferred from the originating storage system. A leader/follower implementation can establish the ordering of a set of object PUT operations and the source for the compounding or copy operations consistently, at least for a leader/follower-based synchronously replicated object store, across the set of storage systems so that mis-ordering will not happen during normal non-faulted operation.

In the context of high availability, where faults and recoveries must be taken into account, a resynchronization operation might have to repair a transient inconsistency for both a copy operation and a compound object creation by either deleting the compound object creation (which is possible if it had not yet been acknowledged) or retransmitting the finalized content for the compound object to any storage system which does not store the version used for that finalized content. This can be more complex in the case of nearly synchronous or periodic replication. However, if nearly synchronous or periodic replication is directional, then ordering of compounding operations, copy operations, and PUT operations can be consistently established by the storage system, or storage systems, that are operating as directional sources.

In the case of symmetric (e.g., bi-directional) non-synchronous (including eventually consistent) replication, the model can fall back to the original storage system processing the request establishing the object version which is the chosen source for the copy or compounding, including that object version’s identity when replicating the operation and

111

transmitting the missing content in the (unlikely) case where the version is not present on the paired storage system.

Azure blob stores have much more problematic variations of object store semantics. Specifically, blobs (Azure's rough equivalent of objects) can either be "append" blobs where objects can be modified by appending to them over time, or they can be "block" blobs which operate as an array of specifically created chunks which can be accessed by an index but where the chunk at any index can be replaced as a way of modifying the content of the blob, or they can be "page" blobs which are essentially 512-byte-sector random read/write entities that can be extended or modified essentially like a file of a file system. All three types of blobs may be too complicated to model in the same simplistic ways as the objects described above. For AWS and Google Cloud Platform content for a version, any content apart from the content associated with a particular object name (and some metadata) is immutable. The only common exception to this, multipart uploads, have very restricted behaviors which make replication semantics relatively simple.

Fully providing Azure blob store semantics, then, is more likely to require the more carefully structured semantics associated with file or block storage replication, meaning that synchronous replication might be possible if something like a leader/follower model is used to establish ordering for conflicting operations, but otherwise something like lightweight checkpoints might be needed for nearly synchronous replication, or full dataset snapshots might be needed for periodic replication.

For further explanation, FIG. 27 sets forth a flow chart of an example method of high availability and disaster recovery for replicated object stores in accordance with at least one embodiment of the present disclosure. The example of FIG. 27 depicts a replication environment that includes, for ease of explanation, three storage systems 2610, 2612, 2614 that replicate objects in a bucket 2640. In the example of FIG. 27, the storage systems 2610, 2612, 2614 are placed at different localities 2720, 2722, 2724. In some examples, these localities correspond to different availability zones and/or different regions. In one example arrangement, locality 2720 and locality 2722 correspond to different availability zones within a first region and locality 2724 corresponds to a second region that is different from the first region. In another example arrangement, localities 2720, 2722, 2724 correspond to different availability zones of the same region. It will be appreciated that the method of FIG. 27 can be implemented with more or fewer storage systems; further, it is not a requirement that the storage systems be dispersed across multiple availability zones or regions.

In some implementations, replication of objects among the storage systems 2610, 2612, 2614 is symmetrical, in that each storage system receives and services host requests for operations directed to the bucket 2640 and replicates those operations or the result of the operation to other storage systems in the replication group for the bucket 2640. In further variants, replication among at least two storage systems 2610, 2612 is synchronous, in that a request for an operation on the bucket 2640 is not acknowledged as complete until a result of the operation has been consistently applied to each of the storage systems 2610, 2612 synchronously replicating the bucket 2640. In alternative variants, at least two of the storage systems 2610, 2612, 2614 employ a form of non-synchronous replication, such as eventually-consistent non-synchronous replication.

The method of FIG. 27 includes, receiving 2702, by a first storage system 2610 of a plurality of storage systems 2610, 2614, 2616 symmetrically replicating objects of a bucket

112

2640, a request 2750 to establish immutable content for the bucket 2640. The immutable content may be content for an object that is associated with a specific object name or object key. In some examples, the first storage system 2610 receives 2702 the request 2750 to establish immutable content in the form of a host application's request for an operation to store specific content in the bucket 2640 (such as an operation to PUT an object) or an operation to copy other immutable content (such as an operation to copy another object). As discussed above, two PUTs of objects of the same name can result in different versions of the object which are identifiably different. In a versioned bucket, two PUTs of objects of the same name generally result in two versions being stored for one object of that name, where one of the two objects is considered current. In a non-versioned bucket, two PUTs of objects of the same name will generally result in the content from one of the two PUTs being discarded due to the first object being replaced by the second.

The method of FIG. 27 also includes indicating 2704, by the first storage system 2610 to a second storage system 2612 of the plurality of storage systems 2610, 2612, 2614, the request 2750 to establish immutable content, wherein the second storage system 2612 establishes an ordering for conflicting requests of different storage systems to establish immutable content for the bucket 2640. In some examples, the first storage system 2610 indicates 2704 the request by sending a result of the requested operation to the second storage system or by sending a representation of the operation itself (e.g., by sending a copy operation without sending the copied data). In some examples, at least the first storage system 2610 and the second storage system 2612 are part of a symmetrical and synchronous replication cluster. In these examples, the first storage system 2610 does not usually acknowledge to the host application that the request is complete until the second storage system 2612 and other storage systems in the cluster acknowledge that the result of the operation has been applied to their local copies of the bucket 2640. In some arrangements, storage system 2614 may also be a member of the synchronous replication cluster and/or other storage systems not depicted may be members of the synchronous replication cluster.

In some examples, the cluster that includes the first storage system 2610 and the second storage system 2612 utilizes a leader/follower model for ordering operations on the bucket replicated across the storage systems in the cluster. For example, if two or more conflicting requests 2750, 2754 are received by different storage systems symmetrically replicating the bucket 2640, the leader storage system determines the ordering for those conflicting requests 2750, 2754. In the example of FIG. 27, the second storage system 2612 is the leader storage system and the first storage system 2610 is one of the follower storage systems. Although FIG. 27 illustrates that the conflicting request 2754 is received from an application by the second storage system 2612, in fact that conflicting request 2754 could be received by any of the storage systems symmetrically replicating the bucket 2640, where indications of those operations or the result of those operations are replicated to the second storage system 2612.

The second storage system 2612, as the leader, is tasked with the ordering of operations for the putting of objects or parts of objects, for virtual or physical copying of objects or parts of objects to form new objects or new parts of objects, for putting versions or parts of versions, for creating or deleting buckets, for modifications to objects or buckets, for establishing the version of an object used for inclusion in a

113

compound object and so on. Dependent operation ordering can then ensure that conflicting operations are consistently applied or consistently discarded. To aid illustration, consider an example that includes creating a bucket associated with a name, putting of an object into a bucket associated with the name, and an operation to delete a bucket with the same name. Even if the requests for these operations are received by different storage systems, the operations are ordered consistently such that, for example, the bucket delete happens before the putting of the object so that the put of the object consistently fails because the bucket does not exist, or so that the delete of the bucket happens before the creation of the bucket, so that the bucket delete consistently fails (because no such bucket exists) and the other two operations succeed, or the object put happens first resulting in consistent failure because no such bucket yet exists.

In one example, the request 2750 is a first operation to establish first immutable content for an object using an object name and a conflicting request 2754 is a second operation to establish second immutable content for another object using the object name. For example, the conflicting request may be a request received by storage system 2612 or by another storage system in the cluster to PUT an object with the same name as the object associated with request 2750. In such examples, the ordering established by the second storage system 2612 indicates that a first object (e.g., the object associated with request 2750) replaces a second object (e.g., the object associated with the conflicting request) for the object name in the bucket 2640. For example, if the first storage system 2610 receives a first request to PUT an object named 'ObjectA' and the second storage system 2612 receives a second request to PUT an object named 'ObjectA,' the second storage system 2612 determines an ordering for the two objects named by determining whether the 'ObjectA' corresponding to the first storage system 2610 will replace the 'ObjectA' corresponding to the second storage system 2612, or vice versa. In a versioned bucket, the ordering indicates which received object becomes the top or current version in the version stack, and which received object is retained as a non-current version.

In another example, the request 2750 is a first operation to establish first immutable content for a first object as a copy of a second object associated with an object name. A conflicting operation is a second operation to establish third immutable content for a third object using the same object name as the second object to replace the second object that includes second immutable content. In this example, the ordering determines whether to copy either the second immutable content or the third immutable content. For example, consider that initially an object having a name 'ObjectA' includes initial immutable content and that a request is an operation to create a new object having a name 'ObjectB' by copying the content of 'ObjectA.' Consider also that there is another operation to PUT another object with the name 'ObjectA' with new immutable content, thus replacing the initial immutable content of 'ObjectA' with the new immutable content. These operations conflict in that different storage systems might apply the operations in different order. That is, does the copy operation result in the copying of the initial immutable content of 'ObjectA' or the new immutable content resulting from the PUT operation to an object of the same name 'ObjectA'? In such examples, the second storage system 2612 establishes the order in which these two conflicting operations are applied. In some examples, where the bucket is a versioned bucket, the copy operation can be associated by the storage system first

114

performing the copy with a specific version of 'ObjectA' to ensure that all storage systems that receive the replicated copy operation know which version's immutable content should be copied, thus resulting in the copy operation being consistently applied by all storage systems in the cluster. The leader can indicate in the ordering information the source version for a copy operation.

The method of FIG. 27 also includes processing 2706, by the first storage system 2610, the request 2750 based on ordering information 2752 received from the second storage system 2612. The conflicting request 2754 is also replicated to the first storage system 2610, thus the first storage system must also process the request 2750 and the conflicting request 2754 in a way that is consistent with all other storage systems replicating the bucket. The first storage system 2610 receives the ordering information 2752 from the second storage system 2612. For example, the second storage system 2612 can send a message to the first storage system 2610 and other storage systems replicating the bucket that describes the ordering information. In some implementations, the ordering information is included in a message that acknowledges that the result of the request 2750 has been applied by the second storage system 2612.

In some examples, the first storage system processes 2706 the request 2750 based on the ordering information 2752 by applying the operations of the conflicting requests based on the ordering information to achieve a consistent result of the conflicting operations. A consistent result is as if the operations were all performed and were performed in the same order, whether or not all the operations were actually performed. Although, in some cases, a consistent result is achieved by applying each operation in the order indicated, in other cases an operation need not be applied at all if that operation is obviated by a subsequent operation. For example, for two operations to store an object of the same name in a non-versioned bucket, one of the operations does not need to be applied at all. For a copy operation in a versioned bucket, where the copy request itself names a version for the copy operation, or where the leader selects a version, then the order of a copy operation and a conflicting PUT operation on the first storage system may be irrelevant.

In some examples, immutable content of the bucket is further replicated to at least one additional storage system (not shown). The additional storage system can be an additional storage system that replicates the bucket but does not do so symmetrically, an additional storage system that replicates the bucket non-synchronously, or any other storage system that is not part of the leader/follower cluster of storage systems described in the preceding description. For such storage systems, any immutable content associated with conflicting operations is established consistently with the ordering determined by the second storage system 2612. Here, consistent means that the conflicts are not resolved in an order, or will not produce an end result, that is different from that determined by the second storage system. For example, updates can be batched to the additional storage system so that a set of already resolved updates happen together rather than happening one after the other.

For example, when the two or more conflicting requests 2750, 2754 include operations for creating a bucket associated with a name, putting of an object into a bucket associated with the name, and deleting a bucket with the same name, the ordering information 2752 indicates the order in which those operations should be applied, and the first storage system 2610 processes the conflicting requests accordingly by applying the operations on its local copy of the bucket 2640 to achieve a result that is consistent with

that ordering. In another example, where the two or more conflicting requests **2750**, **2754** include operations for putting two objects of the same name, the ordering information describes which object replaces the other, and the first storage system **2610** processes the conflicting requests accordingly by applying the operations on its local copy of the bucket **2640** to reflect that a first object replaces a second object. In yet another example, where the two or more conflicting requests **2750**, **2754** include operations to copy an object and to replace the same object, the ordering information describes whether the to apply the PUT of the object first or to apply the copy of the object first, and the first storage system **2610** processes the conflicting requests accordingly by applying the operations on its local copy of the bucket **2640** to reflect a result consistent with that ordering.

In various implementations, the storage systems **2610**, **2612**, **2614** can apply the various fault handling models discussed above. For further explanation, FIG. **28** sets forth a flow chart of another example method of high availability and disaster recovery for replicated object stores in accordance with at least one embodiment of the present disclosure. The method of FIG. **28** extends the method of FIG. **27**, in that the method of FIG. **28** also includes detecting **2802**, by the first storage system **2610**, a fault in communicating with at least one other storage system **2614** of the plurality of storage systems **2610**, **2612**, **2614**. The first storage system **2610** can detect **2802** a communications fault through any of the techniques discussed above with respect to detecting communications faults, such as a network disruption or a storage system temporarily faulting.

The method of FIG. **28** also includes determining **2804**, by the first storage system **2610**, to continue operating with the at least one other storage system **2614** as at least temporarily removed from symmetrically replicating objects of the bucket **2640**. In some examples, the first storage system determines **2804** that it should continue operating based on winning a request for mediation from a mediator. For example, a partition identifier can be used to identify a cluster prior to one member being taken offline, and when a first storage system **2610** can no longer communicate with another storage system **2614** in the cluster, that storage system **2610** can attempt to establish a new partition identifier through an exchange with a mediator which either succeeds, in which case the storage system continues running under a new partition identifier and with the second storage system now excluded from the cluster, or the exchange fails (presumably because another storage system **2614** exchanged a different partition identifier with the mediator first) and the first storage system stops operating for the object store, or at least the part of the object store that is subject to the mediation. The removed storage system **2614** may attempt to rejoin the cluster when communication resumes.

In other examples, a subset (that includes the first storage system **2610**) of the plurality of storage systems determines that it should continue operating based on determining that the subset comprises a quorum of storage systems symmetrically replicating objects of the bucket. For example, the first storage system **2610** and the second storage system **2612** may each detect that communication with a third storage system **2614** has faulted, but also identify that communication between the first storage system **2610** and the second storage system **2612** remains intact. Thus, the first storage system **2610** and the second storage system **2612** may determine that two out of three storage systems replicating the bucket constitutes a quorum, and both storage systems

continue to service requests directed to the bucket and replicate updates between each other, while discontinuing replication to the temporarily removed storage system **2614**.

In some implementations, upon the temporarily removed storage system resuming communication, the bucket is resynchronized to the temporarily-removed storage system, and the temporarily-removed storage system is rejoined such that the plurality of storage systems currently symmetrically replicating objects of the bucket again includes the storage system that had been temporarily removed. In some examples, resynchronizing comprises copying missing immutable content of objects of the bucket to the temporarily-removed storage system and establishing content and names of missing objects and content, identities, and orderings of missing versions of objects. In some implementations, when a temporarily offline storage system rejoins an online cluster of storage systems that continued to symmetrically replicate the bucket **2640**, the rejoining storage system **2614** can be caught up to match the currently online state of the cluster, which may involve backing out operations that had not successfully made it into the online cluster prior to failure, and then receiving all modified state from the online cluster. This can be based on differencing from some checkpoint that predated the rejoining storage system having gone temporarily offline, or it can be based simply on knowledge of what buckets, objects, versions, and modifications had been created, modified, or deleted recently enough that they might not have been stored on (or removed from) the rejoining storage system.

As discussed above, catching up a joining storage system **2614** so that it has all the same content as the online storage systems **2610**, **2612**, while the online storage systems are serving the bucket **2640** and receiving new updates, does require some care. For example, in some implementations, the cluster may go into a mode where the joining storage system receives new updates even though it may not yet have some of the objects or buckets (or modifications) which those updates rely on. To handle this, there may be a transitional state where updates are received by the joining storage system but stored for later application once the transfer of prior buckets and objects have completed.

In some implementations, resynchronization associated with rejoining is carried out by tagging buckets, objects, and versions with some unique identity that is not the bucket, object, or version name and that is not reused. Then, replication of updates associated with new requests can indicate that they depend on some prior existing bucket, object, or version as identified by those unique identifiers, and if a bucket, object, or version with a unique identifier required of an update is not yet present on the joining storage system, then that update can be left to wait for that identifier to be received. In the meantime, a background task can replicate all buckets, objects, and versions that may have been added while the joining storage system was out of the cluster.

Bucket, object, and version deletions can be handled in a variety of ways. In one example, the online storage systems keeps a list of deleted buckets, objects, and versions (with the list recording at least those deleted since the online storage systems had removed the now joining storage system from the cluster). In another example, the online storage systems use snapshots and snapshot differencing to notice deleted buckets, objects, and versions. In yet another example, the joining storage system determines which buckets, objects, and versions it has (with their unique identifiers), sends that list of unique identifiers to the online storage systems of the cluster, and the online storage systems

117

respond by sending the buckets, objects, and versions that the joining storage system does not have, and also by sending back the list of buckets, objects, and versions that no longer exist for the online storage systems so that the joining storage system can delete them as part of joining the cluster. Implementations may also include a unique update identifier associated with metadata updates to a bucket, object, or version (such as changes to properties like tags, policies, or authorizations) where, for example, each bucket or object, and perhaps versions, have an associated unique update identifier that is changed whenever the bucket, object, or version is modified. Then, the joining storage system's list of the buckets, objects, and versions that it has can also indicate the identifiers for their last updates, so that the metadata can be copied to the joining storage system if the joining storage system doesn't have the most up-to-date metadata for a bucket, object, or version.

Once caught up, the rejoining storage system **2614** can then become an online member of the cluster of storage systems servicing the bucket and can then receive requests or can take over in case of future faults. Further, the now-joined storage system can participate with mediator or quorum models depending on how the cluster operates and how many storage systems are replicating the object store or are replicating within a particular cluster of storage systems replicating the object store.

The above example implementations of high availability and disaster recovery for replicated object stores focus on a leader/follower model for a cluster of storage systems (e.g., storage systems **2610**, **2612**, **2614**) that symmetrically and synchronously replicate the bucket **2640**. However, alternative arrangements may use mixed modes of replication based on locality. For example, pairs of storage systems located in the same region may use one mode of replication, while pairs of storage systems located in different regions may use a different mode of replication. For example, where the localities **2720**, **2722** of storage systems **2610**, **2612** correspond to different availability zones of a first region and locality **2724** of storage system **2614** corresponds to a second region, storage systems **2610**, **2612** may form an in-region cluster for symmetrical and synchronous replication of the bucket, while storage systems **2610**, **2612** in the first region non-synchronous replicates updates with storage system **2614** in the second region based on an eventual consistency model.

The above example implementations of high availability and disaster recovery for replicated object stores focus on storage systems that symmetrically replicate the bucket **2640**. However, alternative arrangements may use mixed modes of replication based on both symmetrical bidirectional replication and non-synchronous directional replication. To aid illustration, an additional storage system (not shown) that is not a member of a symmetrical replication cluster for the bucket. In such an example, storage systems **2610**, **2612**, **2614** may form a cluster that symmetrically replicates the bucket, whereas each storage system **2610**, **2612**, **2614** non-synchronously and directionally replicate objects in the bucket to the additional non-symmetrical storage system. Such models can be used to distribute objects or a bucket to alternate locations, such as for test/dev or for applications like media streaming where copies of data at alternate locations can reduce cross-region network traffic. Such models can also be used for disaster recovery where some loss of service at one location can result in takeover of the service at another location, and where there is some acceptance of data loss in exchange for getting the service back up and running. For example, if some failure

118

results in storage systems **2610**, **2612**, **2614** being taken offline for the replicated bucket, the additional storage system that is the target of the directional replication can take over the servicing of requests directed to the bucket, and the direction of replication can be reversed.

Although some embodiments are described largely in the context of a storage system, readers of skill in the art will recognize that embodiments of the present disclosure may also take the form of a computer program product disposed upon computer readable storage media for use with any suitable processing system. Such computer readable storage media may be any storage medium for machine-readable information, including magnetic media, optical media, solid-state media, or other suitable media. Examples of such media include magnetic disks in hard drives or diskettes, compact disks for optical drives, magnetic tape, and others as will occur to those of skill in the art. Persons skilled in the art will immediately recognize that any computer system having suitable programming means will be capable of executing the steps described herein as embodied in a computer program product. Persons skilled in the art will recognize also that, although some of the embodiments described in this specification are oriented to software installed and executing on computer hardware, nevertheless, alternative embodiments implemented as firmware or as hardware are well within the scope of the present disclosure.

In some examples, a non-transitory computer-readable medium storing computer-readable instructions may be provided in accordance with the principles described herein. The instructions, when executed by a processor of a computing device, may direct the processor and/or computing device to perform one or more operations, including one or more of the operations described herein. Such instructions may be stored and/or transmitted using any of a variety of known computer-readable media.

A non-transitory computer-readable medium as referred to herein may include any non-transitory storage medium that participates in providing data (e.g., instructions) that may be read and/or executed by a computing device (e.g., by a processor of a computing device). For example, a non-transitory computer-readable medium may include, but is not limited to, any combination of non-volatile storage media and/or volatile storage media. Exemplary non-volatile storage media include, but are not limited to, read-only memory, flash memory, a solid-state drive, a magnetic storage device (e.g., a hard disk, a floppy disk, magnetic tape, etc.), ferroelectric random-access memory ("RAM"), and an optical disc (e.g., a compact disc, a digital video disc, a Blu-ray disc, etc.). Exemplary volatile storage media include, but are not limited to, RAM (e.g., dynamic RAM).

Advantages and features of the present disclosure can be further described by the following statements:

1. A method comprising: determining, for an object store replicated across a plurality of storage systems at a plurality of locations, a minimum time interval between requests to modify the object store that are received by differing storage systems of the plurality of storage systems that ensures a predictable result on all storage systems among which those modifications are replicated; and providing, through an application programming interface (API) to a client that utilizes the object store, one or more object store parameters including the minimum time interval.

2. The method of statement 1 wherein at least one pair of the plurality of storage systems bidirectionally replicate using an eventual consistency model.

3. The method of statement 2 or statement 1 wherein clock values are exchanged between pairs of replicating storage systems.

4. The method of statement 3, statement 2, or statement 1 wherein the exchanged clock values are utilized to order operations received by different storage systems among the plurality of storage systems.

5. The method of statement 4, statement 3, statement 2, or statement 1 wherein the exchanged clock values are based on respective local monotonic clocks of the plurality of storage systems.

6. The method of statement 5, statement 4, statement 3, statement 2, or statement 1 wherein a first request to store a first version of an object is received by a first storage system of the plurality of storage systems and a second request to store a second version of the object is received by a second storage system of the plurality of storage systems, and wherein the second version is ordered consistently after the first version by all storage systems that store replicas of the first version and the second version when the first request and the second request are separated by at least the minimum time interval.

7. The method of statement 6, statement 5, statement 4, statement 3, statement 2, or statement 1 wherein the minimum time interval is based, in part, on a round-trip time for messages between pairs of storage systems.

8. The method of statement 7, statement 6, statement 5, statement 4, statement 3, statement 2, or statement 1 wherein the minimum time interval is based, in part, on known physical distances between pairs of storage systems.

9. The method of statement 8, statement 7, statement 6, statement 5, statement 4, statement 3, statement 2, or statement 1 wherein the minimum time interval is modified based on recency since a last completed round-trip between pairs of storage systems.

10. The method of statement 9, statement 8, statement 7, statement 6, statement 5, statement 4, statement 3, statement 2, or statement 1 wherein the one or more object store parameters includes a parameter indicating that the minimum time interval should increase over time if it is not refreshed through subsequent API calls.

11. The method of statement 10, statement 9, statement 8, statement 7, statement 6, statement 5, statement 4, statement 3, statement 2, or statement 1 wherein object store parameters are associated with a timed lease.

12. The method of statement 11, statement 10, statement 9, statement 8, statement 7, statement 6, statement 5, statement 4, statement 3, statement 2, or statement 1 where at least two of the storage systems replicating at least some objects of the object store between them are in different geographic regions.

13. The method of statement 12, statement 11, statement 10, statement 9, statement 8, statement 7, statement 6, statement 5, statement 4, statement 3, statement 2, or statement 1 wherein at least two of the storage systems replicating at least some objects of the object store between them are in different availability zones within a geographic region.

14. The method of statement 13, statement 12, statement 11, statement 10, statement 9, statement 8, statement 7, statement 6, statement 5, statement 4, statement 3, statement 2, or statement 1 wherein different pairs of storage systems among the plurality of storage systems are associated with different minimum time intervals.

15. The method of statement 14, statement 13, statement 12, statement 11, statement 10, statement 9, statement 8, statement 7, statement 6, statement 5, statement 4, statement 3, statement 2, or statement 1 wherein the one or more object

store parameters includes timing parameters related to fault handling and failure recovery in an event where one or more of the plurality of storage systems takes over servicing of objects in the object store in response to a fault.

16. The method of statement 15, statement 13, statement 12, statement 11, statement 10, statement 9, statement 8, statement 7, statement 6, statement 5, statement 4, statement 3, statement 2, or statement 1 wherein a copy operation on an object is applied consistently by all storage systems using the same source version of the object.

One or more embodiments may be described herein with the aid of method steps illustrating the performance of specified functions and relationships thereof. The boundaries and sequence of these functional building blocks and method steps have been arbitrarily defined herein for convenience of description. Alternate boundaries and sequences can be defined so long as the specified functions and relationships are appropriately performed. Any such alternate boundaries or sequences are thus within the scope and spirit of the claims. Further, the boundaries of these functional building blocks have been arbitrarily defined for convenience of description. Alternate boundaries could be defined as long as the certain significant functions are appropriately performed. Similarly, flow diagram blocks may also have been arbitrarily defined herein to illustrate certain significant functionality.

To the extent used, the flow diagram block boundaries and sequence could have been defined otherwise and still perform the certain significant functionality. Such alternate definitions of both functional building blocks and flow diagram blocks and sequences are thus within the scope and spirit of the claims. One of average skill in the art will also recognize that the functional building blocks, and other illustrative blocks, modules and components herein, can be implemented as illustrated or by discrete components, application specific integrated circuits, processors executing appropriate software and the like or any combination thereof.

While particular combinations of various functions and features of the one or more embodiments are expressly described herein, other combinations of these features and functions are likewise possible. The present disclosure is not limited by the particular examples disclosed herein and expressly incorporates these other combinations.

One or more embodiments may be described herein with the aid of method steps illustrating the performance of specified functions and relationships thereof. The boundaries and sequence of these functional building blocks and method steps have been arbitrarily defined herein for convenience of description. Alternate boundaries and sequences can be defined so long as the specified functions and relationships are appropriately performed. Any such alternate boundaries or sequences are thus within the scope and spirit of the claims. Further, the boundaries of these functional building blocks have been arbitrarily defined for convenience of description. Alternate boundaries could be defined as long as the certain significant functions are appropriately performed. Similarly, flow diagram blocks may also have been arbitrarily defined herein to illustrate certain significant functionality.

To the extent used, the flow diagram block boundaries and sequence could have been defined otherwise and still perform the certain significant functionality. Such alternate definitions of both functional building blocks and flow diagram blocks and sequences are thus within the scope and spirit of the claims. One of average skill in the art will also recognize that the functional building blocks, and other

121

illustrative blocks, modules and components herein, can be implemented as illustrated or by discrete components, application specific integrated circuits, processors executing appropriate software and the like or any combination thereof.

While particular combinations of various functions and features of the one or more embodiments are expressly described herein, other combinations of these features and functions are likewise possible. The present disclosure is not limited by the particular examples disclosed herein and expressly incorporates these other combinations.

What is claimed is:

1. A method comprising:
 - determining, for an object store replicated across a plurality of storage systems at a plurality of locations, a minimum time interval between requests to apply modifications to the object store that are received by differing storage systems of the plurality of storage systems that ensures a predictable result on one or more storage systems, of the plurality of storage systems, among which those modifications are replicated; and
 - providing, through an application programming interface (API) to a client that utilizes the object store, one or more object store parameters including the minimum time interval.
2. The method of claim 1, wherein at least one pair of the plurality of storage systems bidirectionally replicate the object store using an eventual consistency model.
3. The method of claim 1, wherein clock values are exchanged between pairs of replicating storage systems.
4. The method of claim 3, wherein the exchanged clock values are utilized to order operations received by different storage systems among the plurality of storage systems.
5. The method of claim 4, wherein the exchanged clock values are based on respective local monotonic clocks of the plurality of storage systems.
6. The method of claim 4, wherein a first request to store a first version of an object is received by a first storage system of the plurality of storage systems and a second request to store a second version of the object is received by a second storage system of the plurality of storage systems, and wherein the second version is ordered consistently after the first version by any storage systems, of the plurality of storage systems, that store replicas of the first version and the second version, wherein the first request and the second request are separated by at least the minimum time interval.
7. The method of claim 1, wherein the minimum time interval is based, in part, on a round-trip time for messages between pairs of storage systems.
8. The method of claim 1, wherein the minimum time interval is based, in part, on known physical distances between pairs of storage systems.
9. The method of claim 1, wherein the minimum time interval is modified based on recency since a last completed round-trip between pairs of storage systems.
10. The method of claim 1, wherein the one or more object store parameters includes a parameter indicating that the minimum time interval should increase over time if it is not refreshed through subsequent API calls.
11. The method of claim 1, wherein object store parameters are associated with a timed lease.

122

12. The method of claim 1, wherein at least two of the plurality of storage systems replicating at least some objects of the object store between them are in different geographic regions.

13. The method of claim 1, wherein at least two of the plurality of storage systems replicating at least some objects of the object store between them are in different availability zones within a geographic region.

14. The method of claim 1, wherein different pairs of storage systems among the plurality of storage systems are associated with different minimum time intervals.

15. The method of claim 1, wherein the one or more object store parameters includes timing parameters related to fault handling and failure recovery in an event where one or more of the plurality of storage systems takes over servicing of objects in the object store in response to a fault.

16. The method of claim 1, wherein a copy operation on an object is applied consistently by all storage systems using a same source version of the object.

17. An apparatus comprising a computer processor, a computer memory operatively coupled to the computer processor, the computer memory comprising computer program instructions that, when executed by the computer processor, cause the apparatus to carry out steps of:

determining, for an object store replicated across a plurality of storage systems at a plurality of locations, a minimum time interval between requests to apply modifications to the object store that are received by differing storage systems of the plurality of storage systems that ensures a predictable result on one or more storage systems, of the plurality of storage systems, among which those modifications are replicated; and providing, through an application programming interface (API) to a client that utilizes the object store, one or more object store parameters including the minimum time interval.

18. The apparatus of claim 17, wherein at least one pair of the plurality of storage systems bidirectionally replicate the object store using an eventual consistency model.

19. The apparatus of claim 17, wherein clock values are exchanged between pairs of replicating storage systems; and wherein the exchanged clock values are utilized to order operations received by different storage systems among the plurality of storage systems.

20. A non-transitory computer readable medium comprising a computer program product, the computer program product comprising computer program instructions that, when executed, cause a computer to carry out steps of:

determining, for an object store replicated across a plurality of storage systems at a plurality of locations, a minimum time interval between requests to apply modifications to the object store that are received by differing storage systems of the plurality of storage systems that ensures a predictable result on one or more storage systems, of the plurality of storage systems, among which those modifications are replicated; and providing, through an application programming interface (API) to a client that utilizes the object store, one or more object store parameters including the minimum time interval.

* * * * *